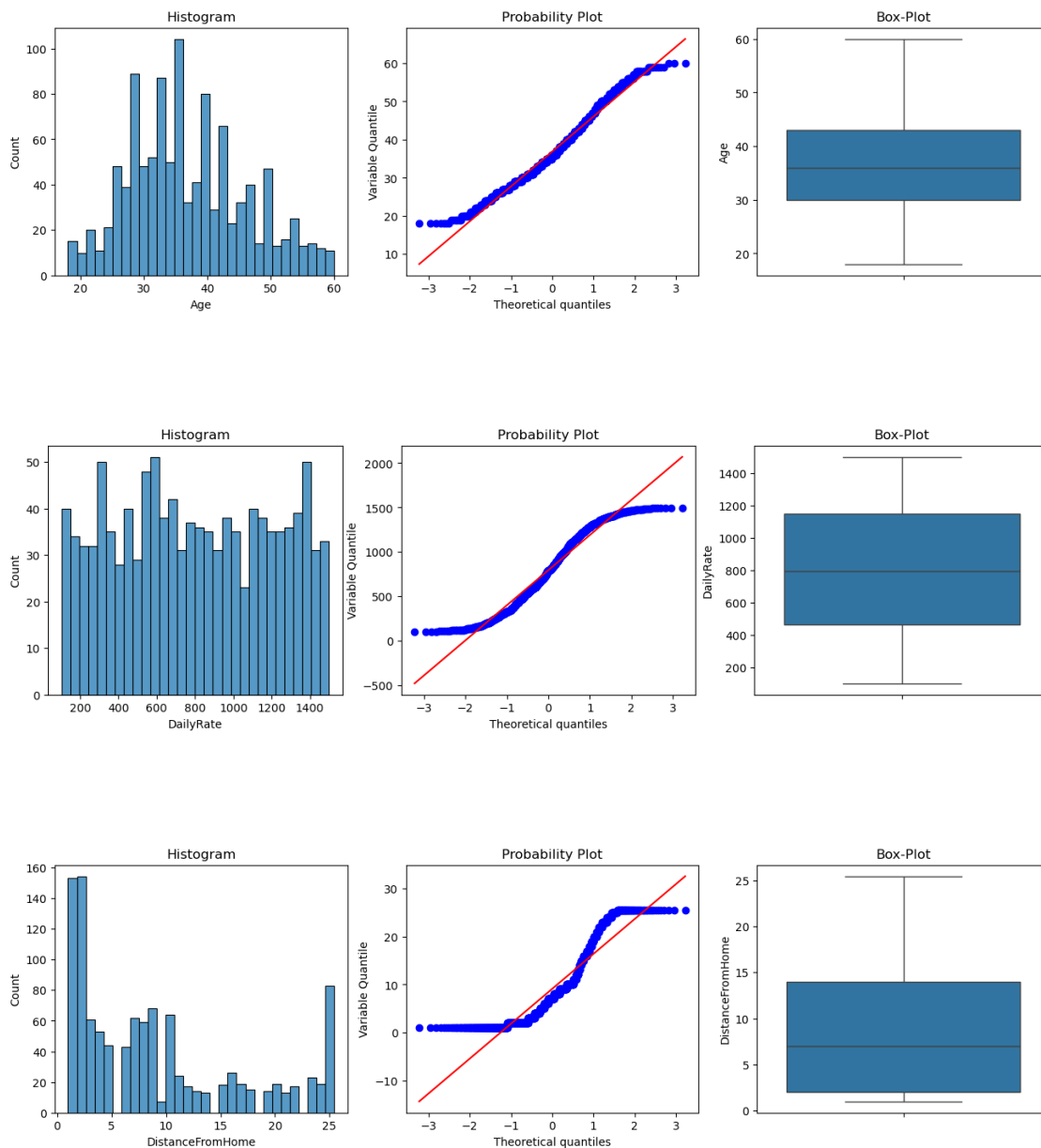
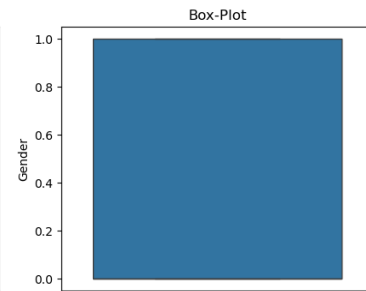
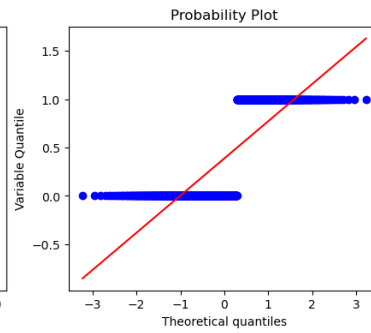
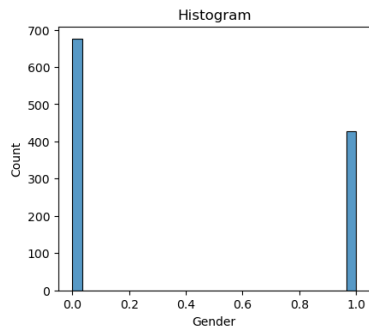
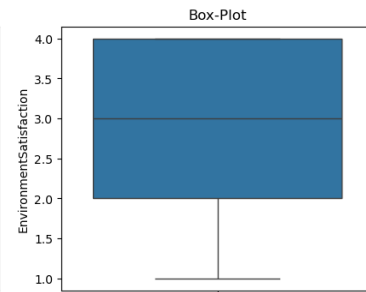
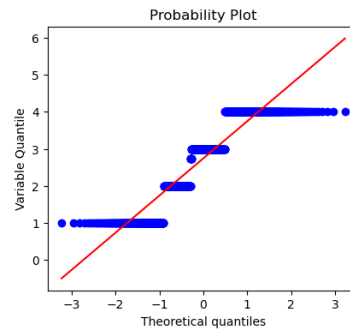
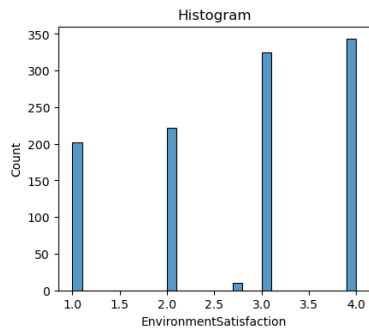
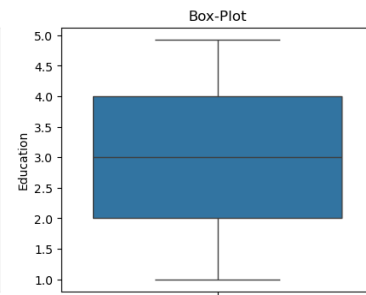
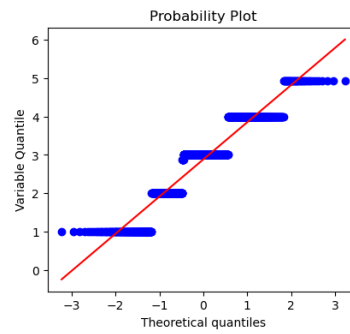
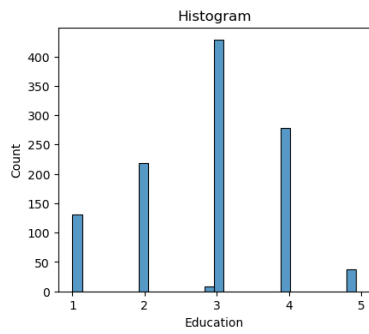


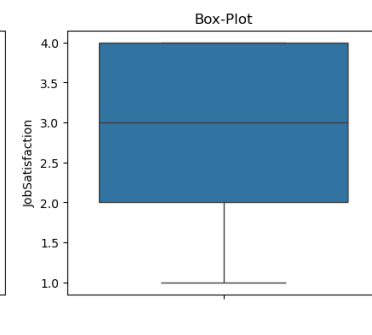
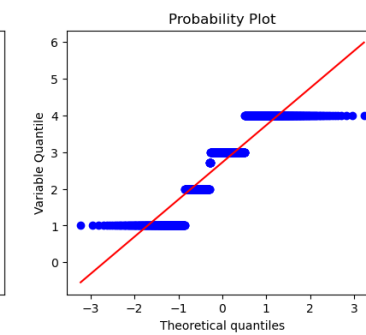
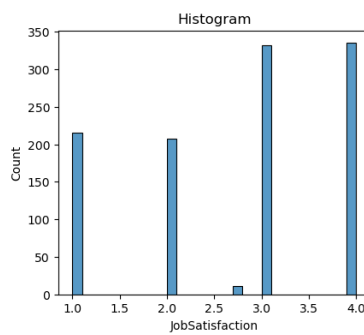
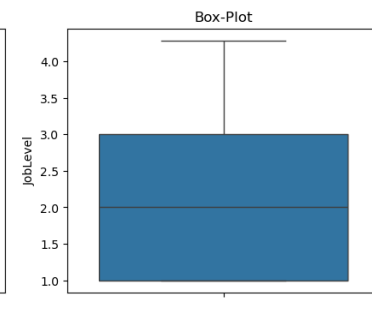
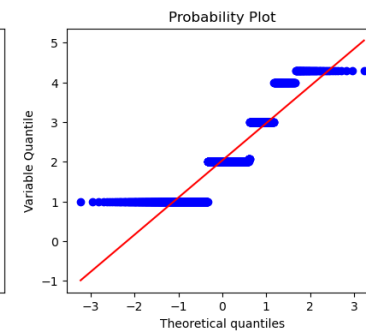
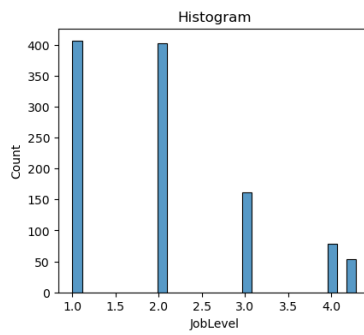
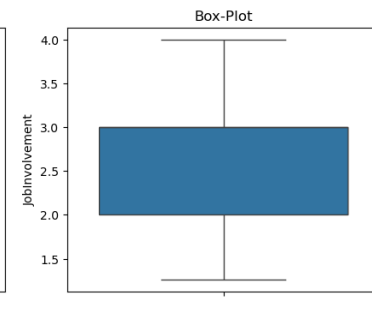
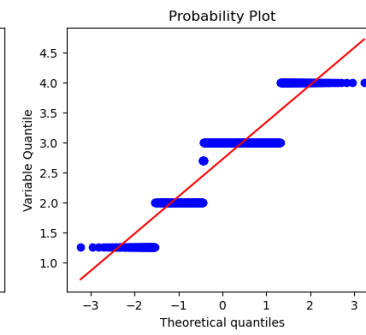
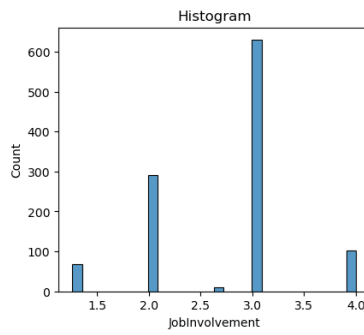
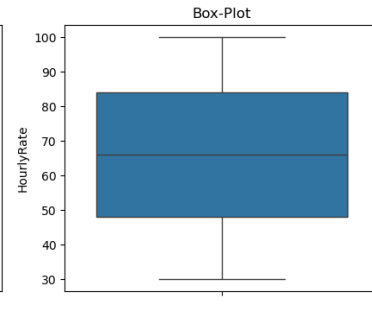
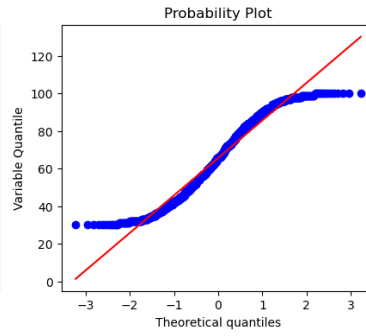
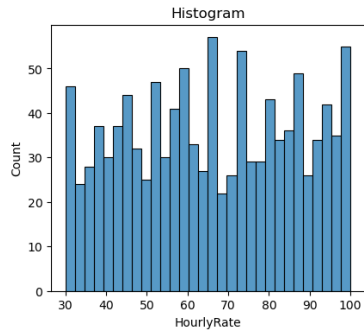
part_2

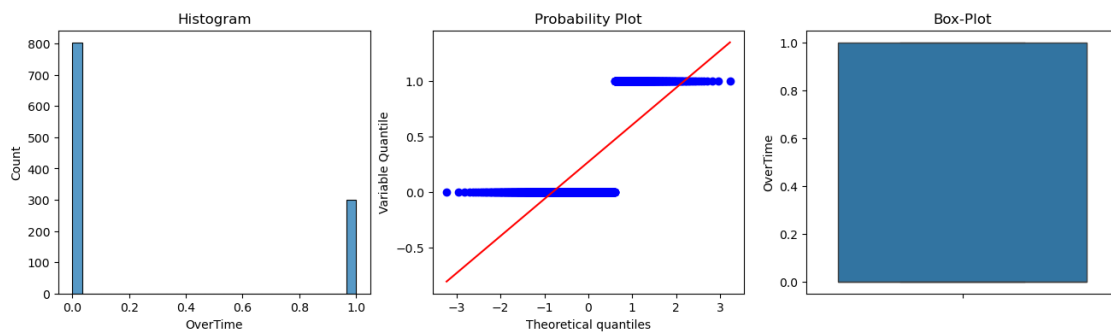
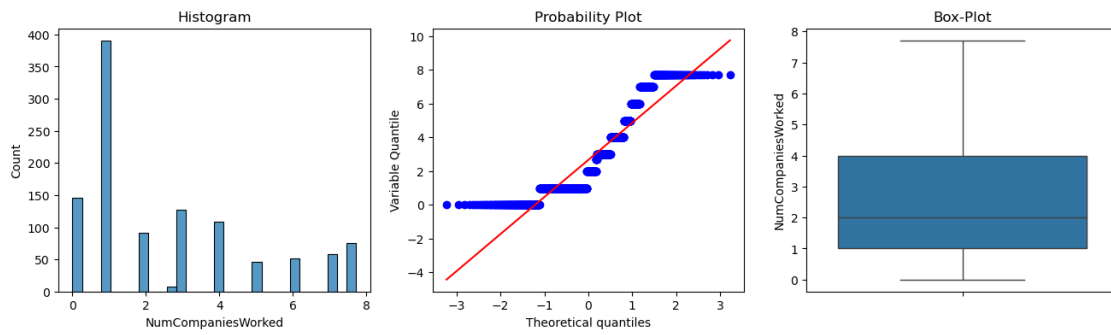
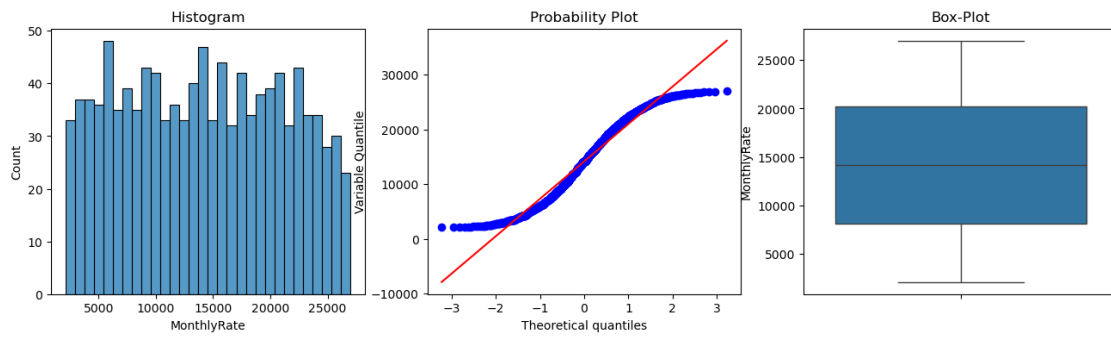
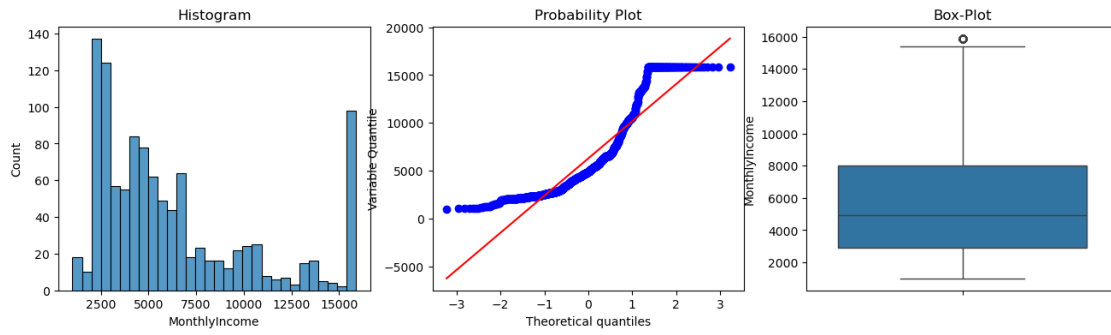
October 25, 2025

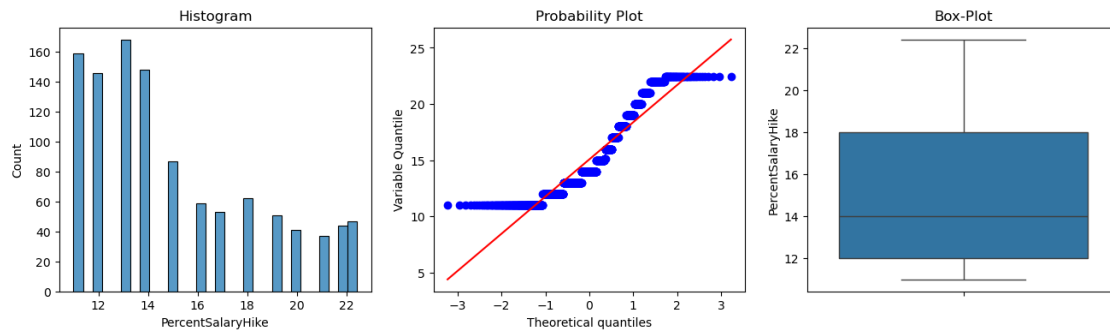
```
[31]: for col in win_train.columns:  
      diagnostic_plots(win_train, col)
```

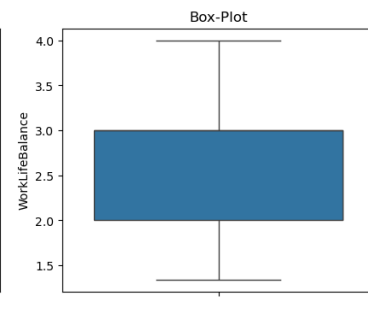
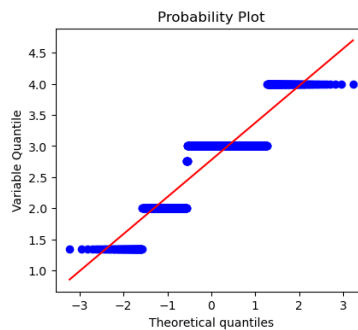
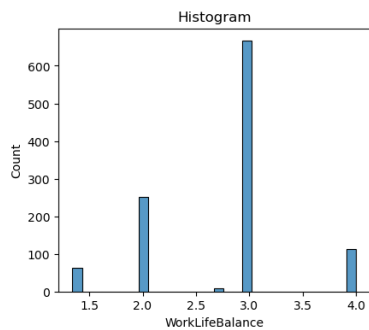
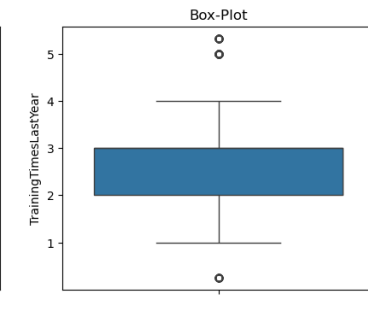
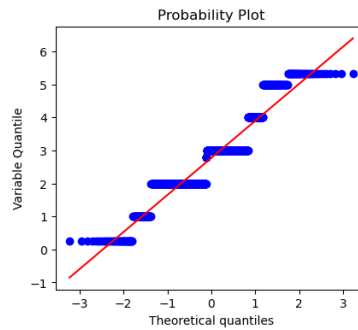
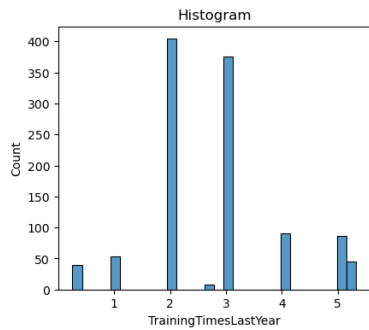
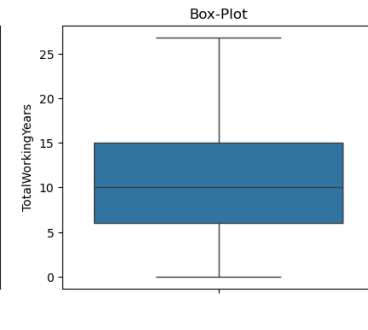
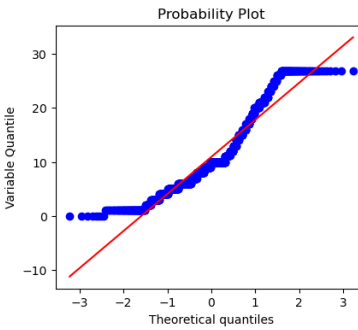
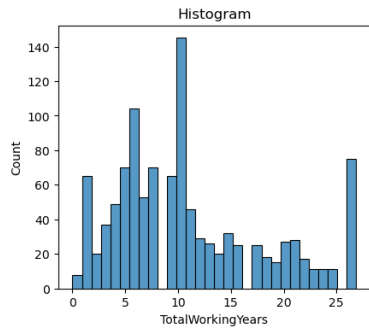
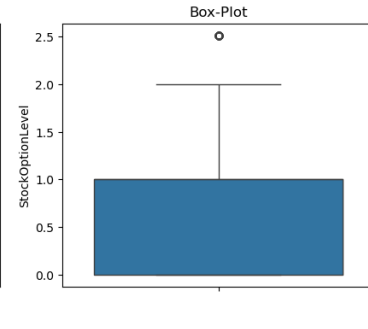
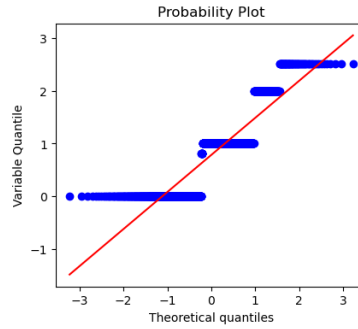
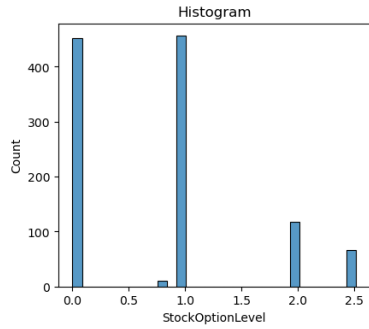


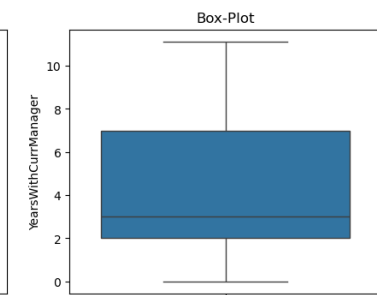
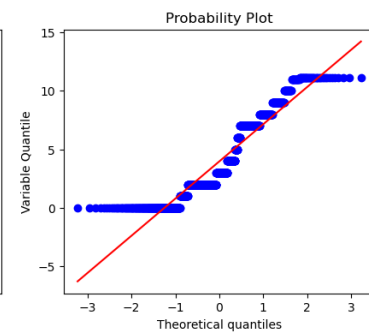
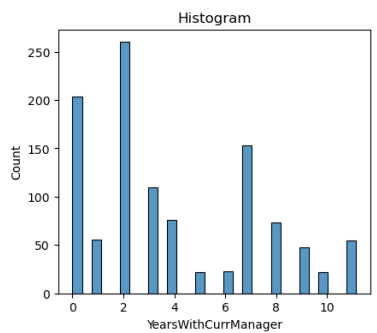
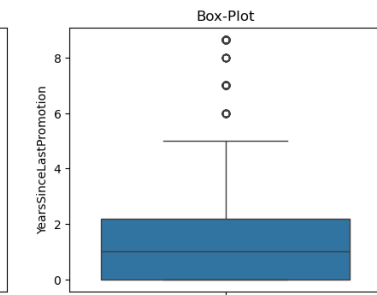
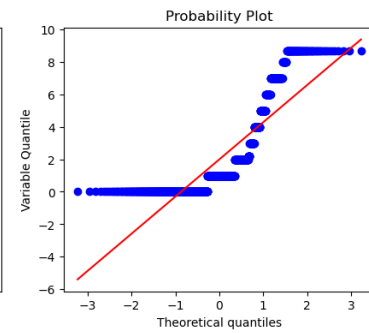
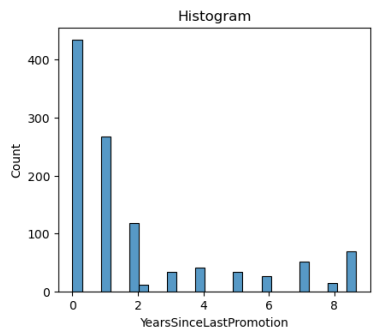
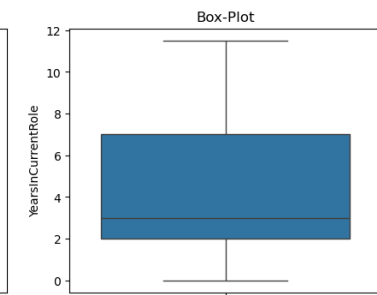
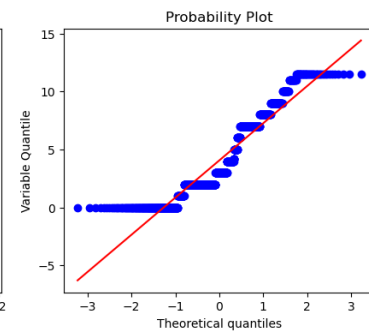
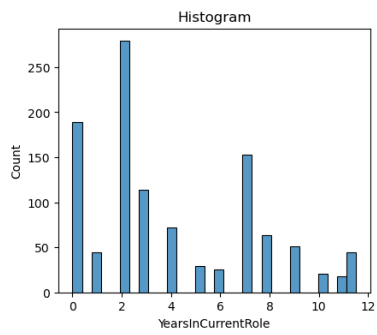
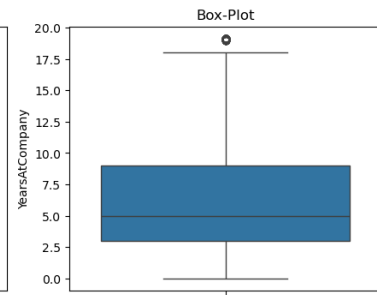
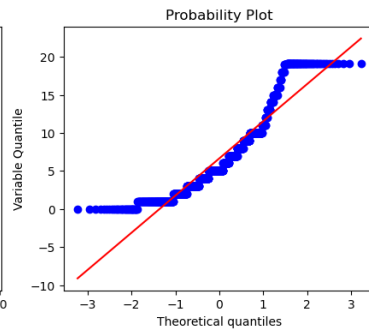
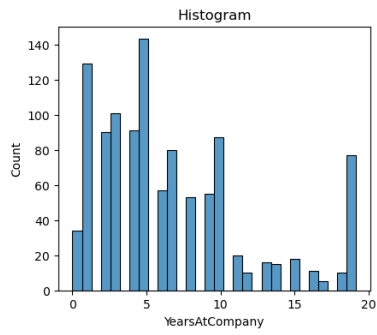


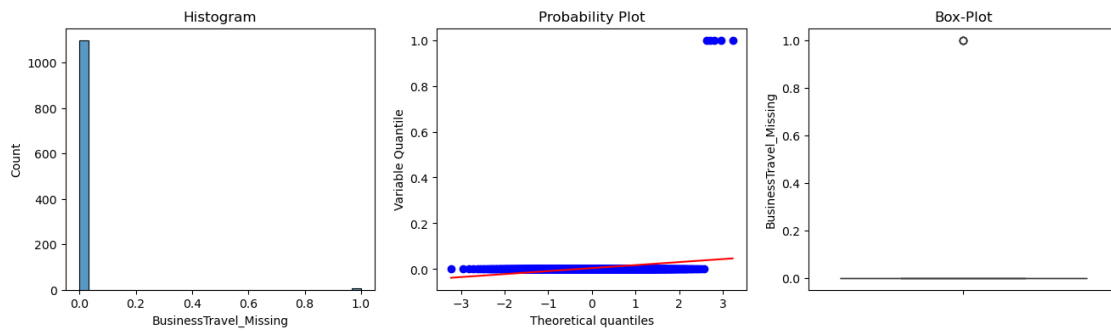
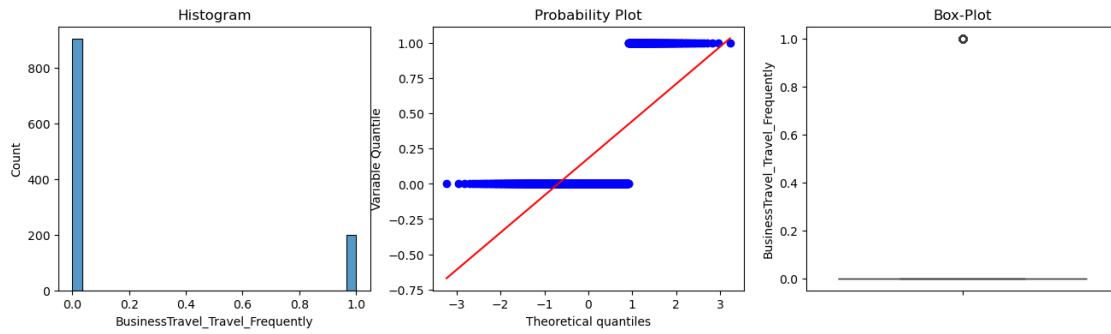
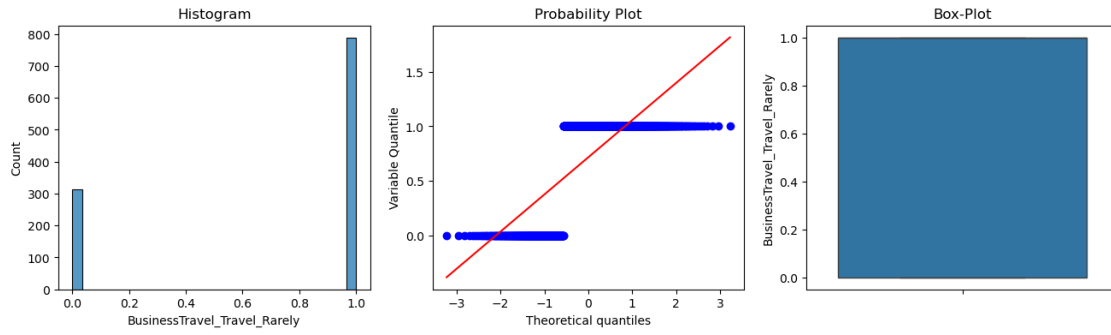


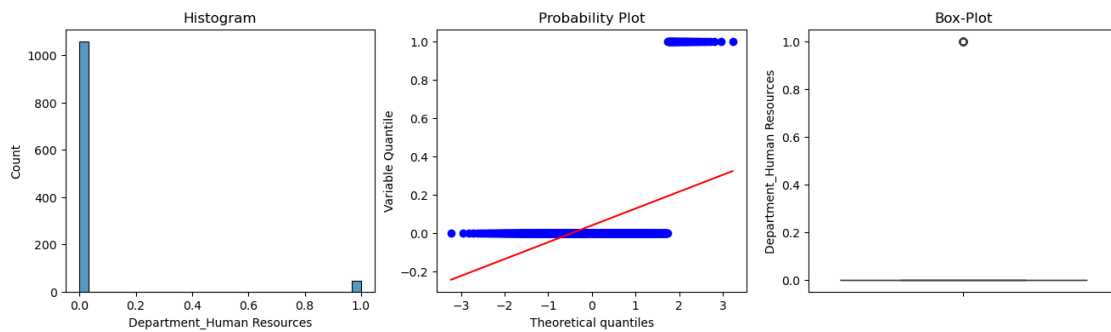
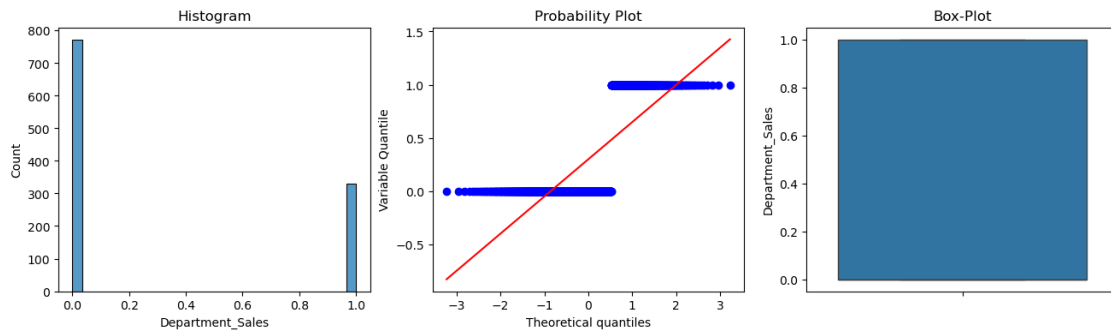
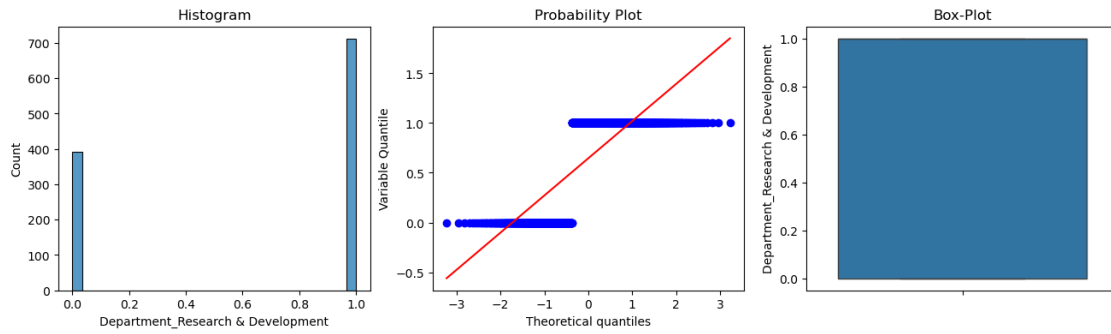
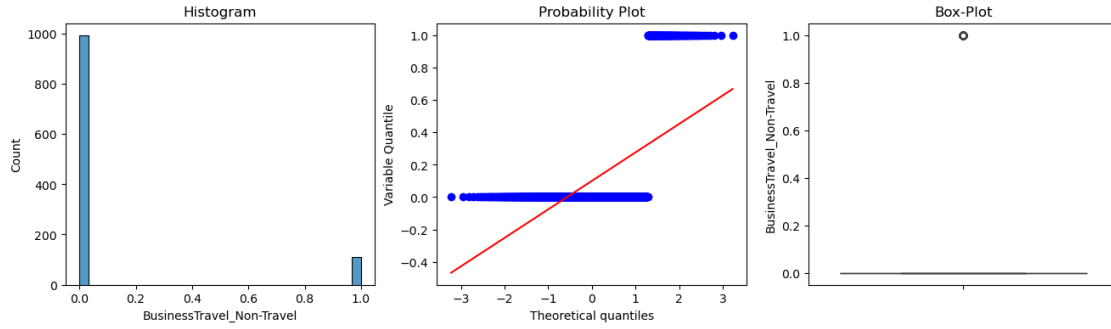


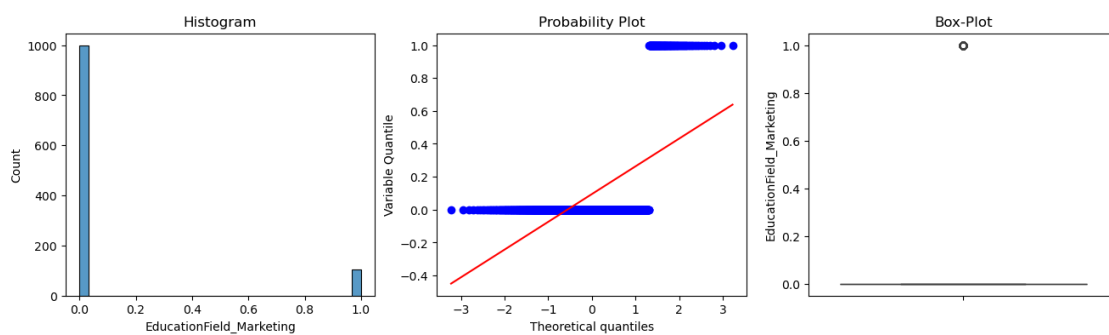
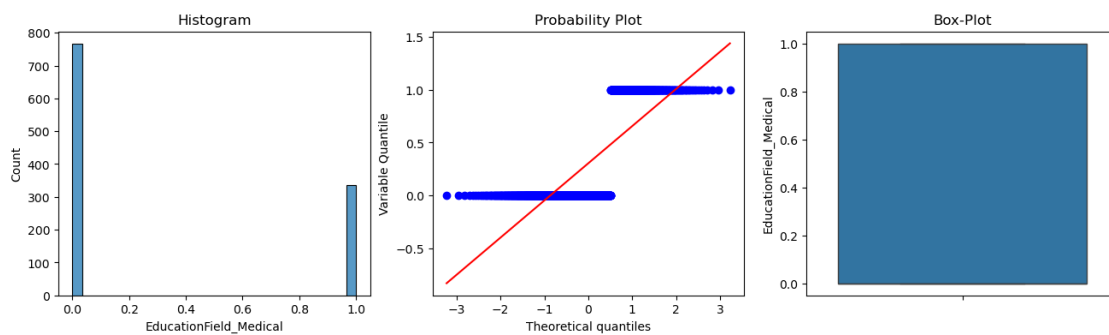
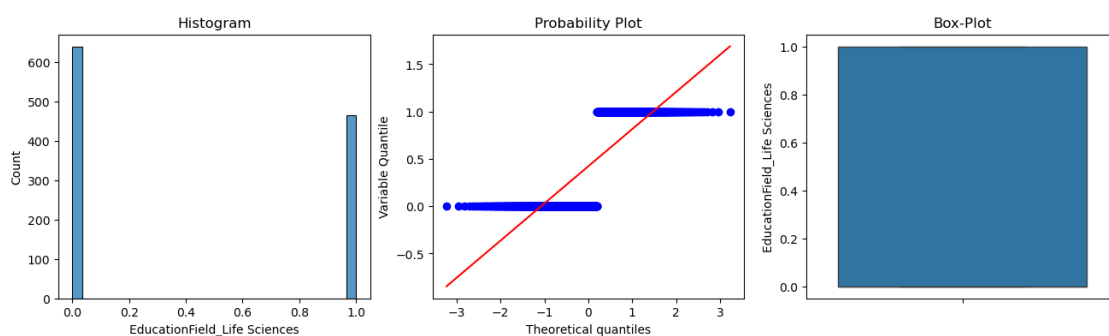
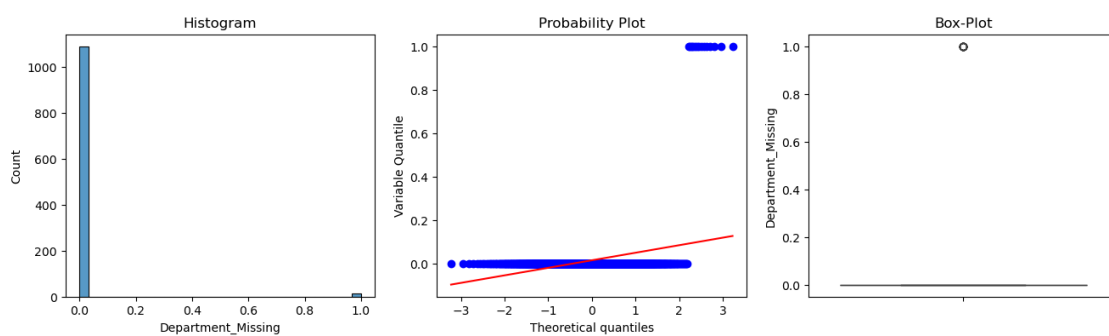


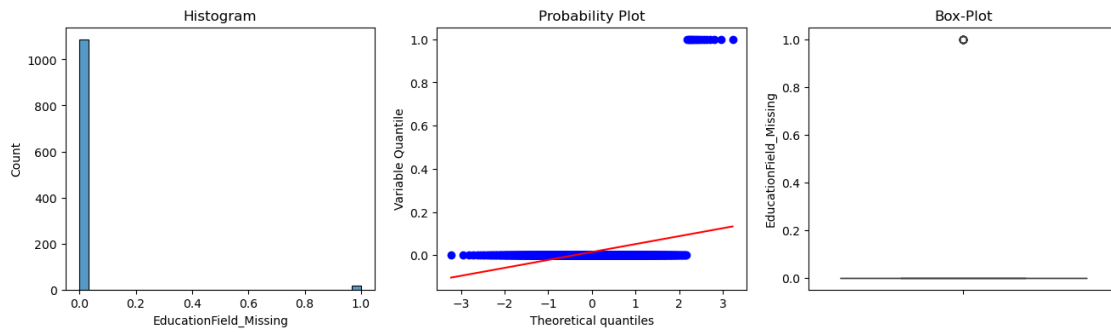
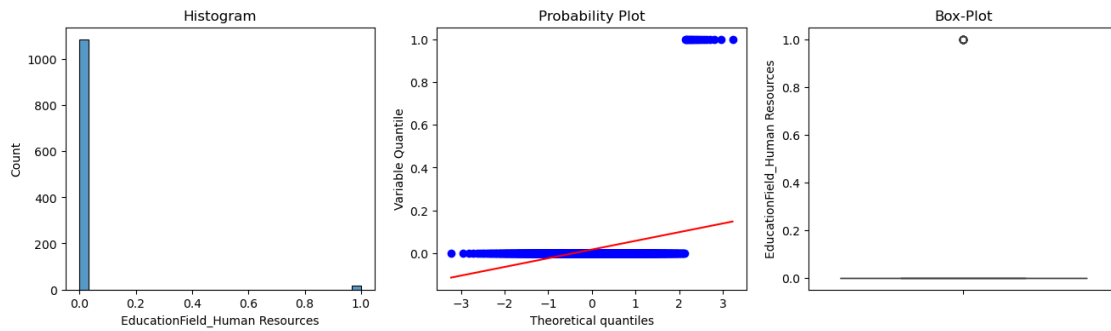
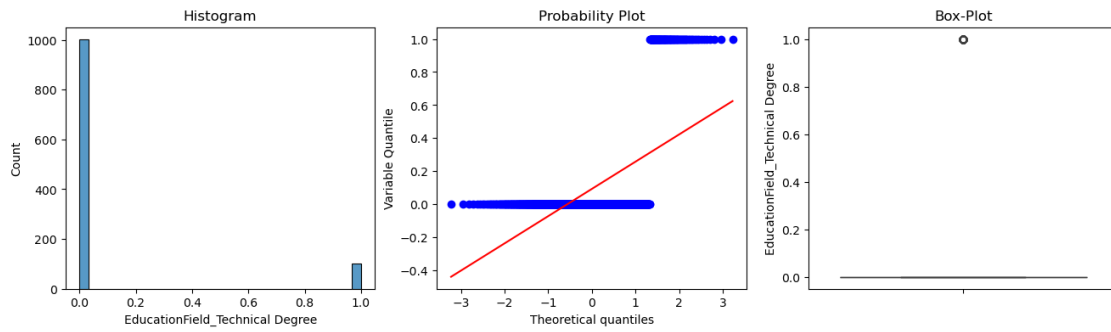


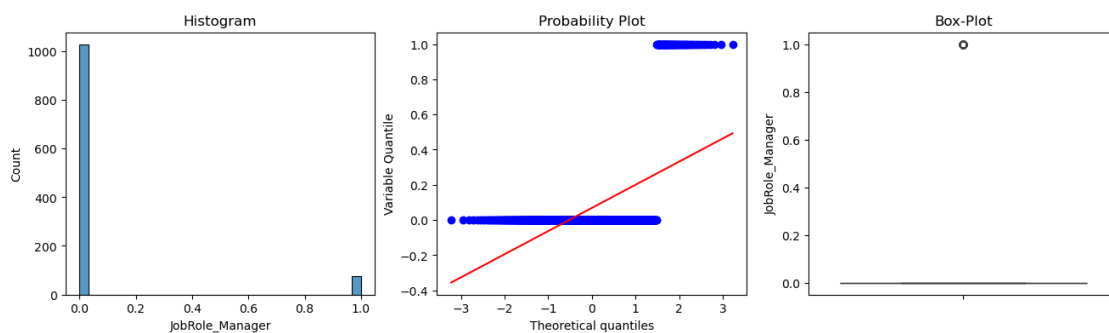
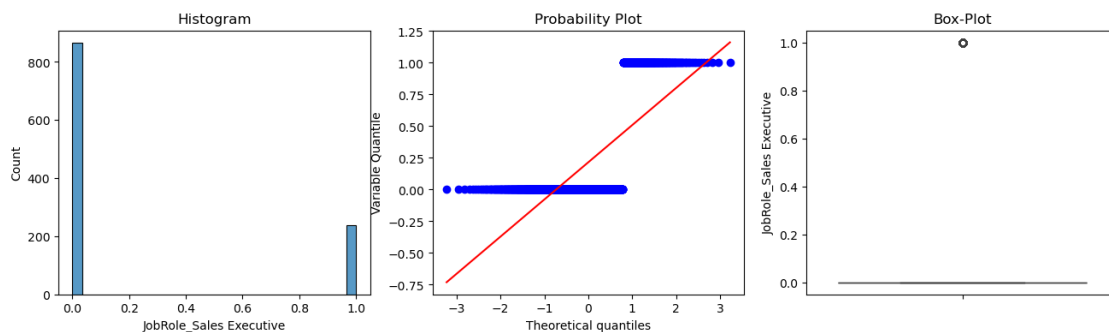
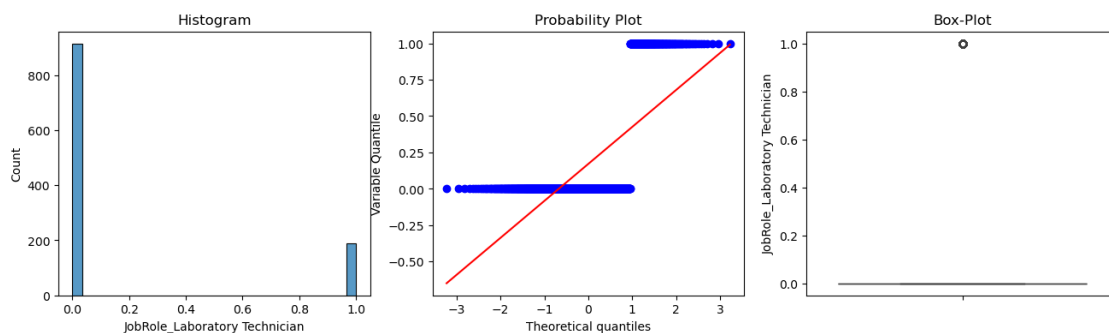
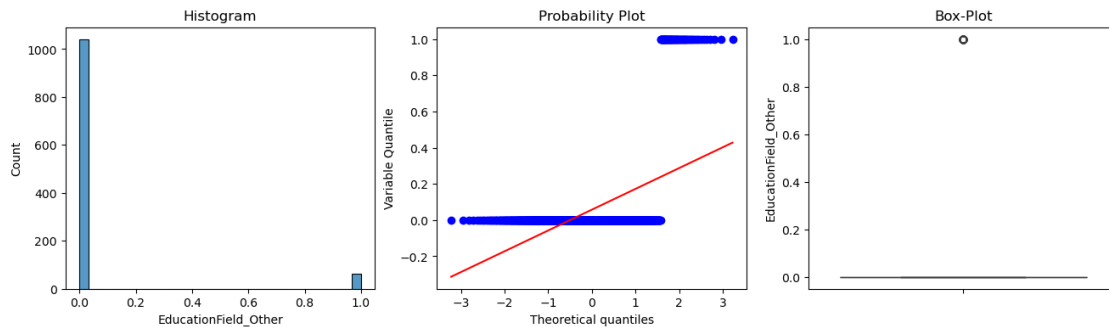


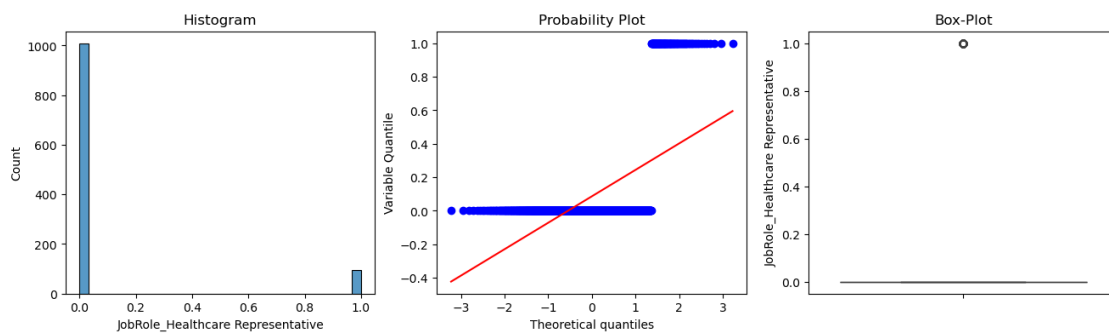
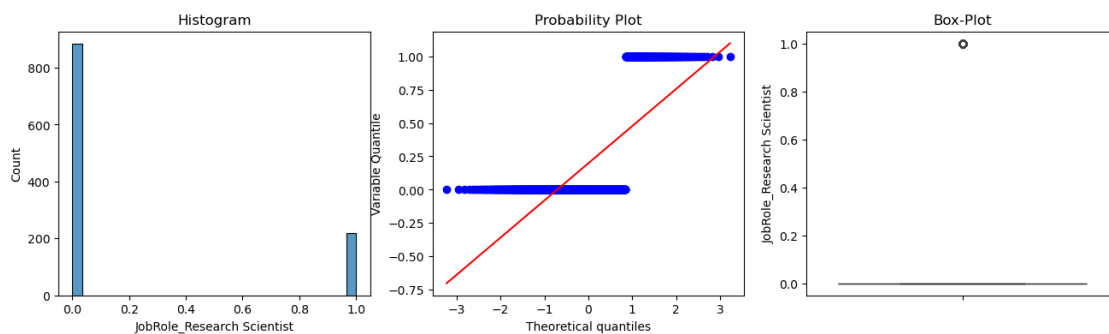
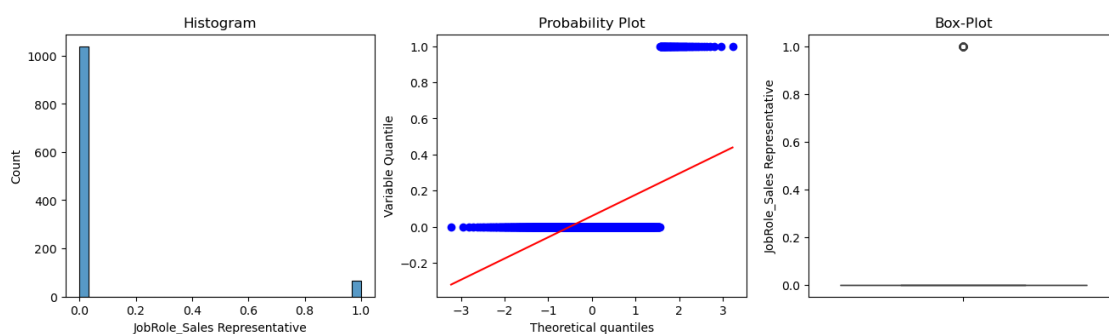
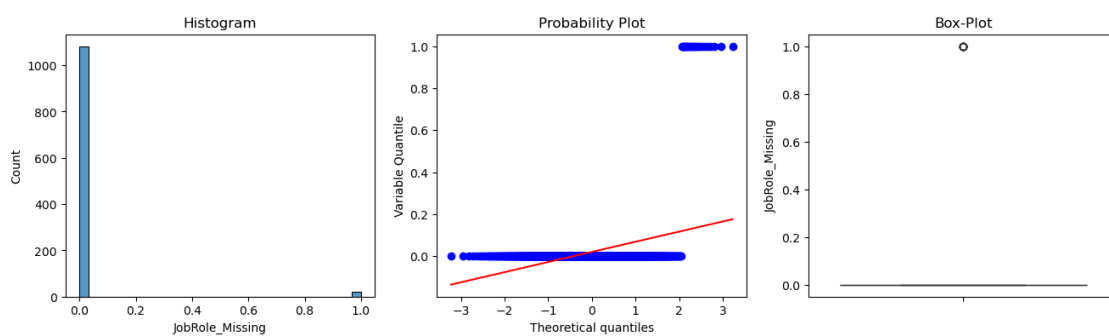


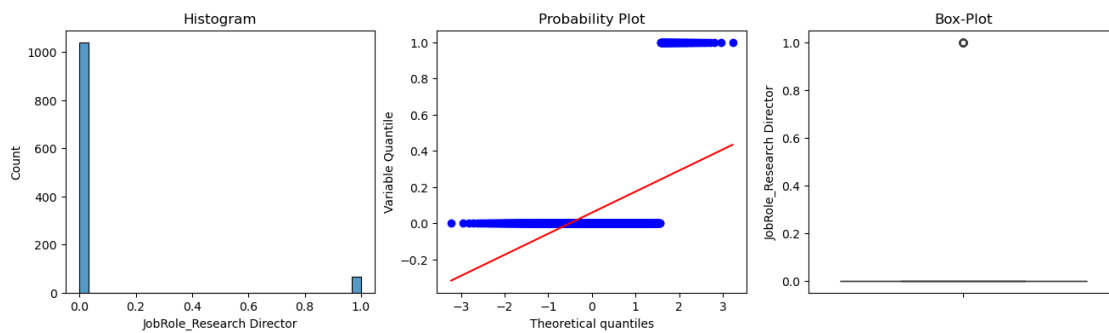
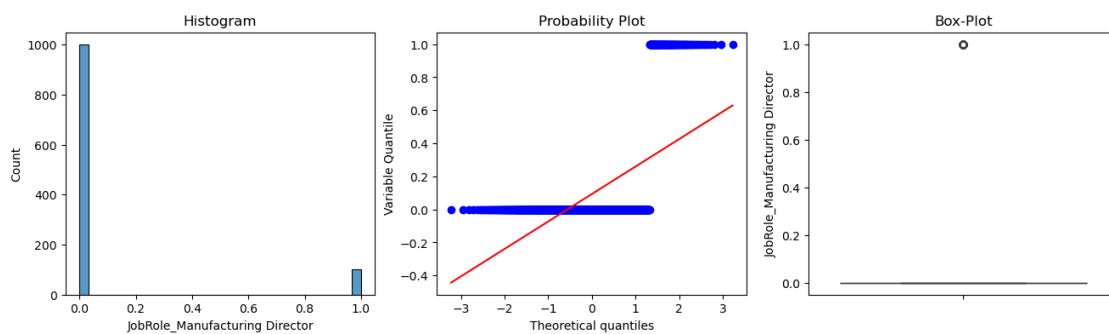
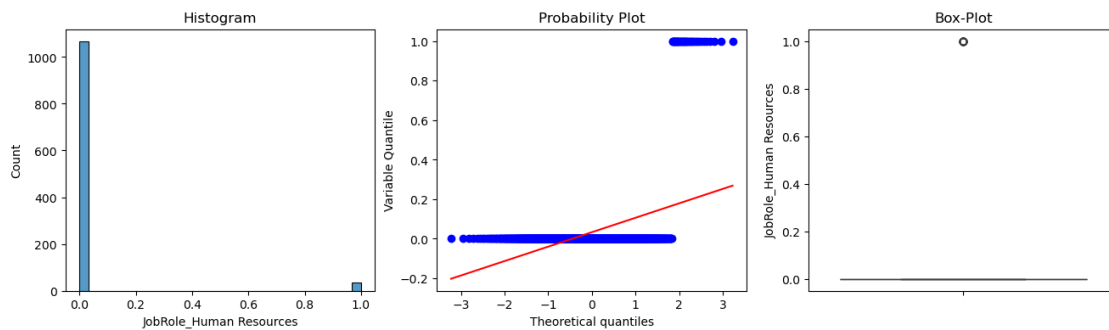


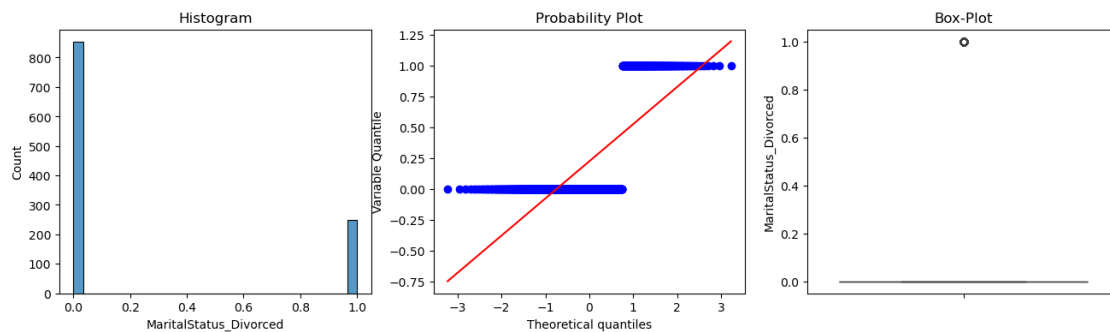
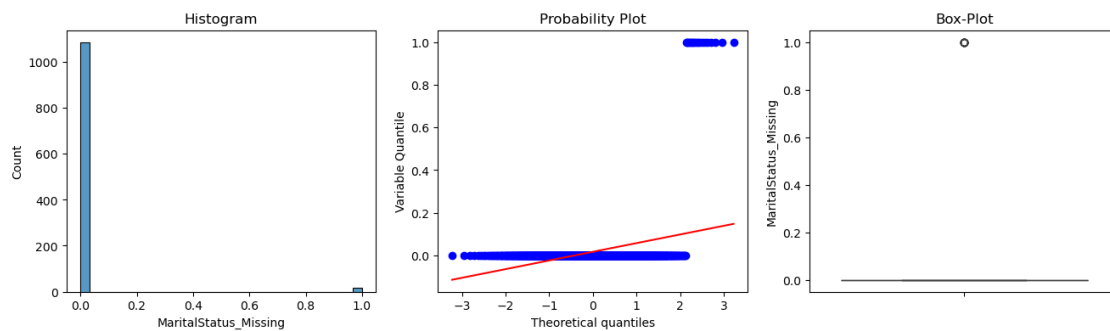
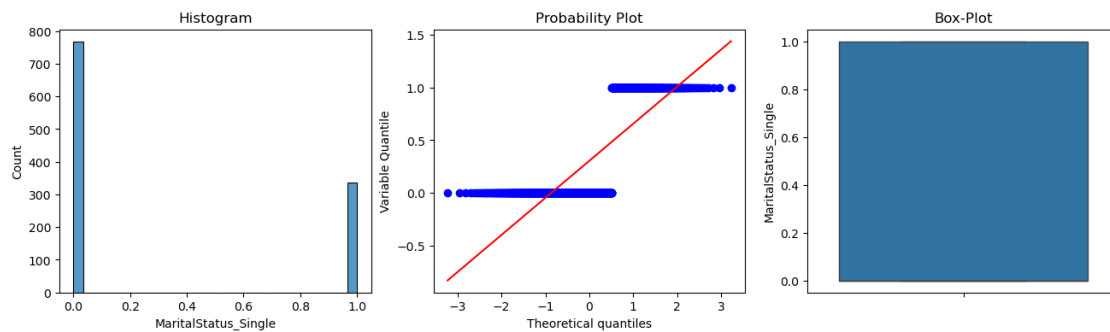
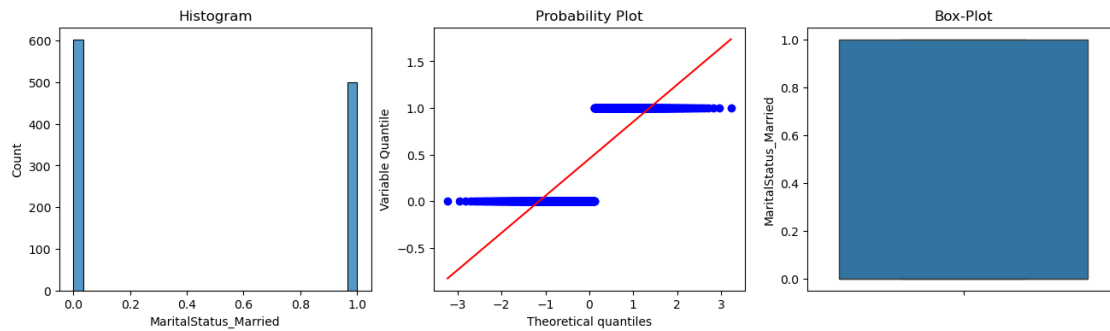








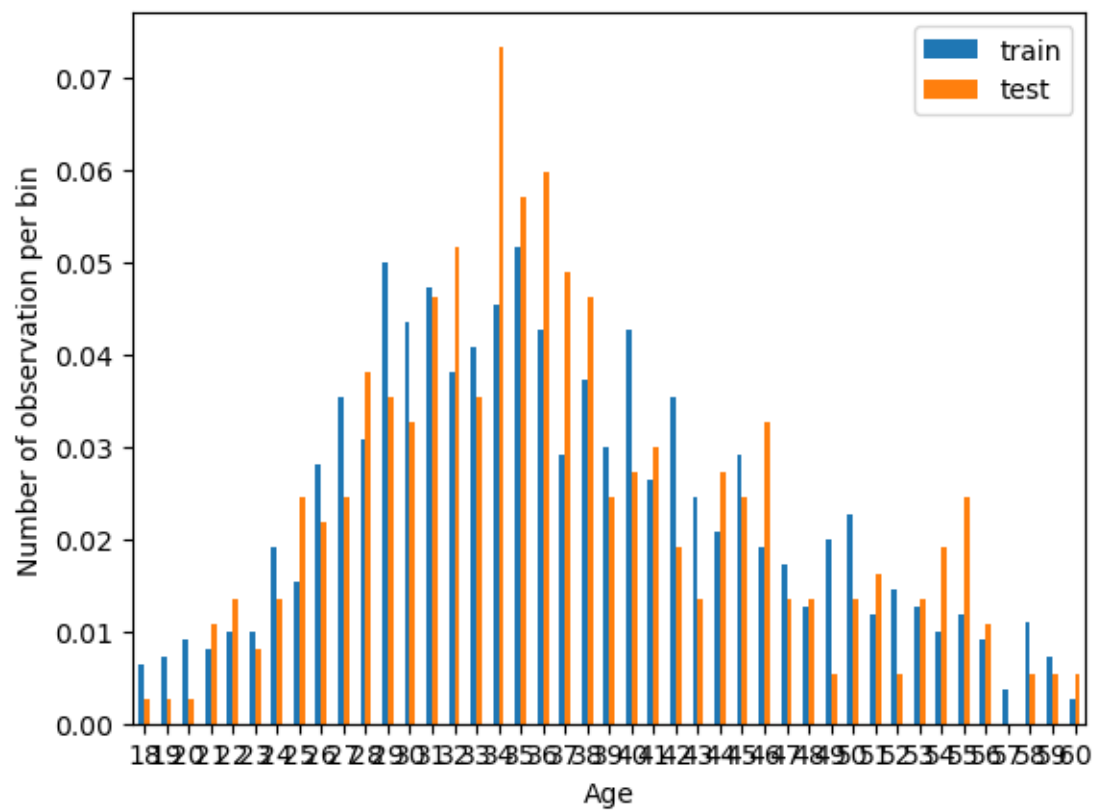


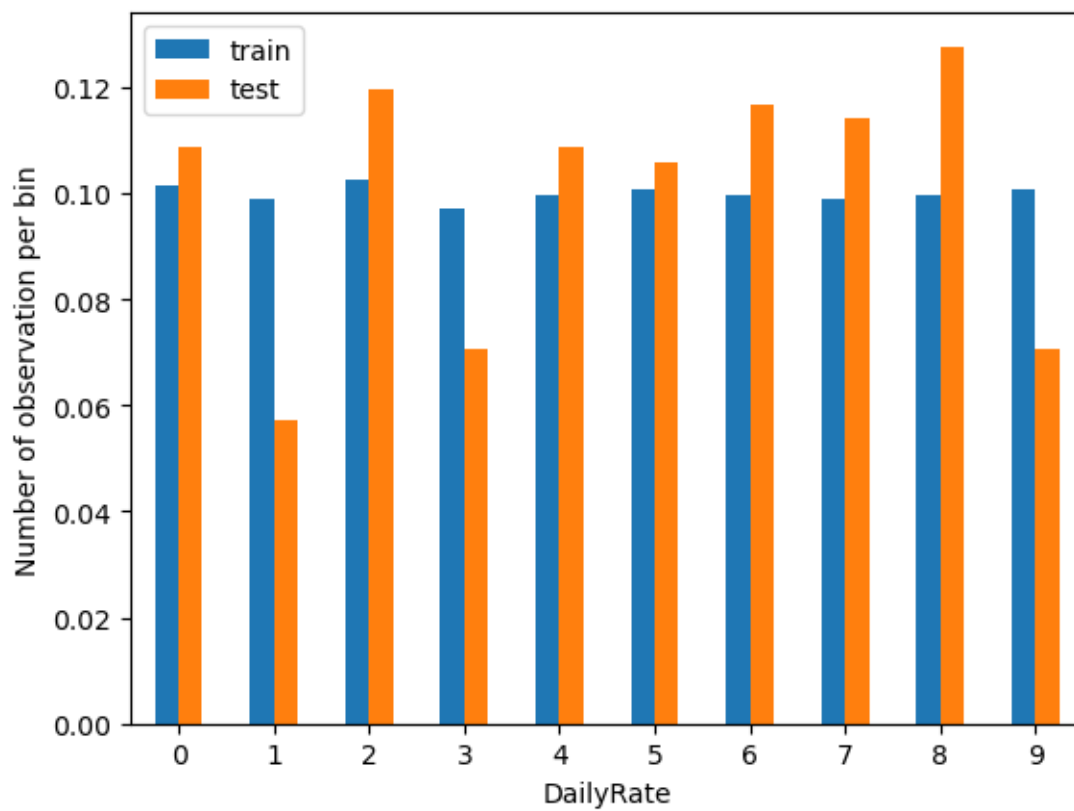


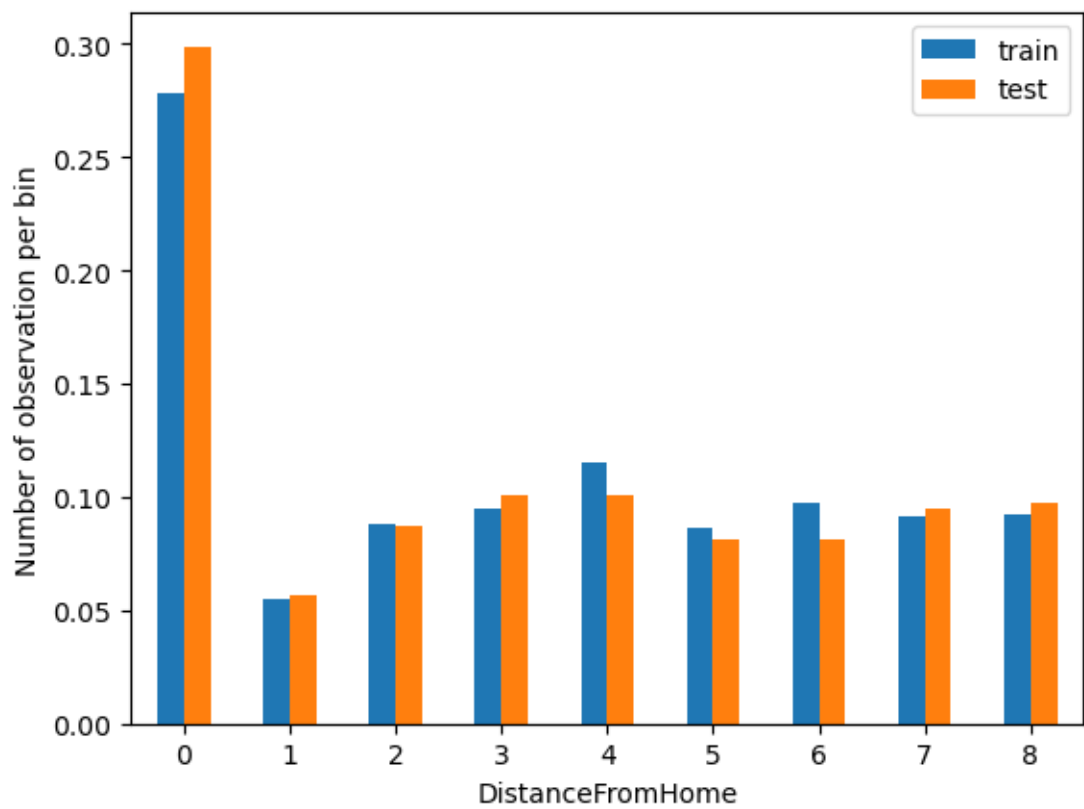
2.7 Discretization with EqualFrequencyDiscretiser

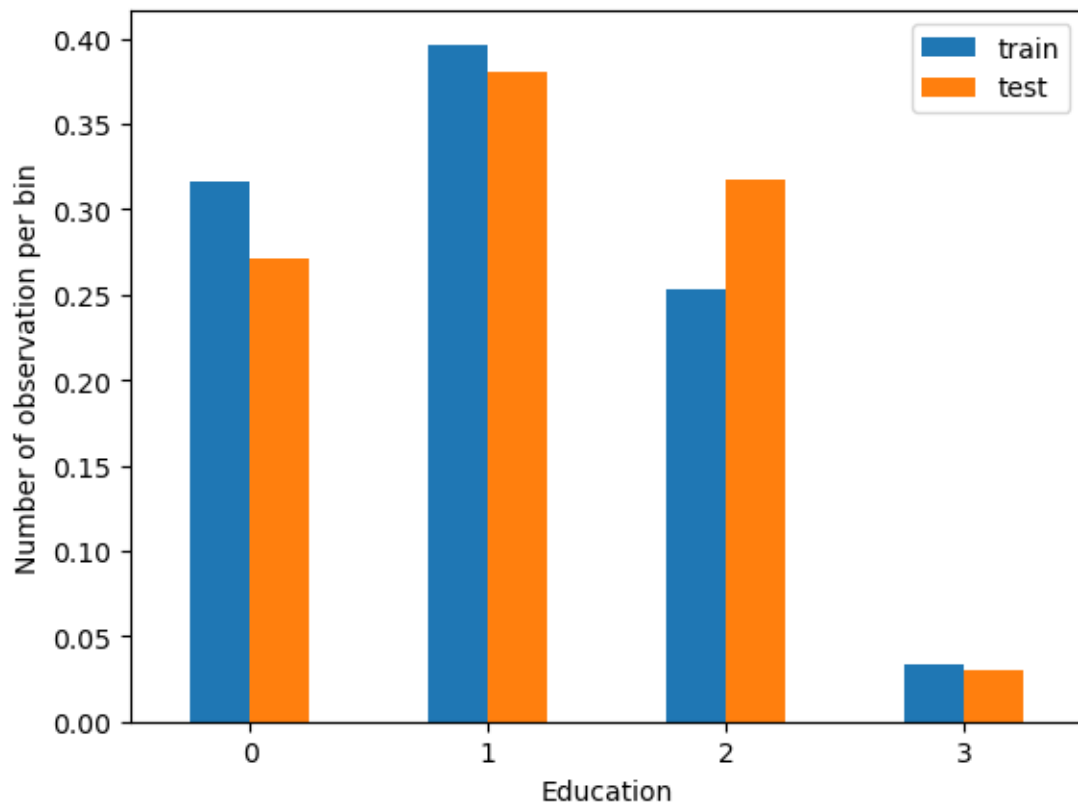
```
[32]: disc = EqualFrequencyDiscretiser(  
        q = 10,  
        variables = continuous  
    )  
disc.fit(tmp_mean_train)  
  
train_disc = disc.transform(tmp_mean_train)  
test_disc = disc.transform(tmp_mean_test)  
  
train_disc.head()
```

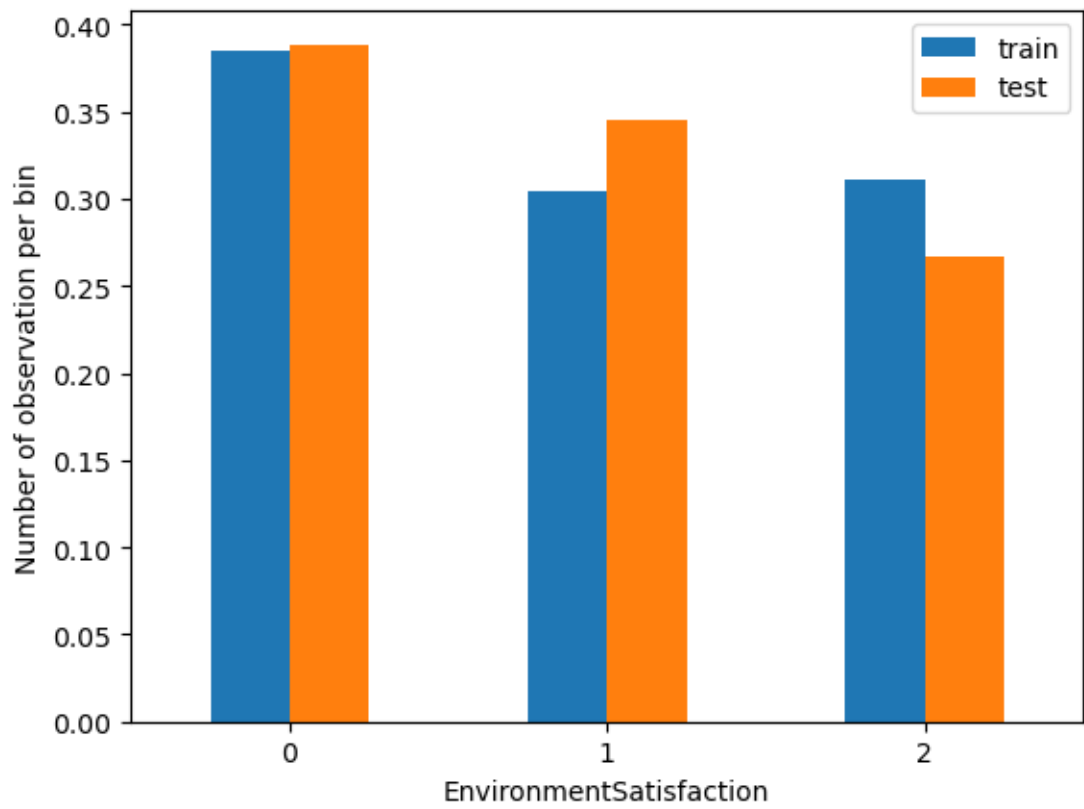
```
[33]: for col in train_disc.columns:  
  
        t1 = train_disc.groupby([col])[col].count() / len(train_disc)  
        t2 = test_disc.groupby([col])[col].count() / len(test_disc)  
  
        tmp = pd.concat([t1, t2], axis = 1)  
        tmp.columns = ['train', 'test']  
        tmp.plot.bar()  
        plt.xticks(rotation = 0)  
        plt.ylabel('Number of observation per bin')  
        plt.show()
```

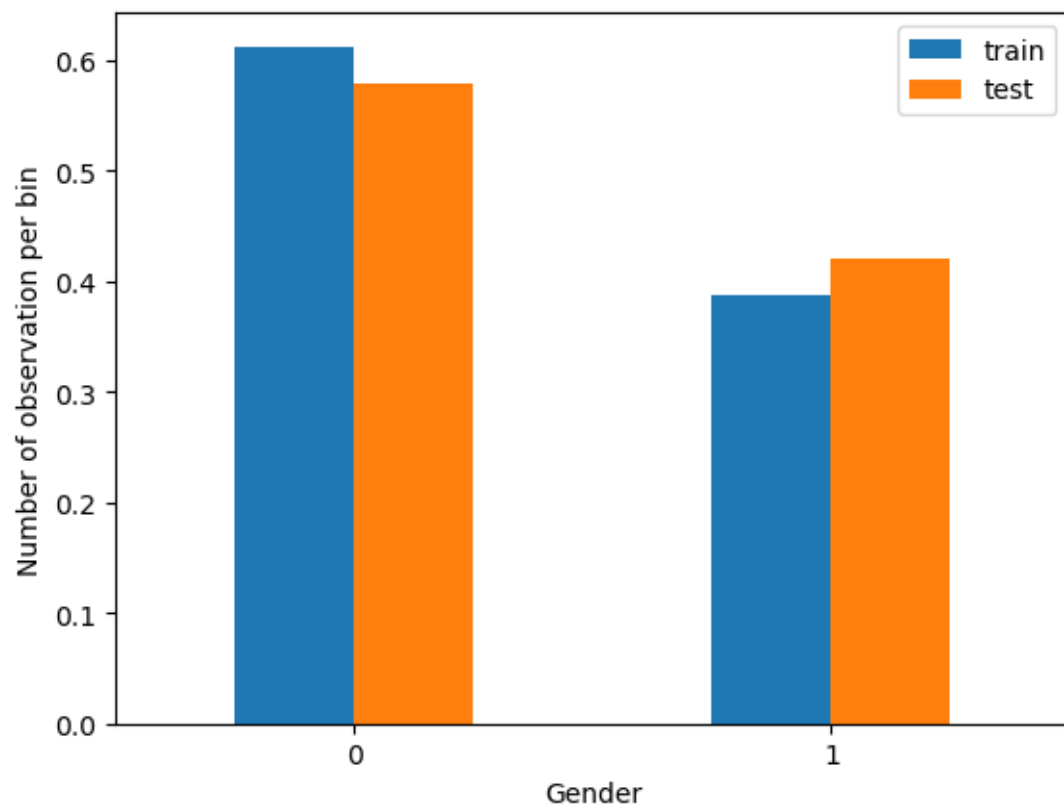


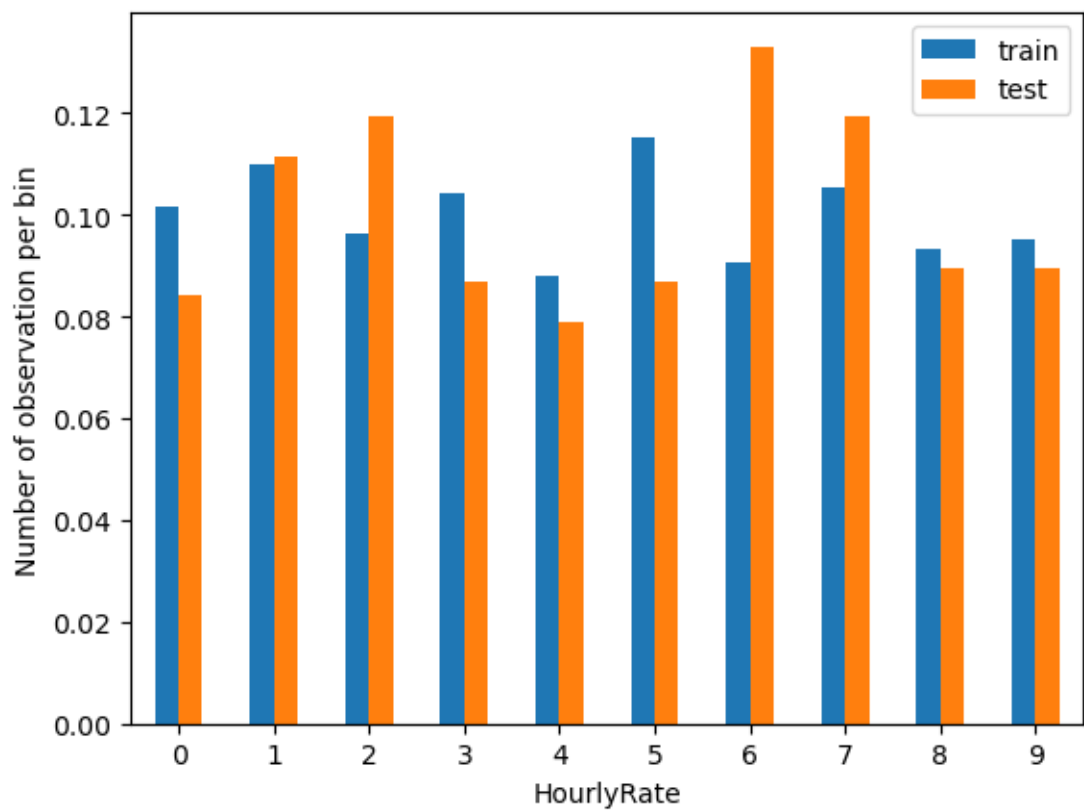


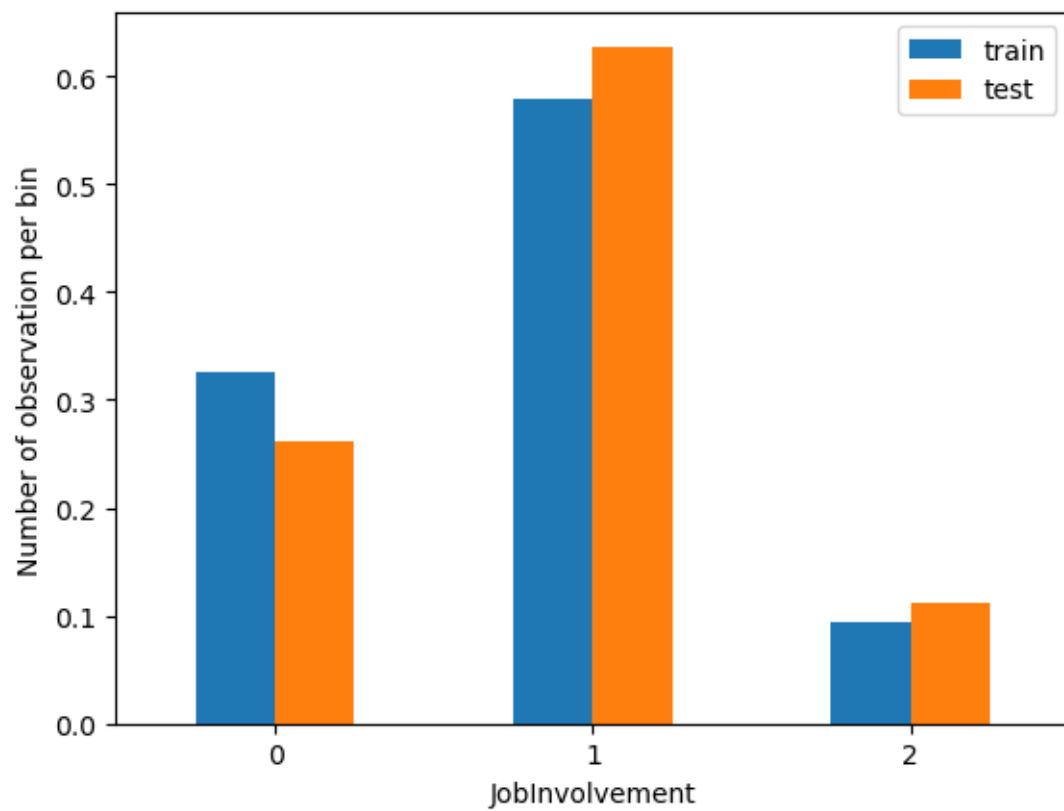


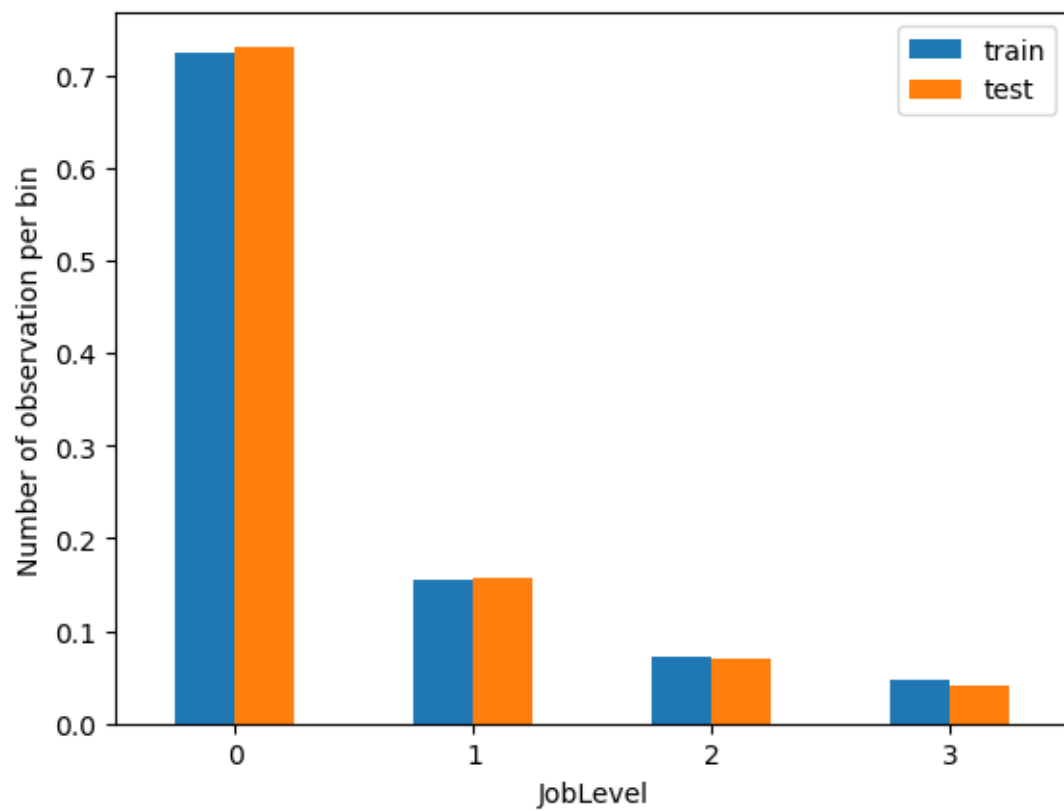


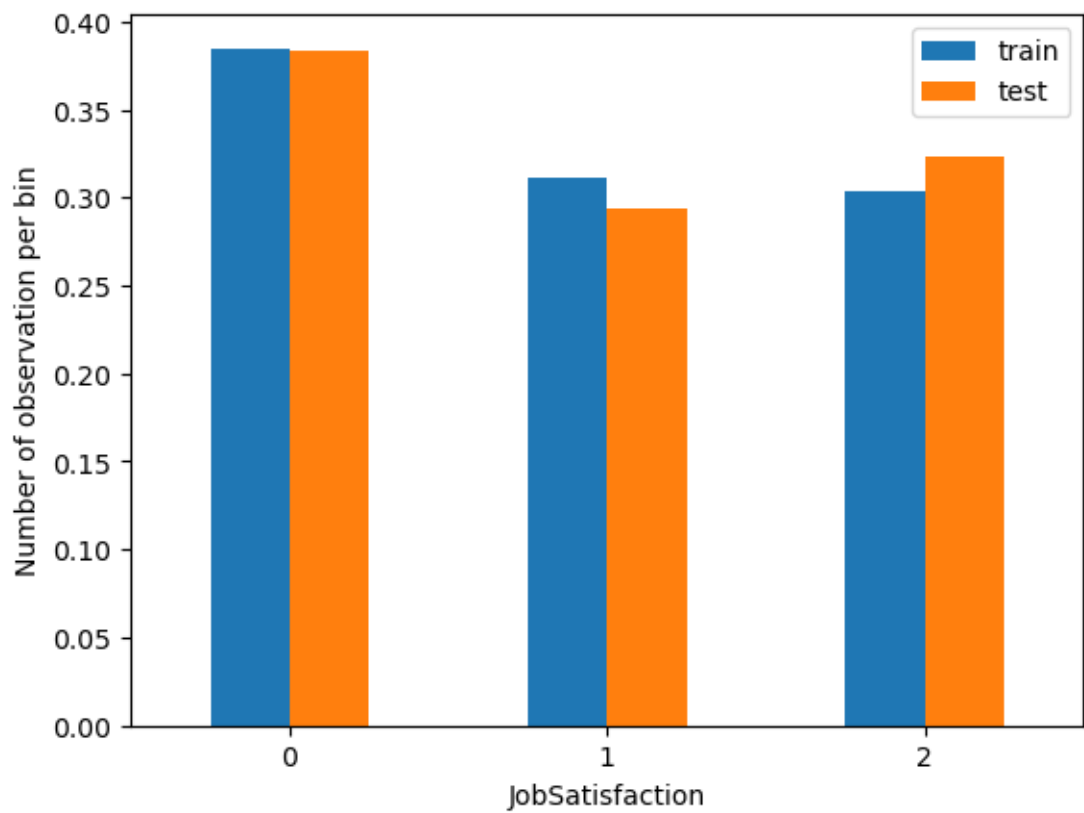


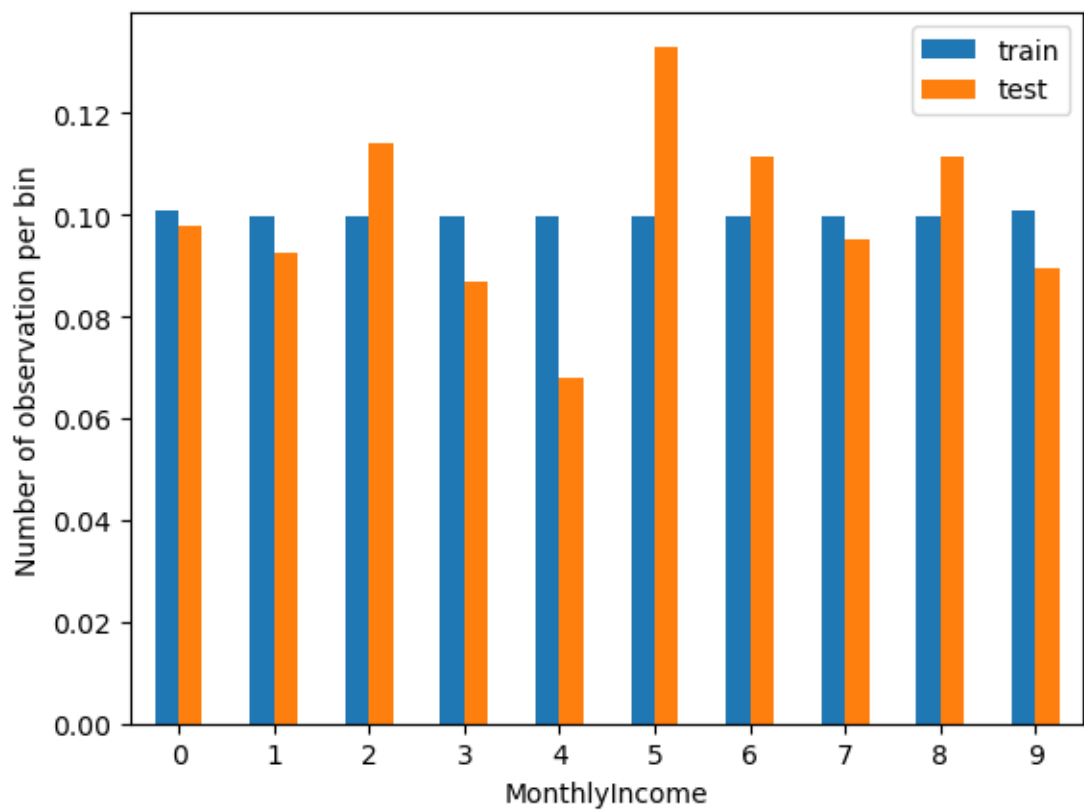


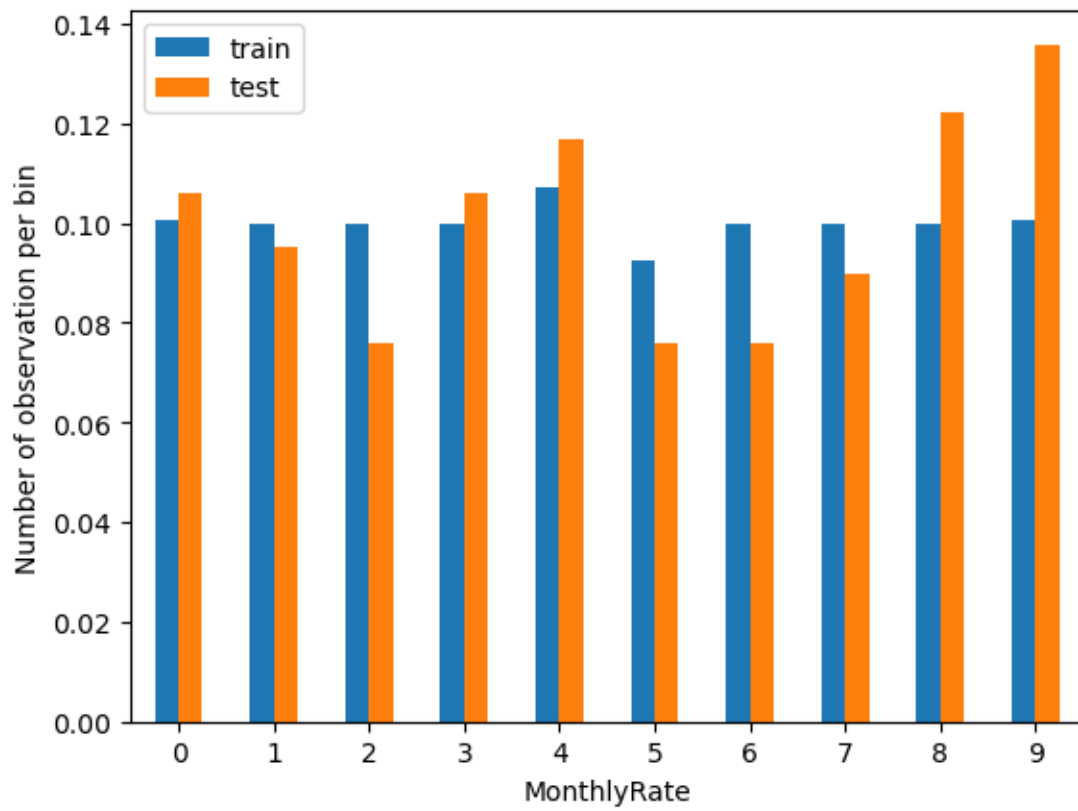


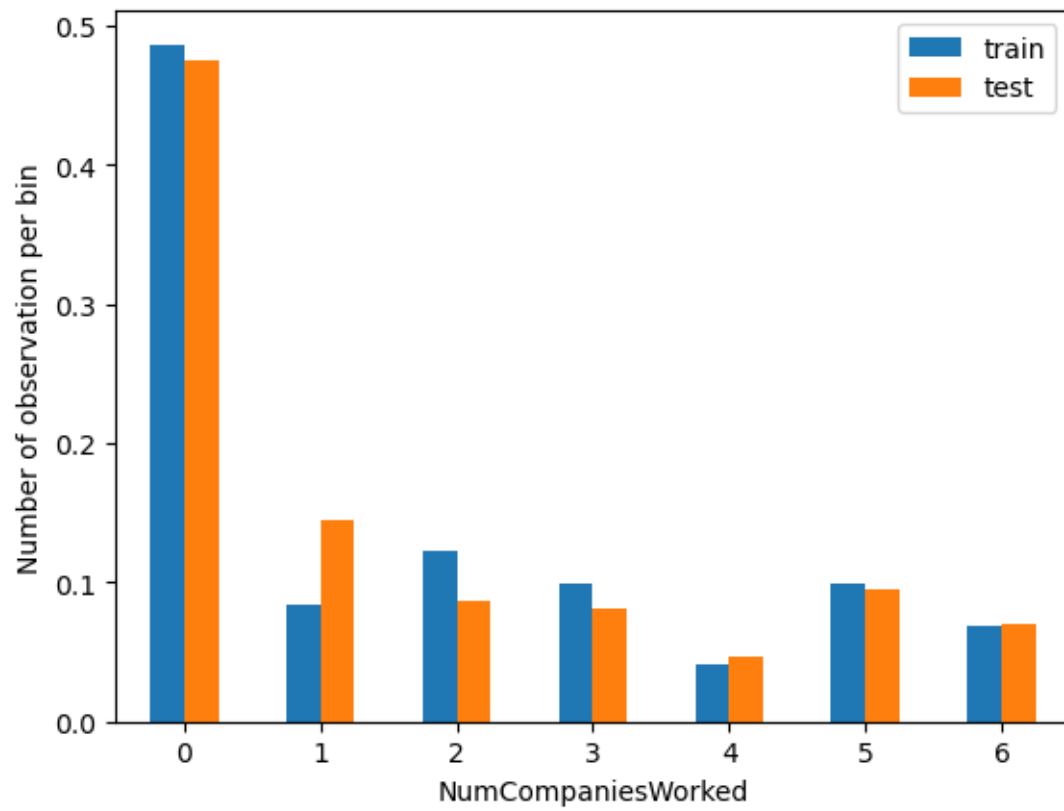


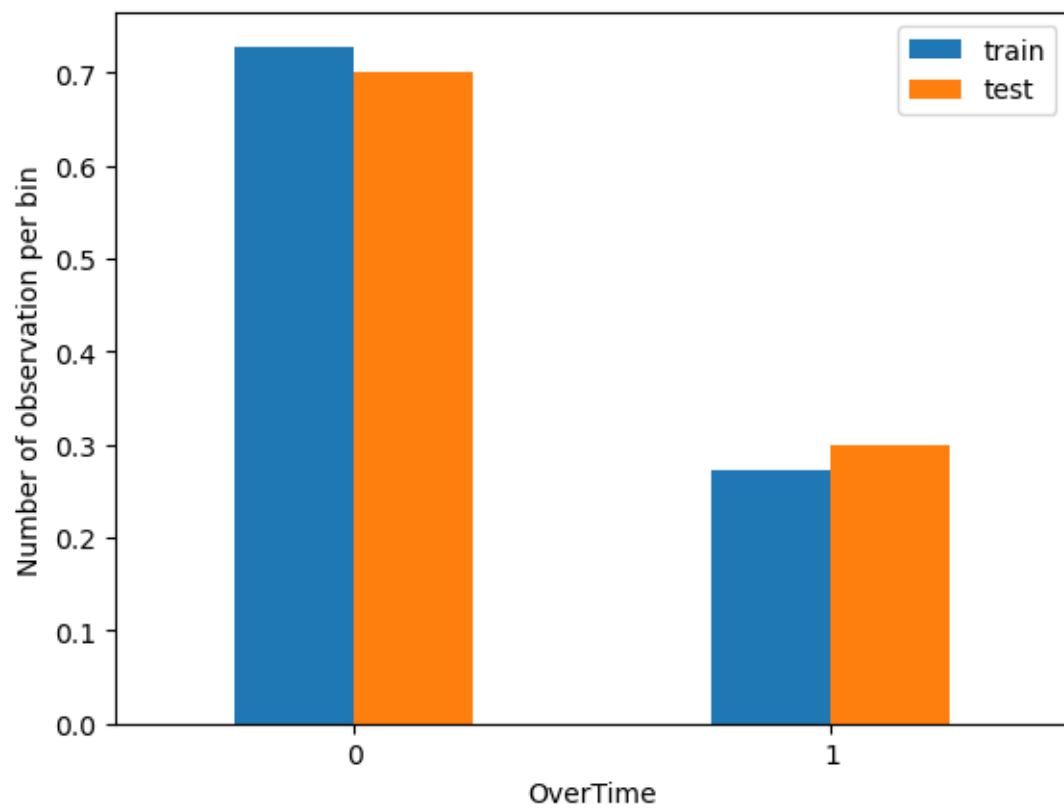


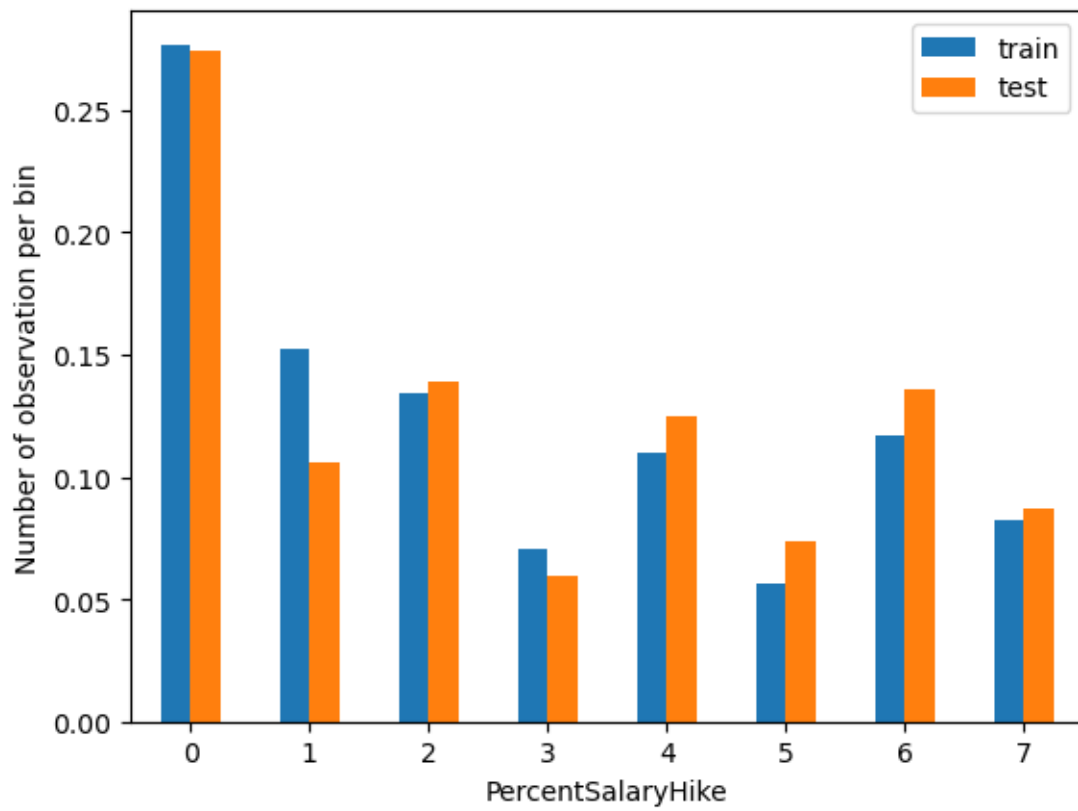


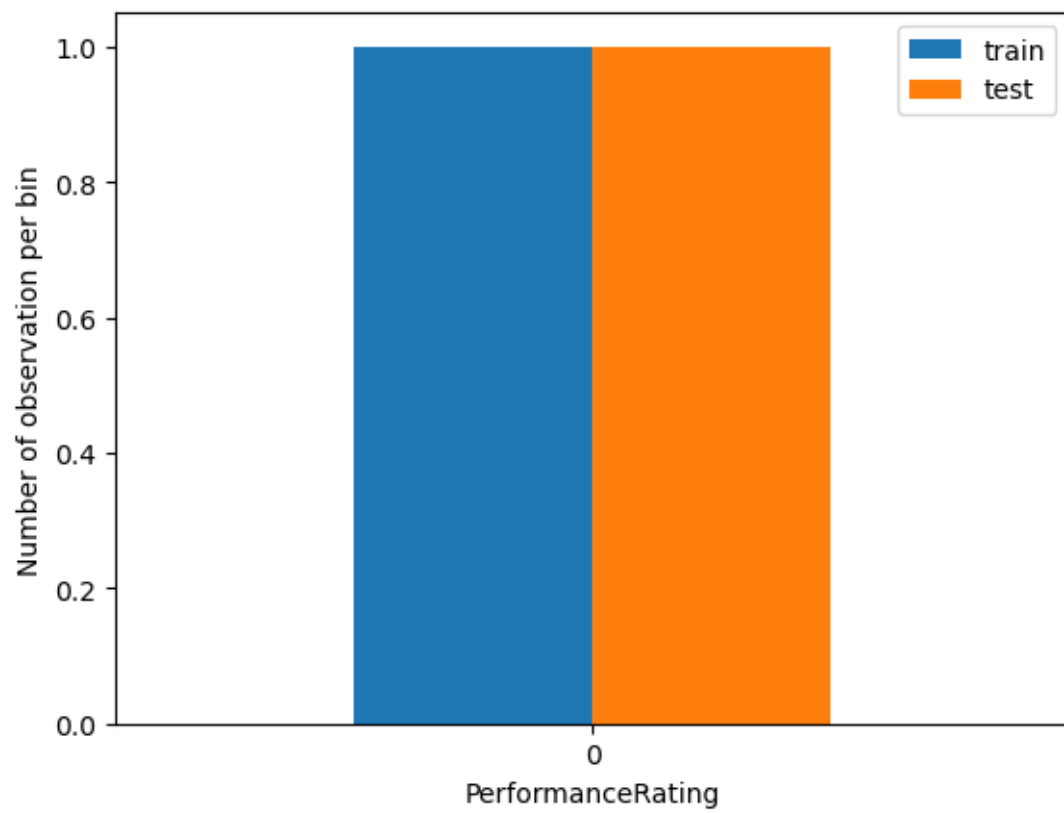


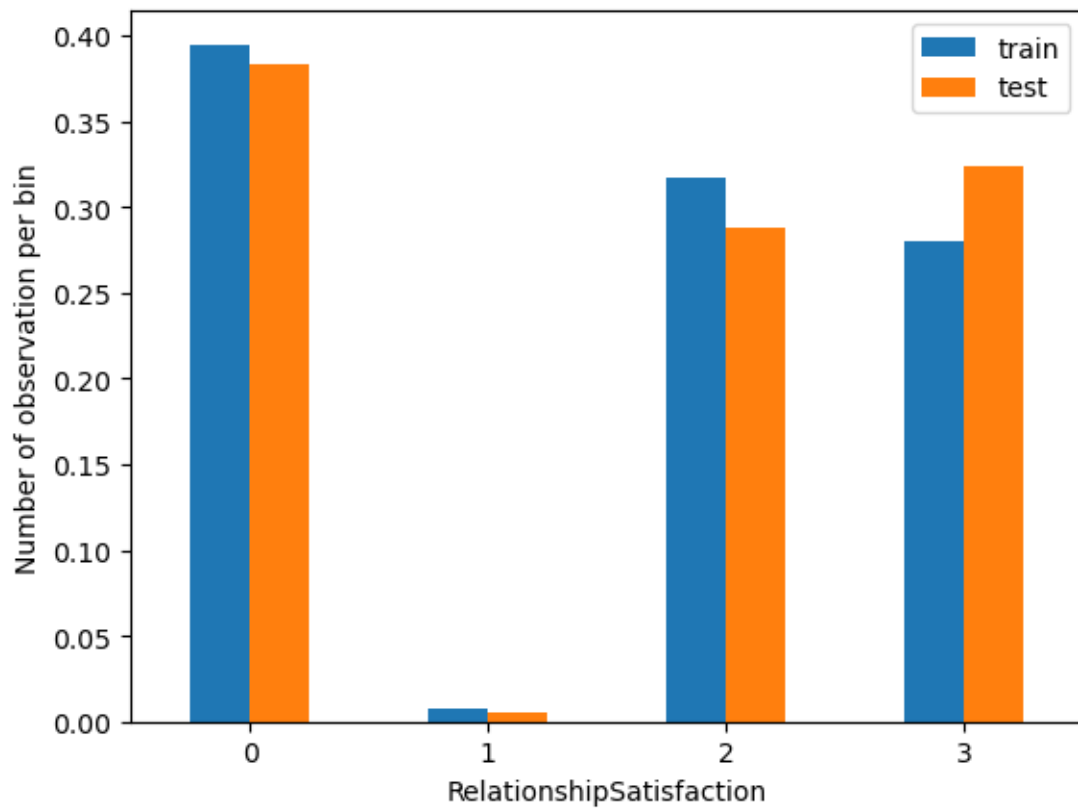


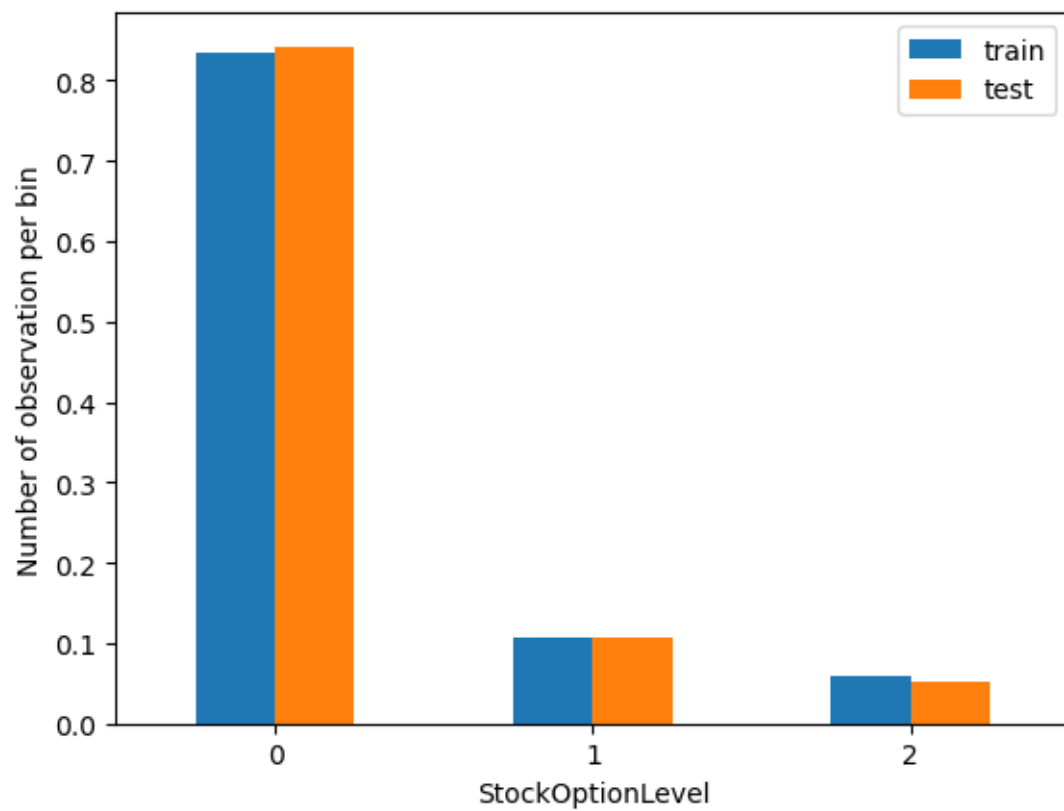


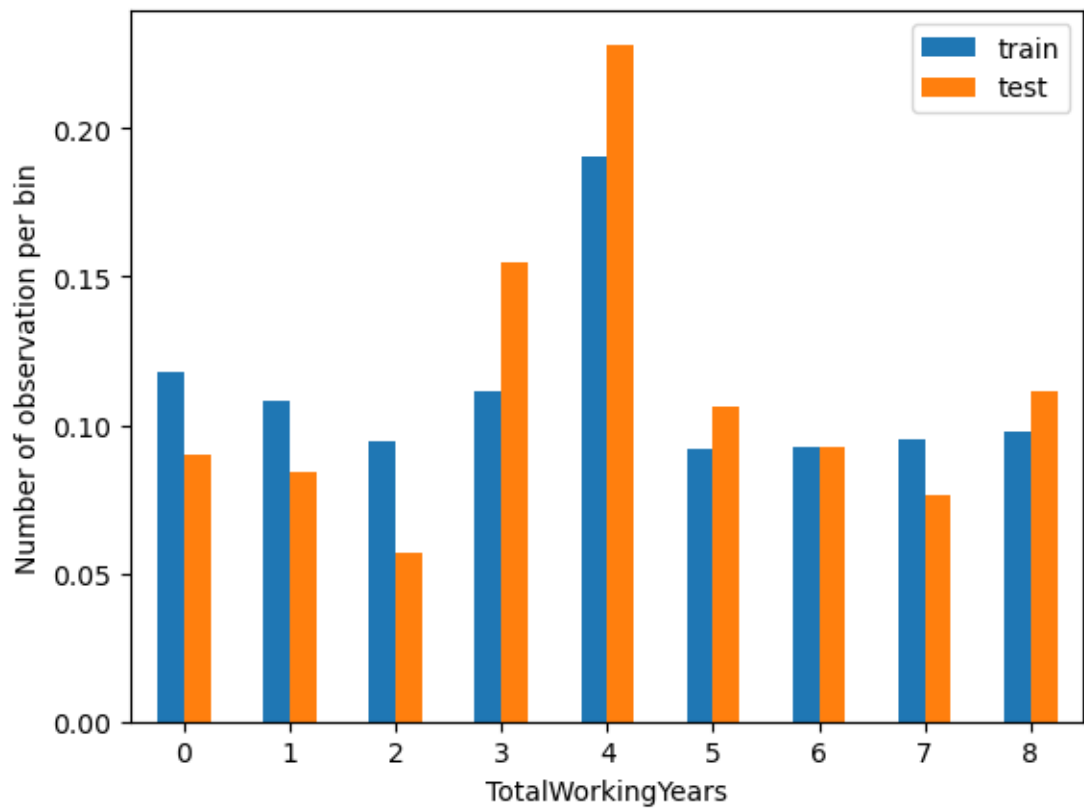


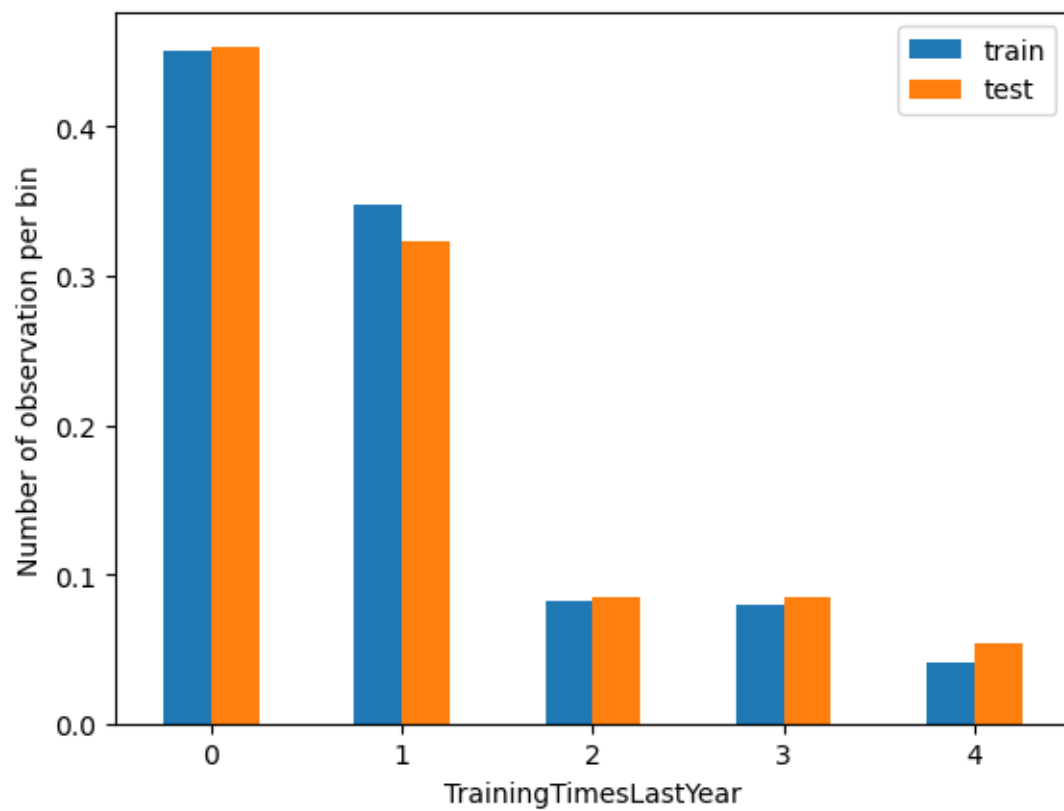


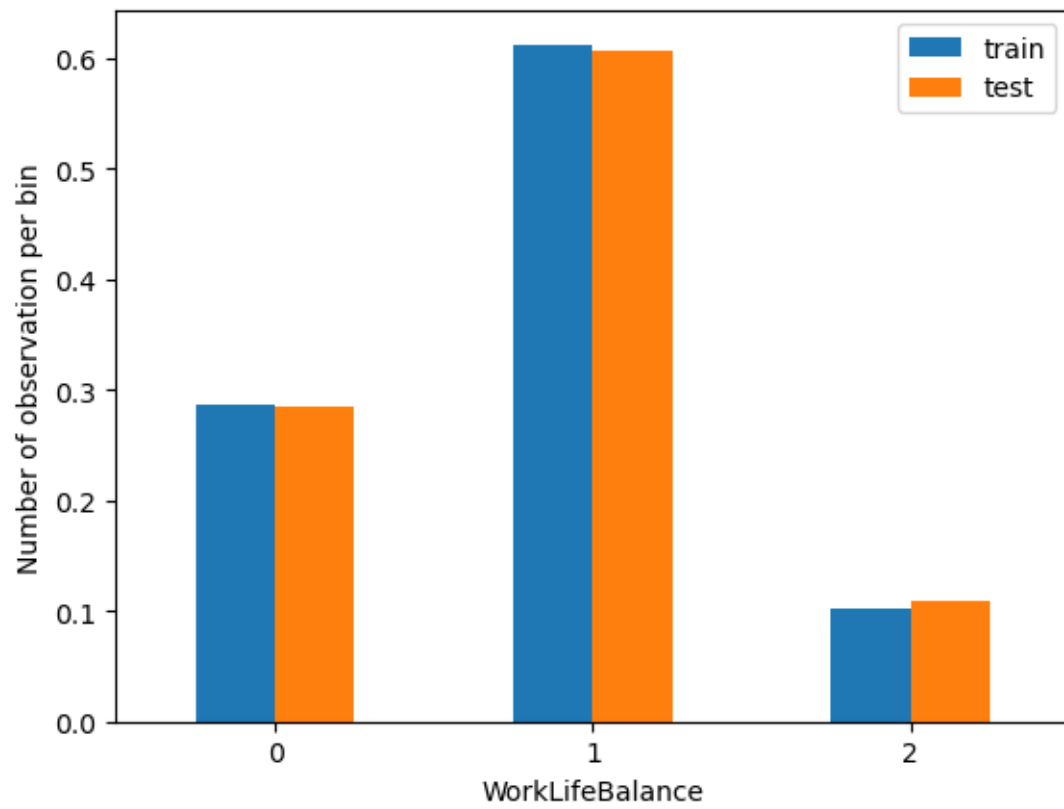


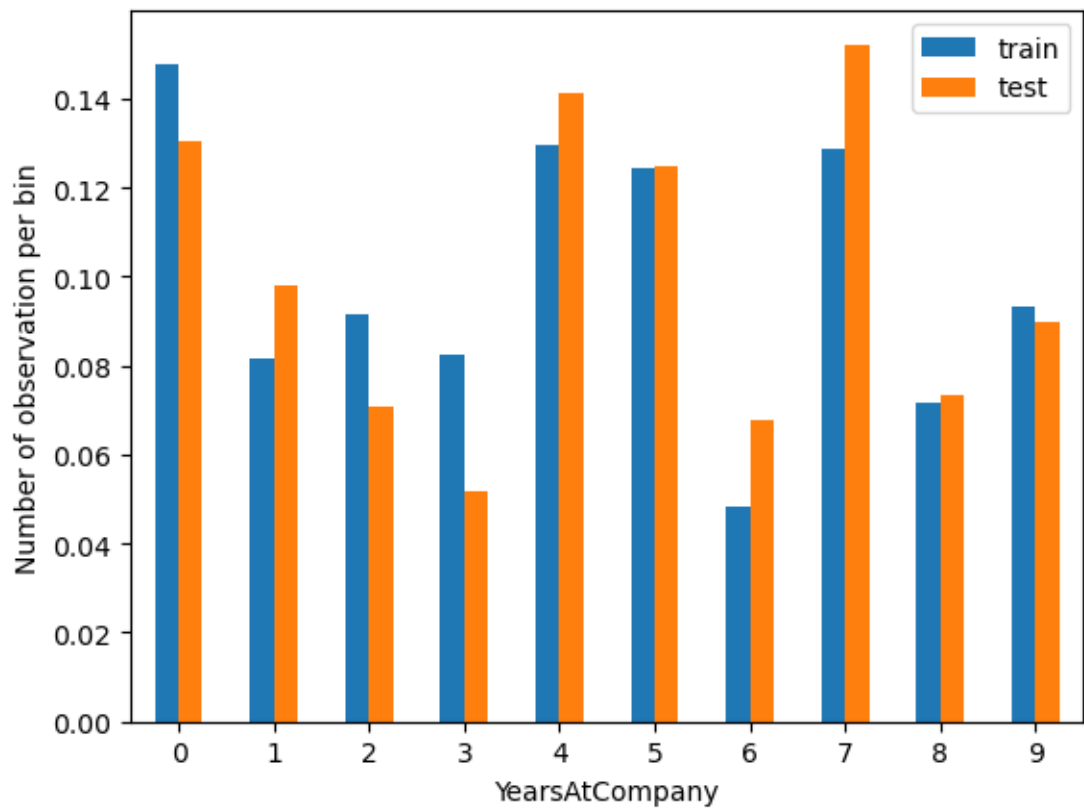


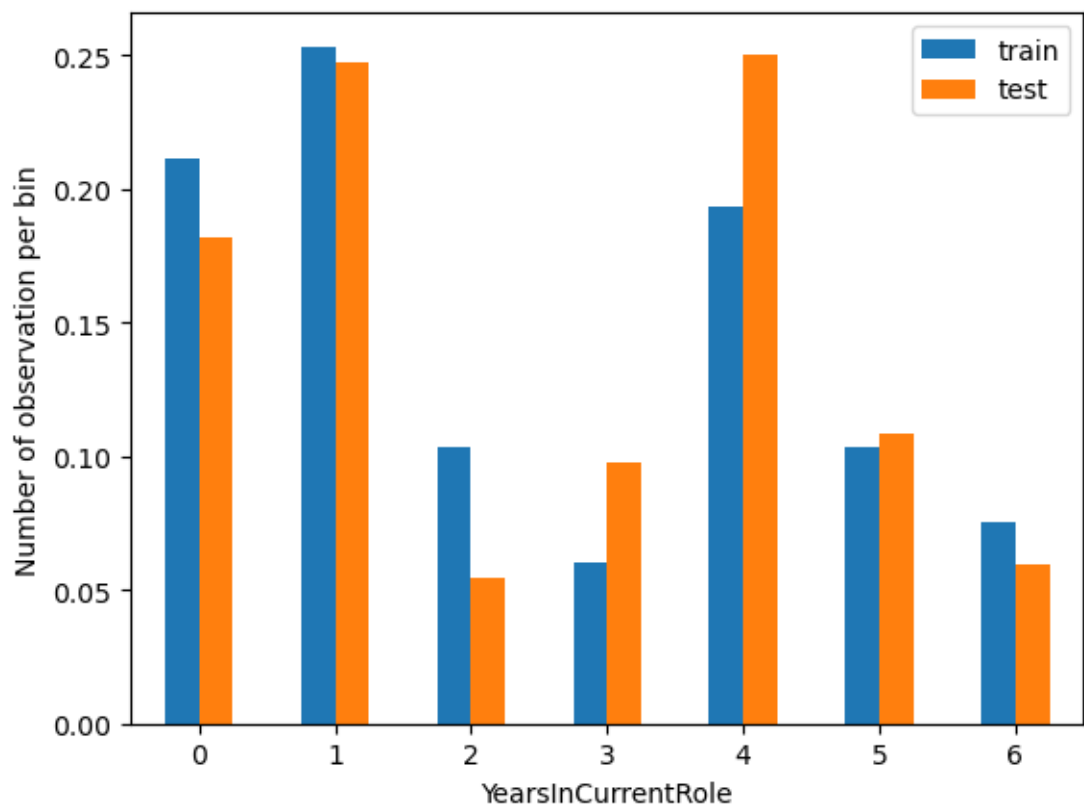


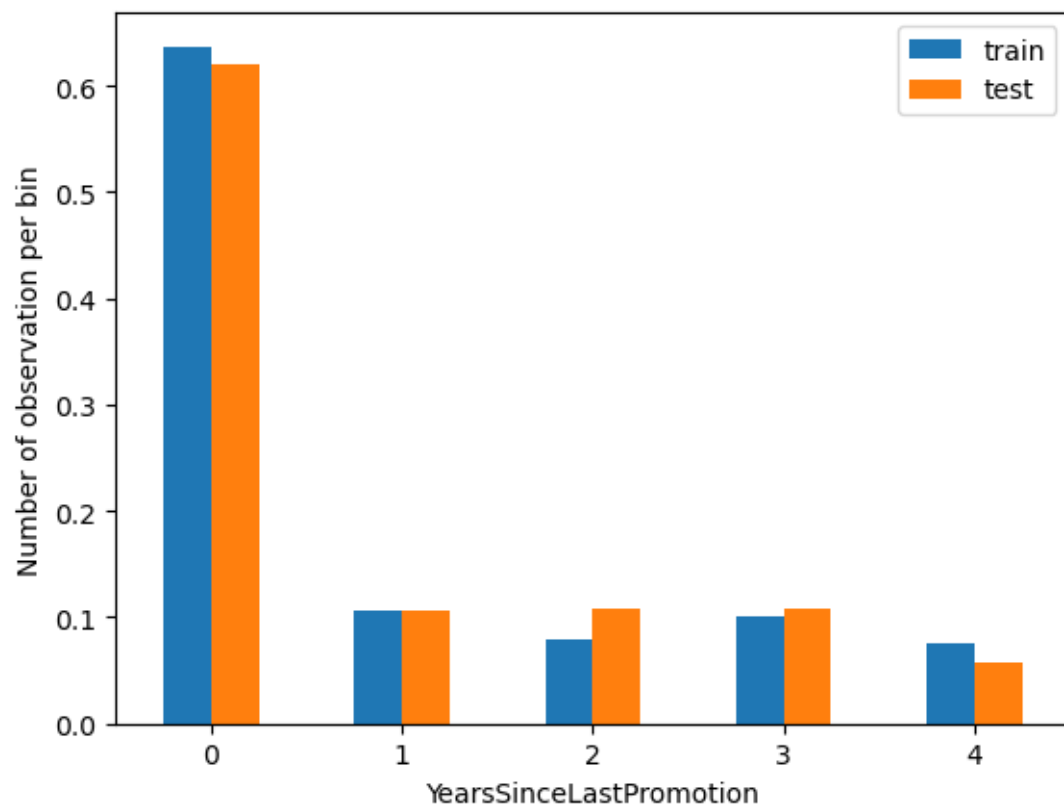


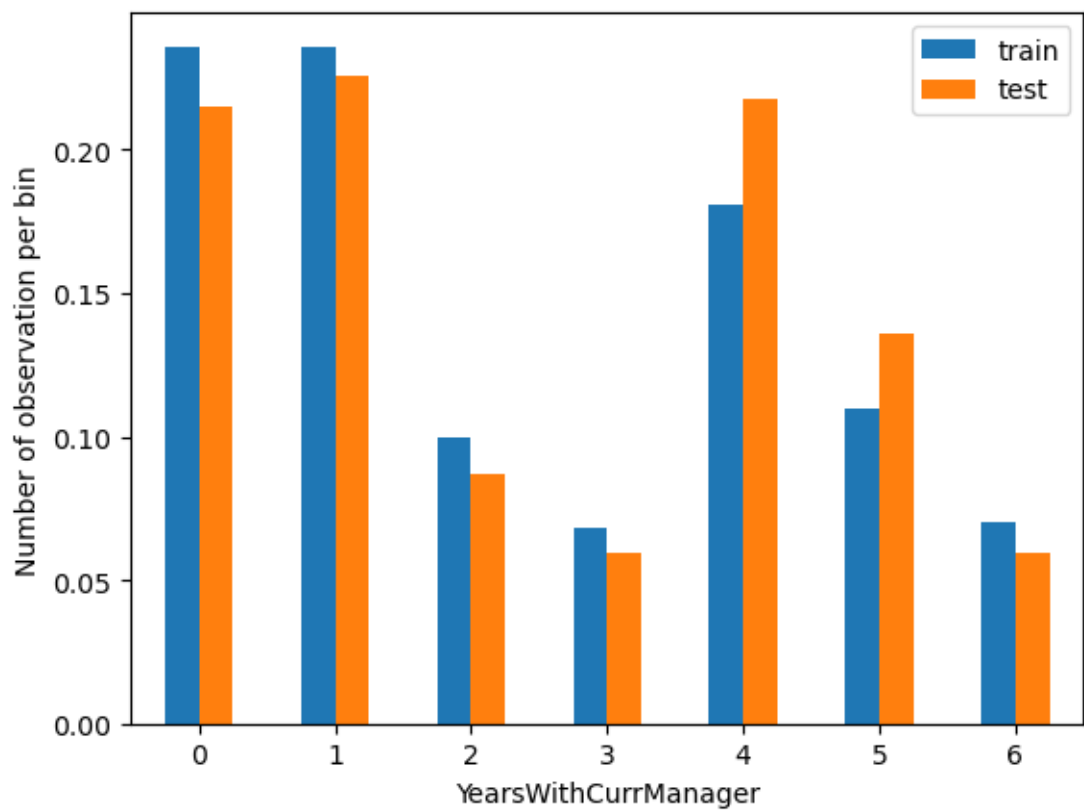


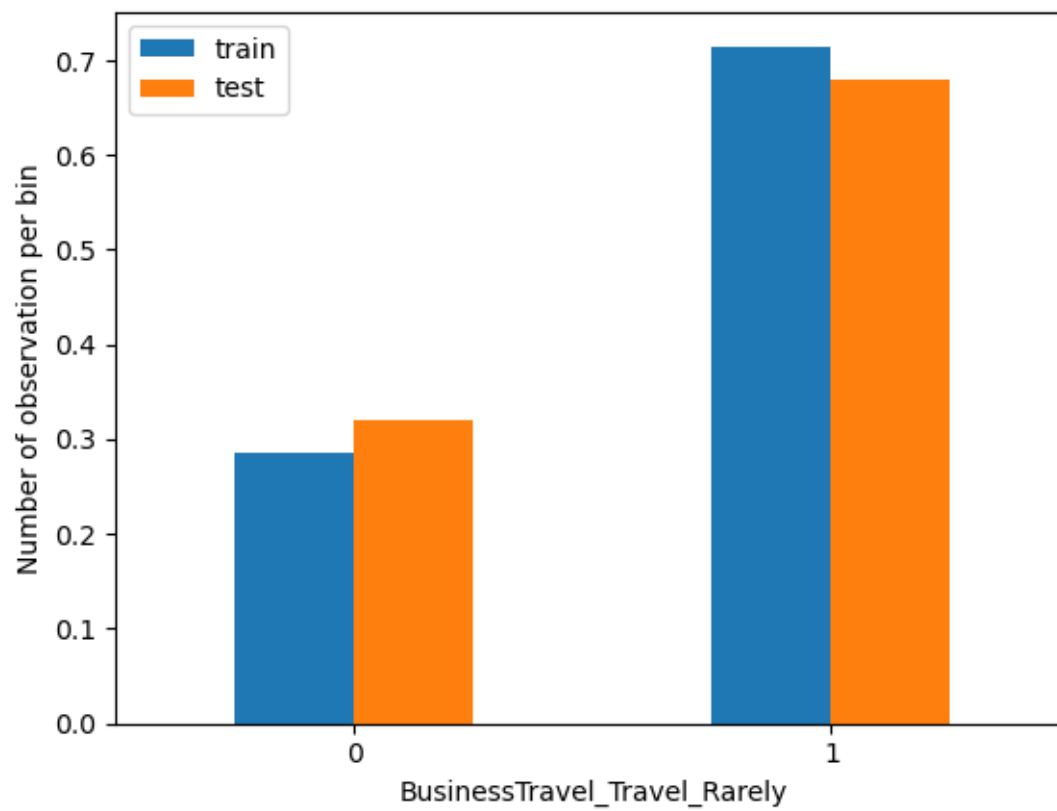


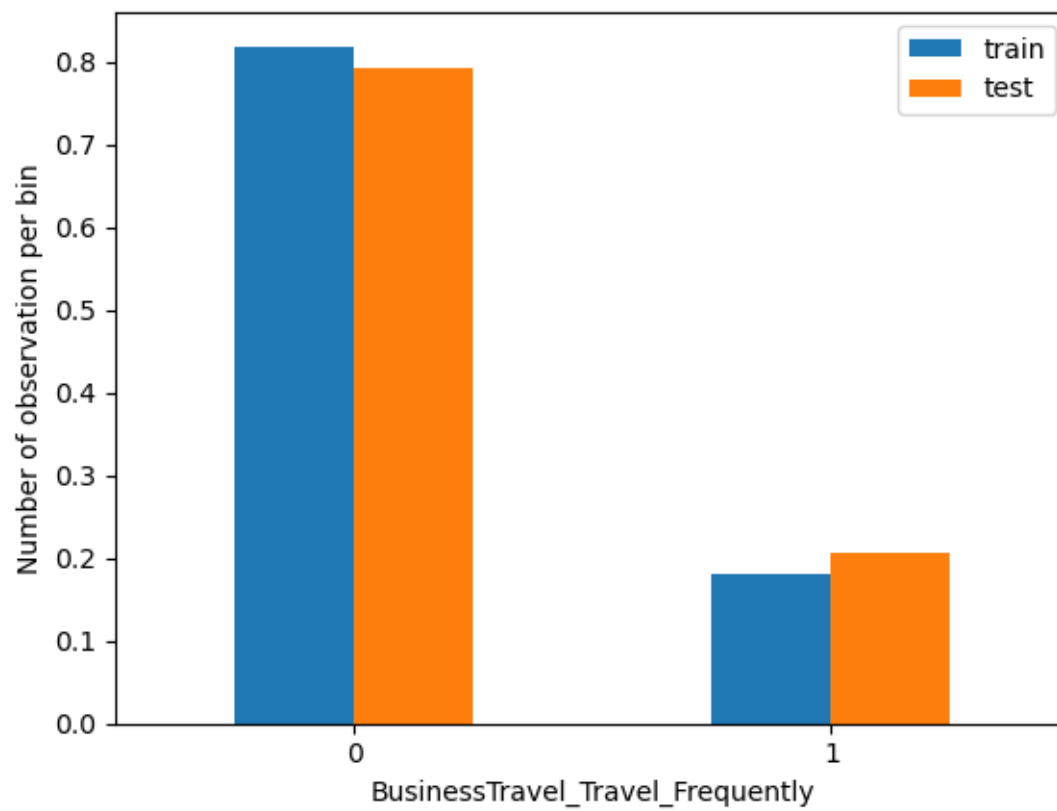


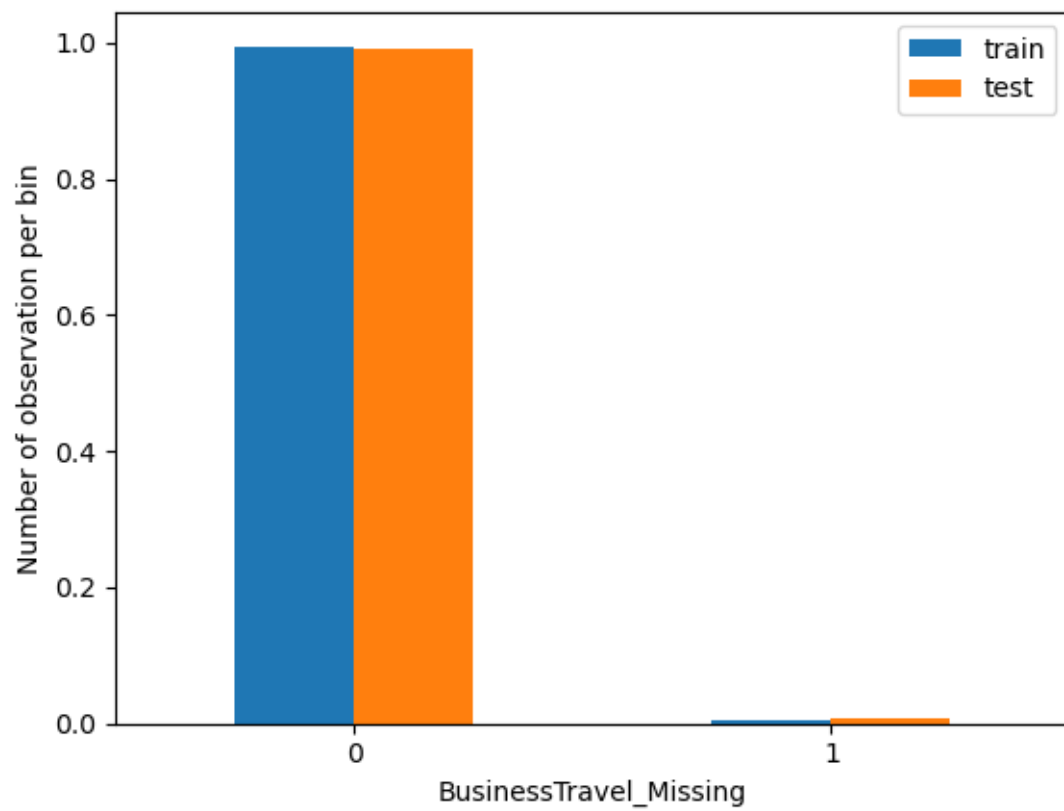


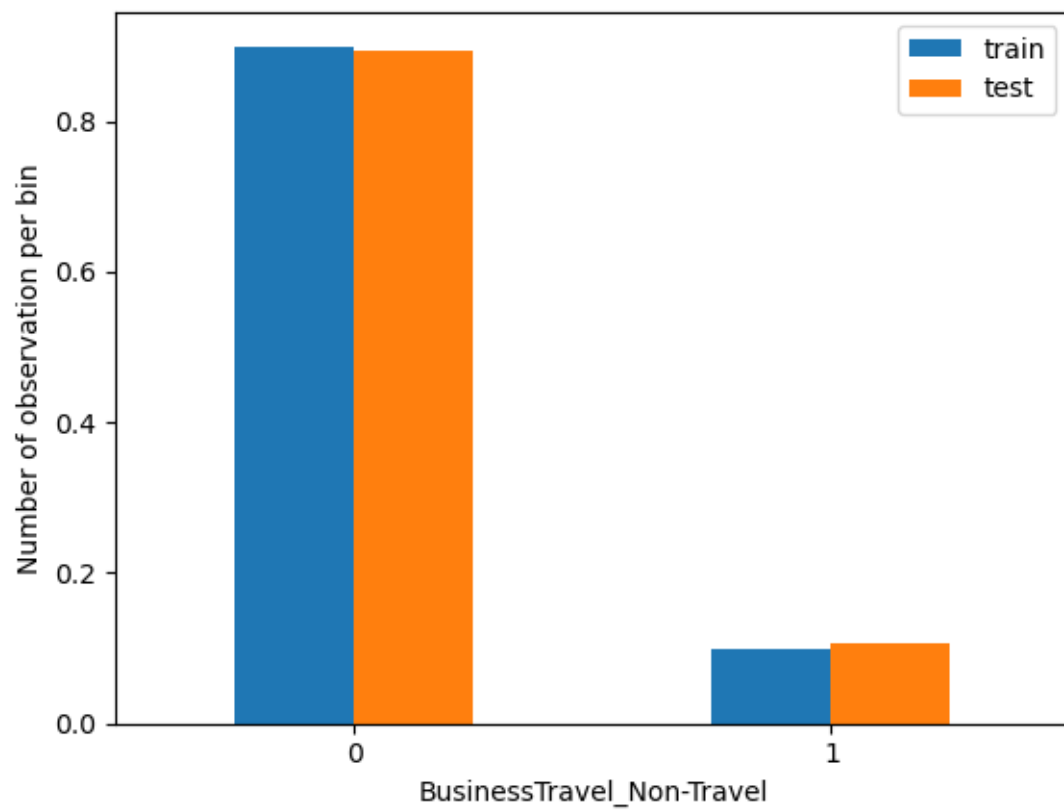


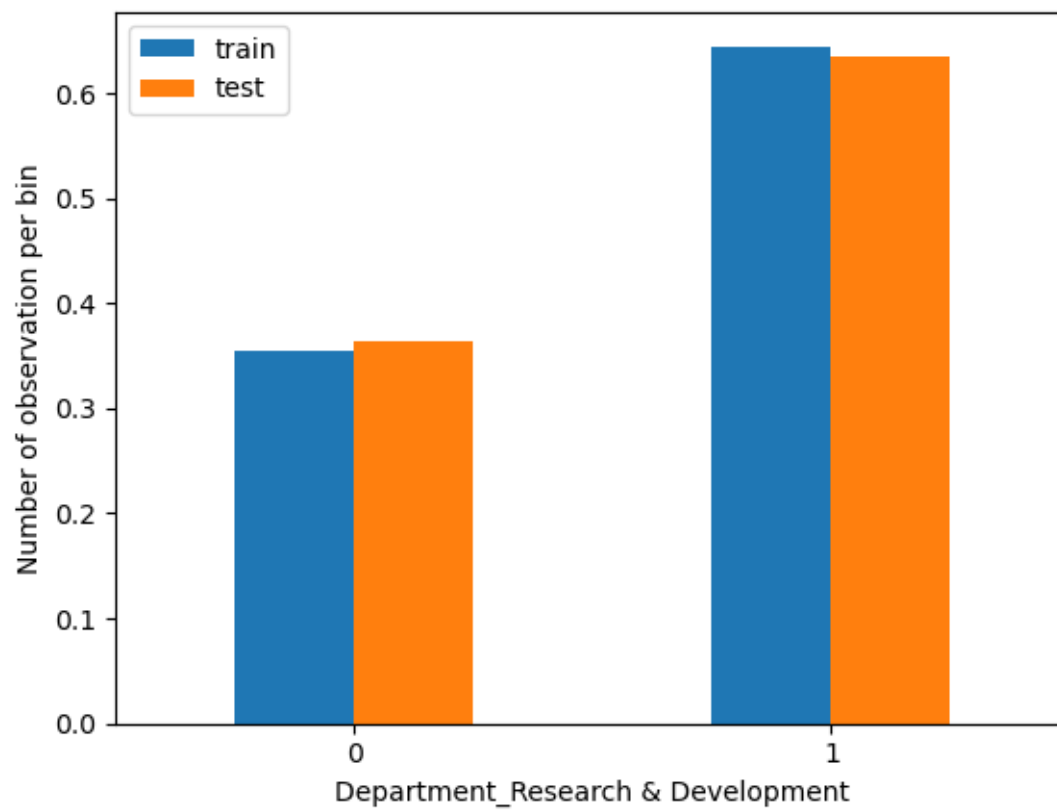


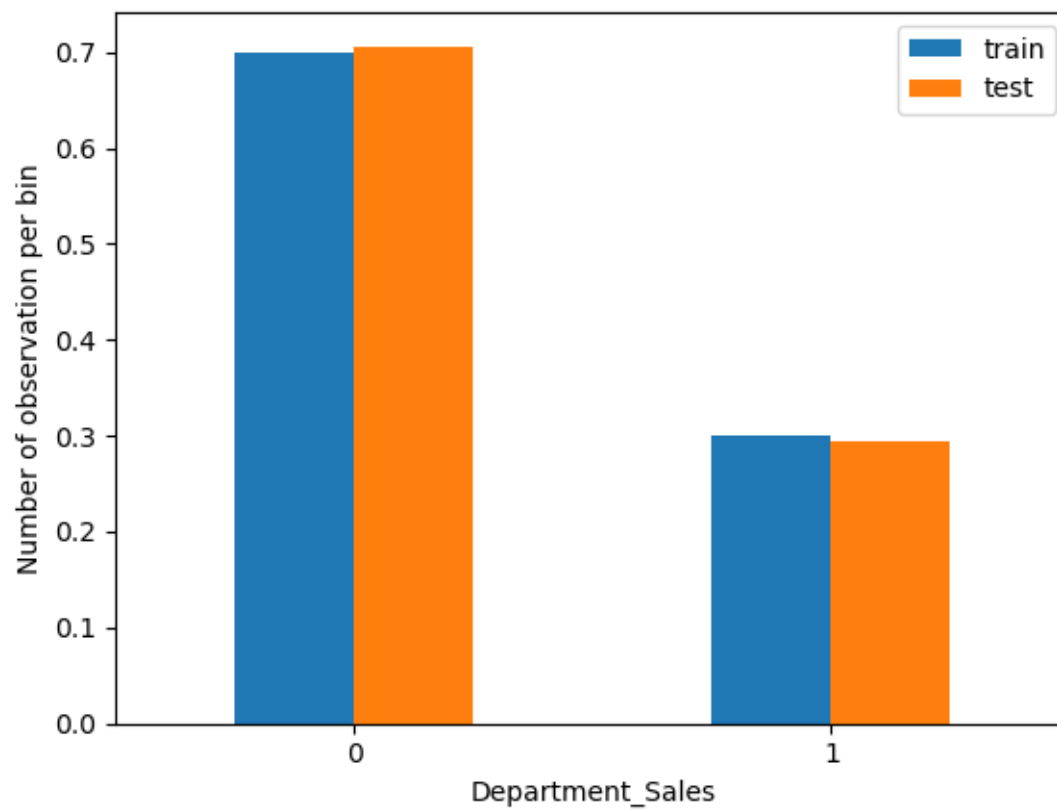


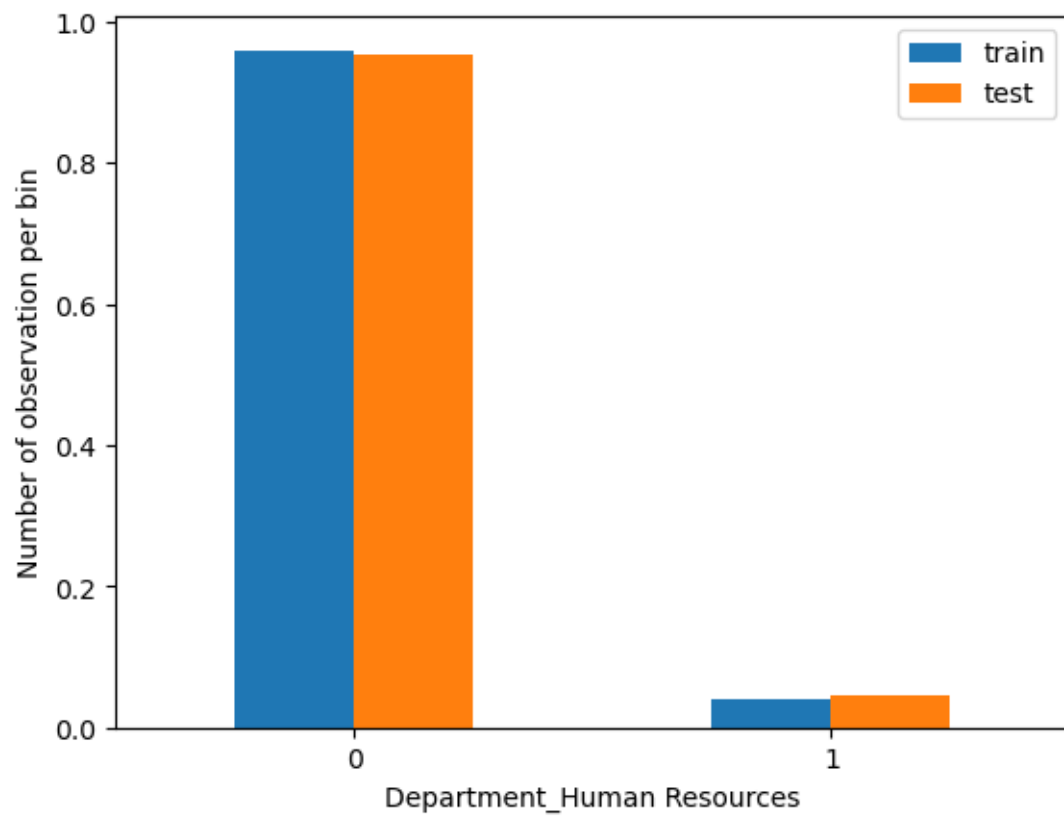


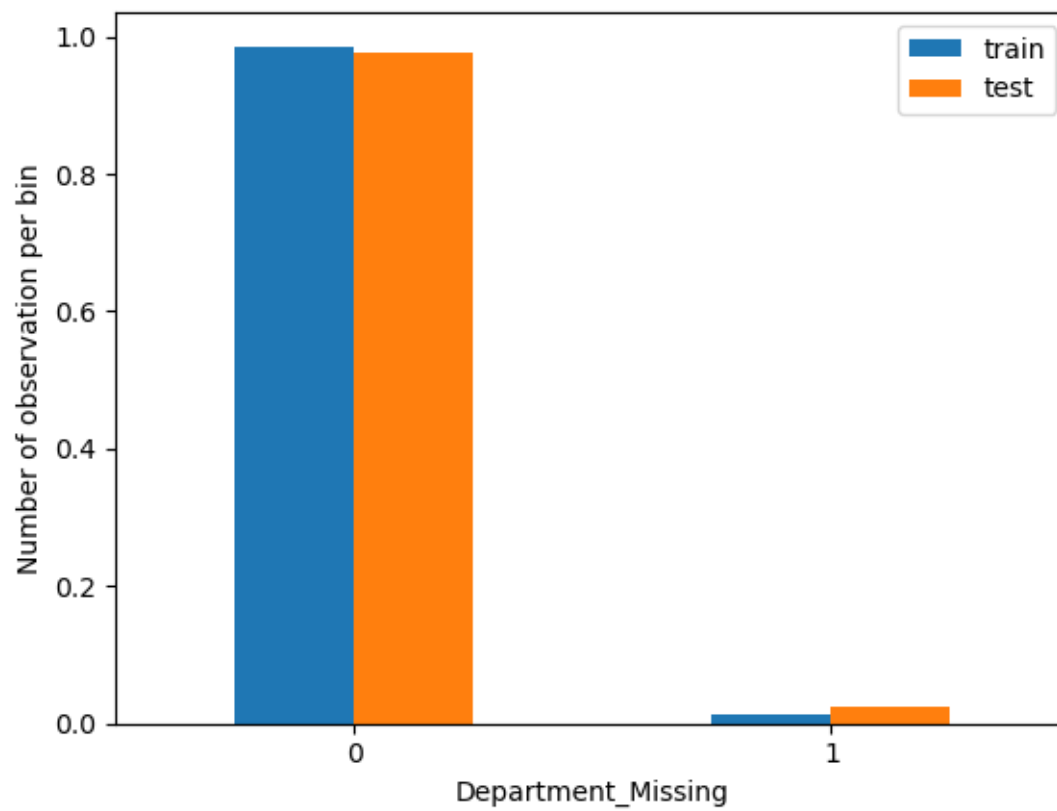


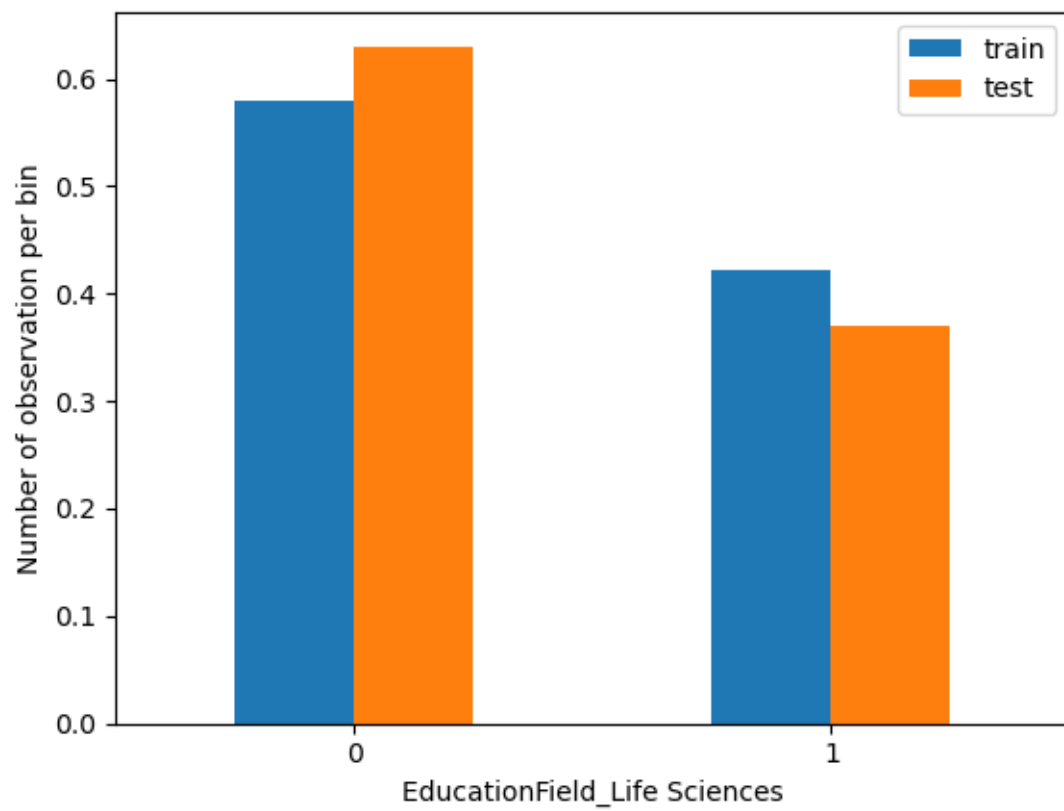


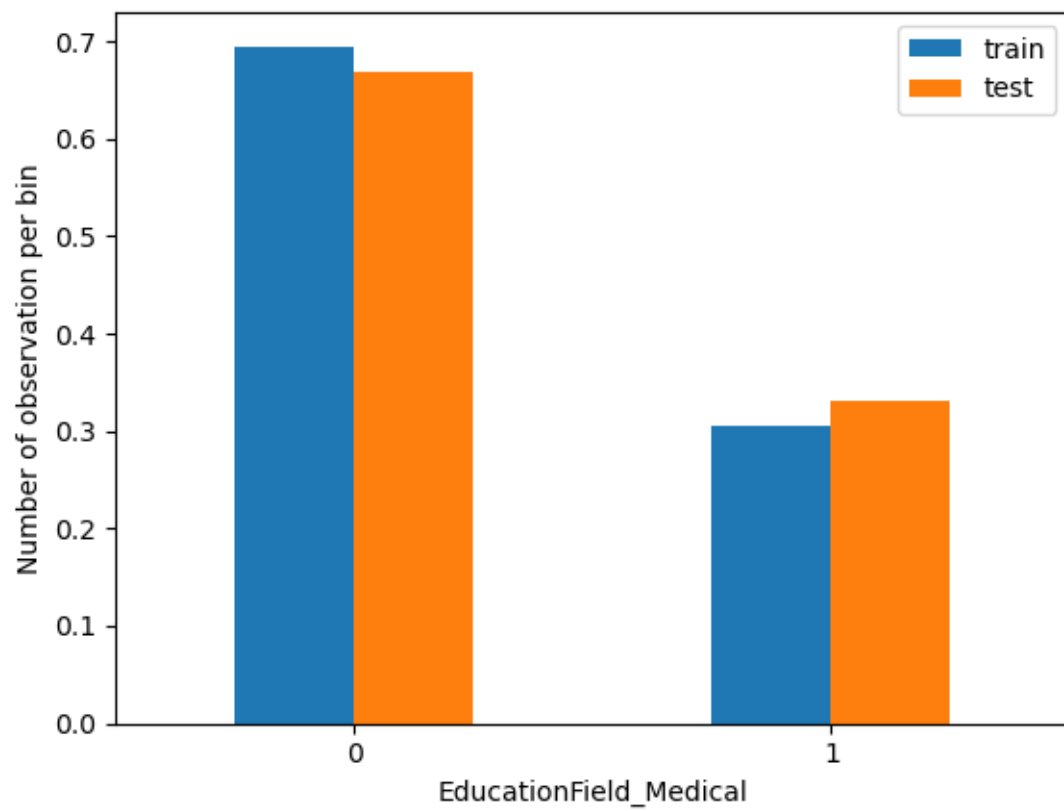


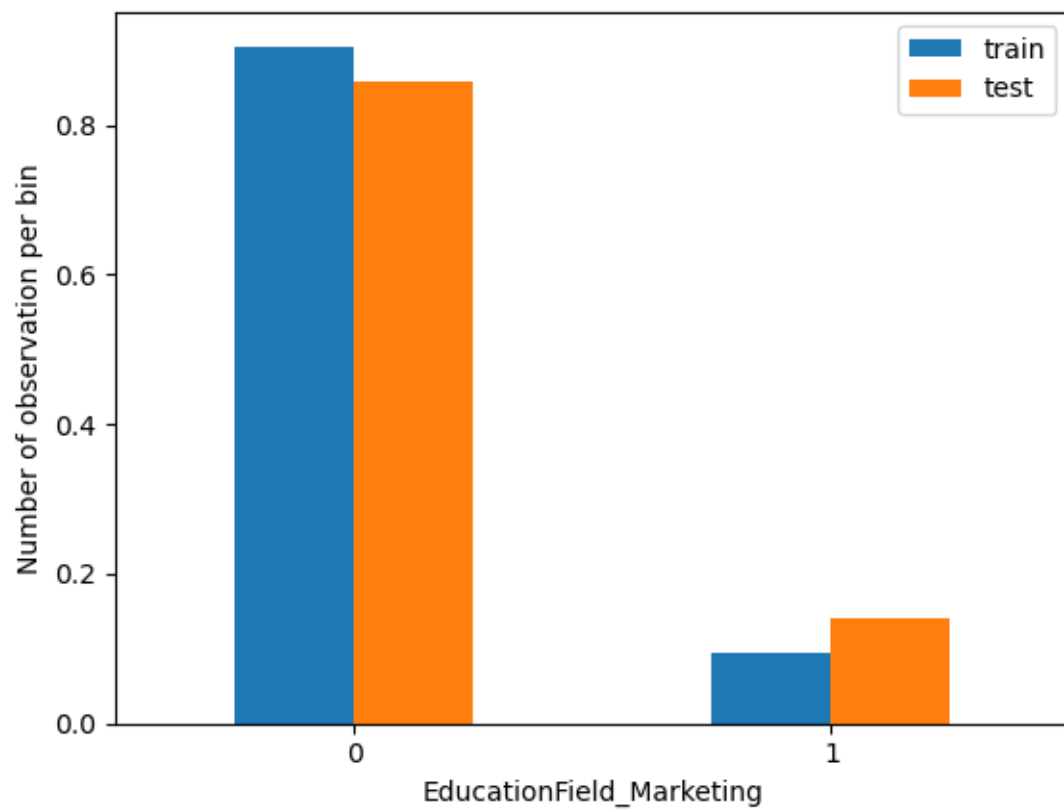


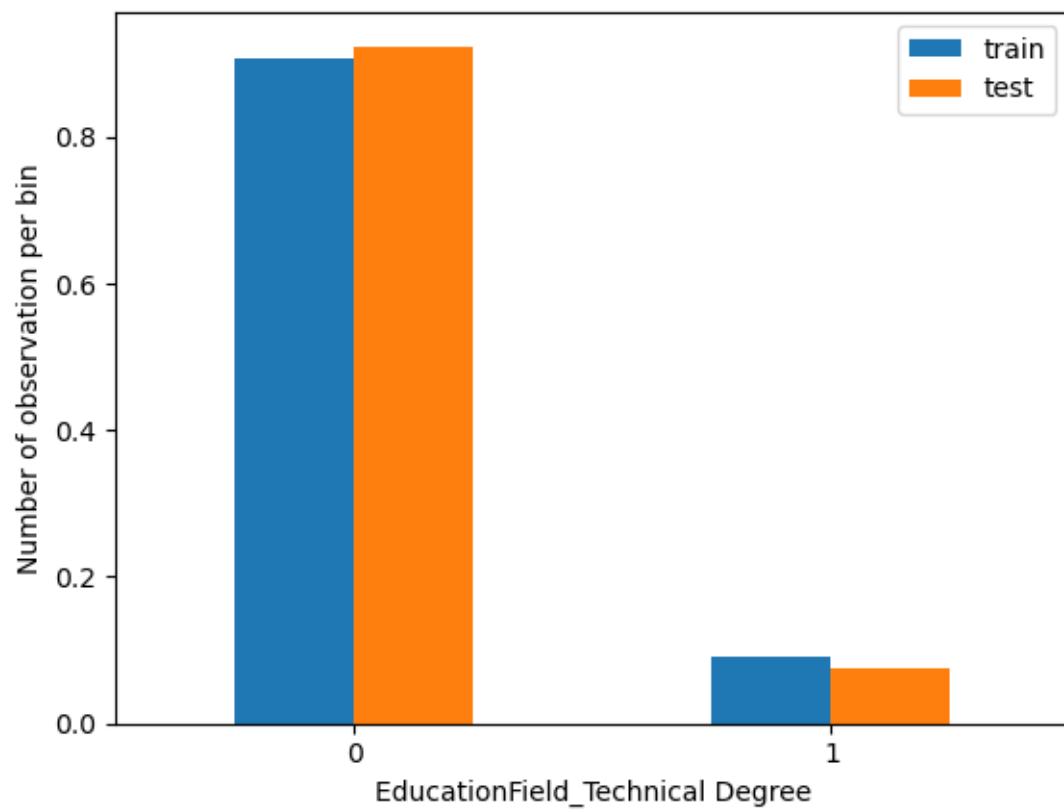


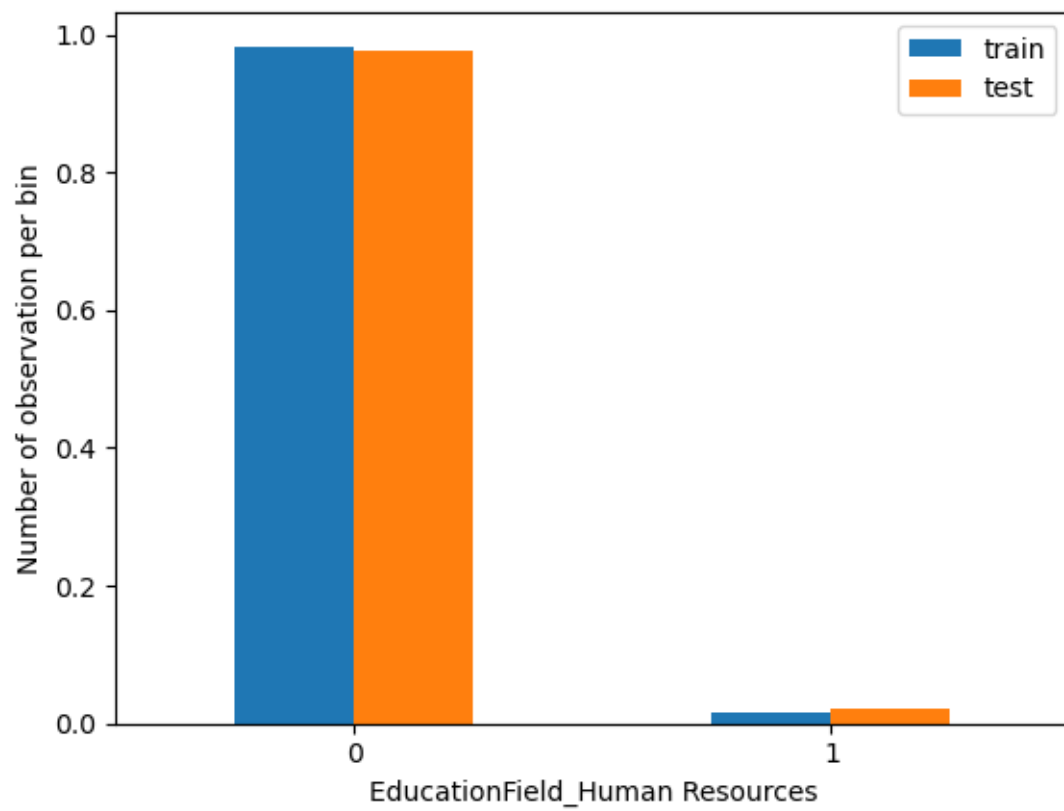


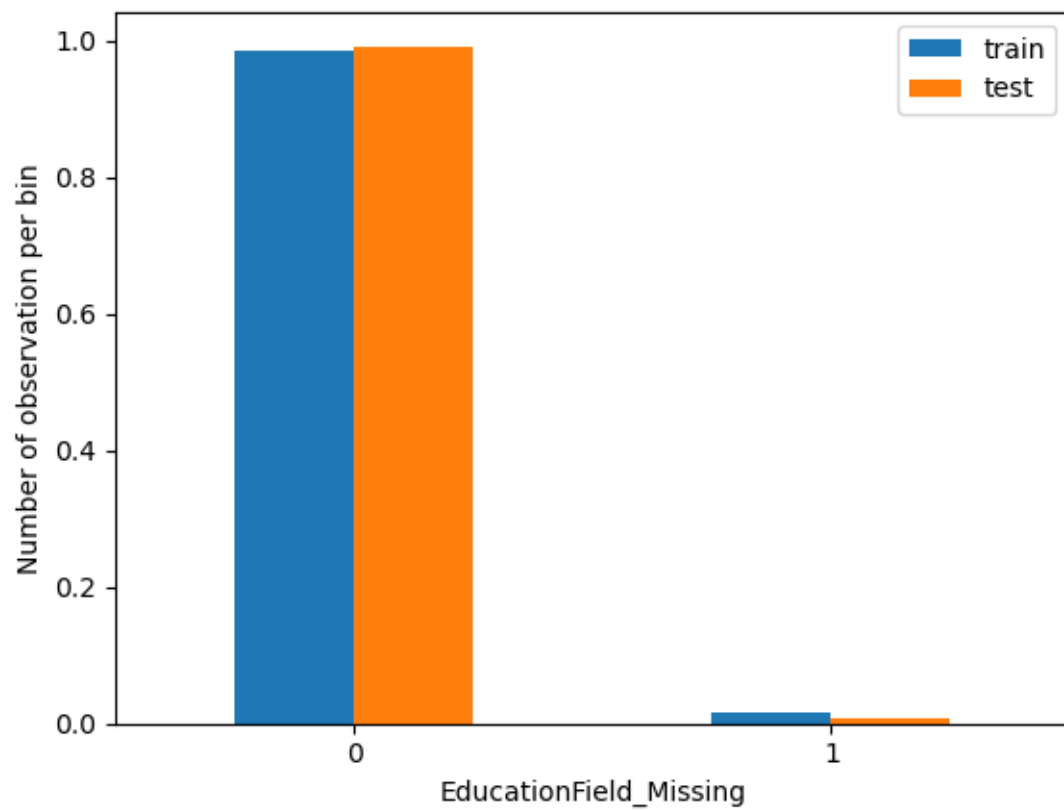


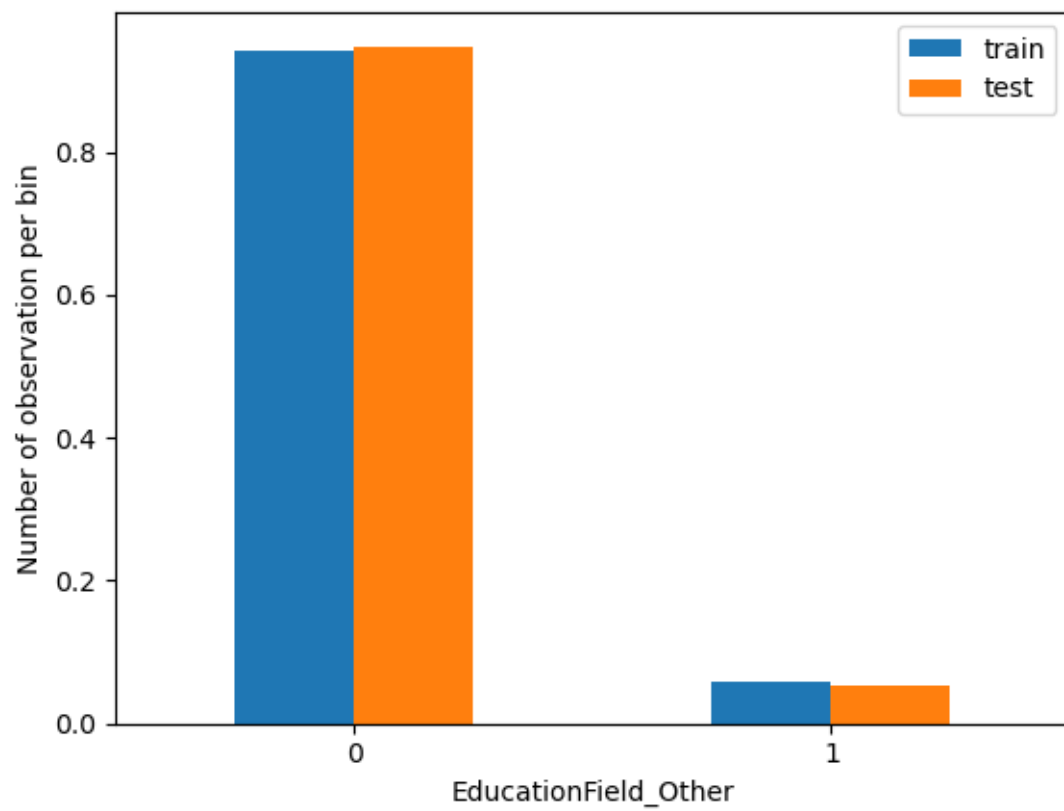


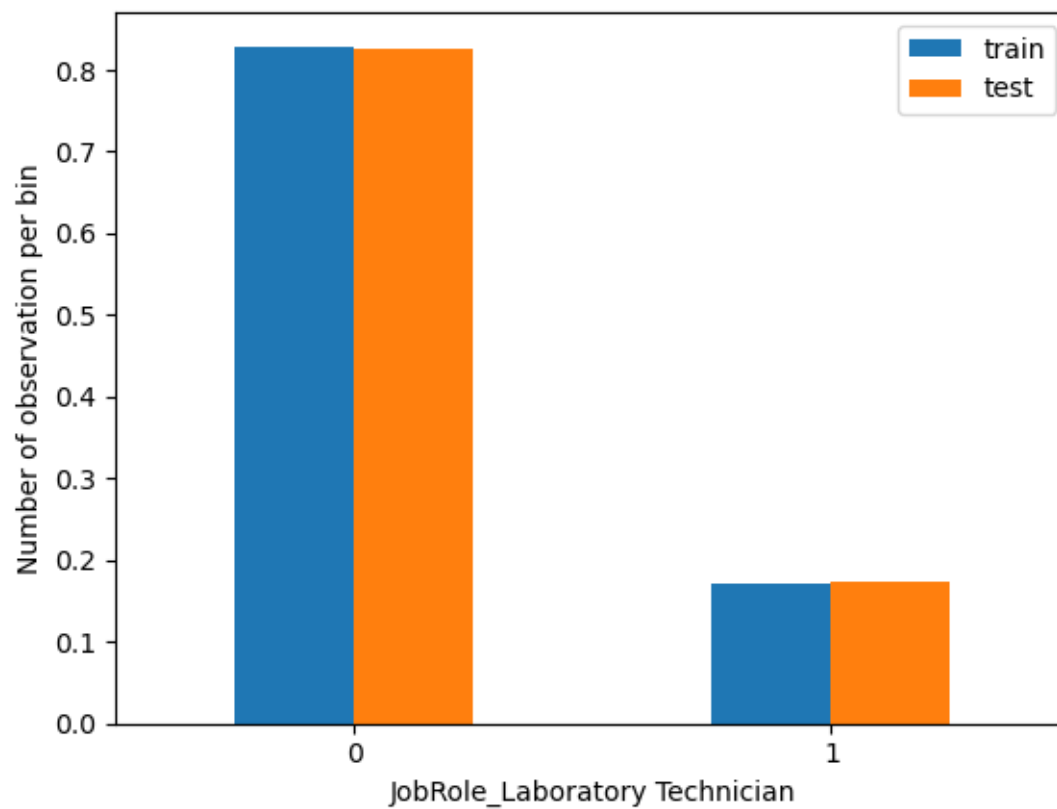


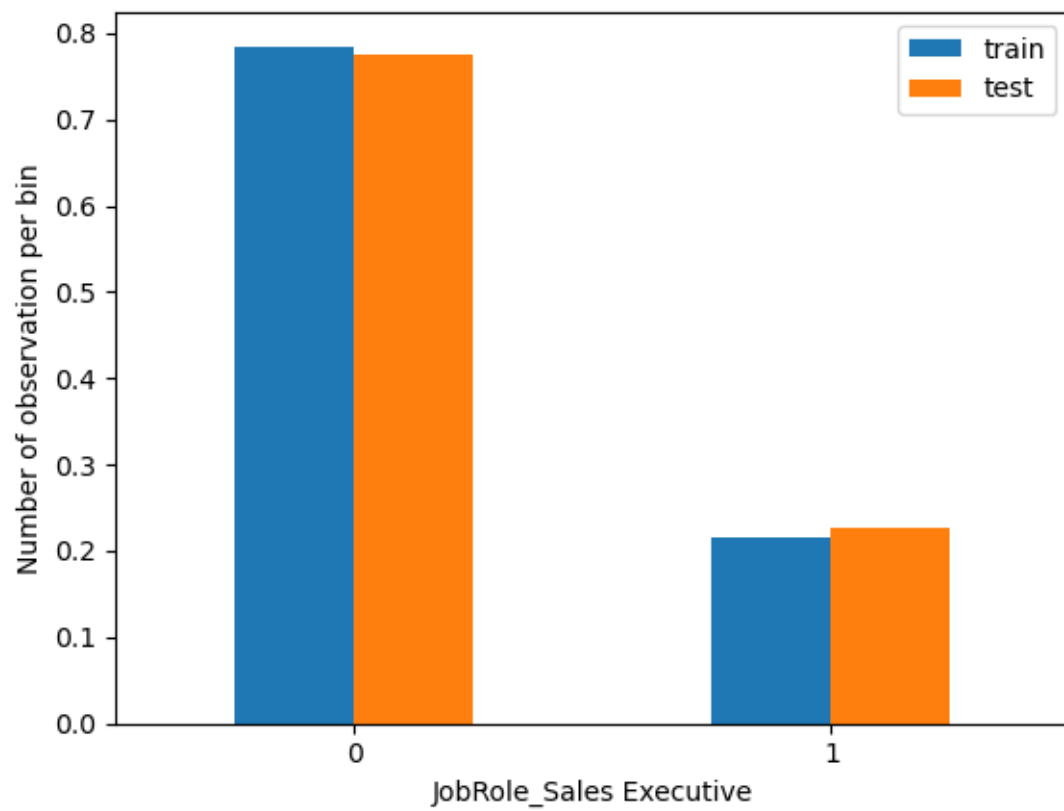


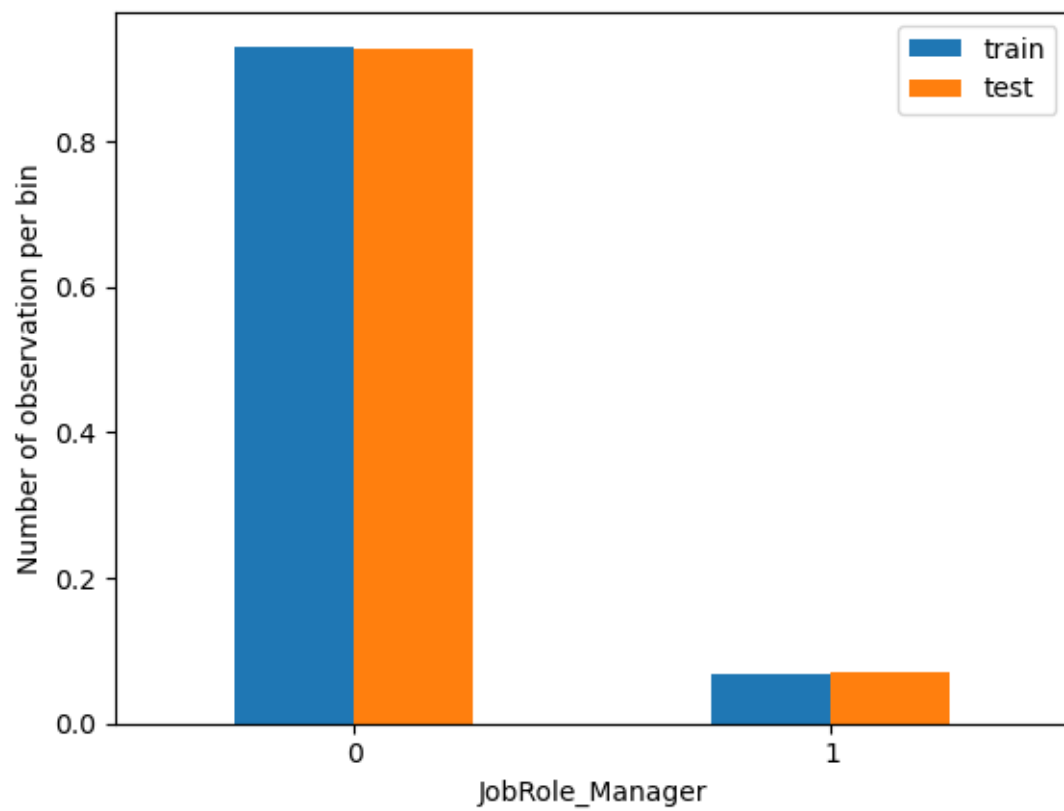


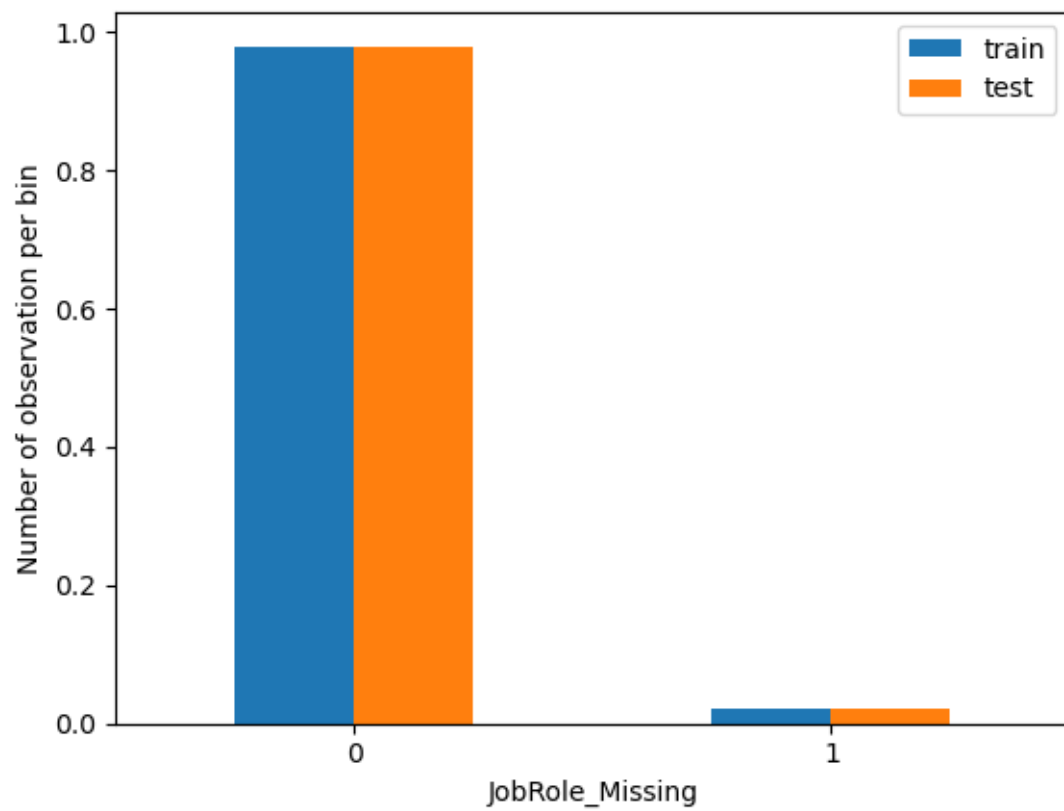


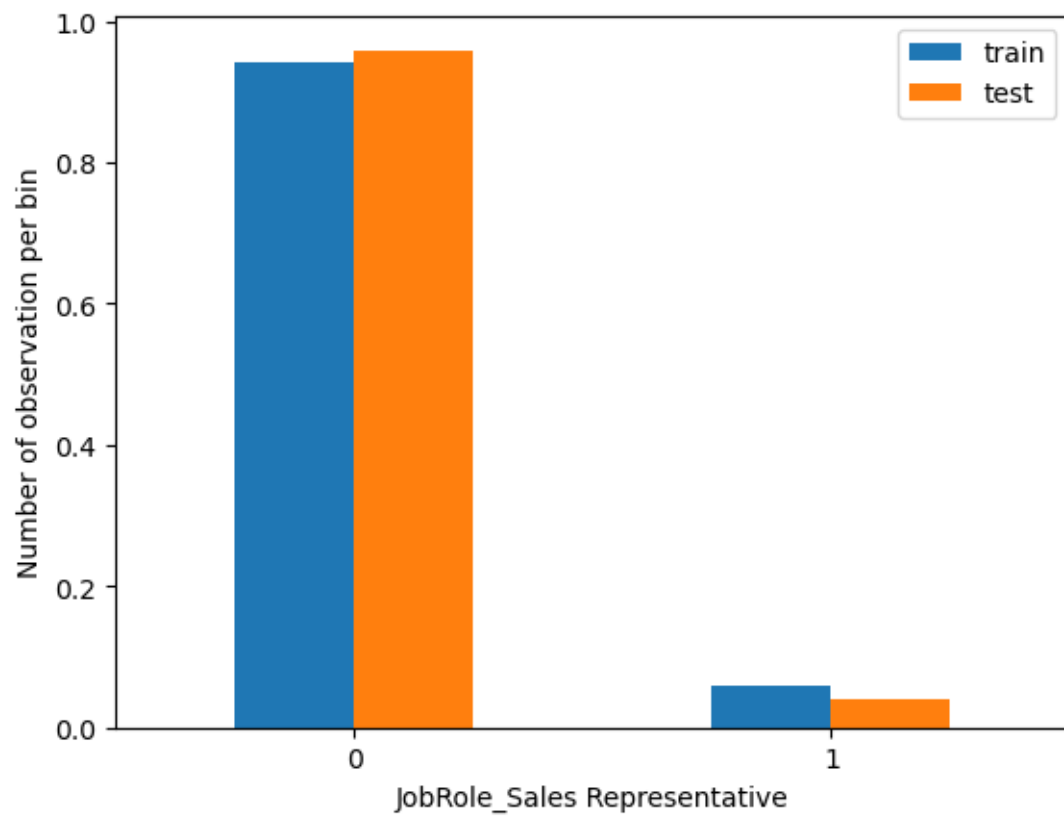


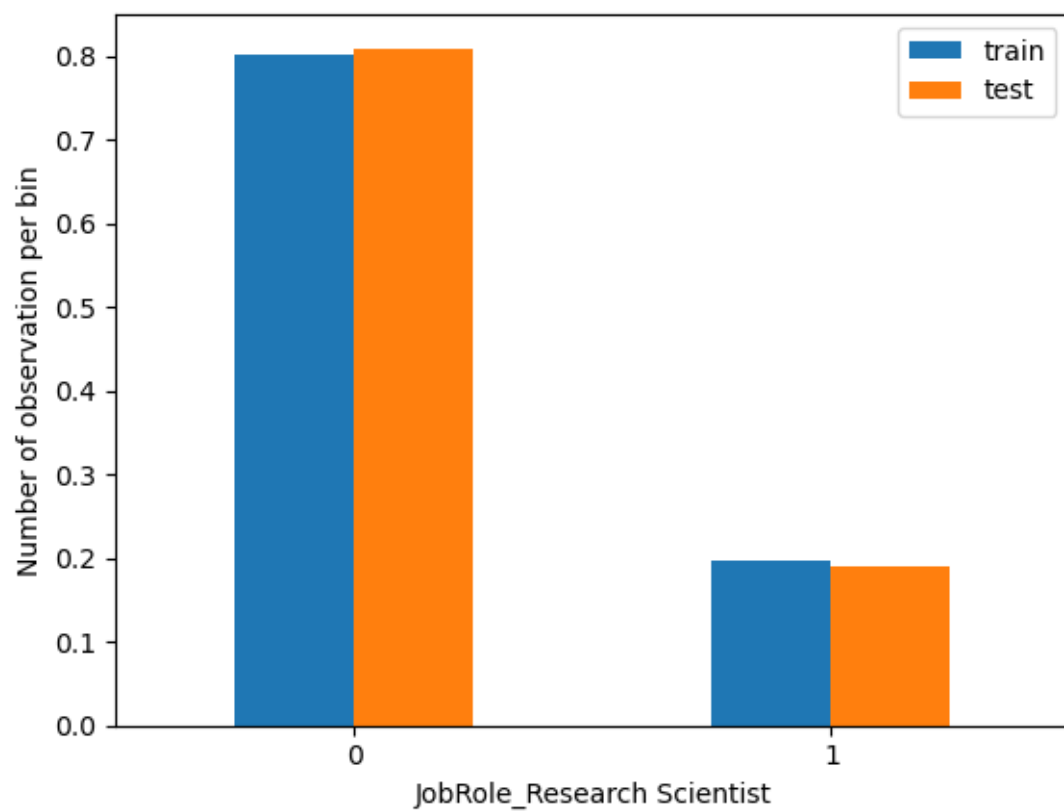


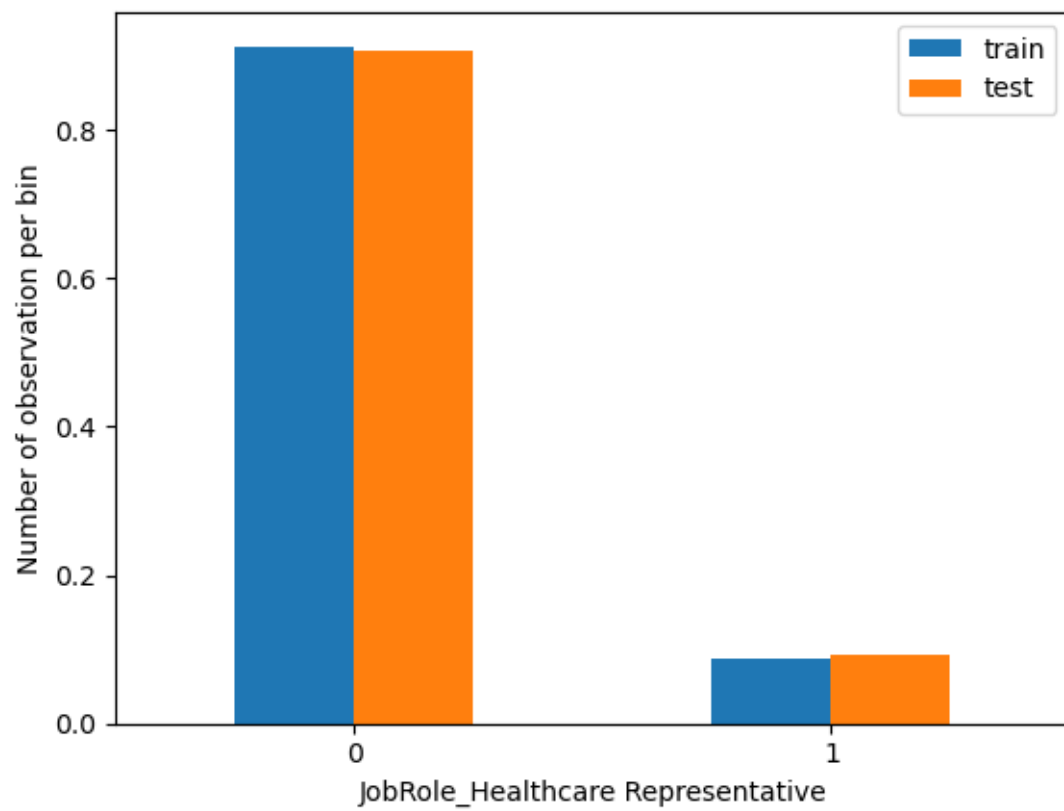


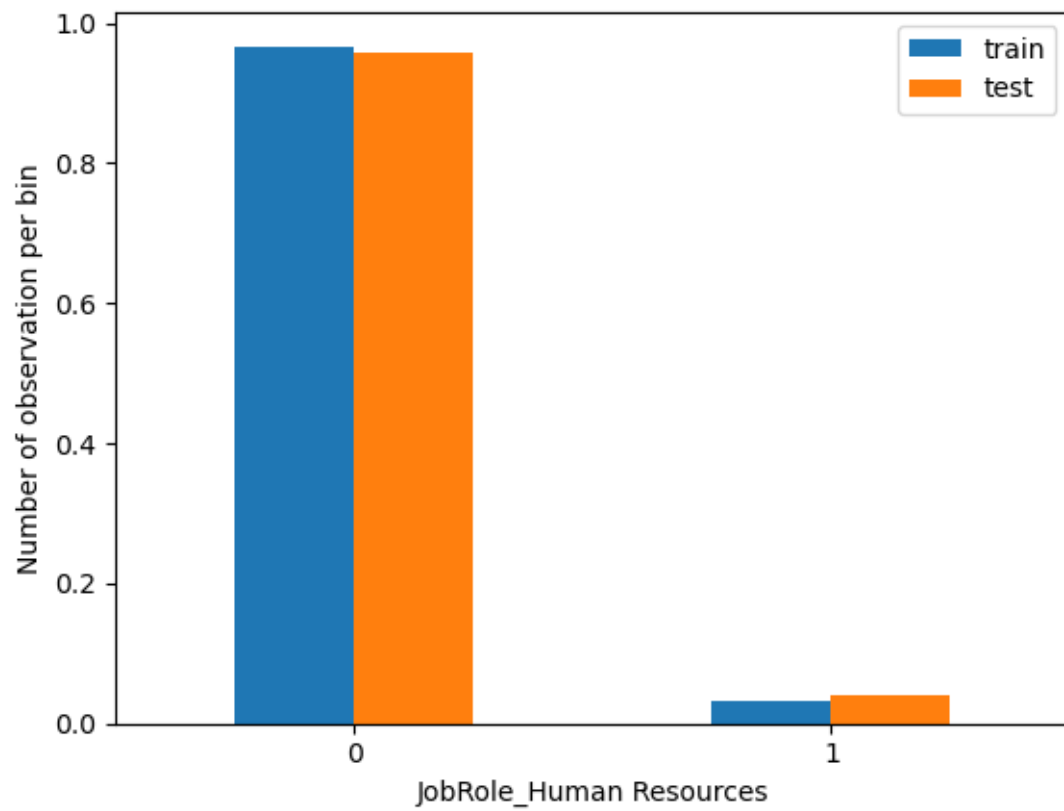


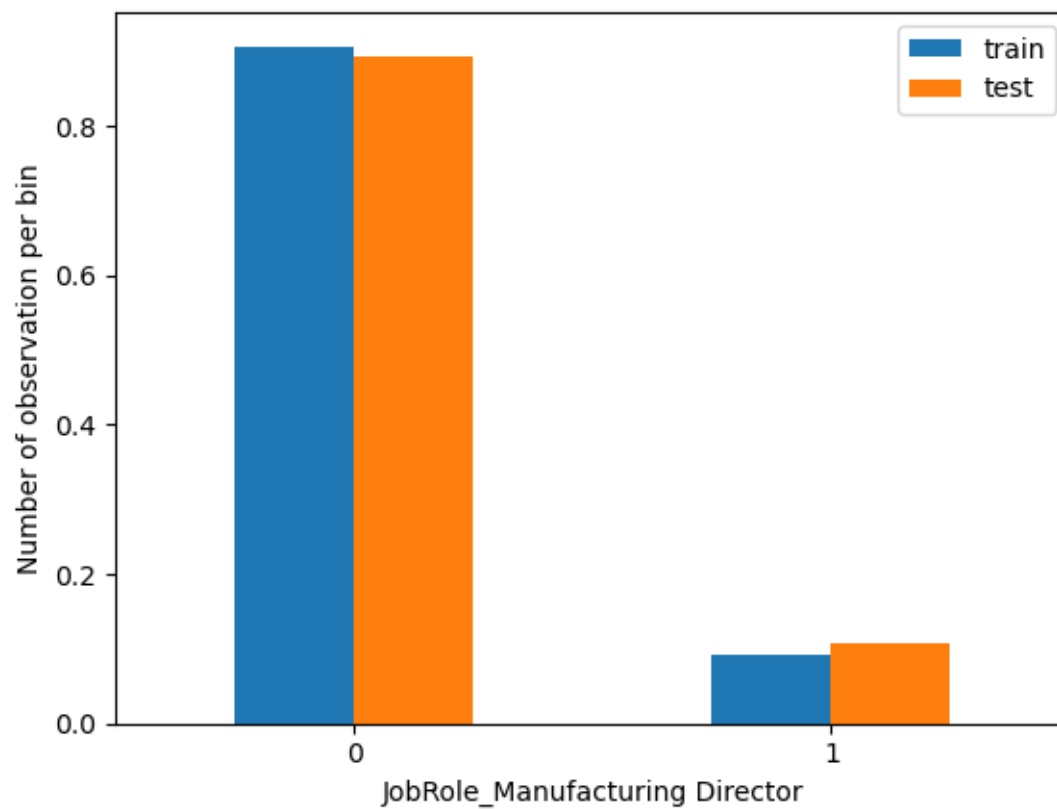


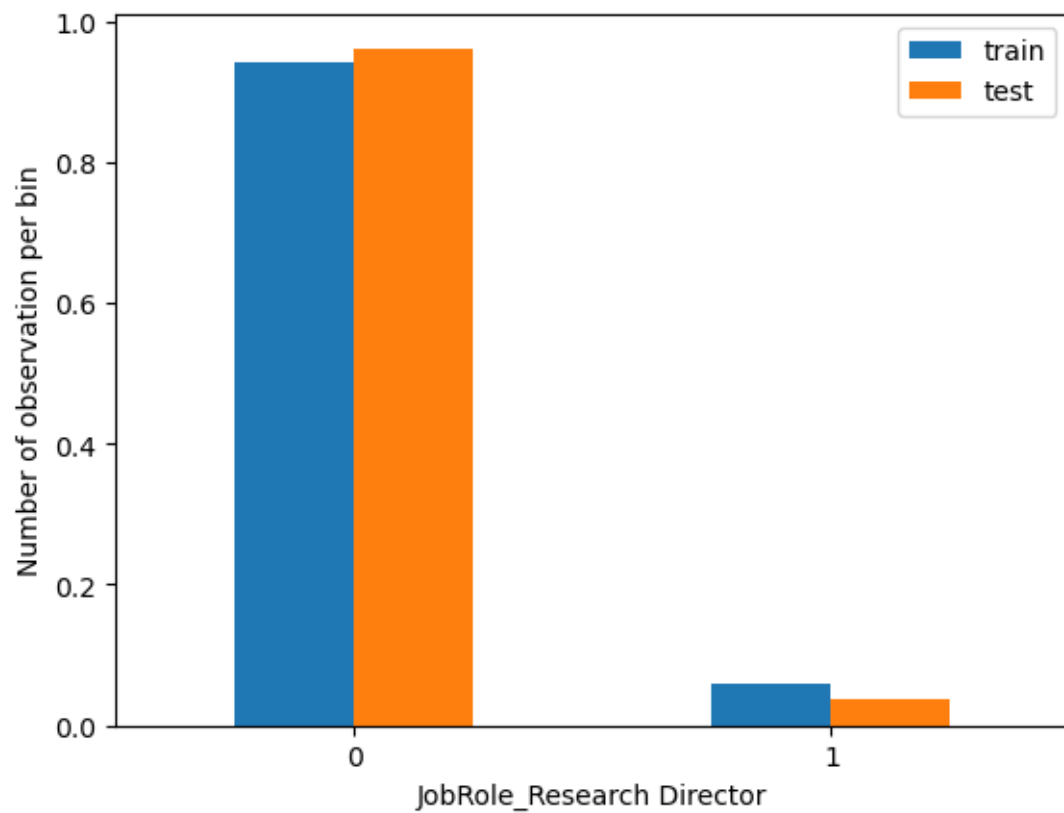


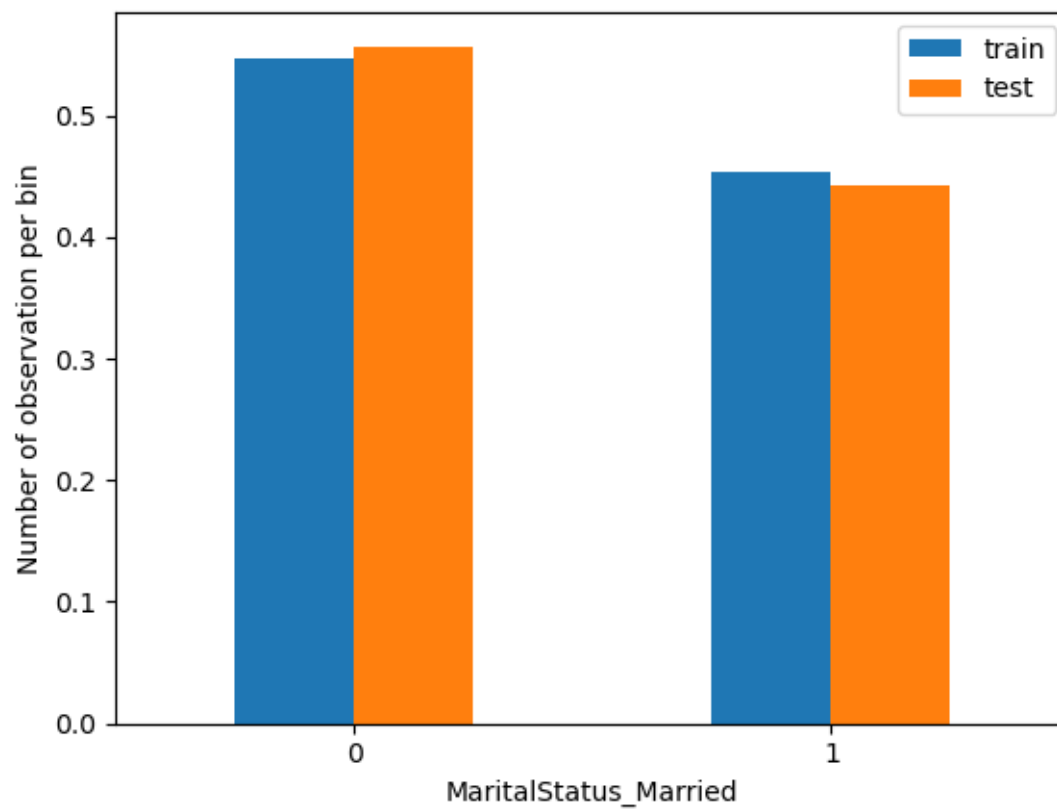


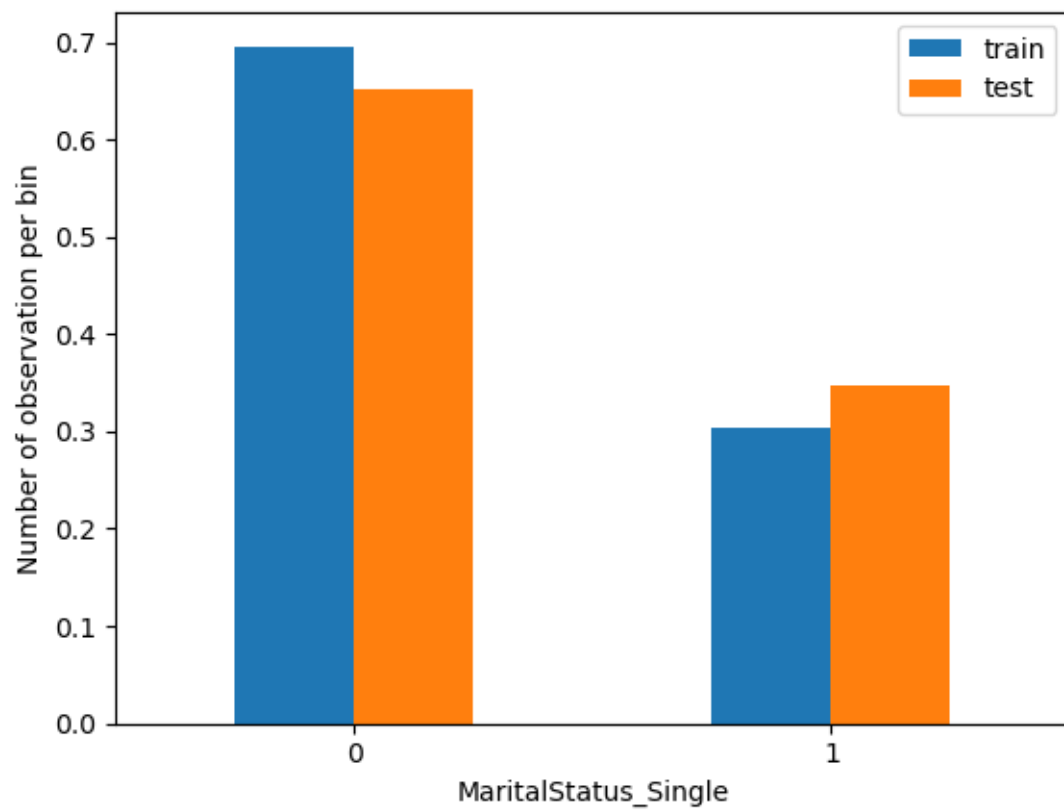


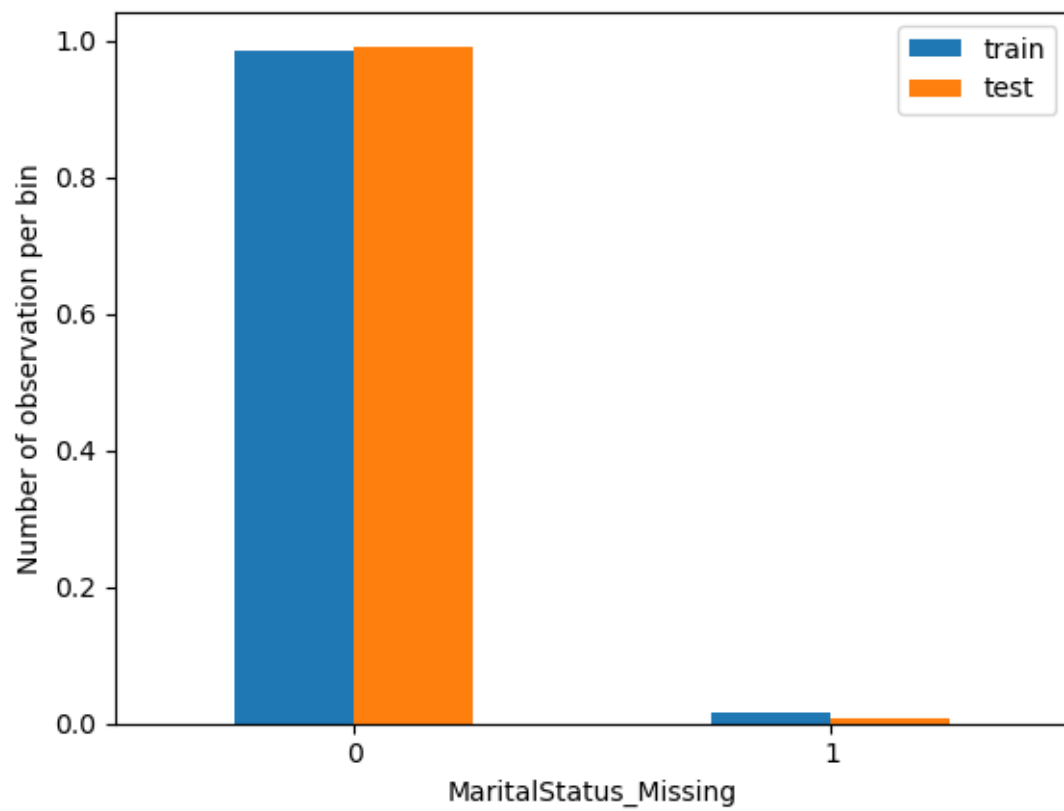


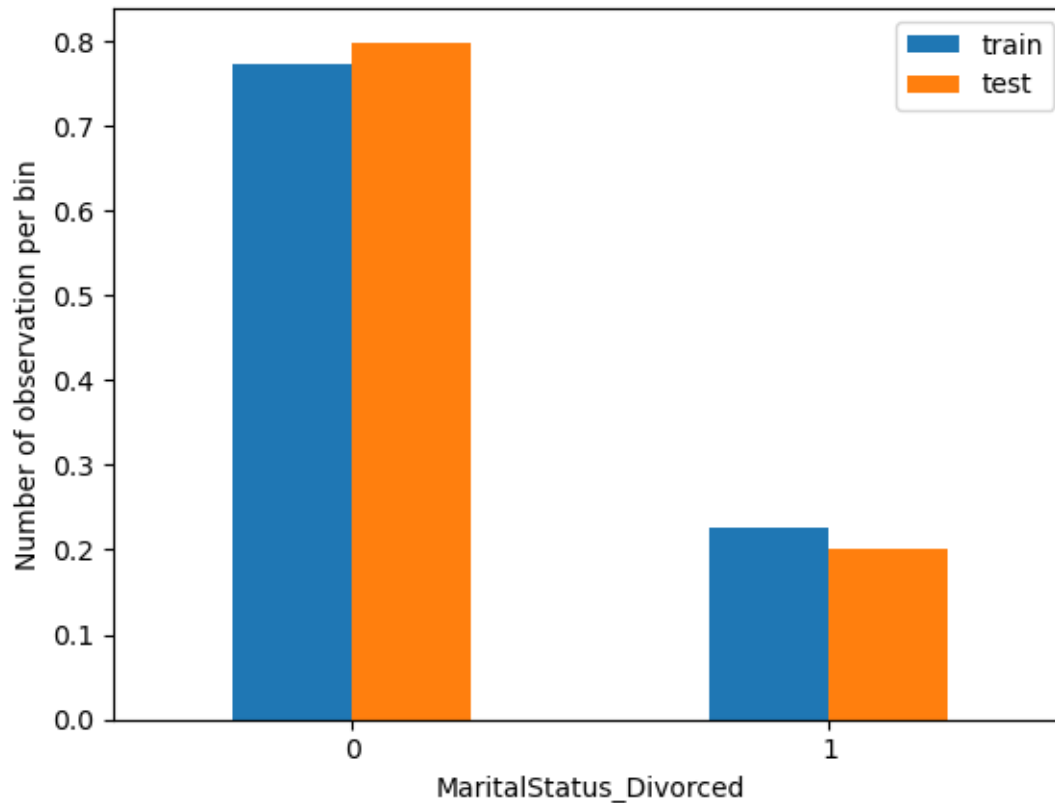












2.8 Discretization with EqualWidthDiscretiser

```
[34]: disc_width = EqualWidthDiscretiser(
        bins = 10,
        variables = continuous
    )
    disc_width.fit(tmp_mean_train)

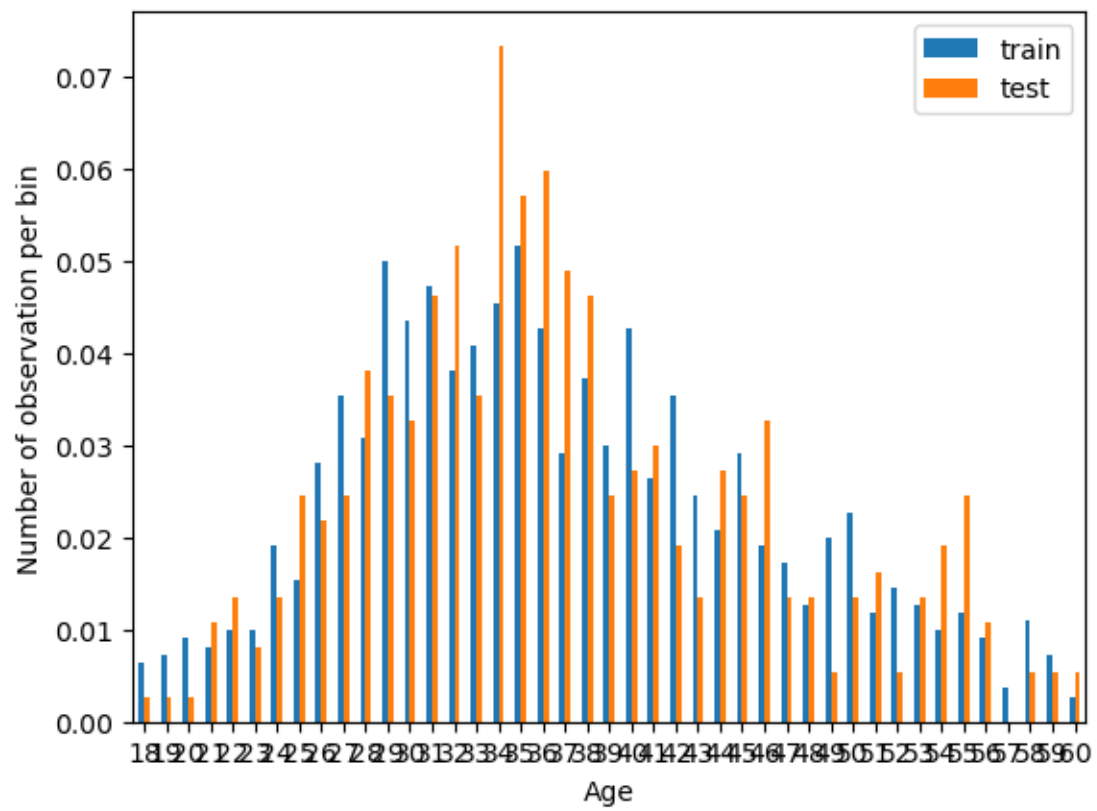
    disc_width_train = disc_width.transform(tmp_mean_train)
    disc_width_test = disc_width.transform(tmp_mean_test)
    disc_width_train.head()

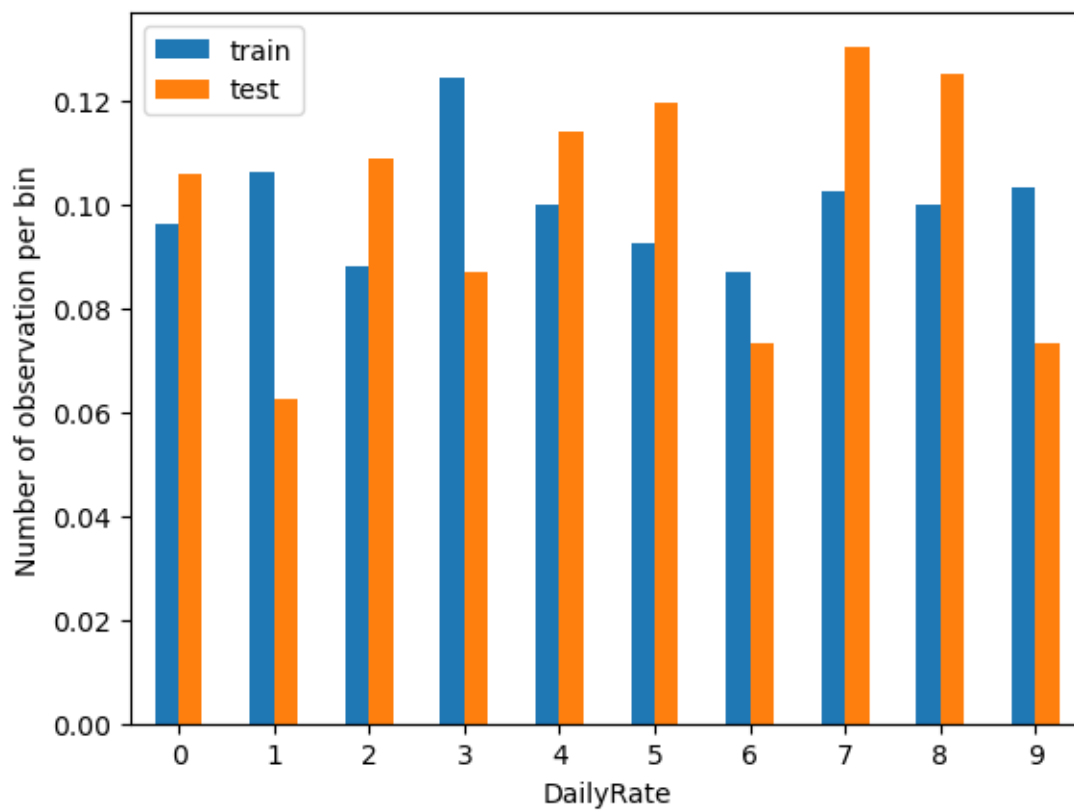
[35]: for col in train_disc.columns:

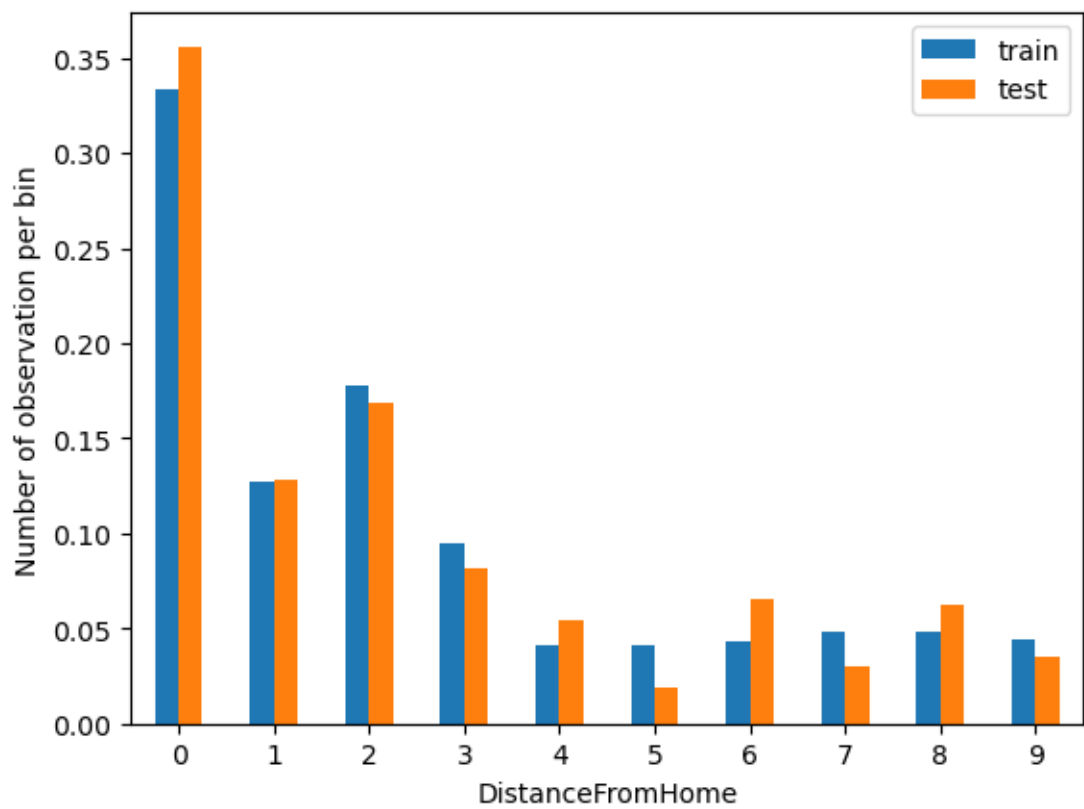
        t1 = disc_width_train.groupby([col])[col].count() / len(disc_width_train)
        t2 = disc_width_test.groupby([col])[col].count() / len(disc_width_test)

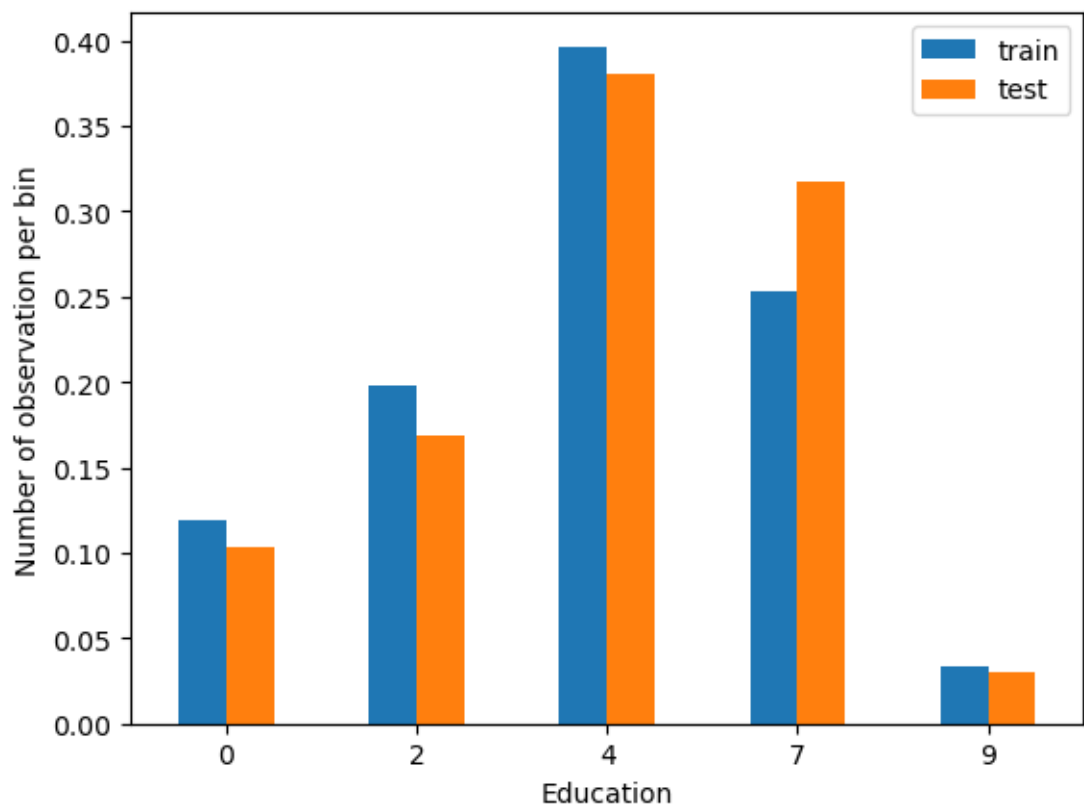
        tmp = pd.concat([t1, t2], axis = 1)
        tmp.columns = ['train', 'test']
        tmp.plot.bar()
        plt.xticks(rotation = 0)
```

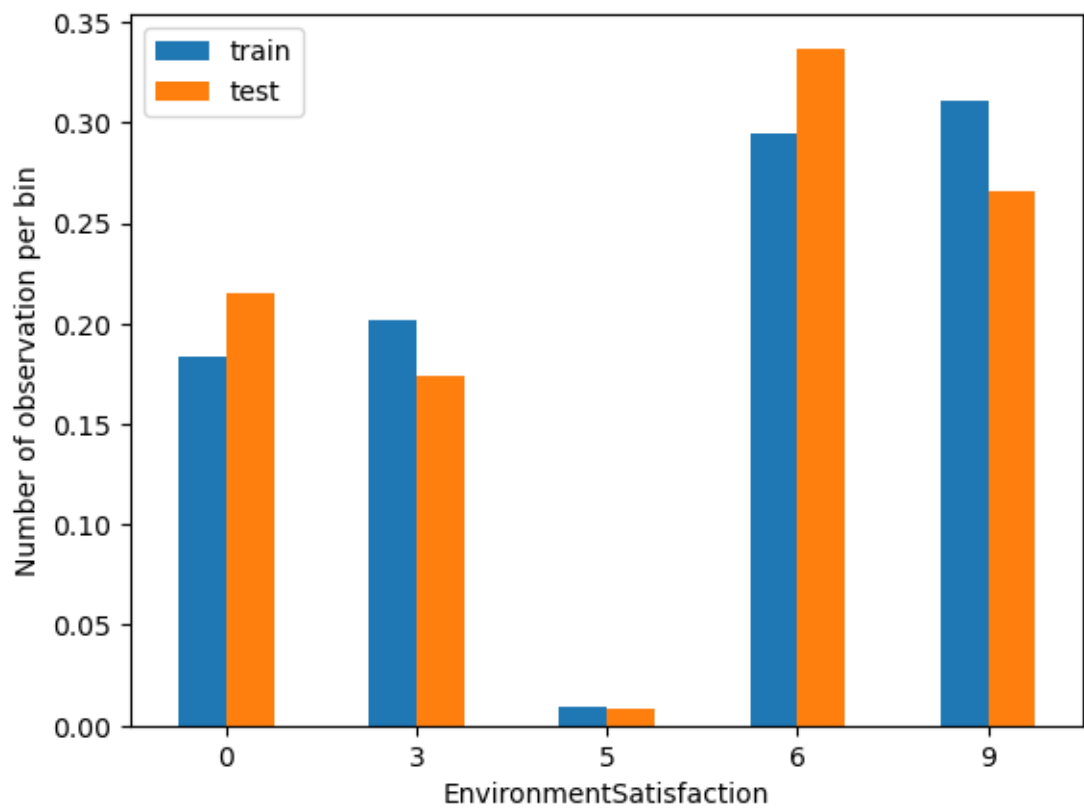
```
plt.ylabel('Number of observation per bin')  
plt.show()
```

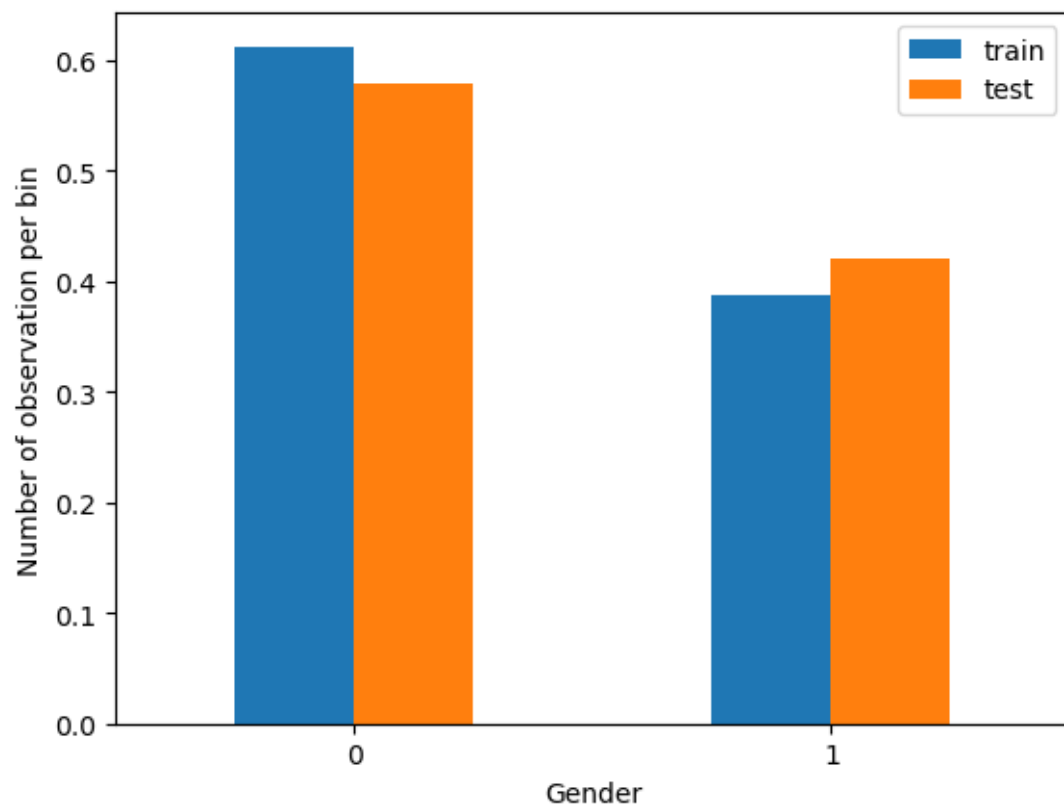


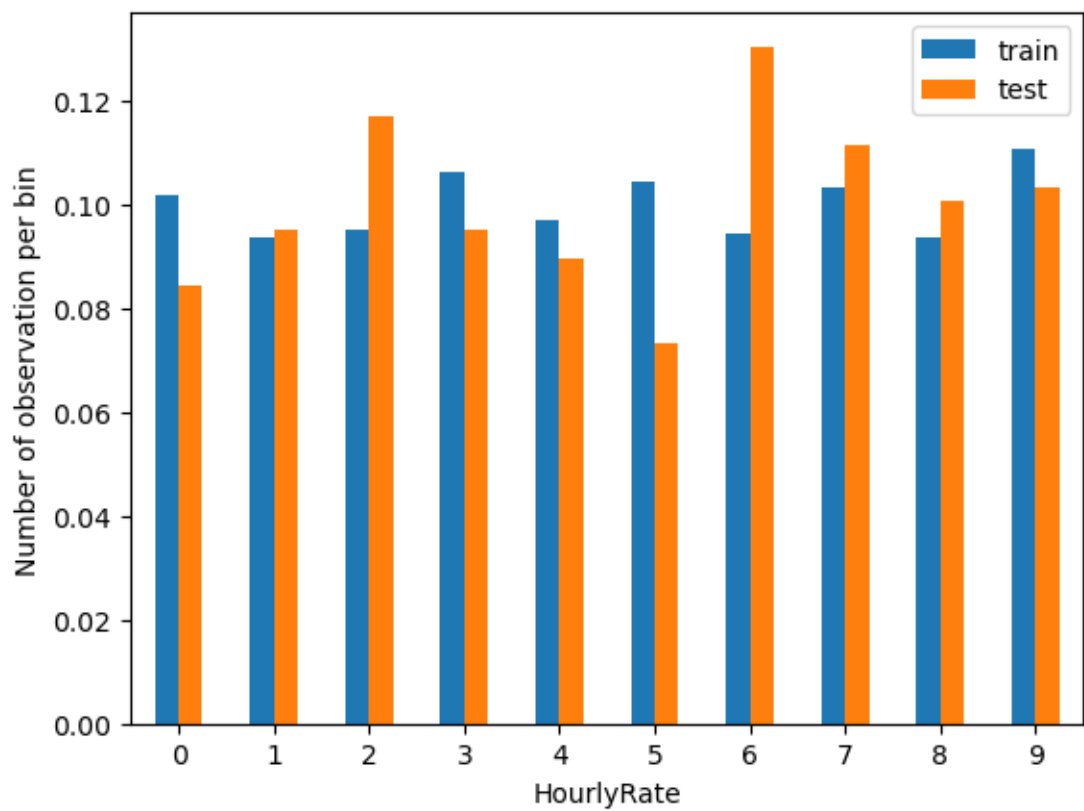


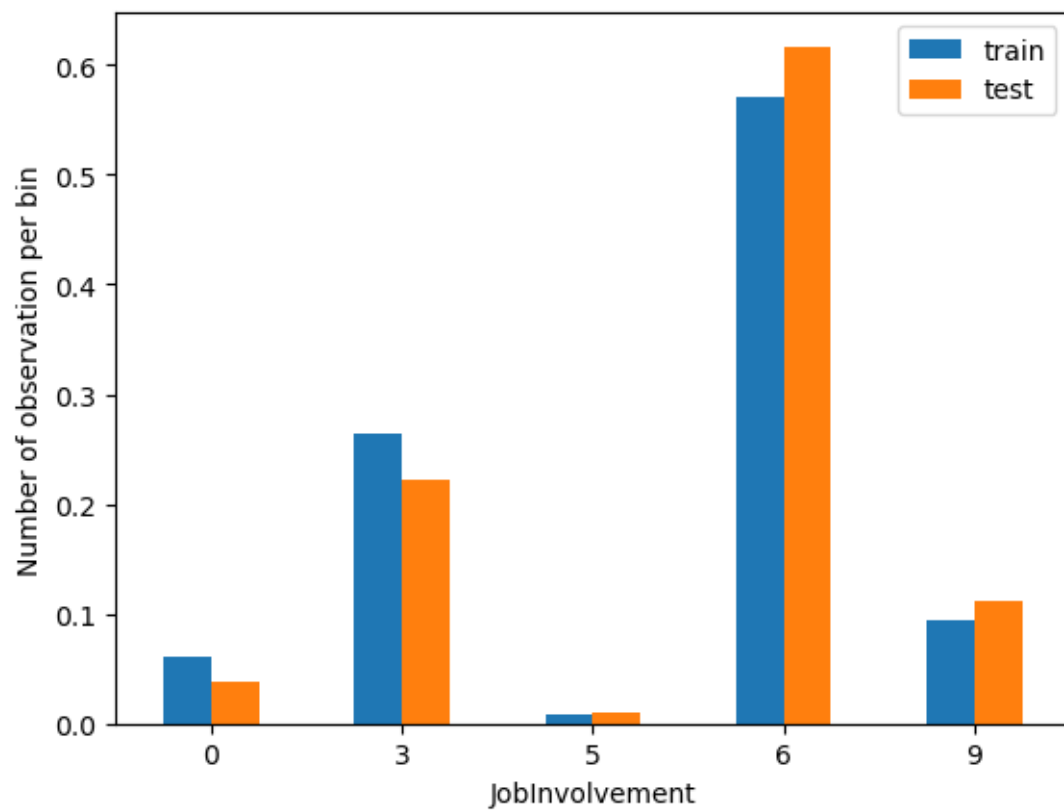


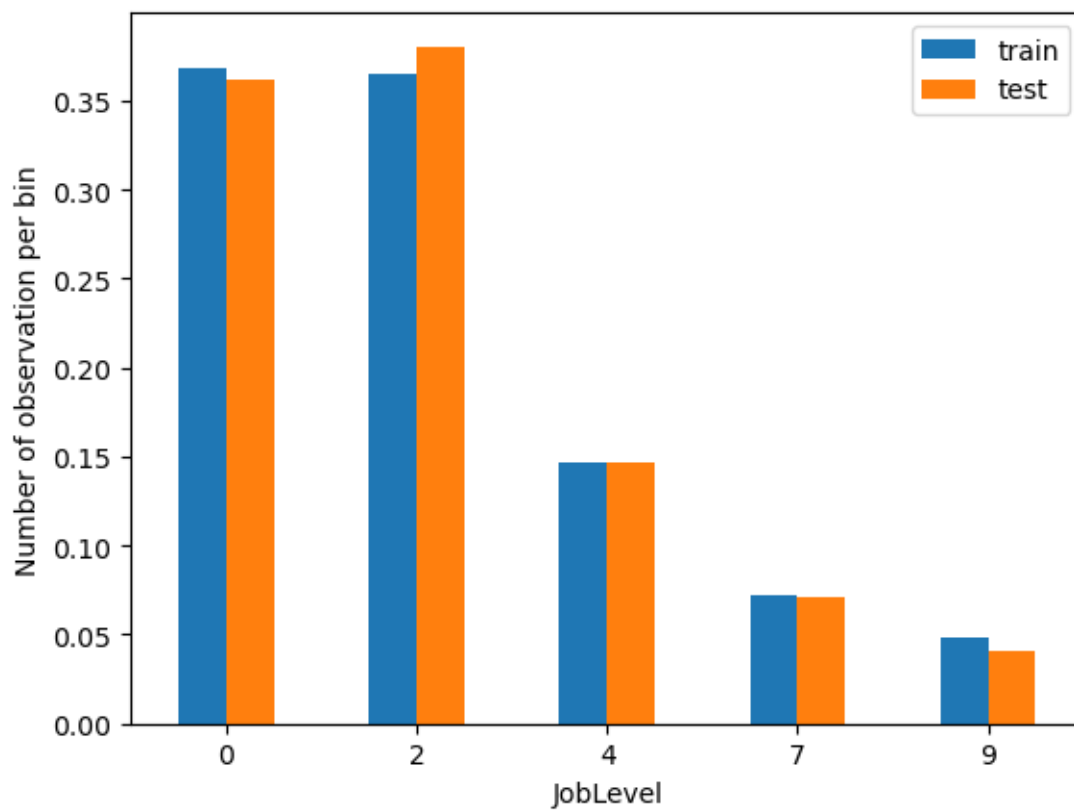


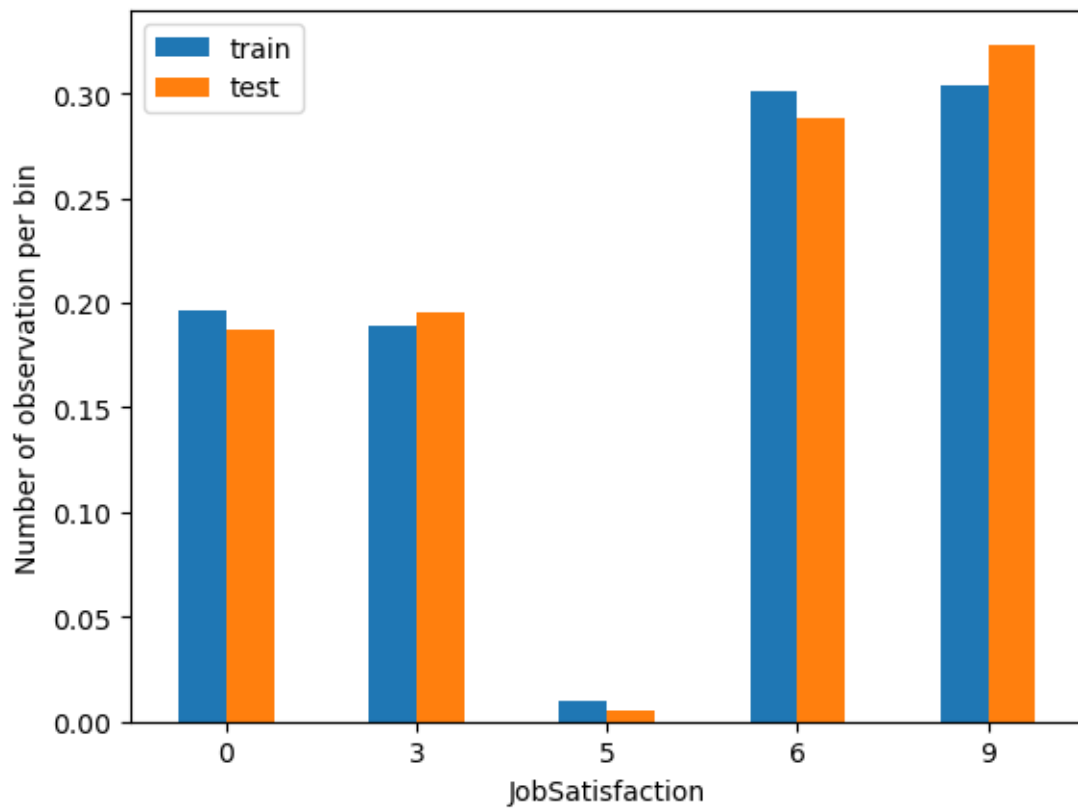


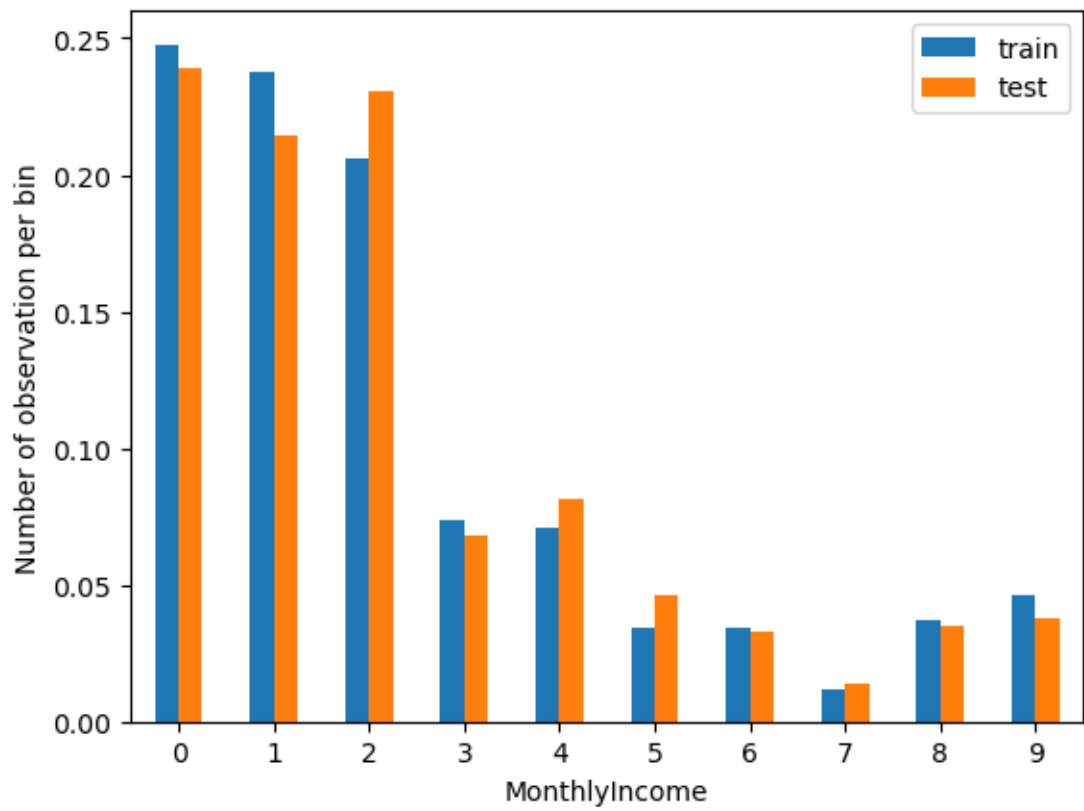


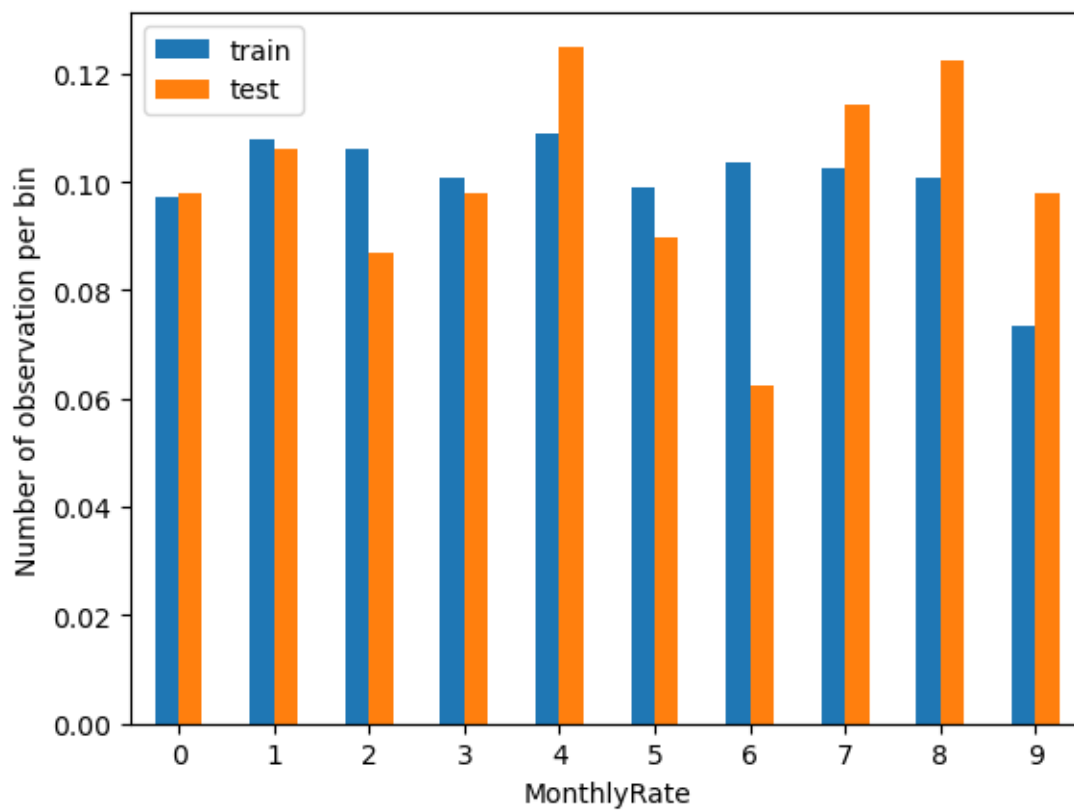


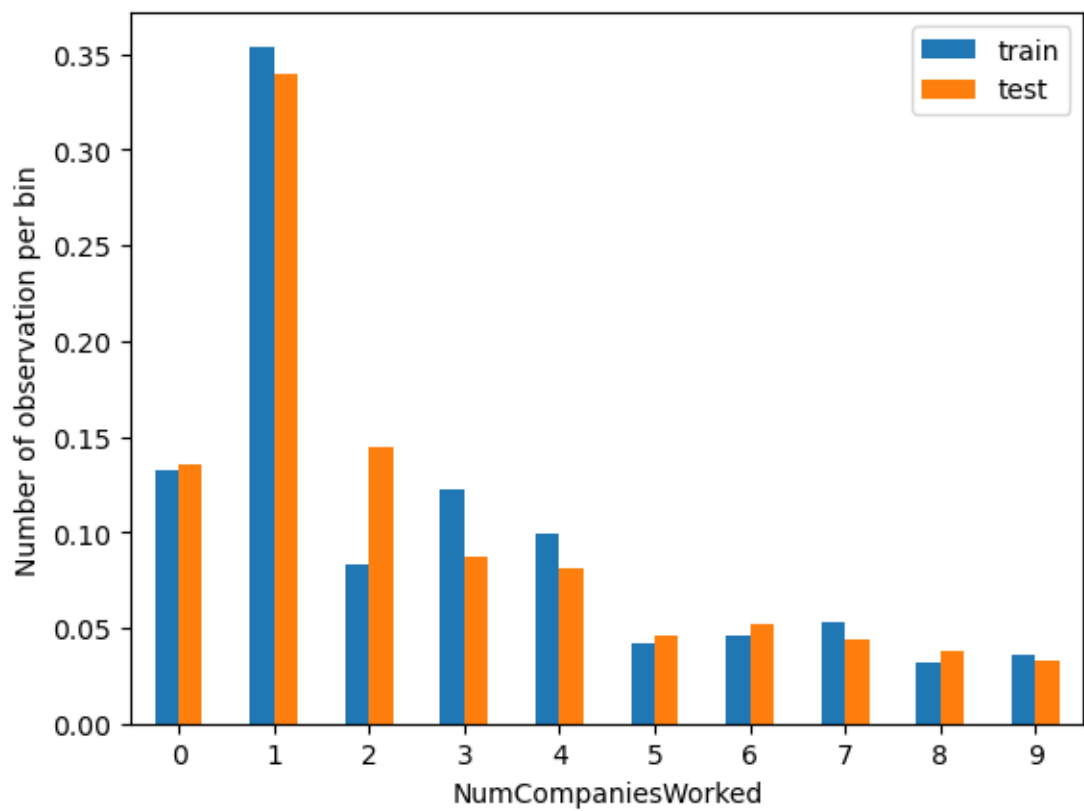


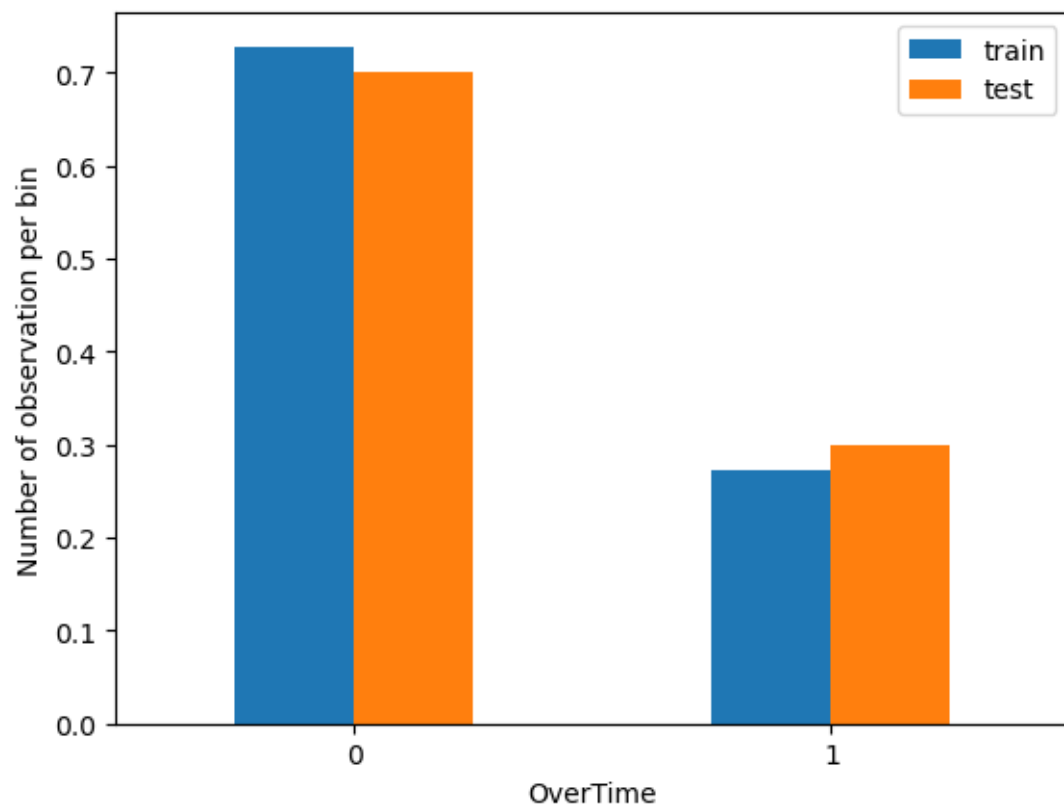


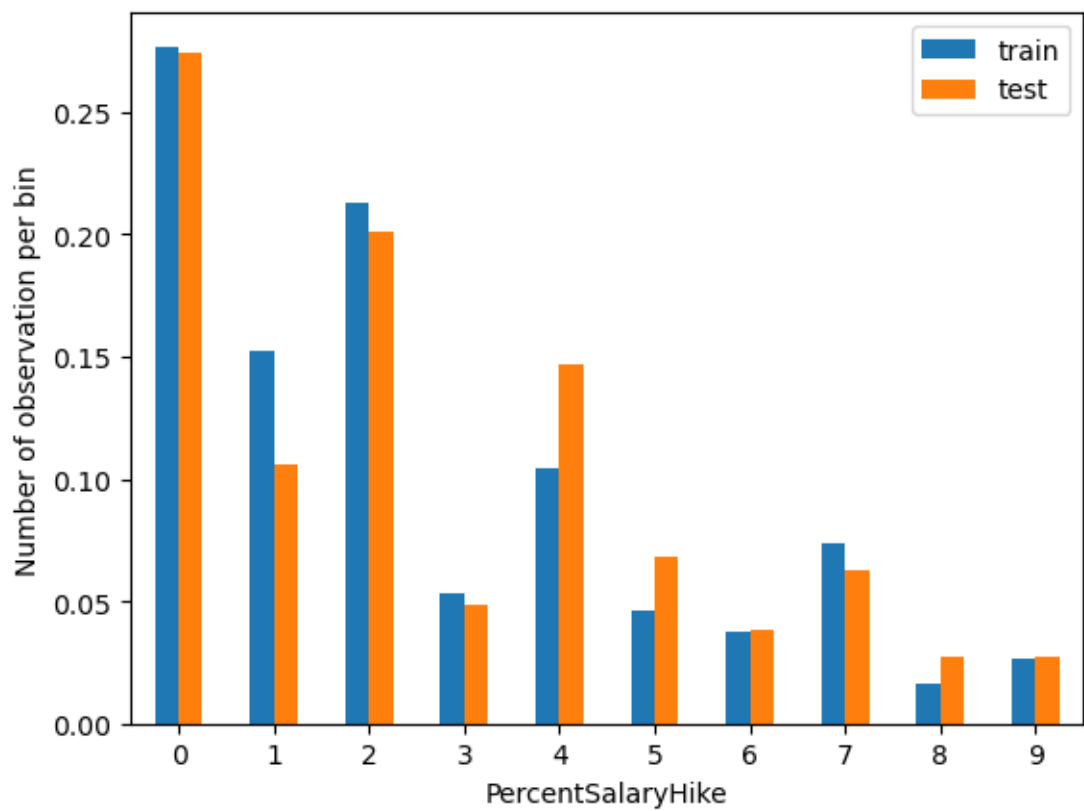


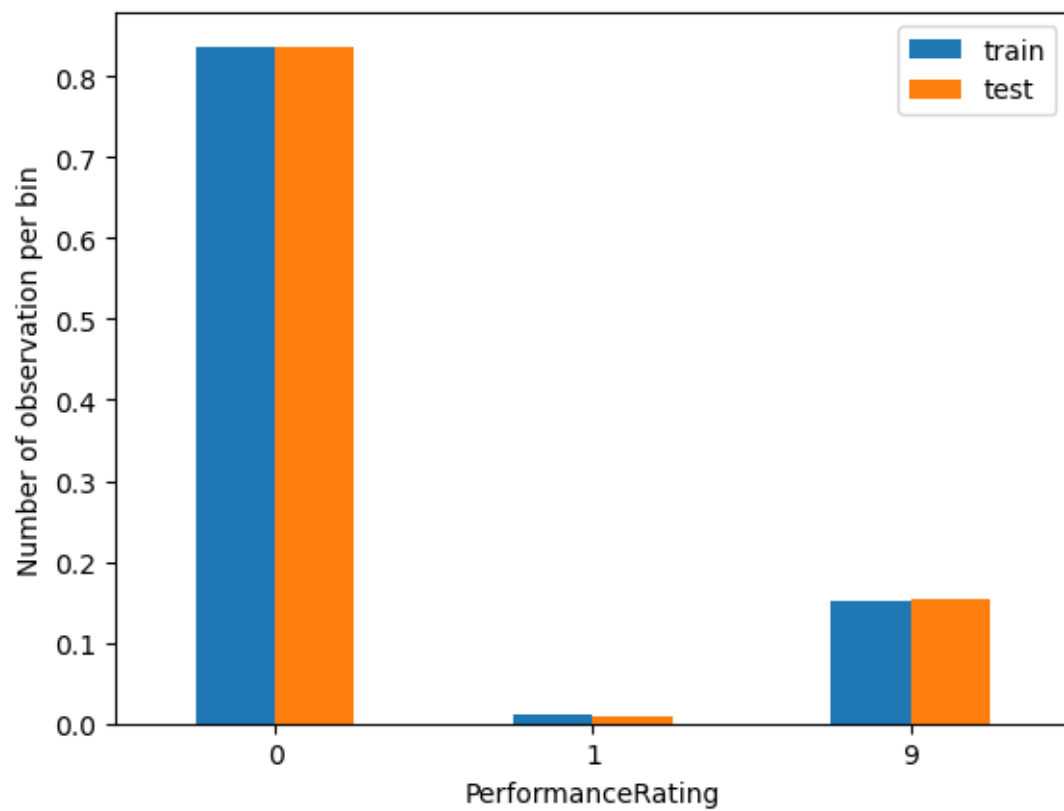


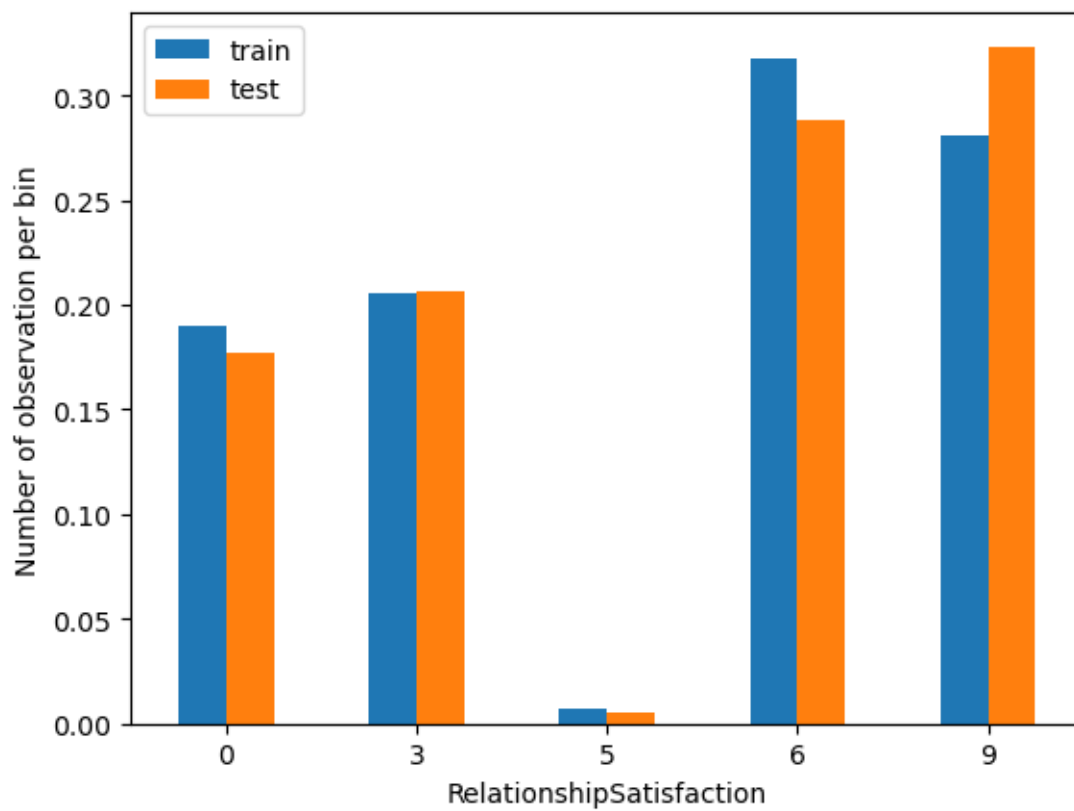


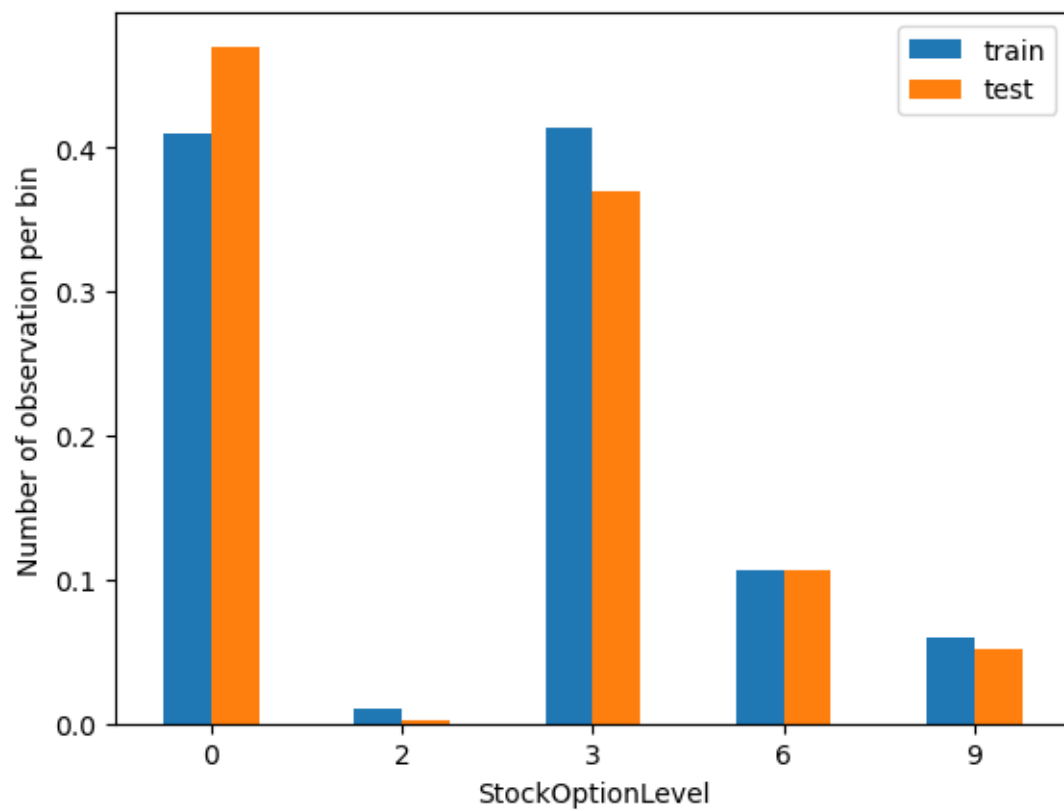


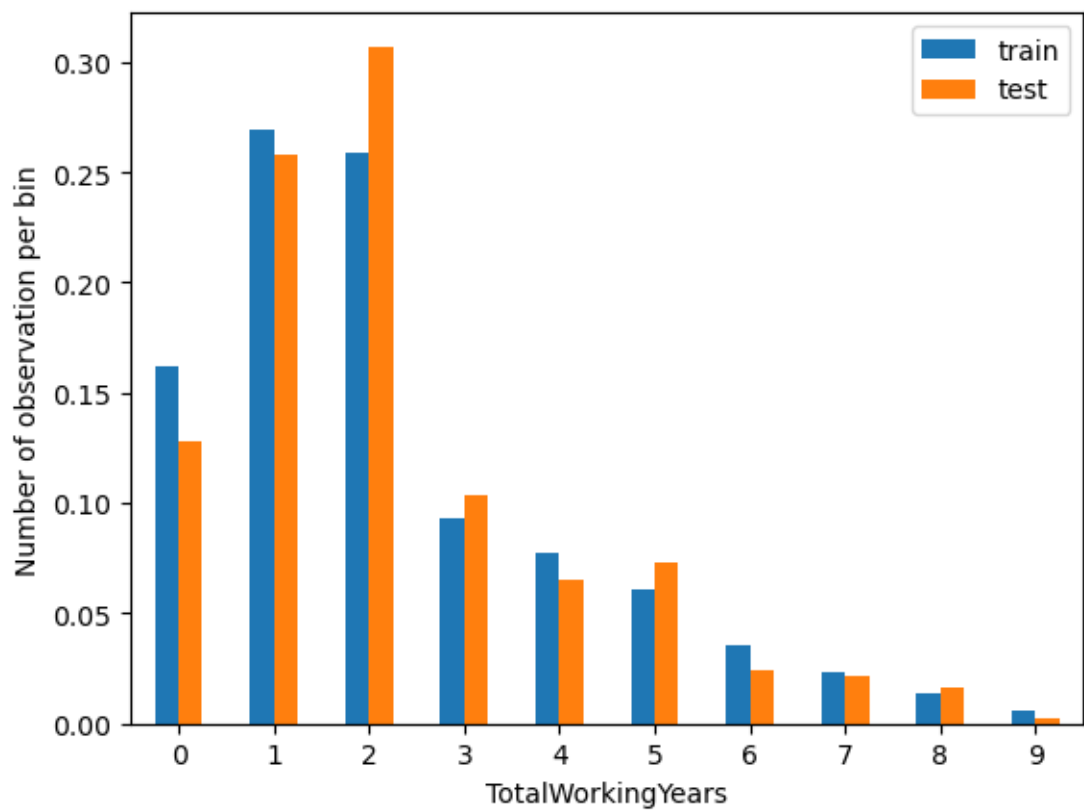


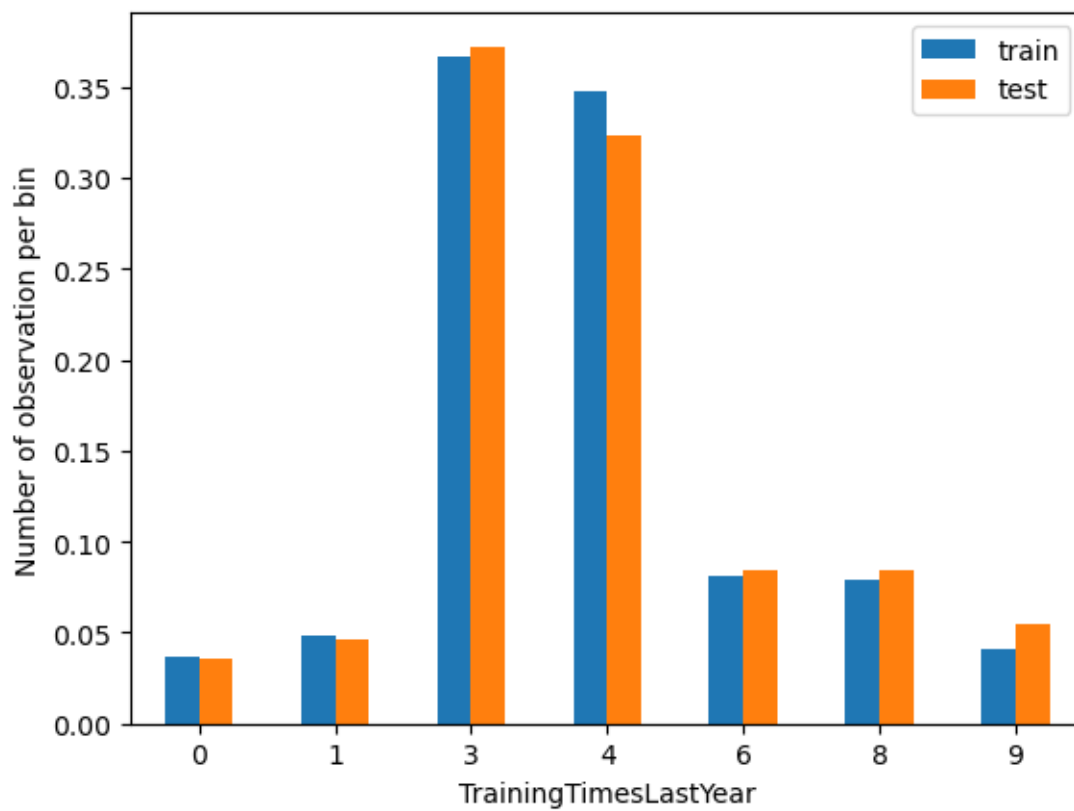


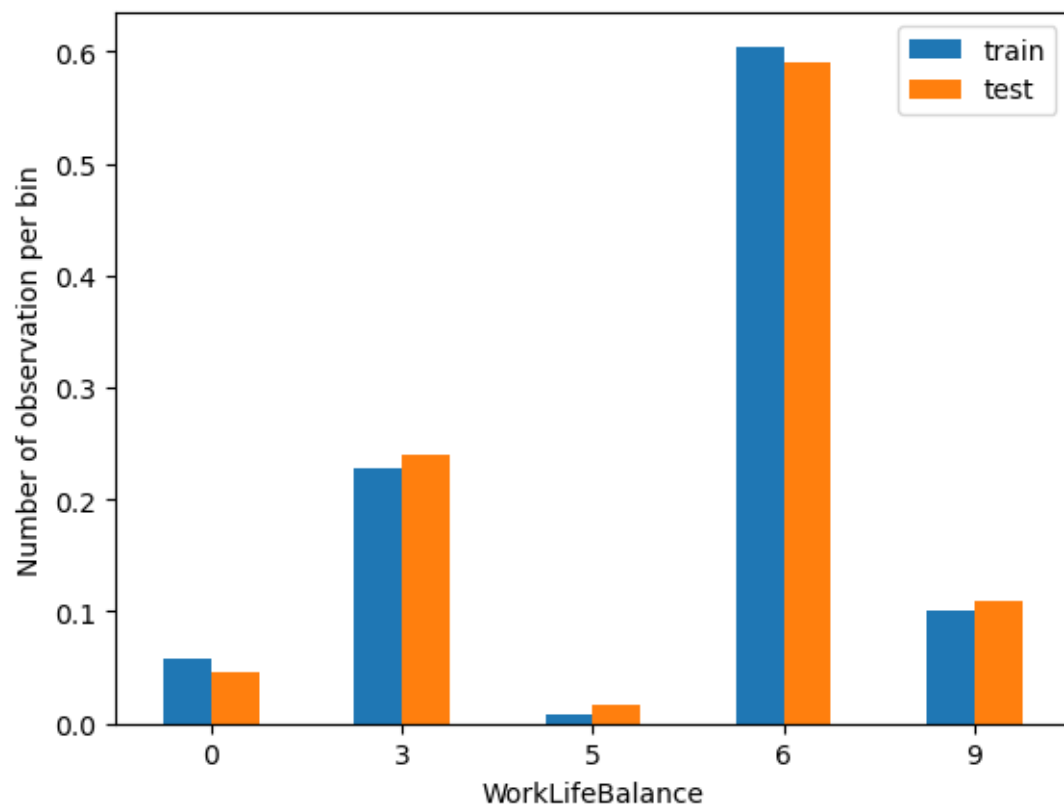


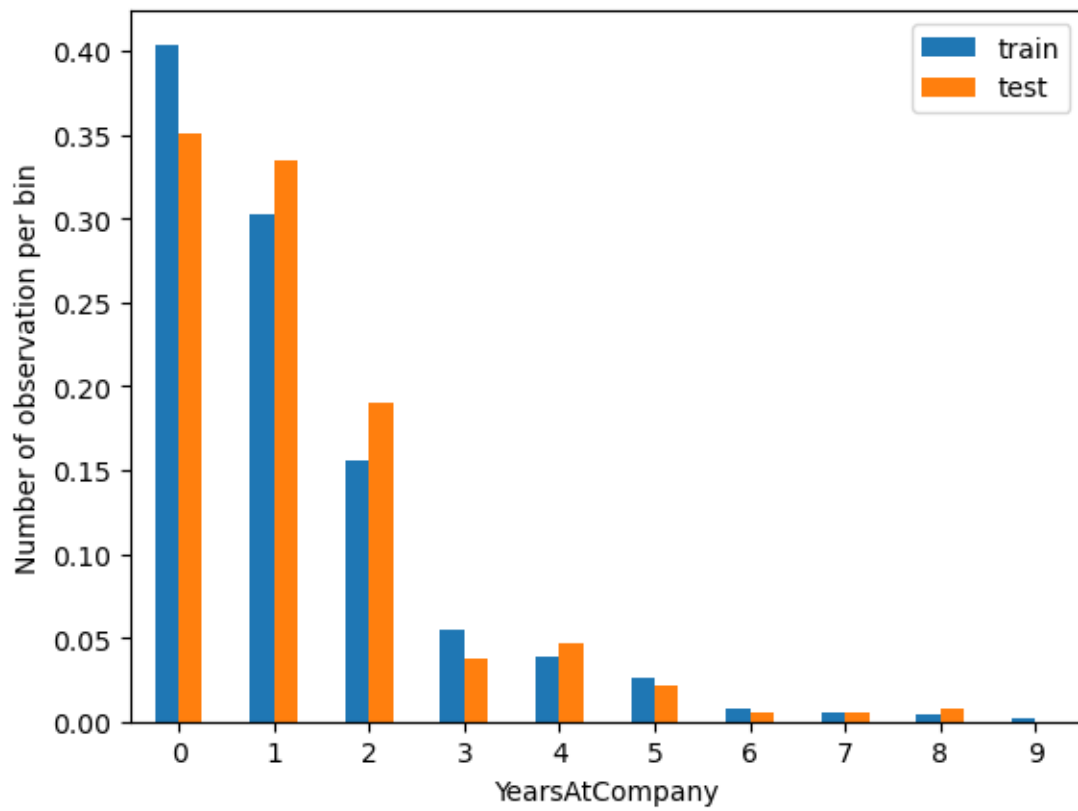


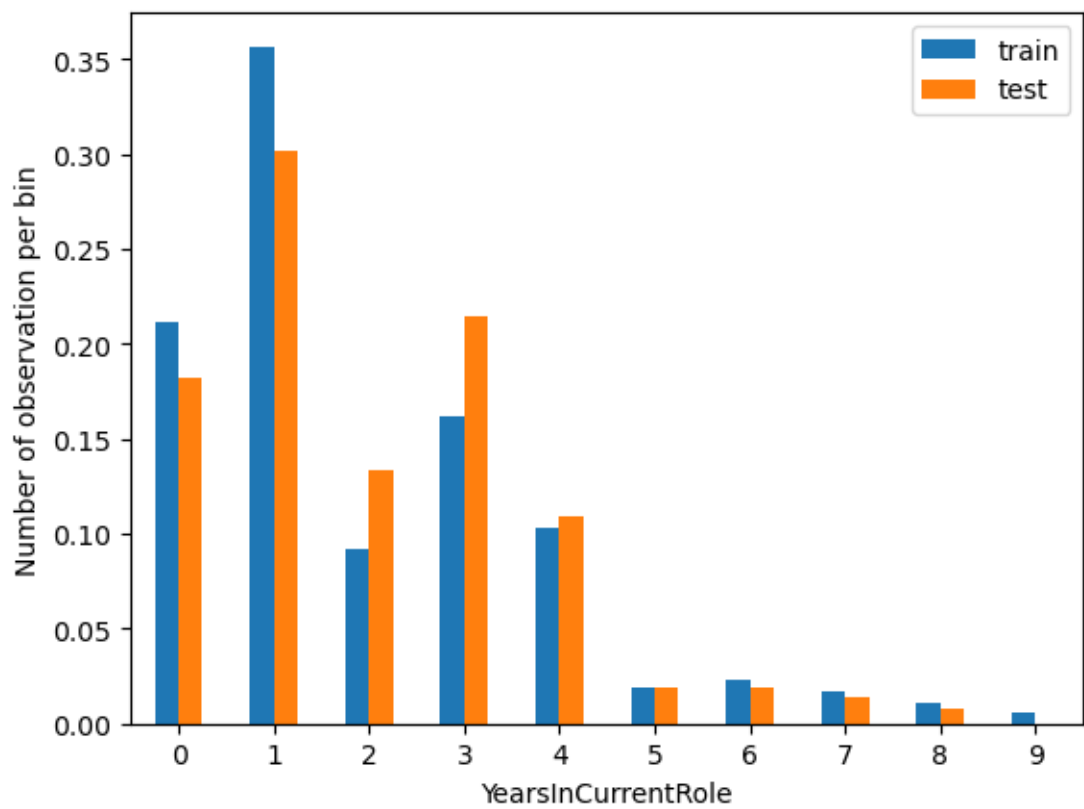


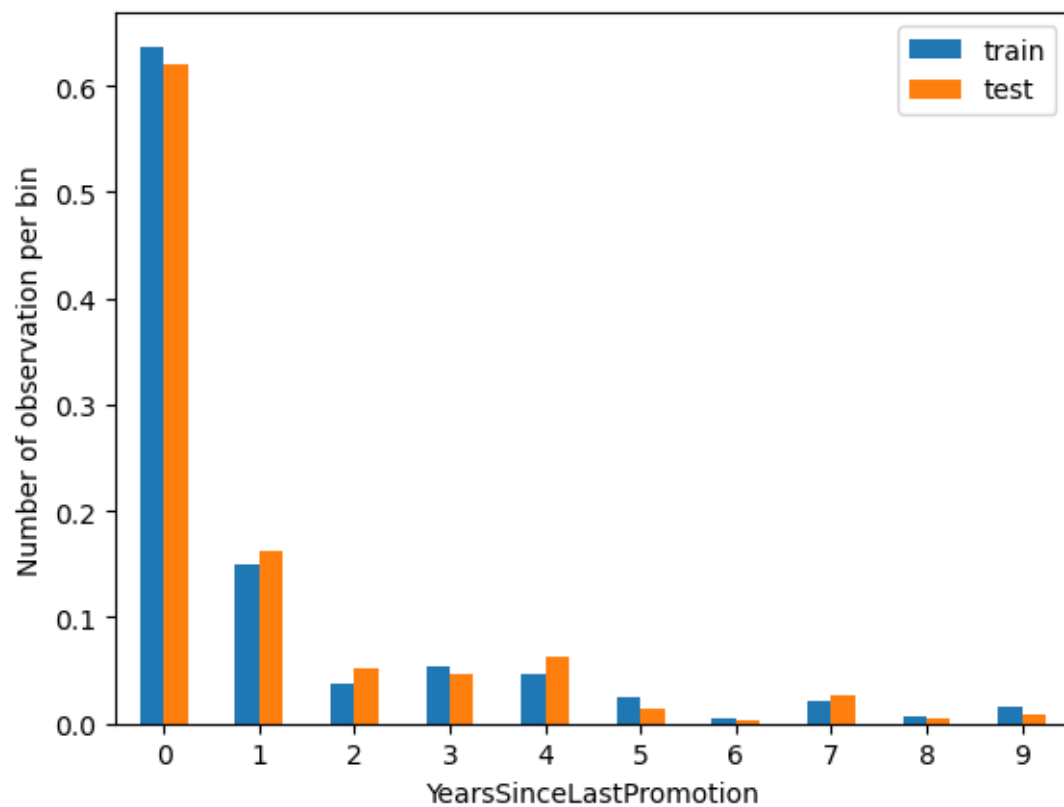


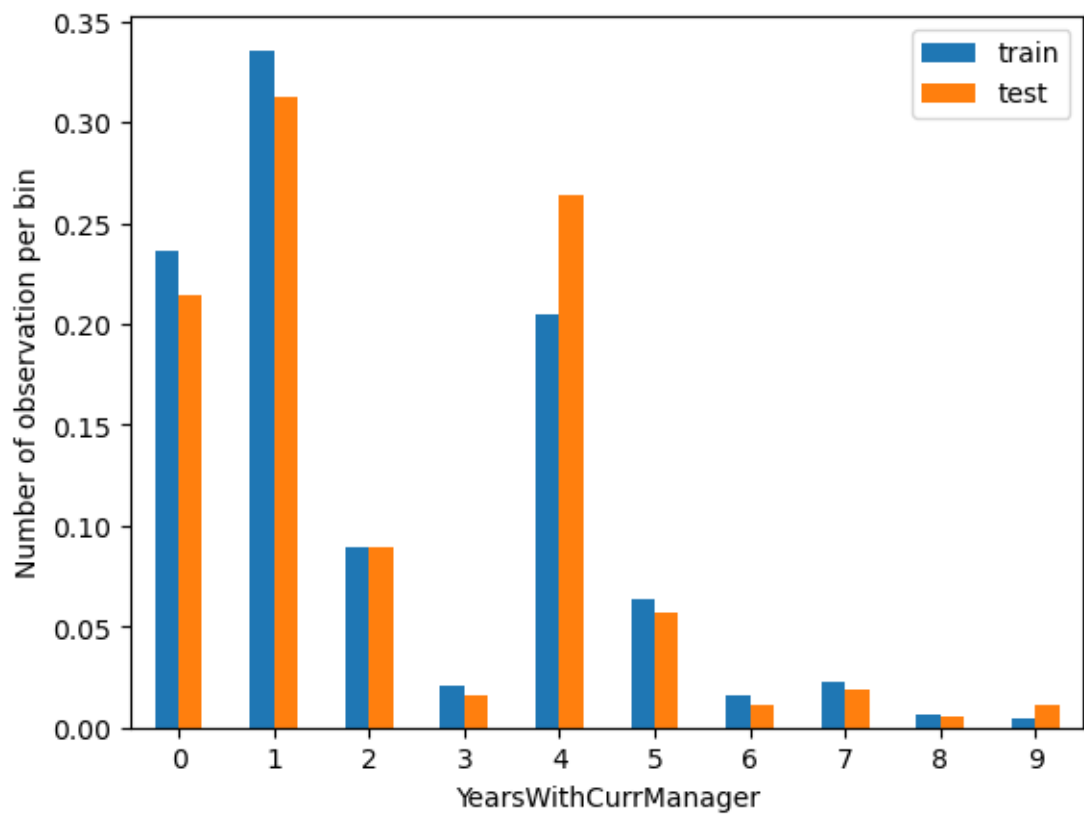


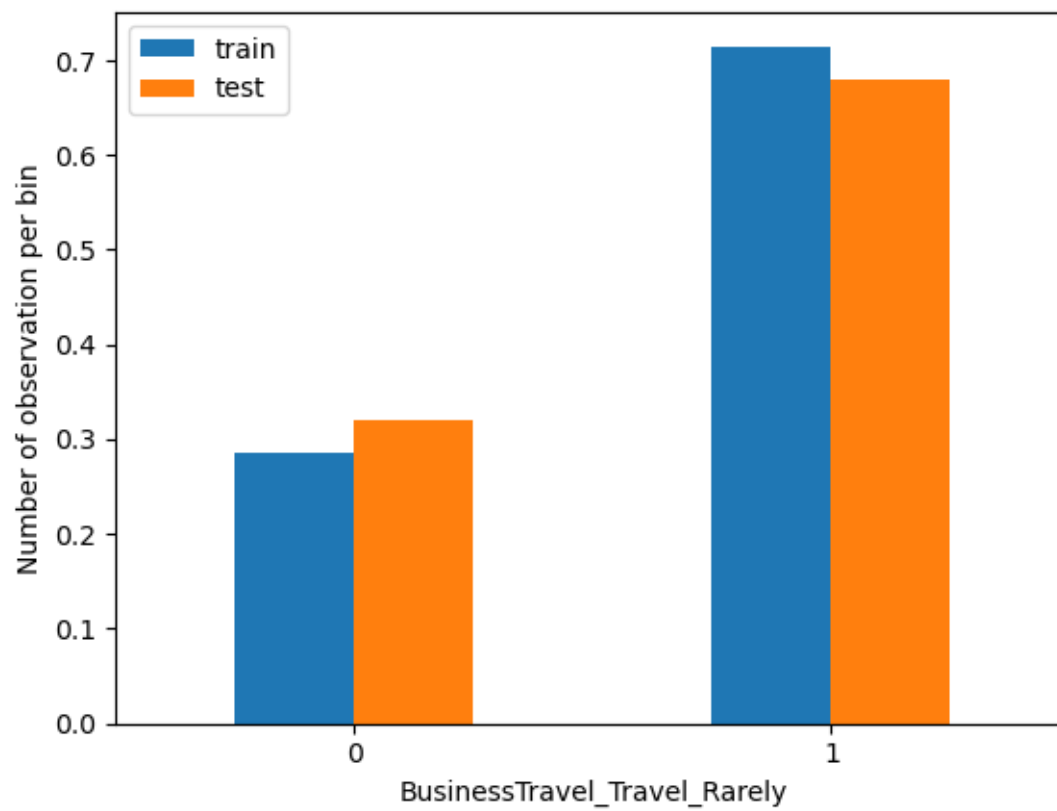


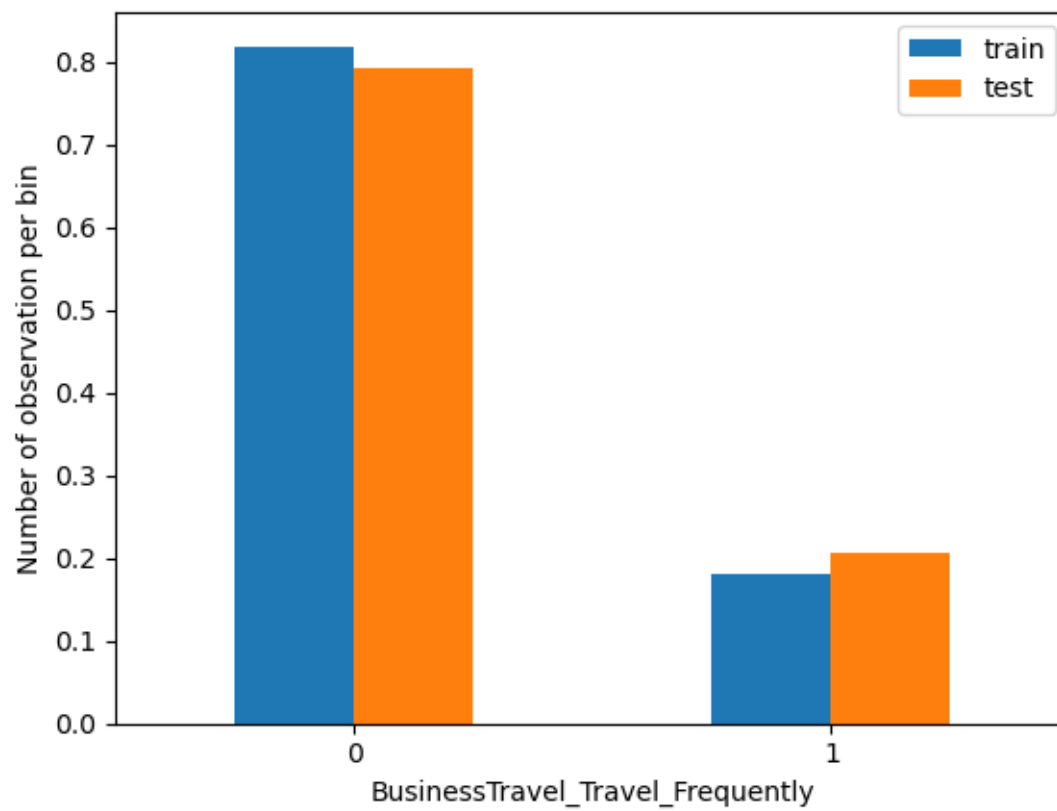


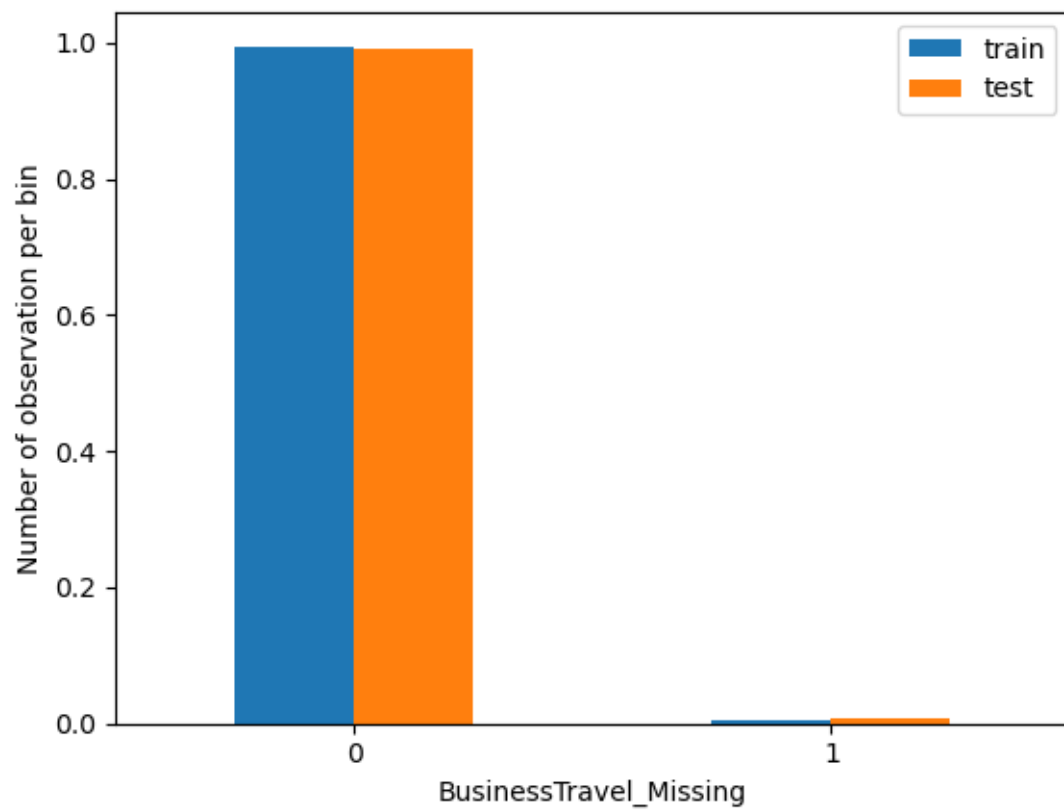


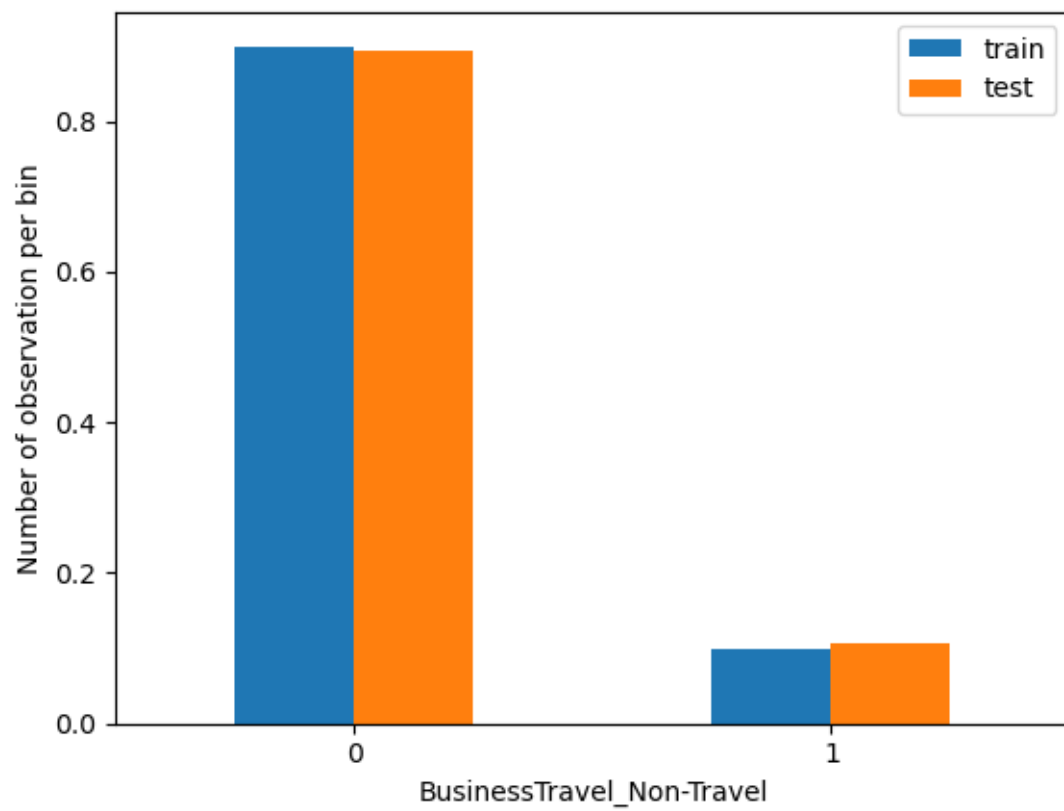


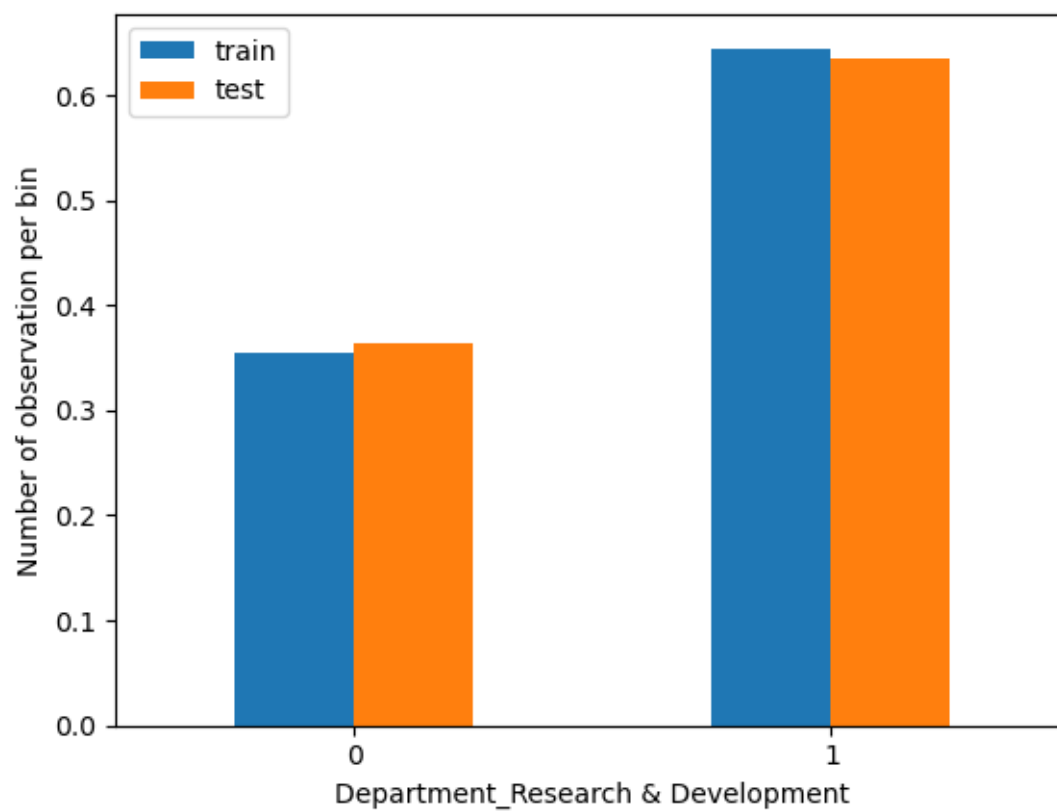


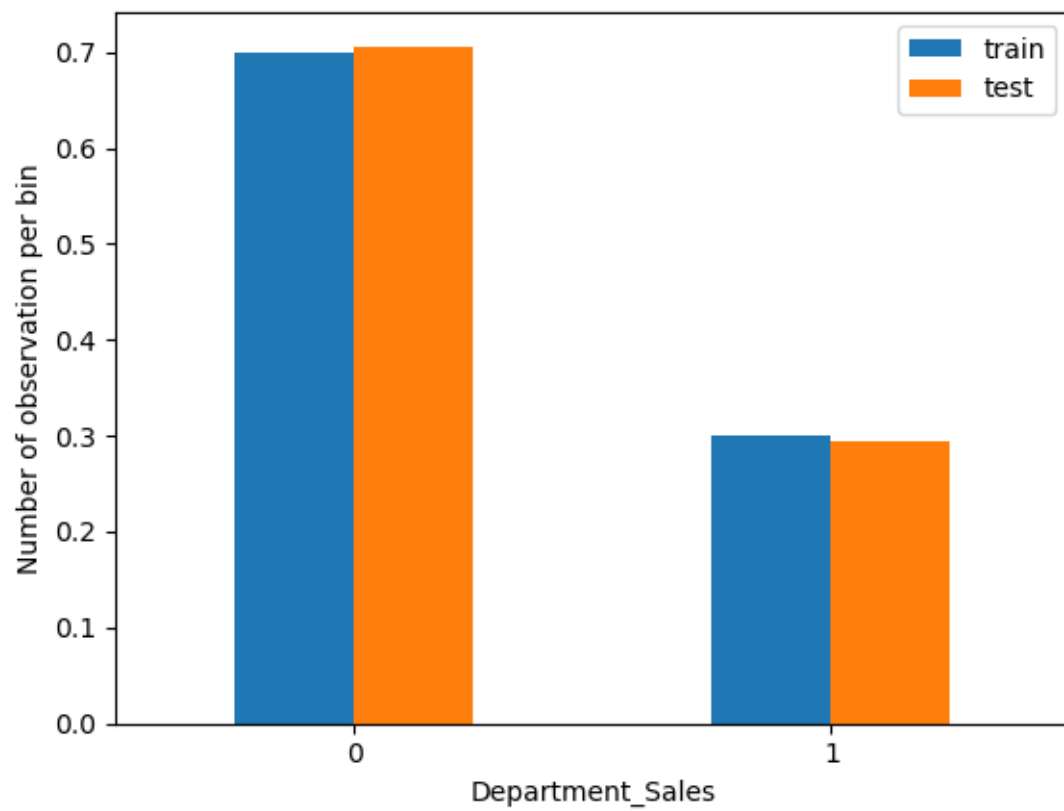


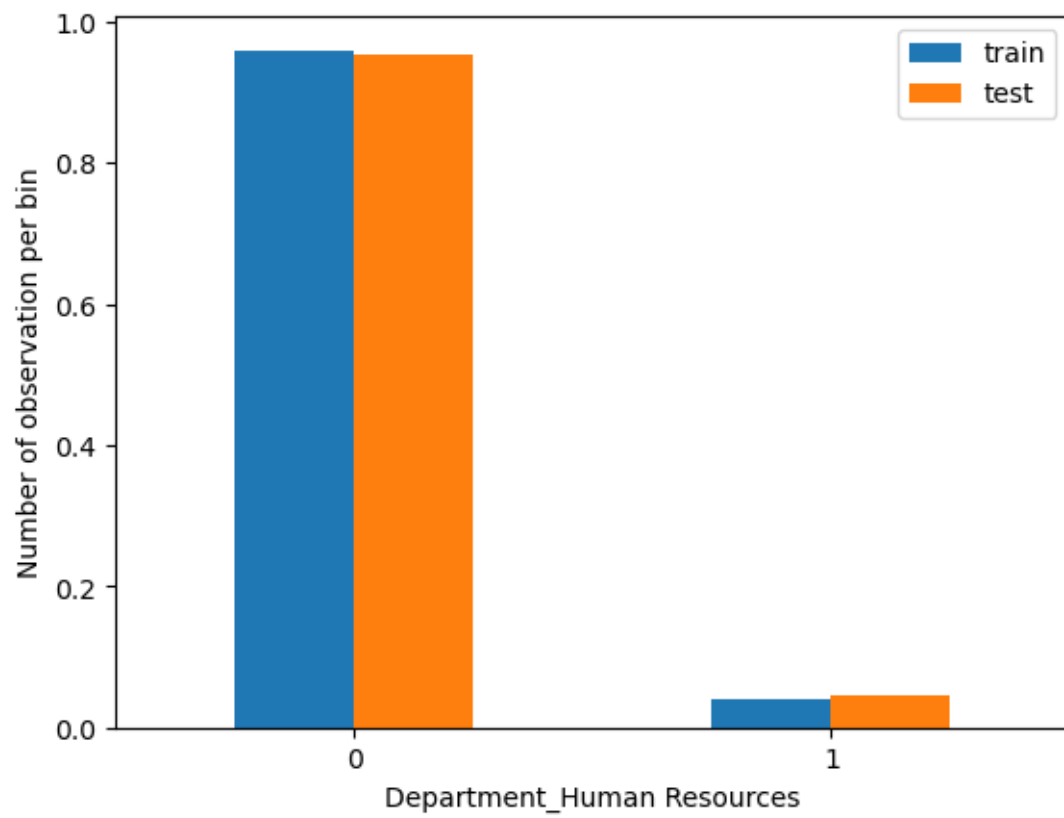


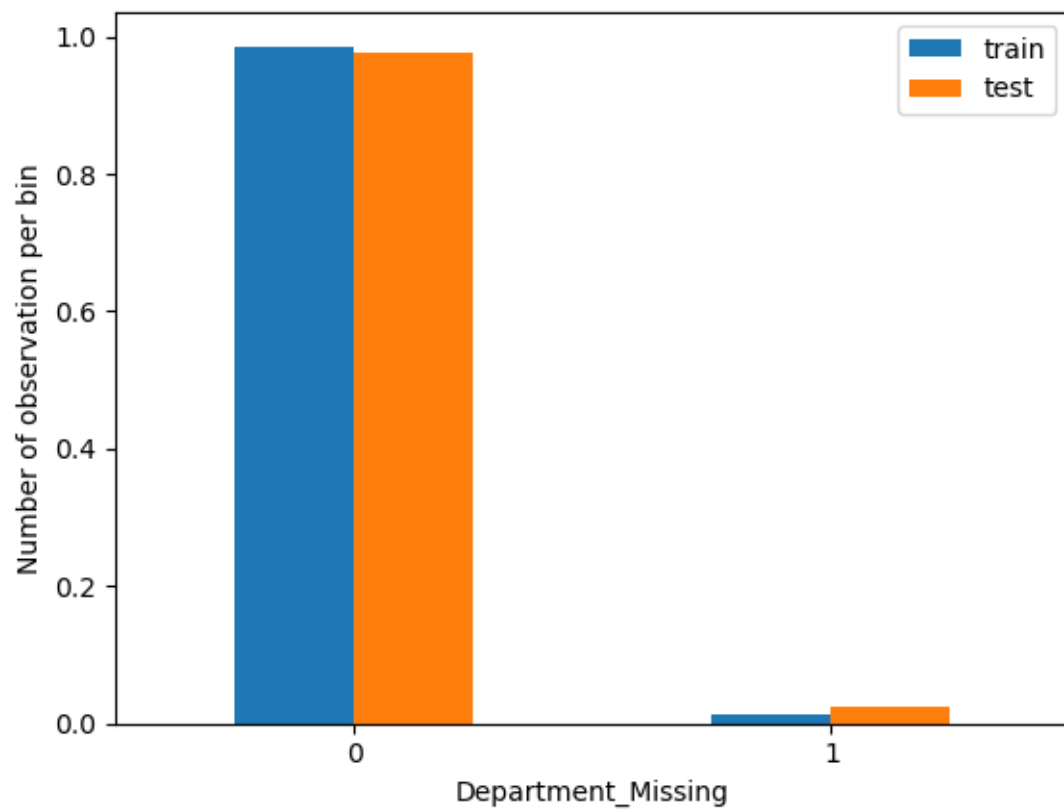


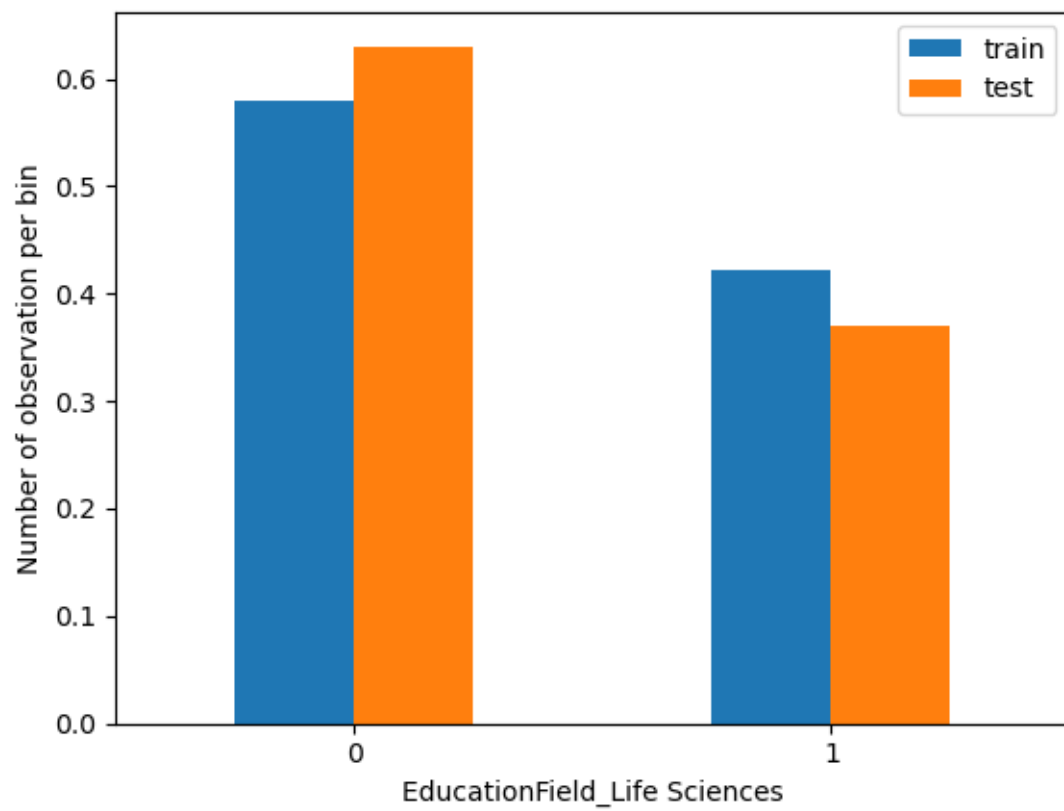


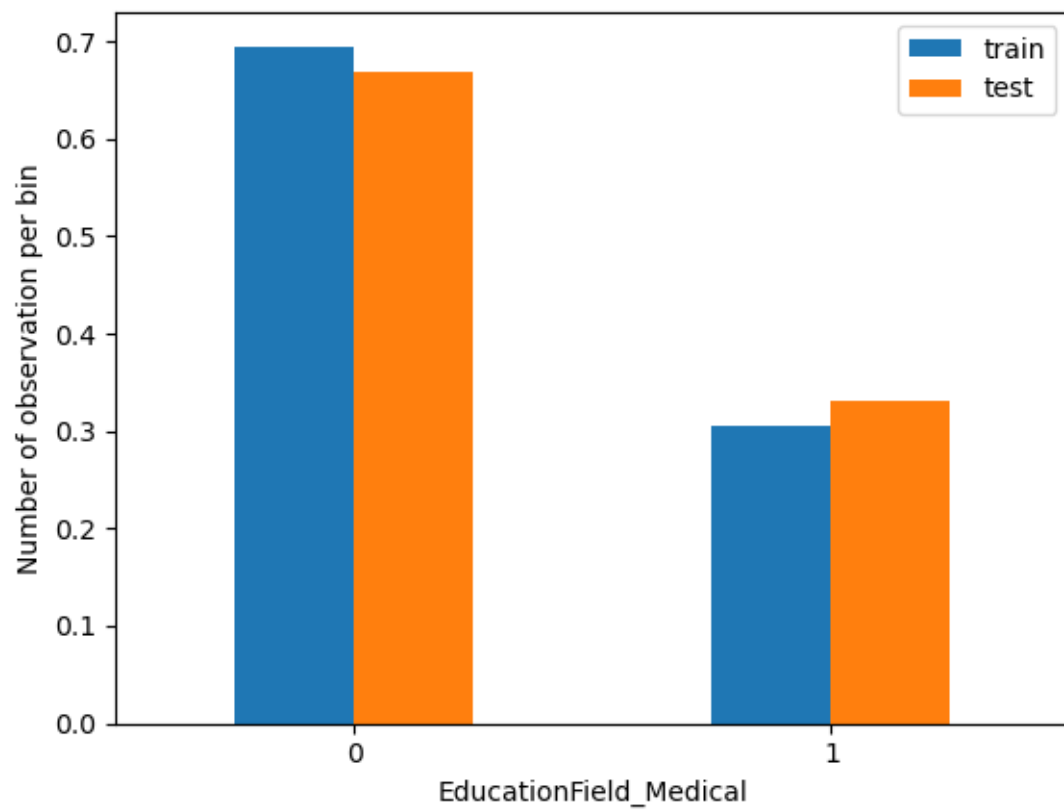


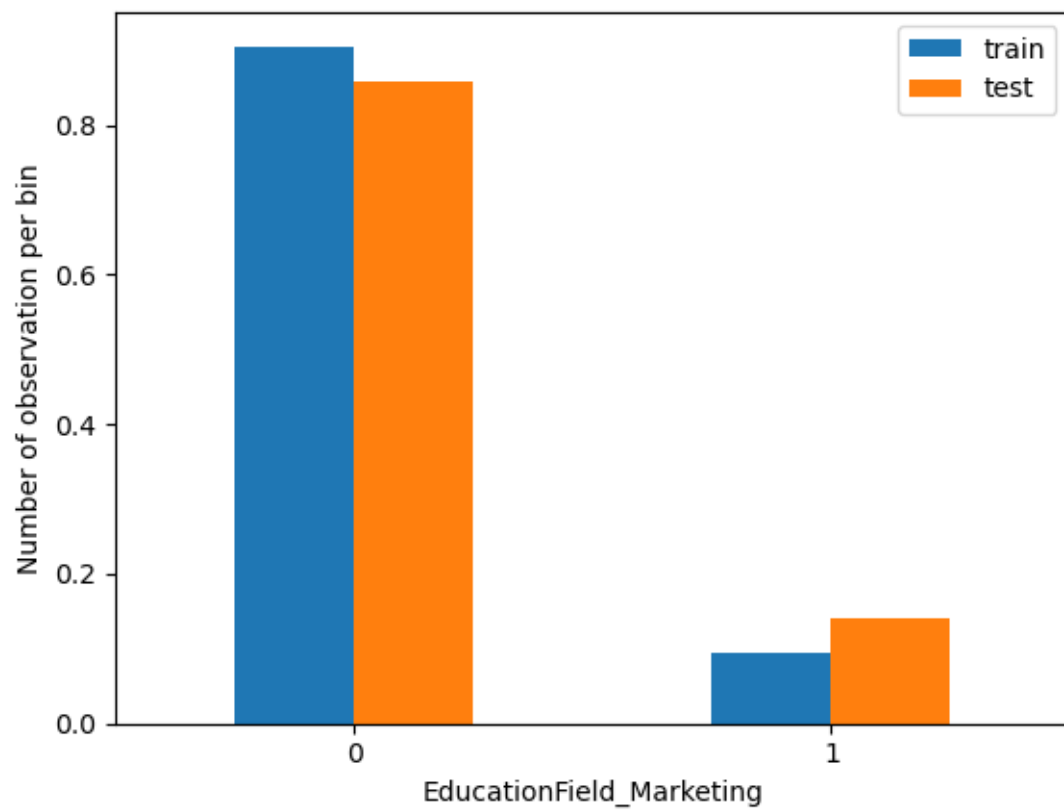


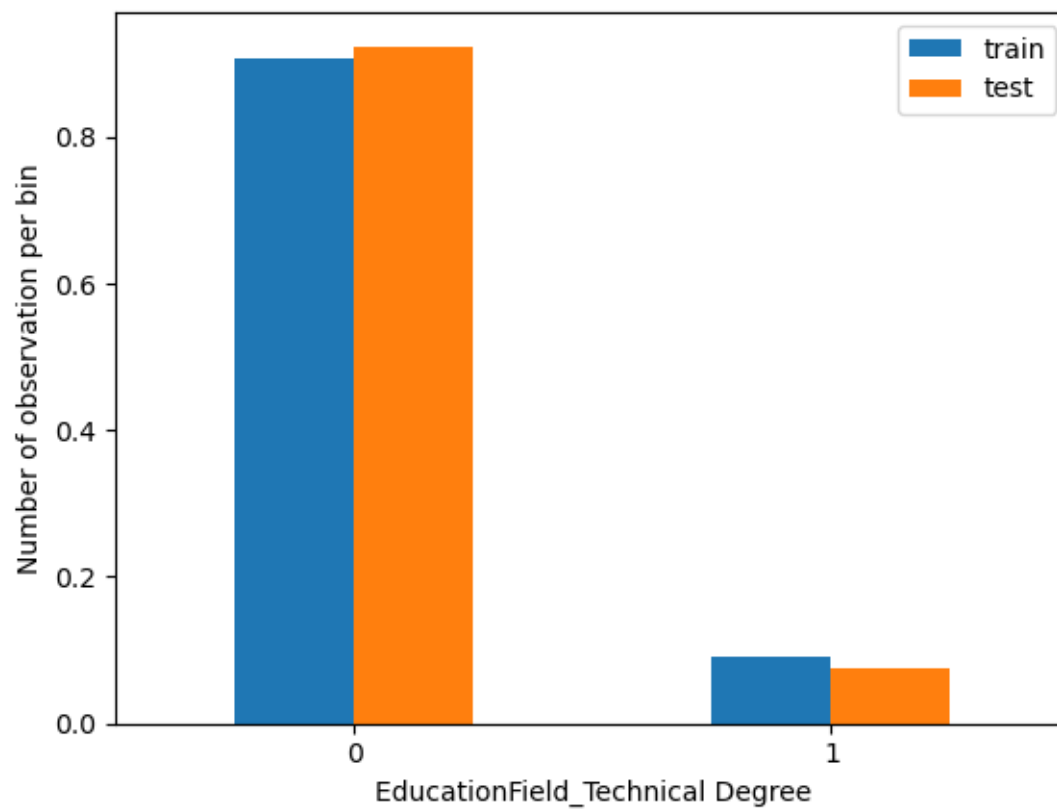


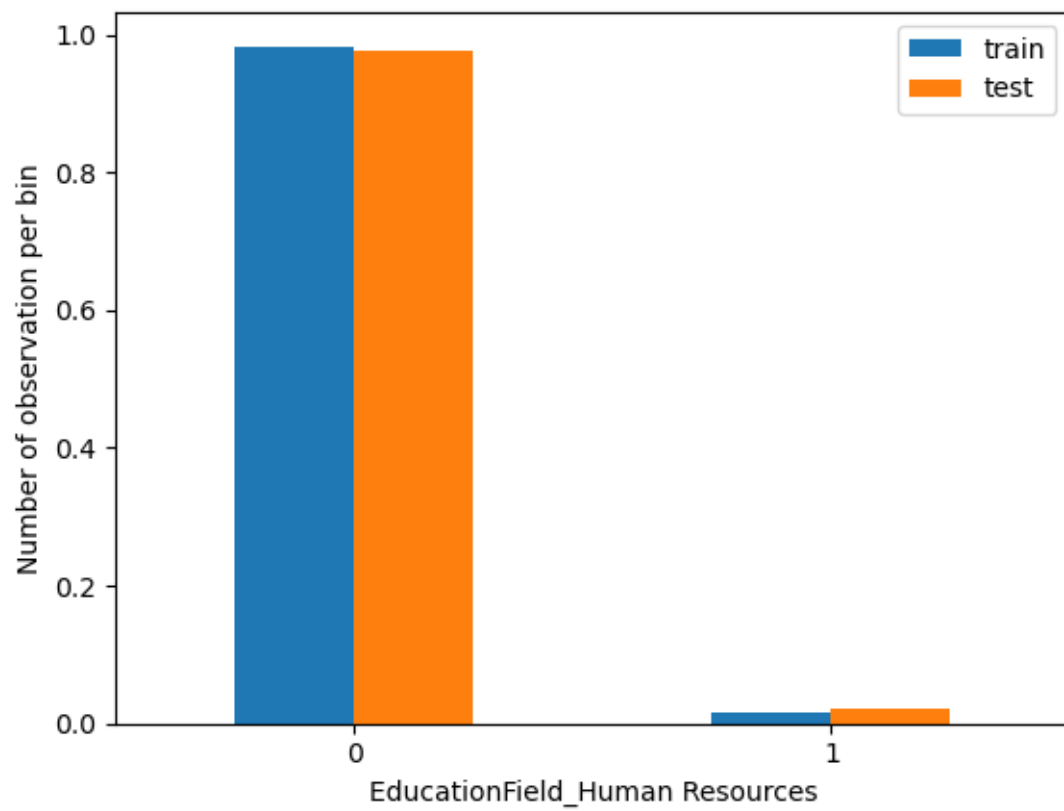


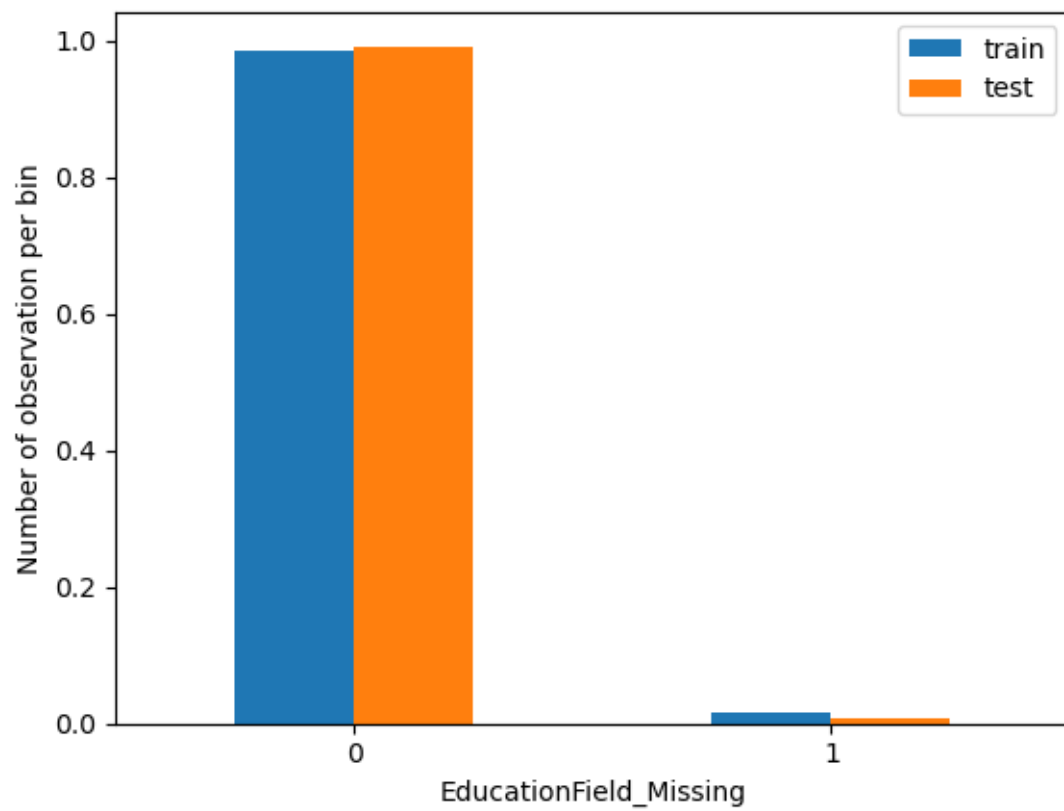


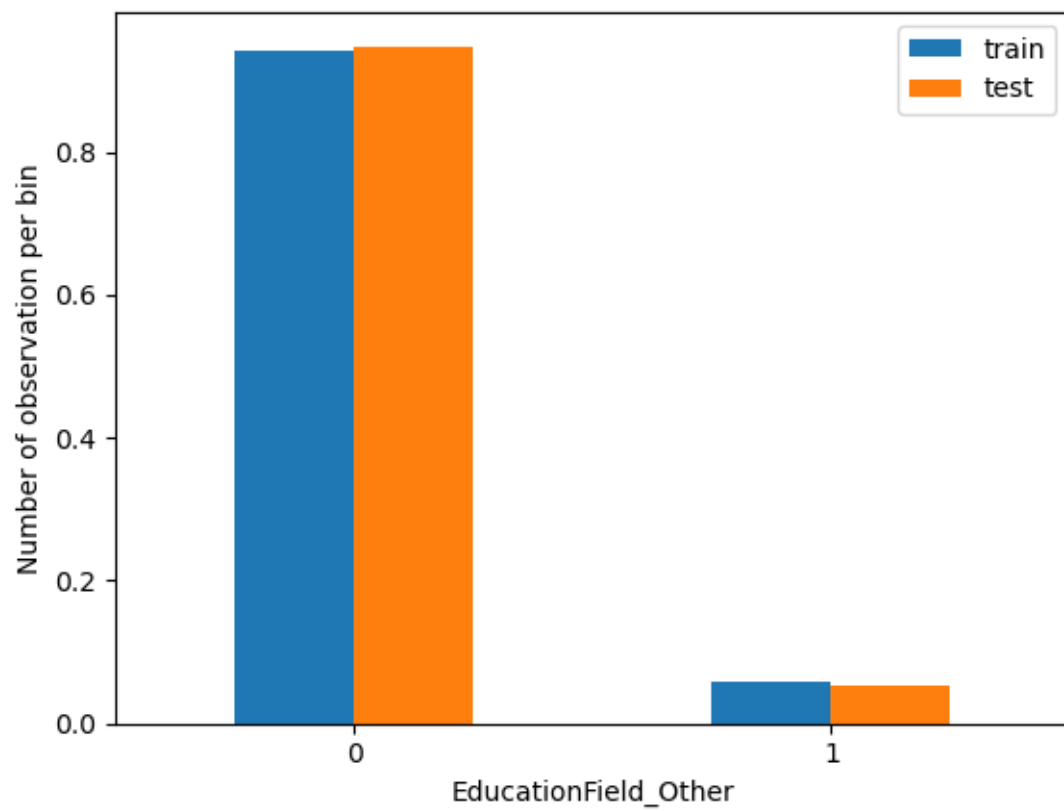


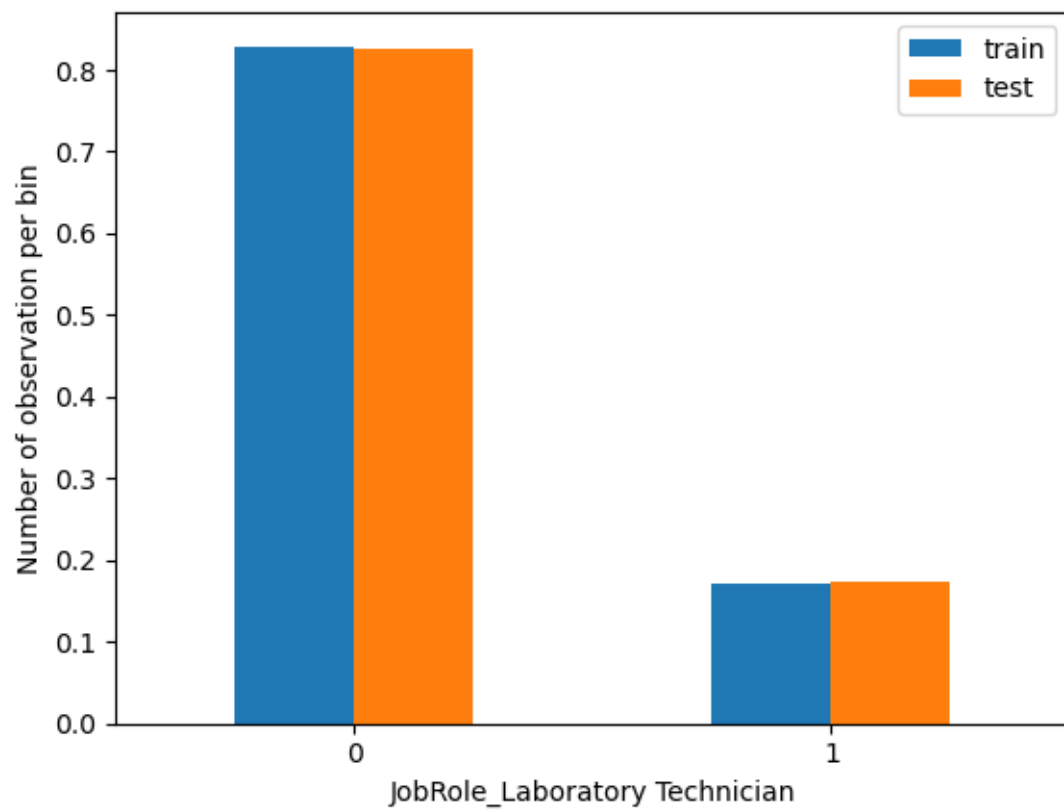


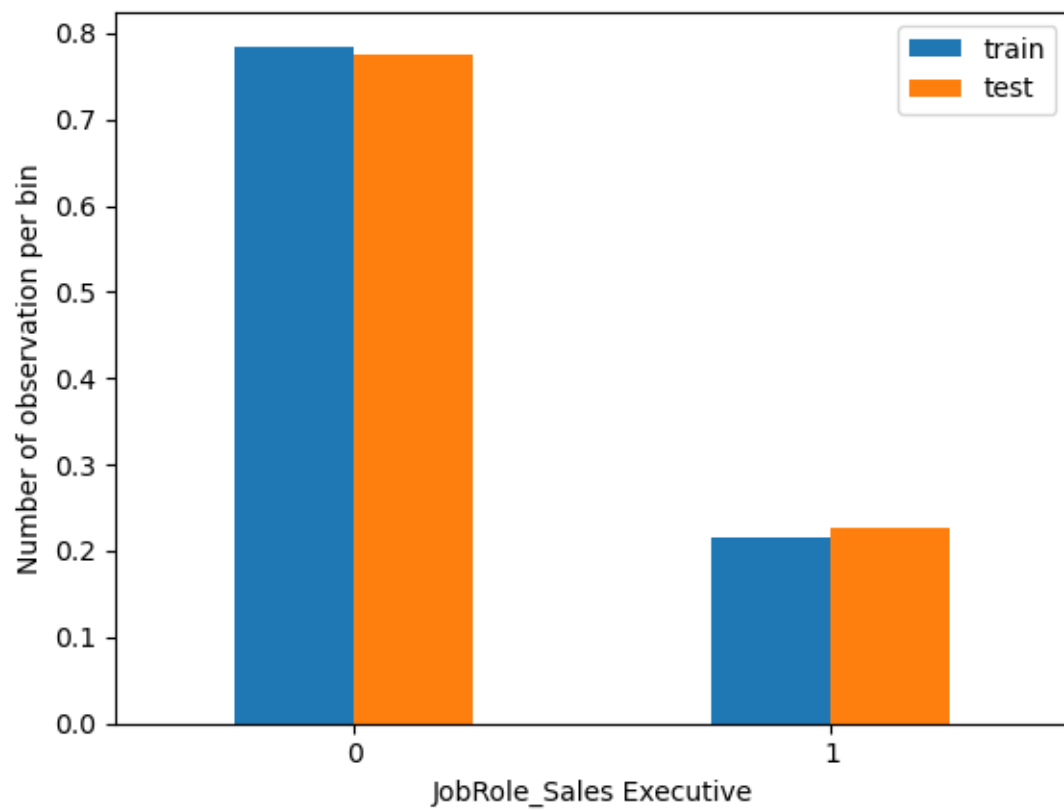


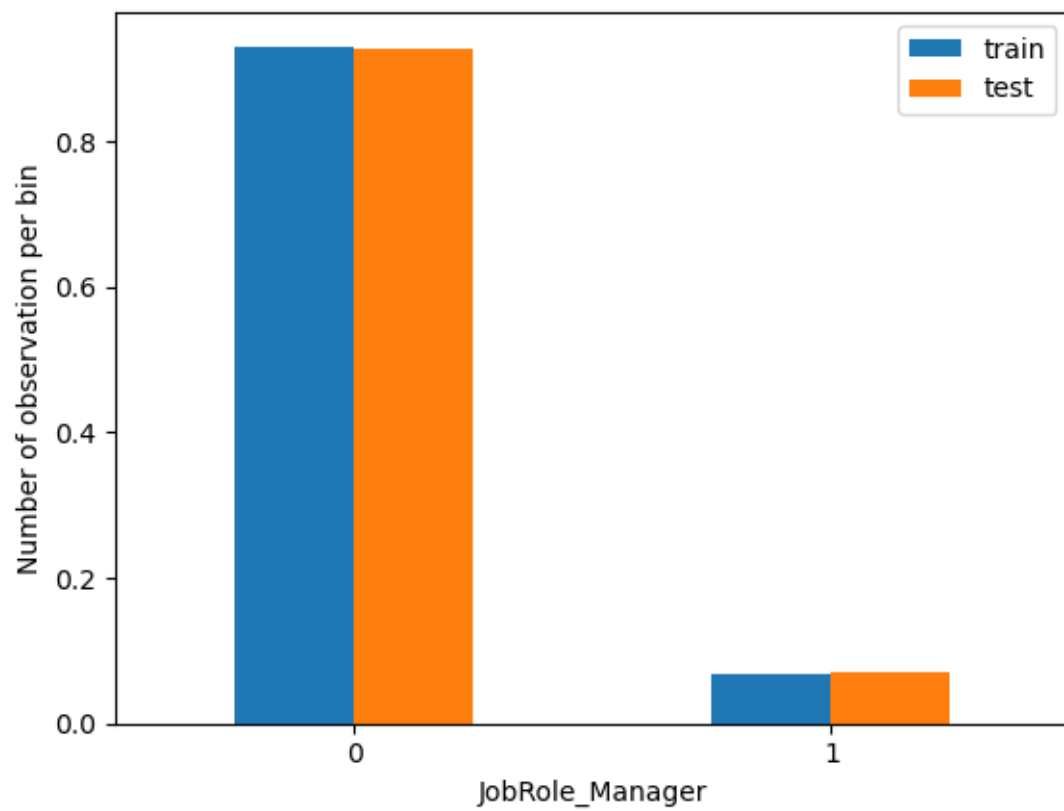


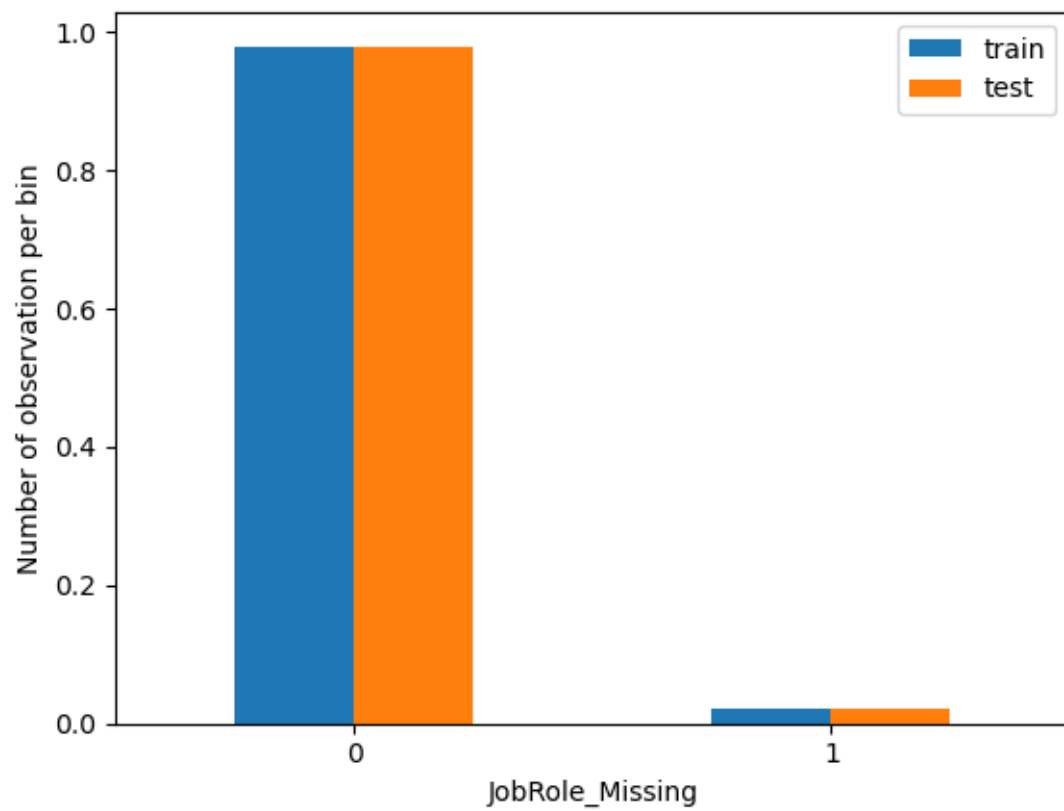


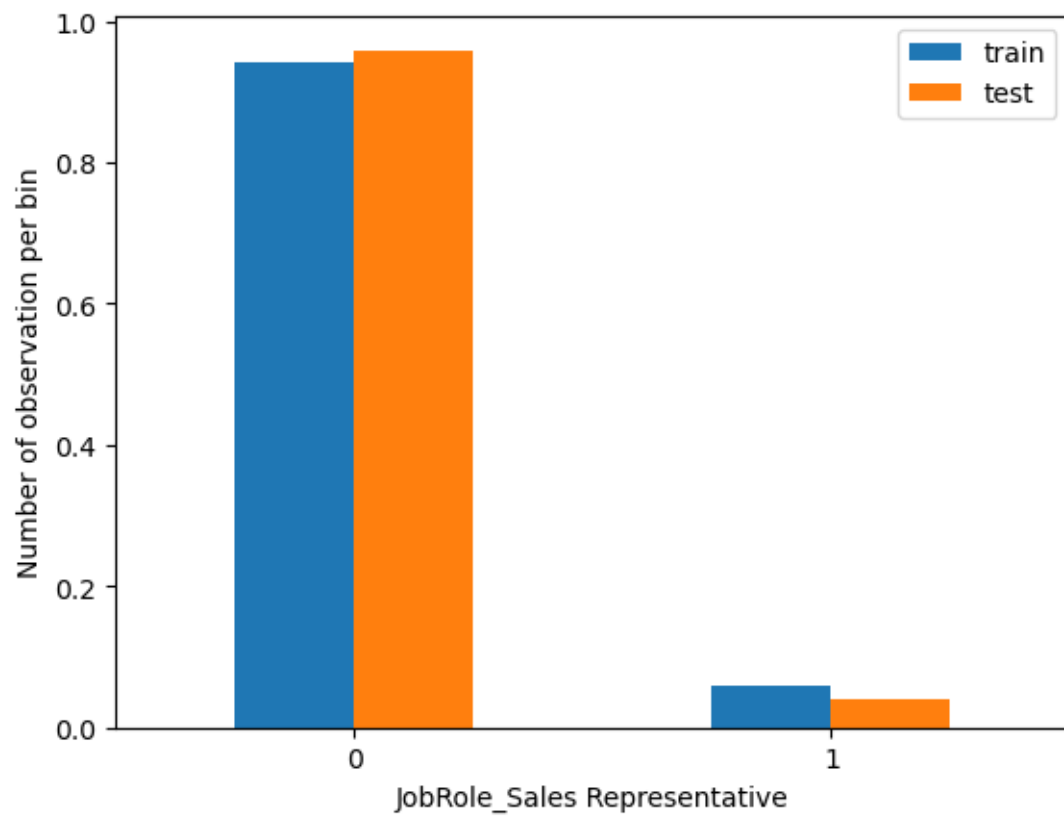


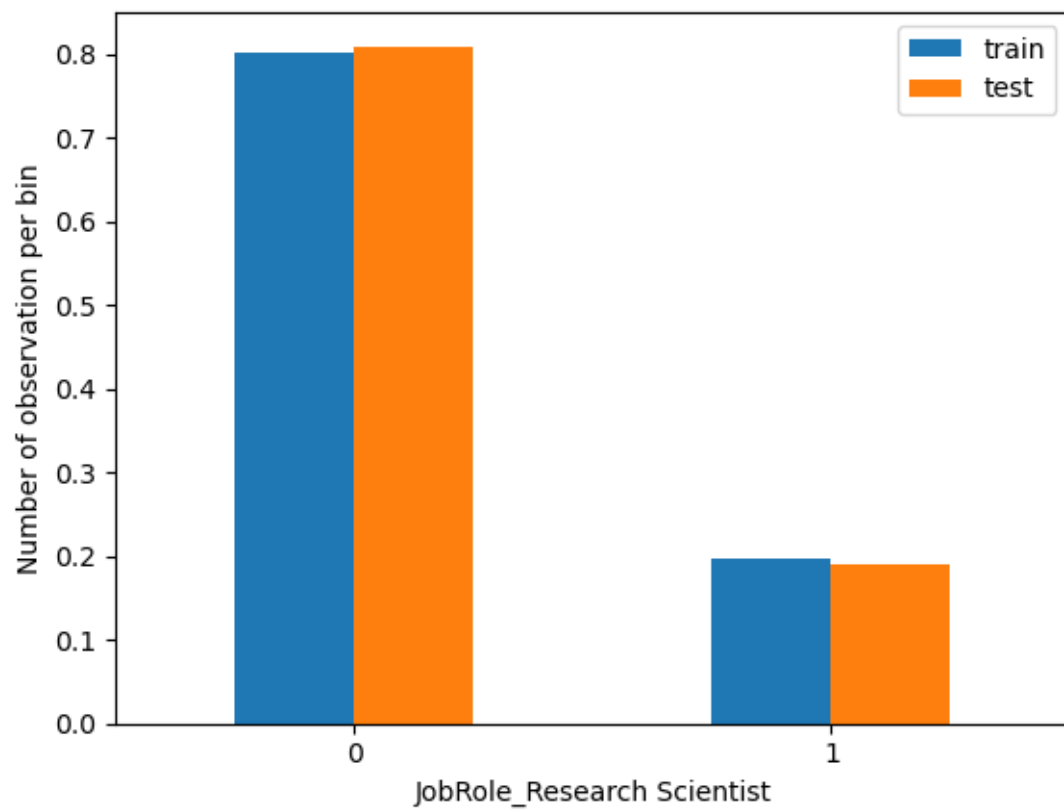


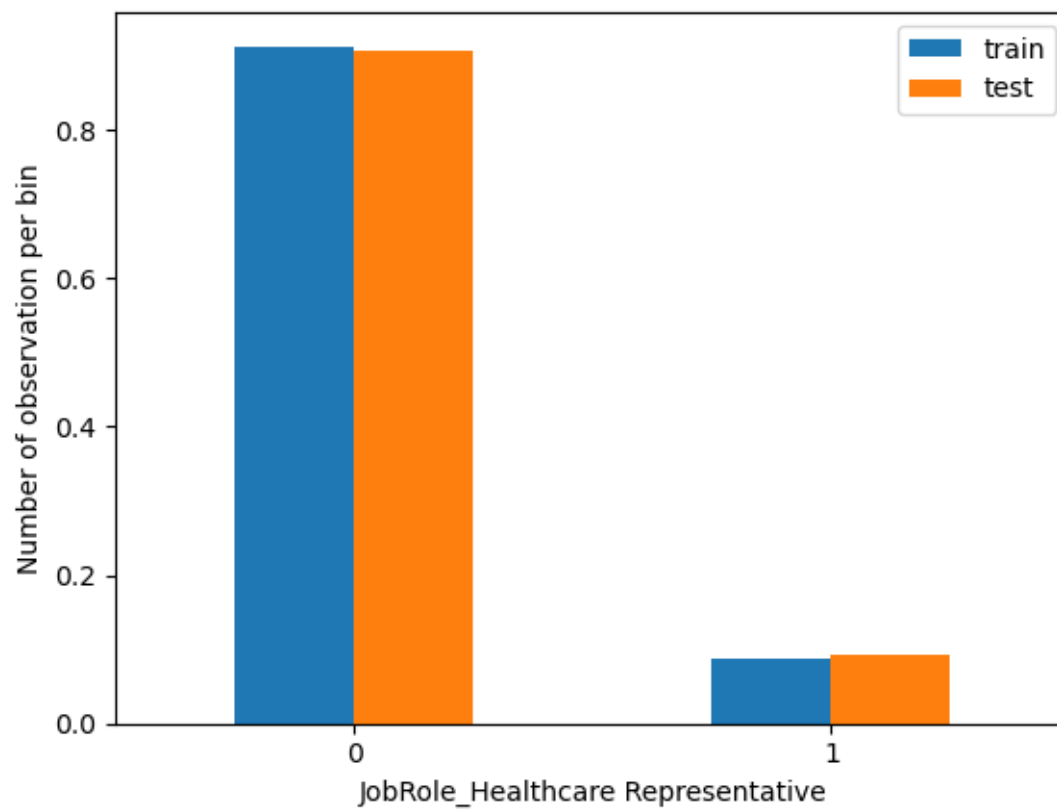


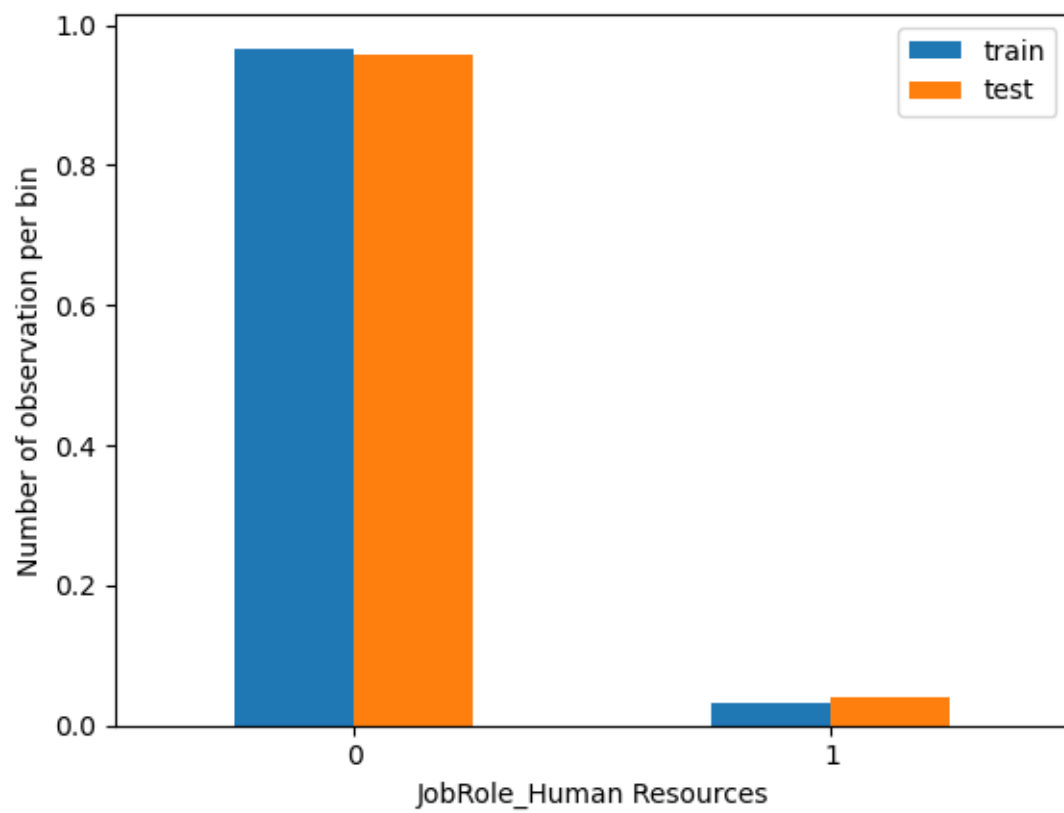


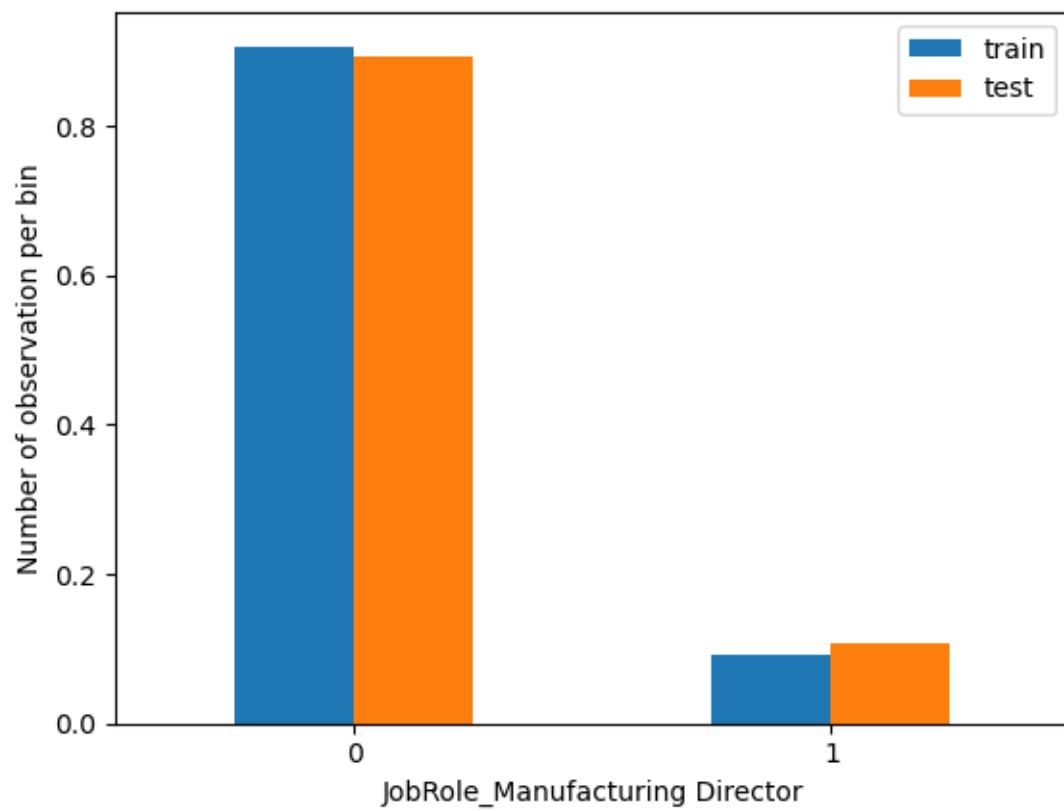


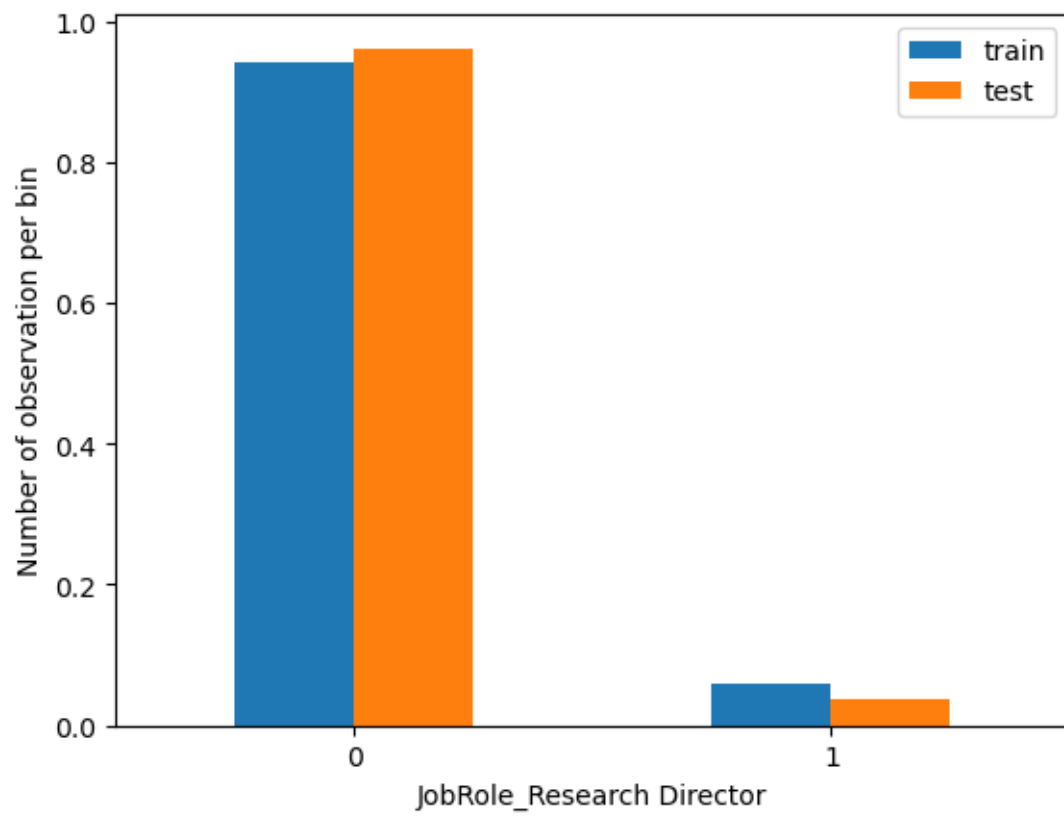


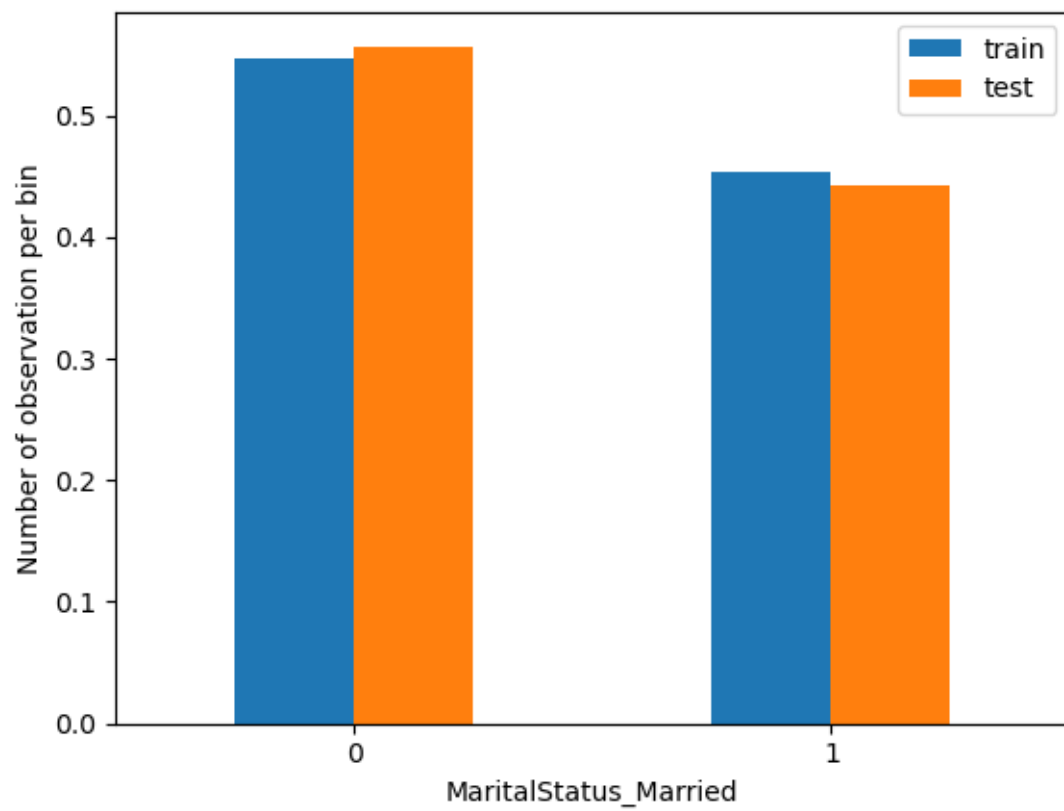


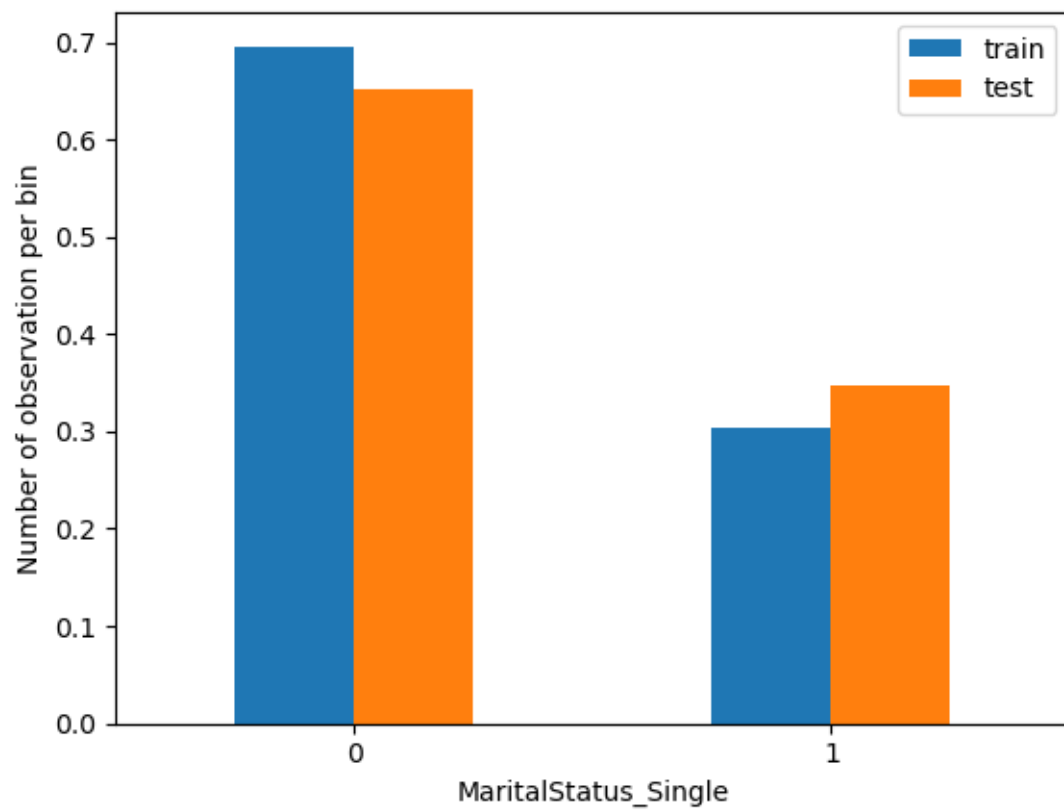


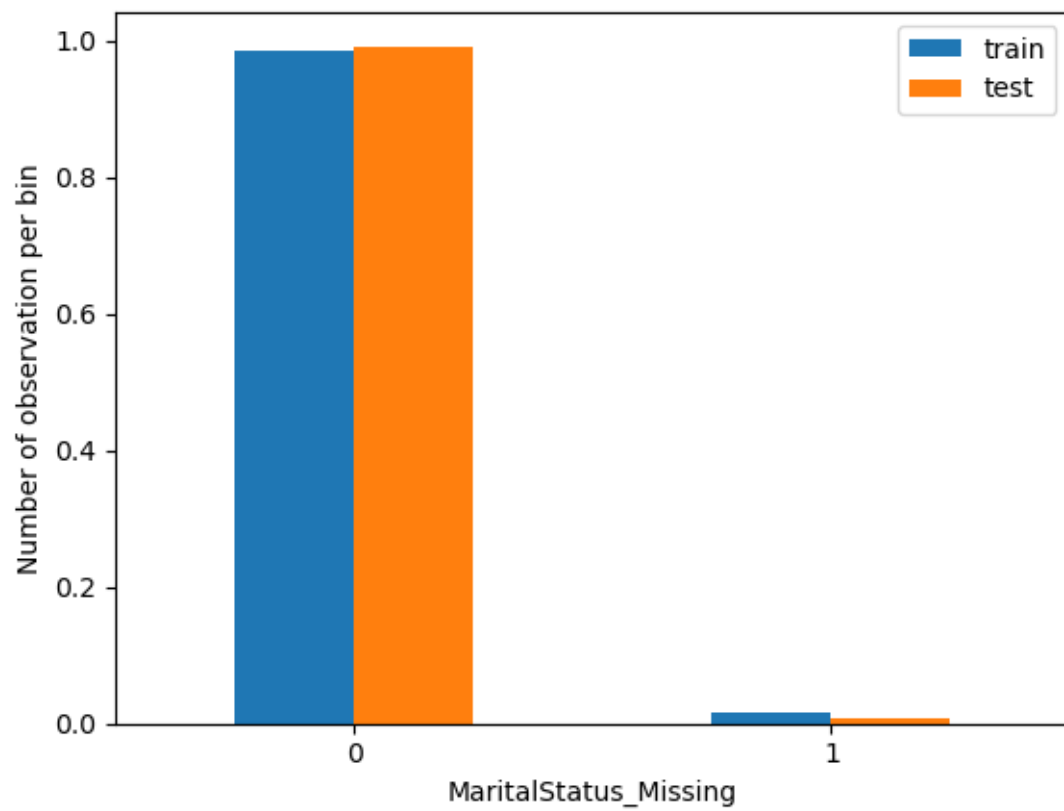


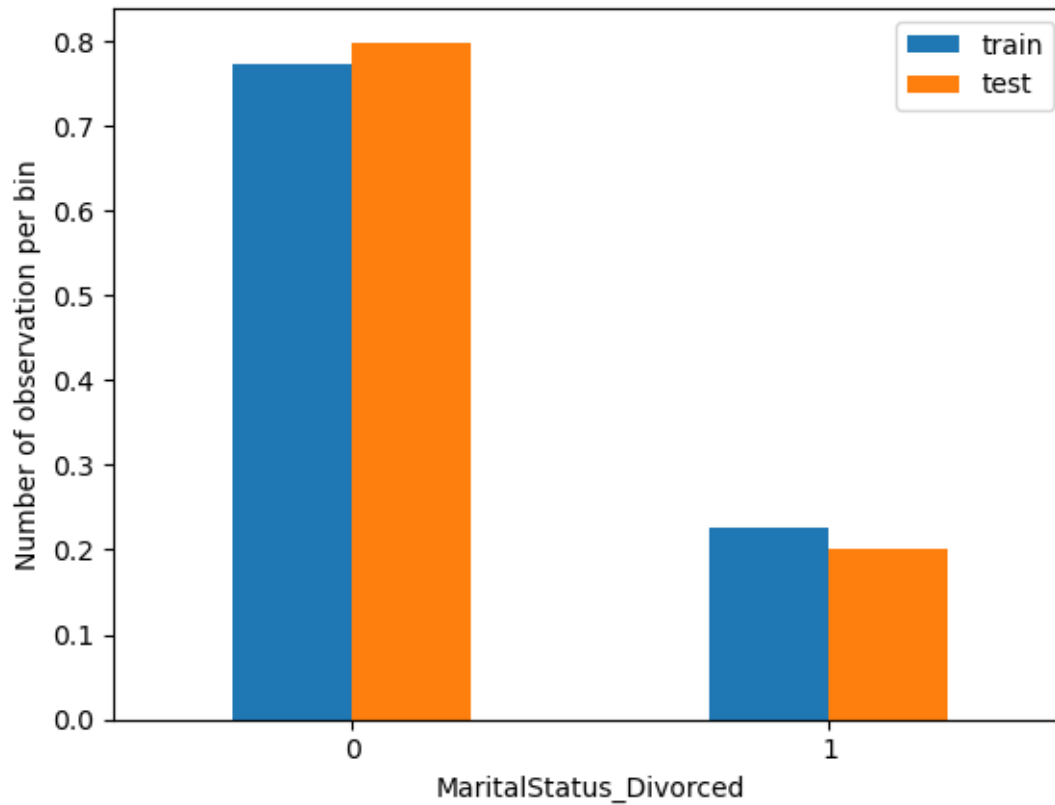












2.9 DecisionTreeDiscretiser

```
[36]: tree_disc = DecisionTreeDiscretiser(
    cv = 10,
    scoring = 'accuracy',
    variables = continuous,
    regression = False,
    param_grid = {
        'max_depth': [2, 3],
        'min_samples_leaf': [4, 6, 8]
    }
)

tree_disc.fit(tmp_mean_train, y_train)
```

```
[37]: train_decision = tree_disc.transform(tmp_mean_train)
test_decision = tree_disc.transform(tmp_mean_test)
```

```
[38]: for col in train_decision.columns:
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 6))
```

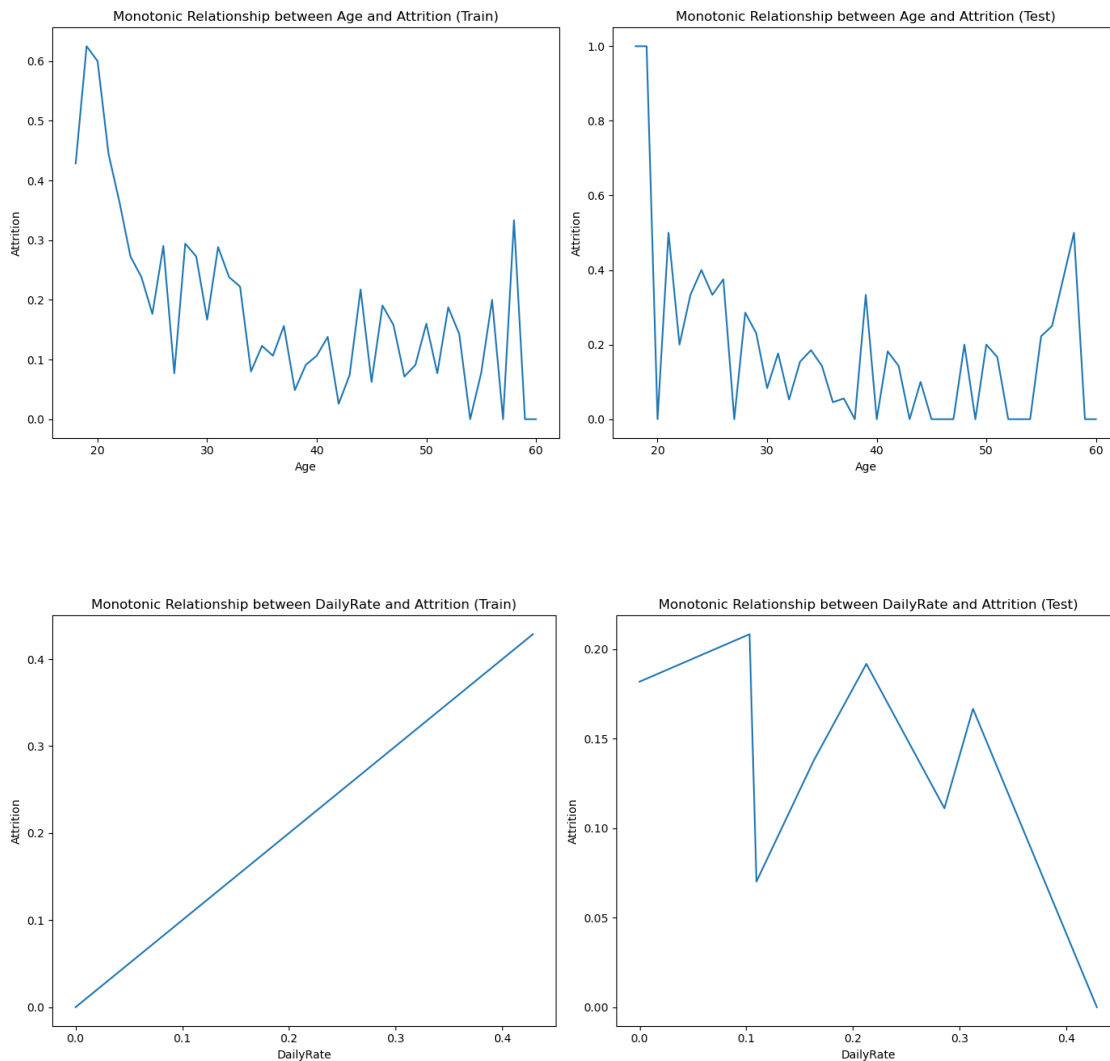
```

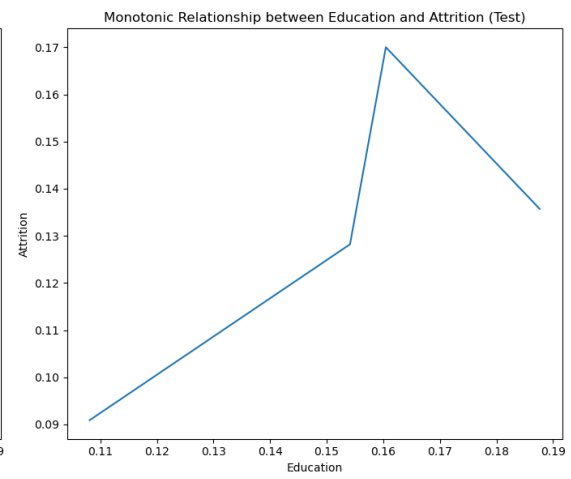
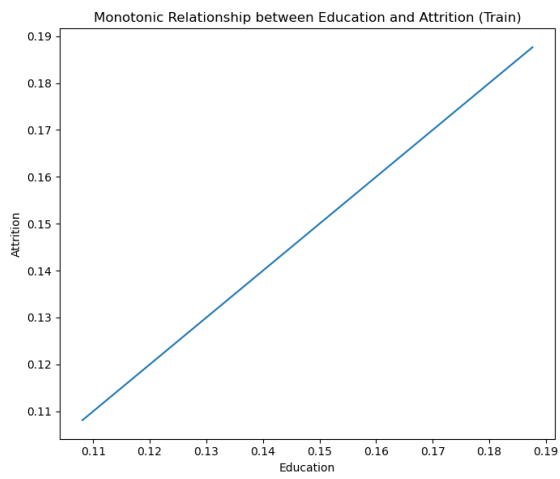
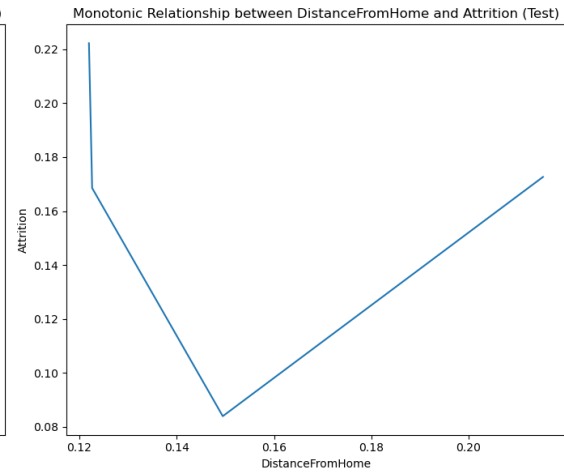
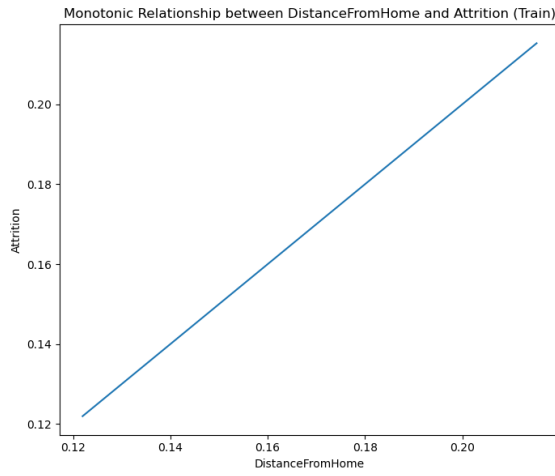
pd.concat([train_decision, y_train], axis=1).groupby([col])['Attrition'].
↪mean().plot(ax=axes[0])
axes[0].set_title(f'Monotonic Relationship between {col} and Attrition_
↪(Train)')
axes[0].set_ylabel('Attrition')
axes[0].set_xlabel(col)

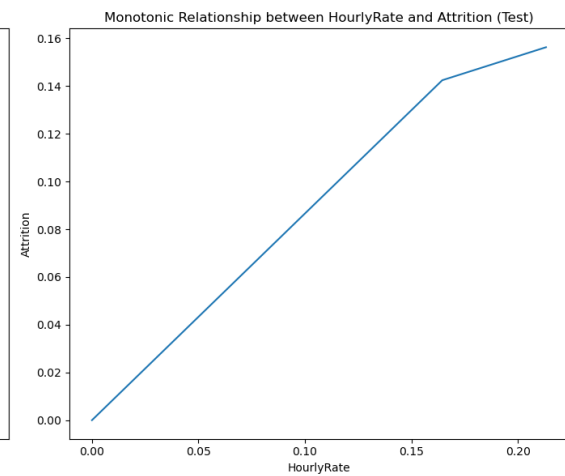
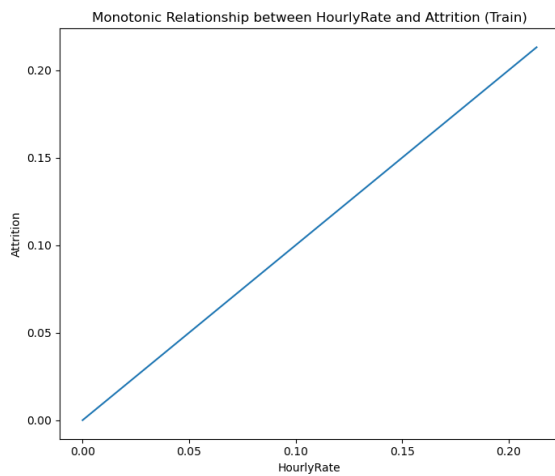
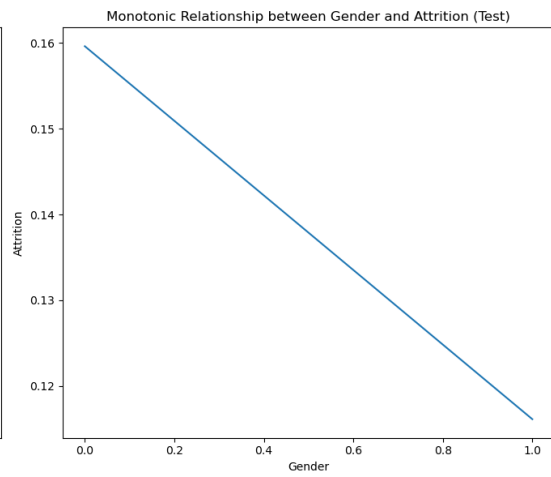
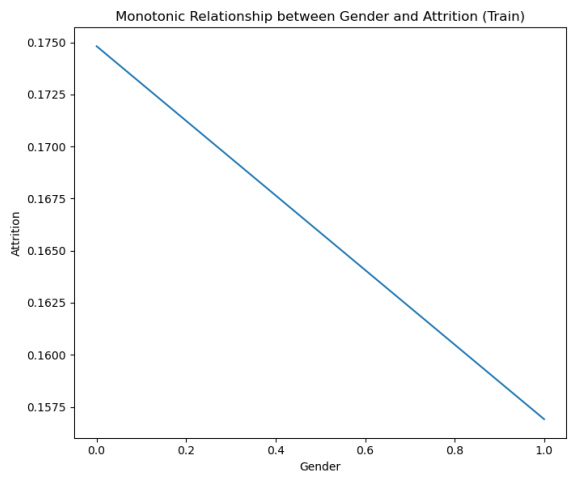
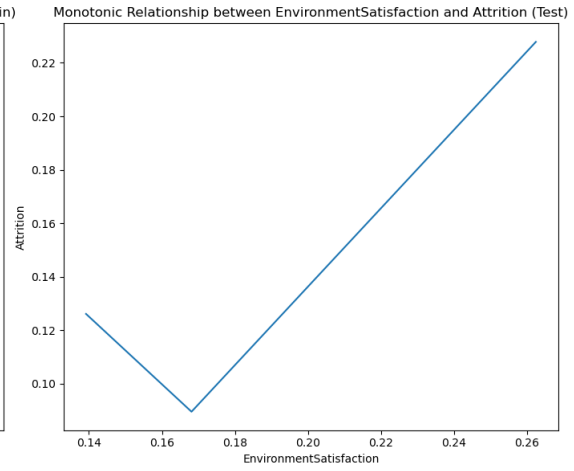
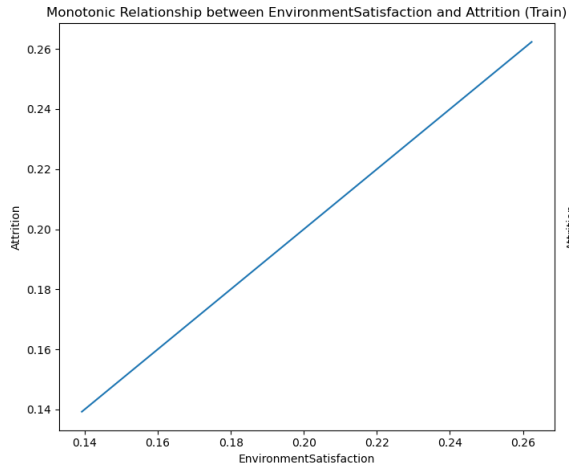
pd.concat([test_decision, y_test], axis=1).groupby([col])['Attrition'].
↪mean().plot(ax=axes[1])
axes[1].set_title(f'Monotonic Relationship between {col} and Attrition_
↪(Test)')
axes[1].set_ylabel('Attrition')
axes[1].set_xlabel(col)

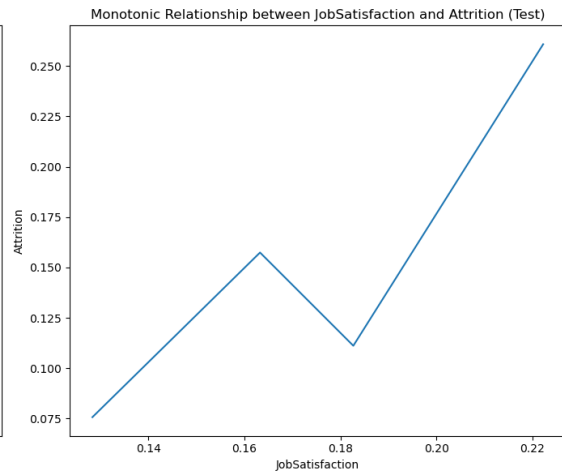
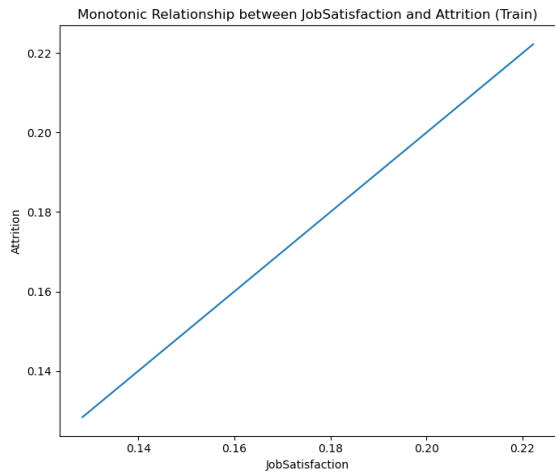
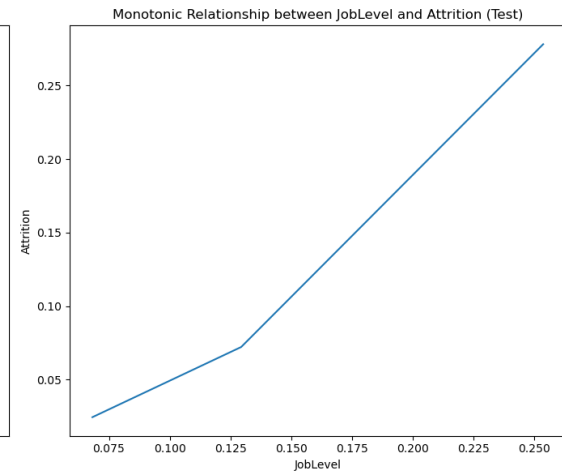
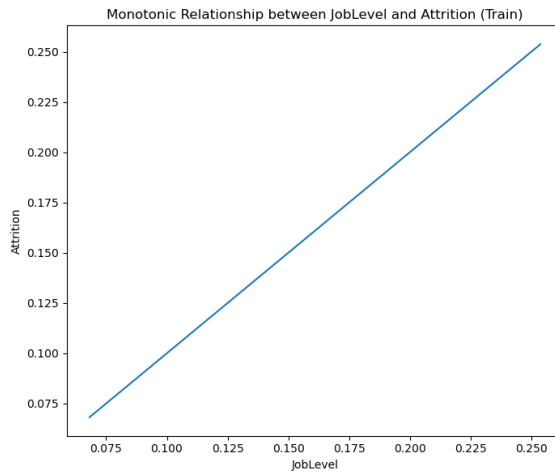
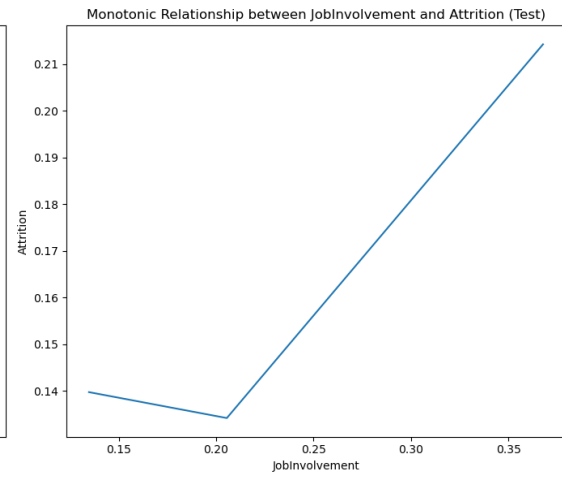
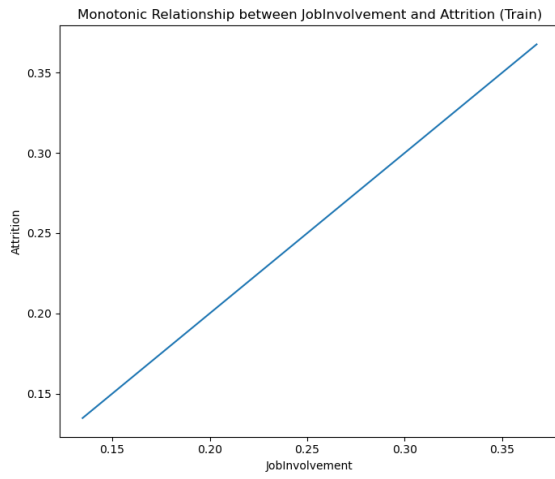
plt.tight_layout()
plt.show()

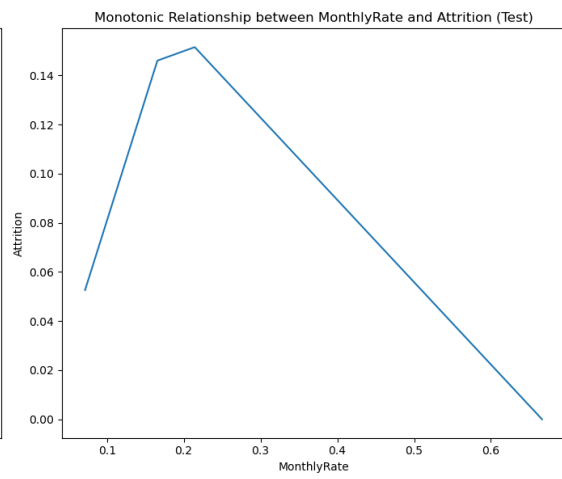
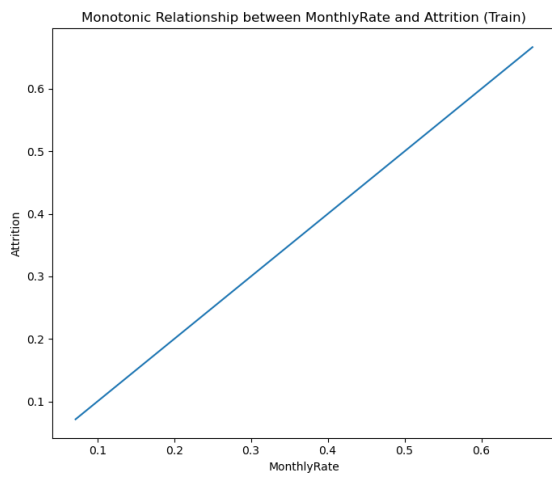
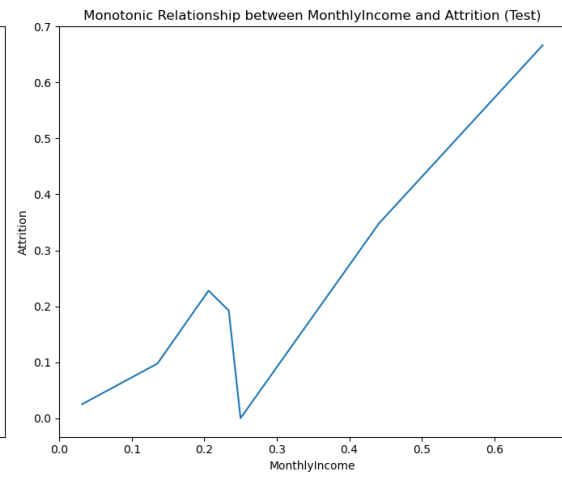
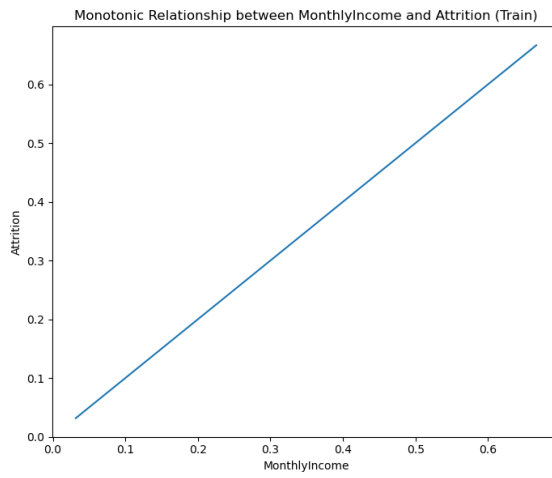
```

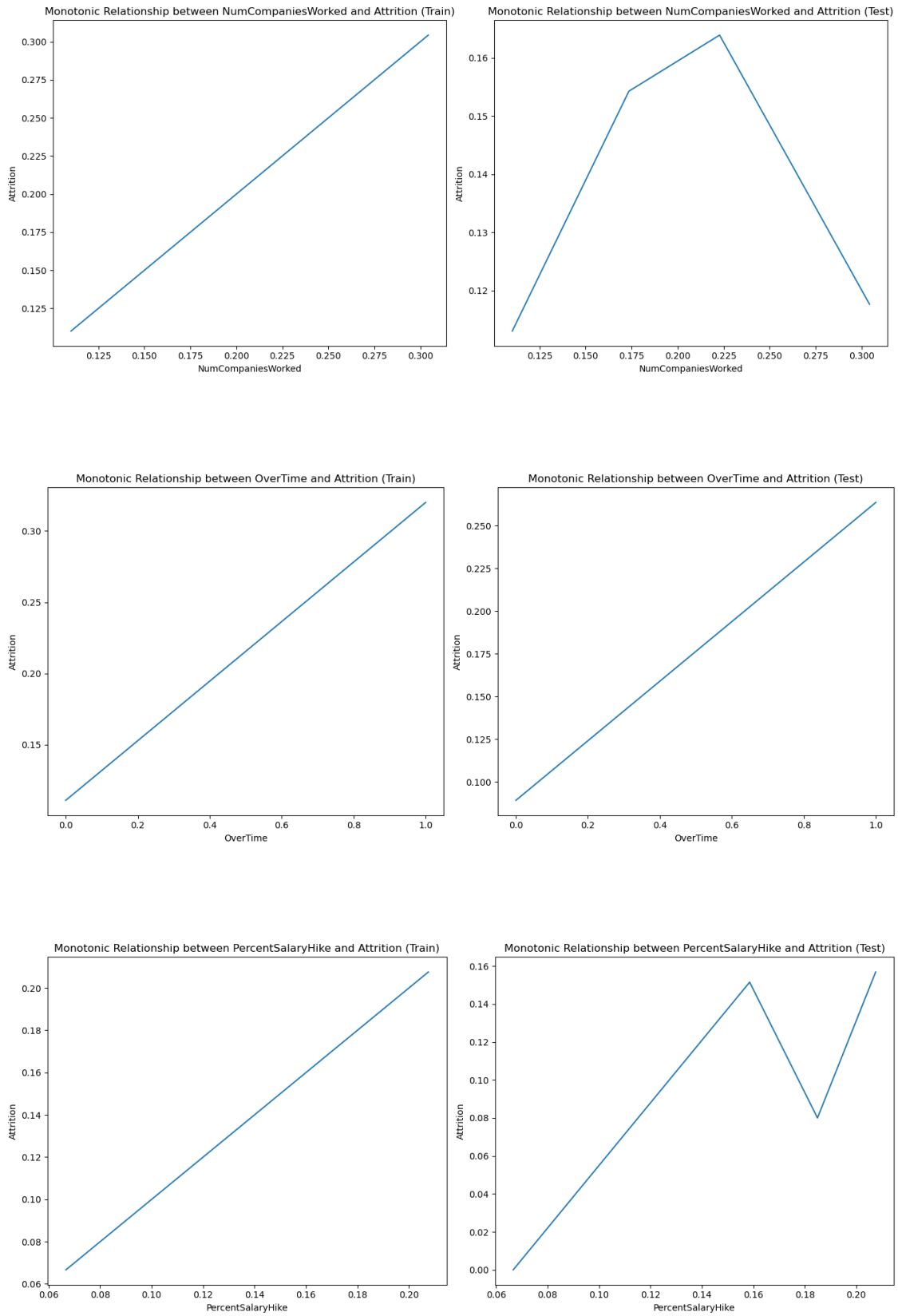


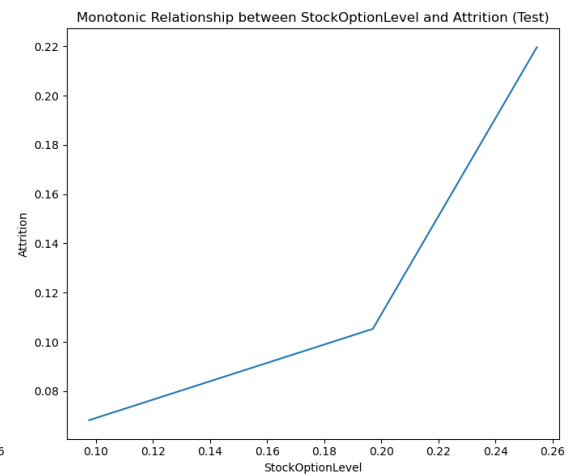
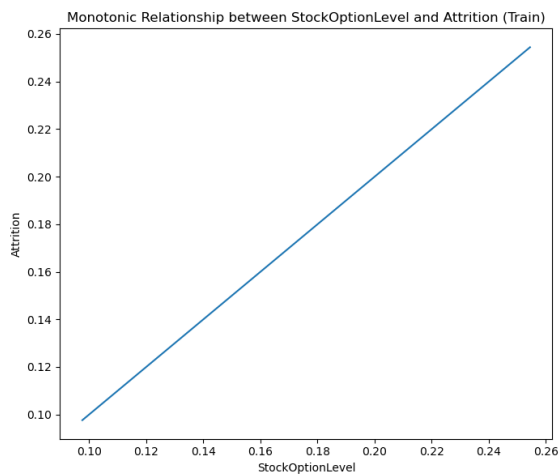
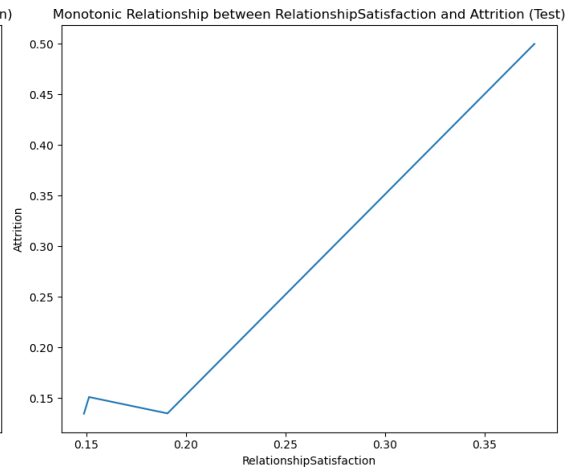
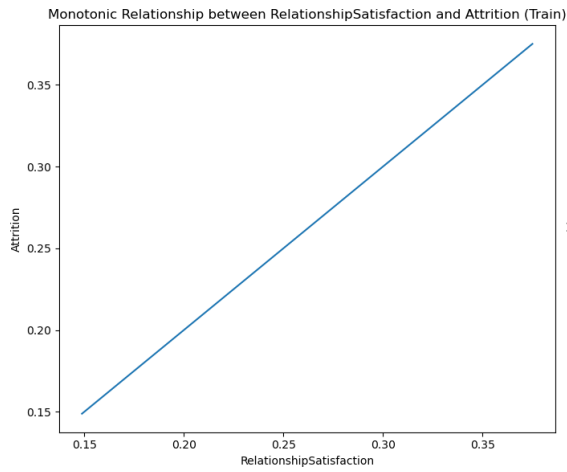
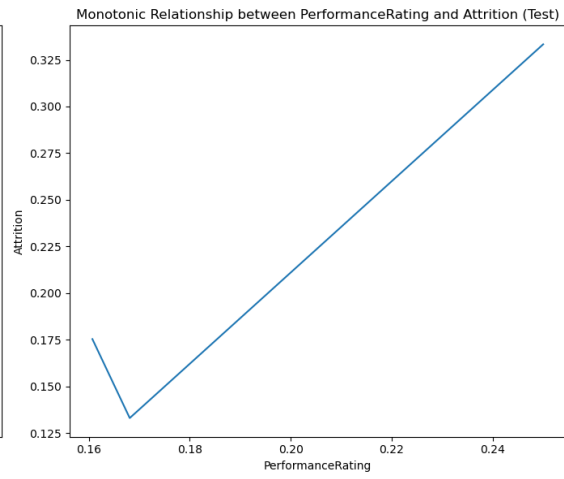
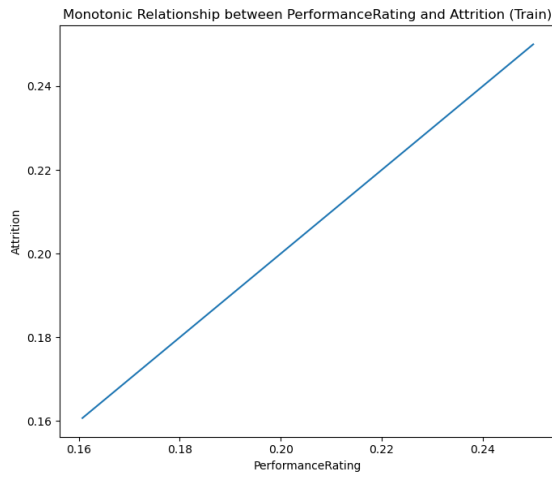


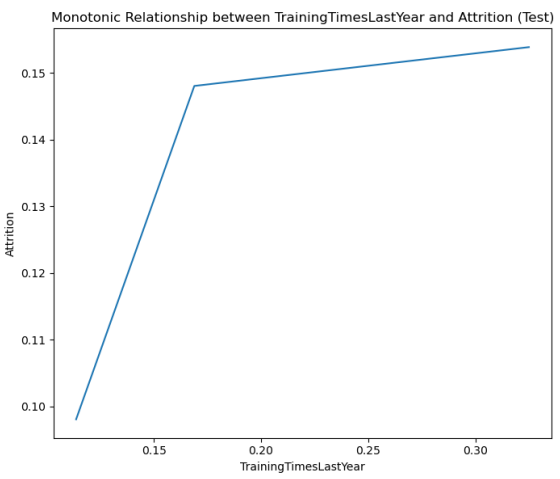
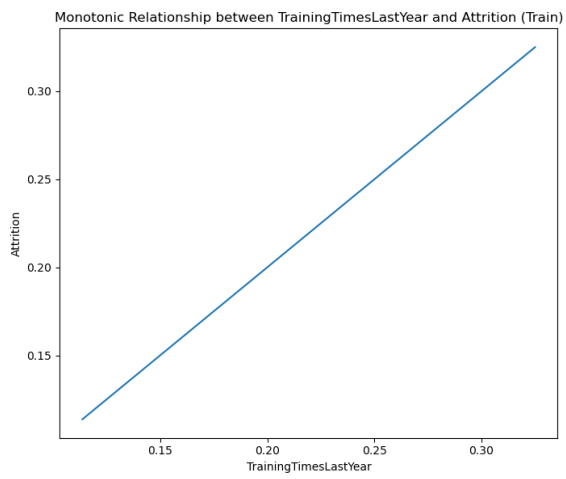
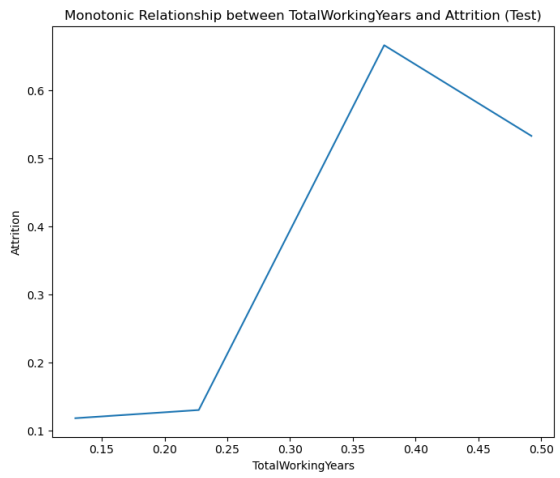
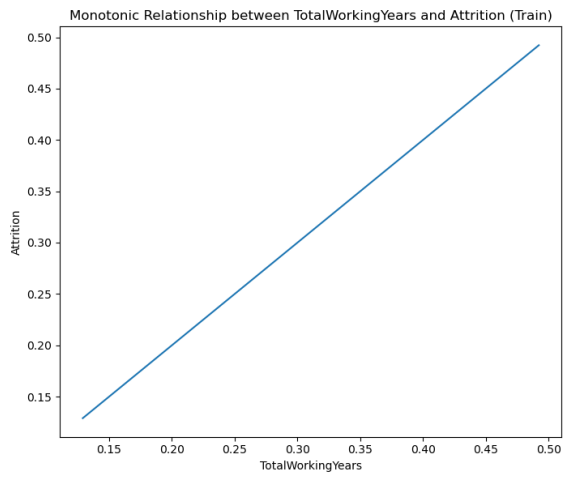


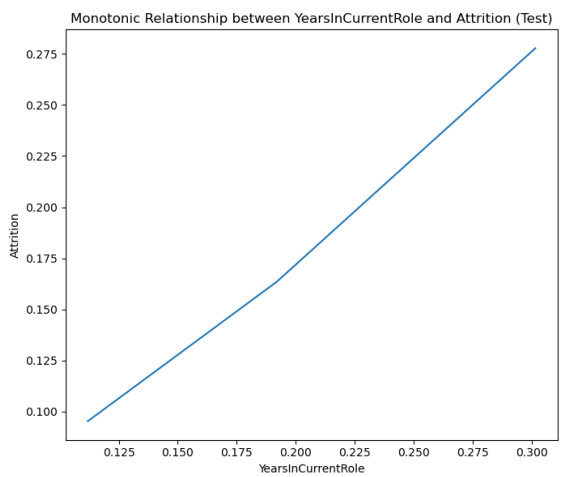
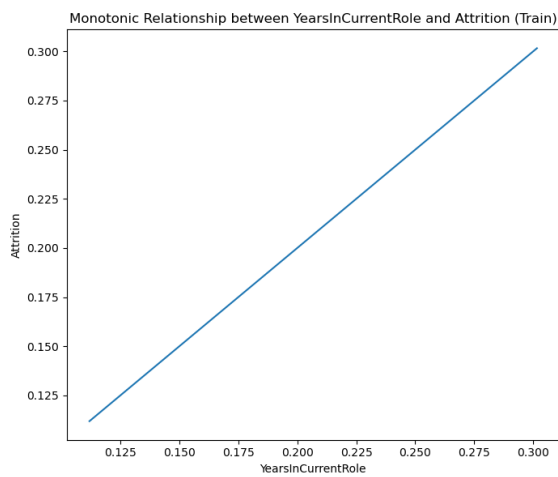
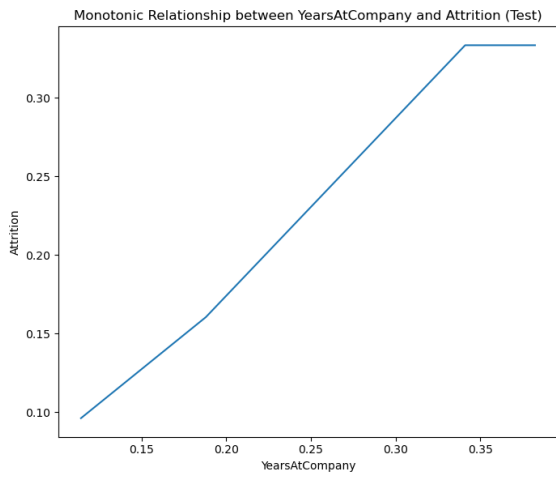
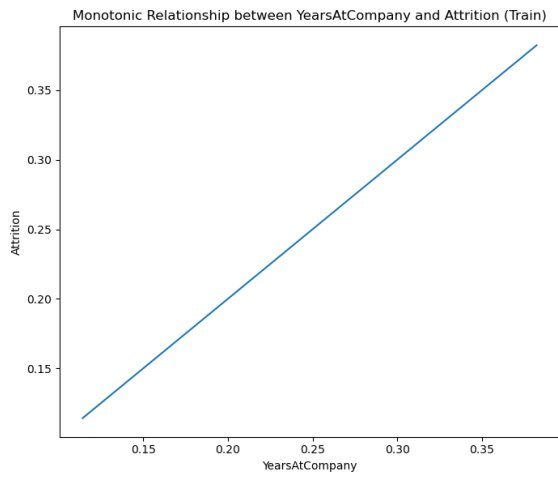
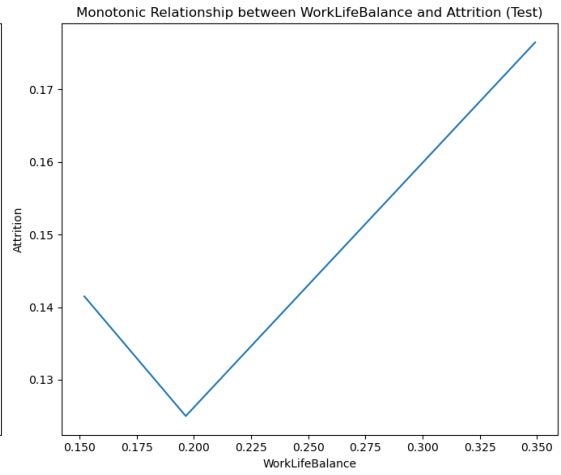
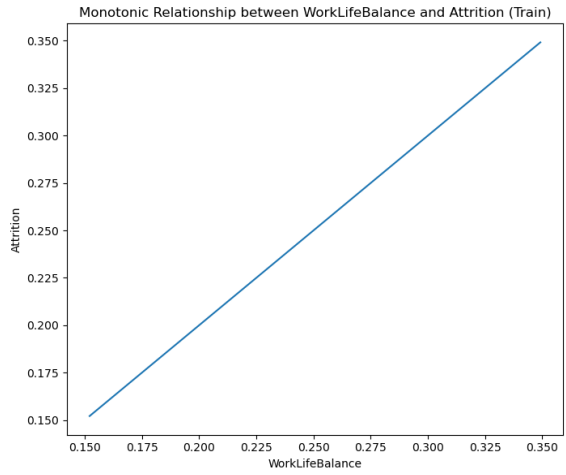


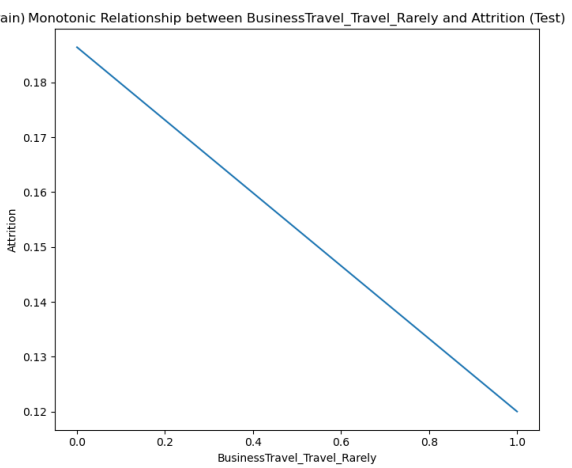
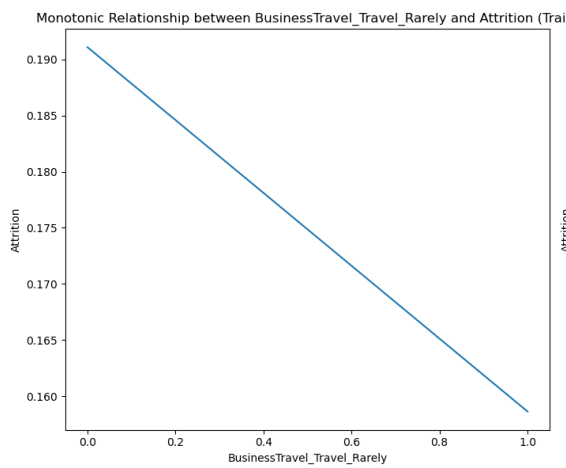
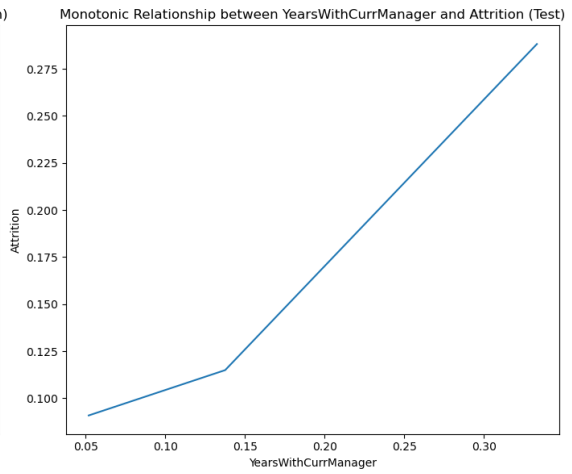
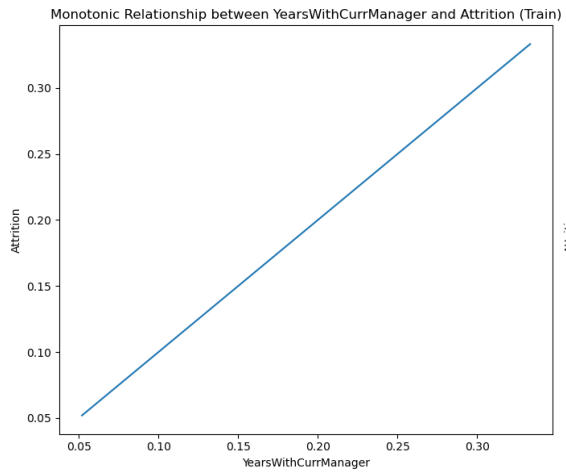
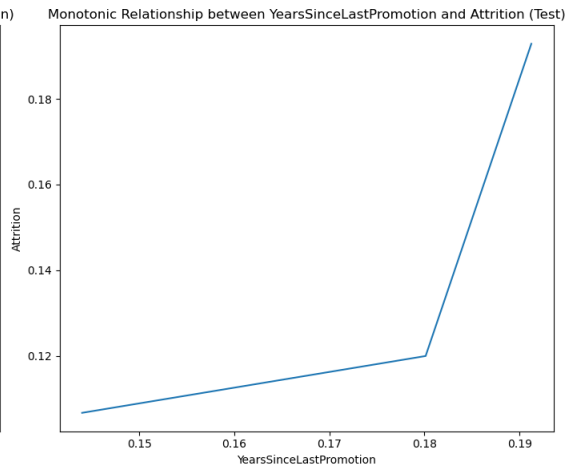
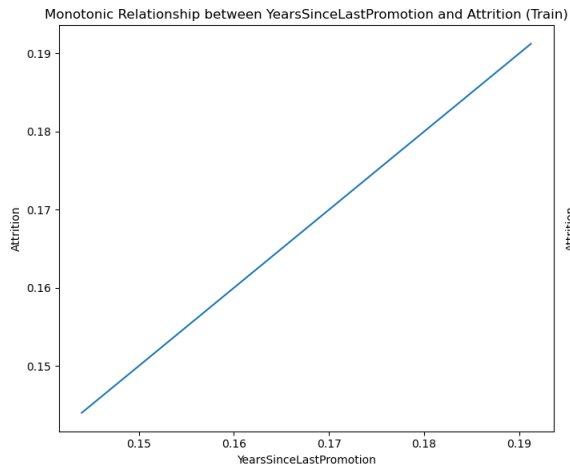




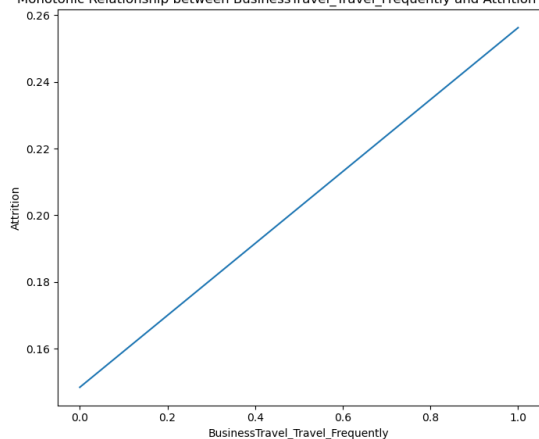




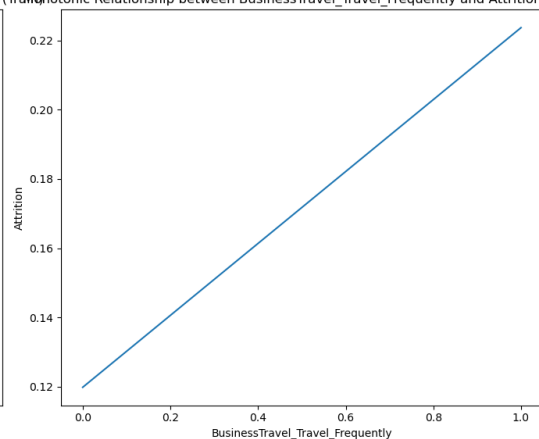




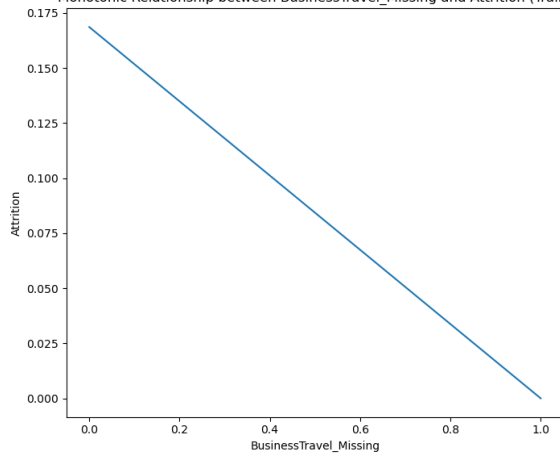
Monotonic Relationship between BusinessTravel_Travel_Frequently and Attrition (Train)



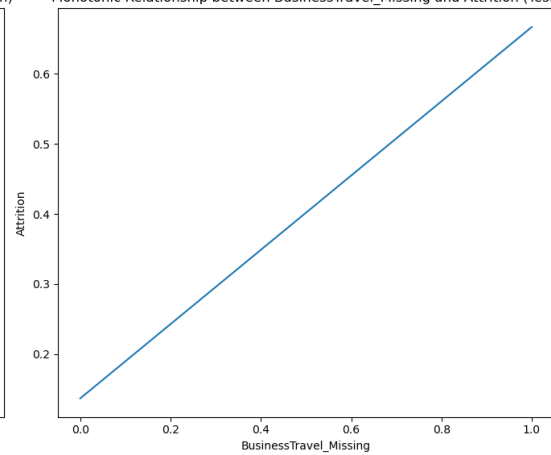
Monotonic Relationship between BusinessTravel_Travel_Frequently and Attrition (Test)



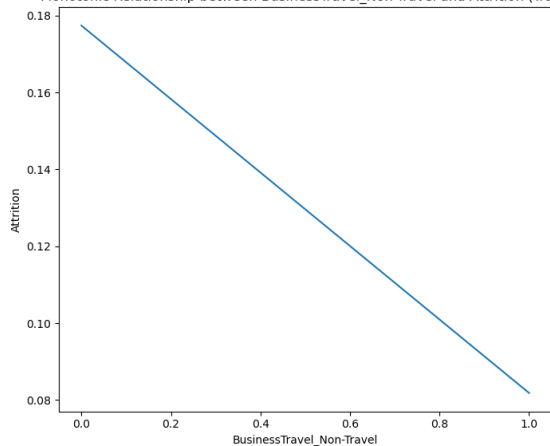
Monotonic Relationship between BusinessTravel_Missing and Attrition (Train)



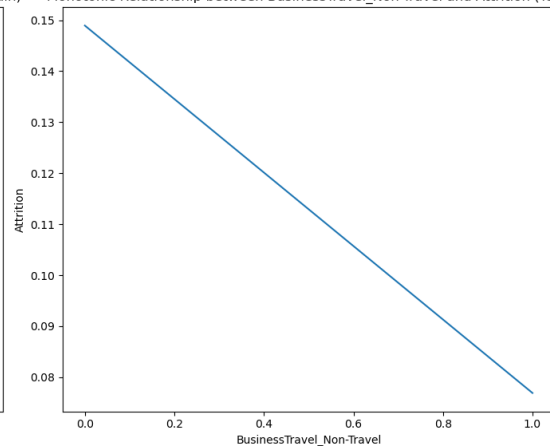
Monotonic Relationship between BusinessTravel_Missing and Attrition (Test)



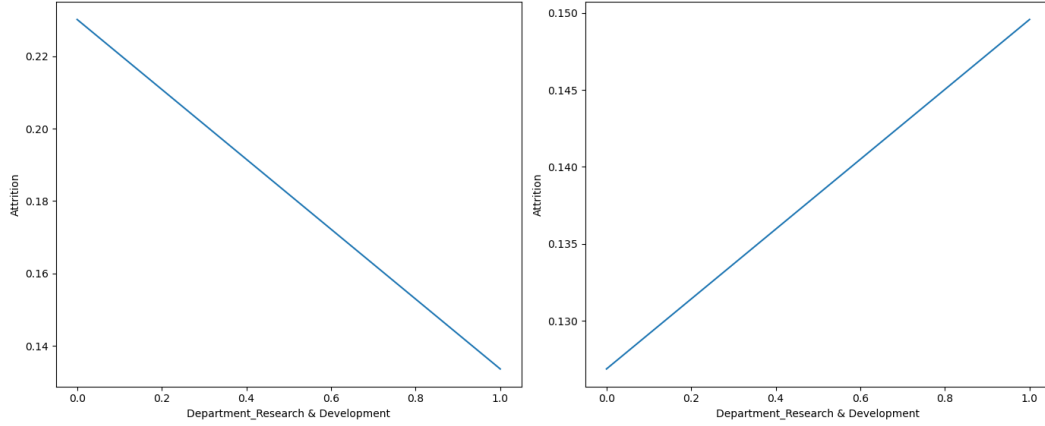
Monotonic Relationship between BusinessTravel_Non-Travel and Attrition (Train)



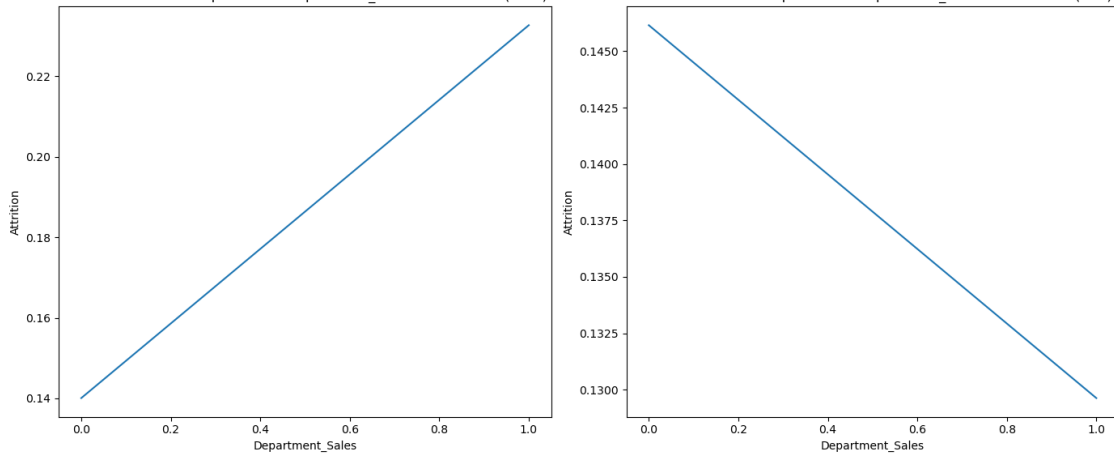
Monotonic Relationship between BusinessTravel_Non-Travel and Attrition (Test)



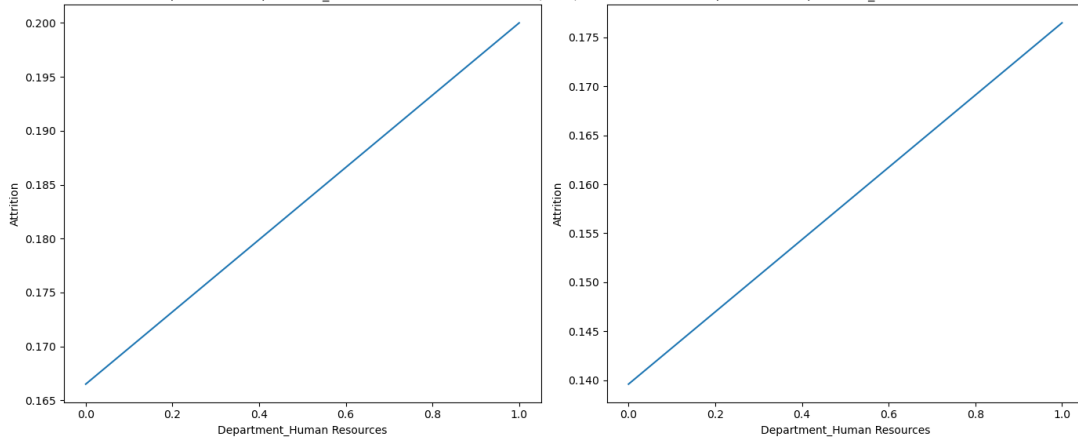
Monotonic Relationship between Department_Research & Development and Attrition (Train) Monotonic Relationship between Department_Research & Development and Attrition (Test)

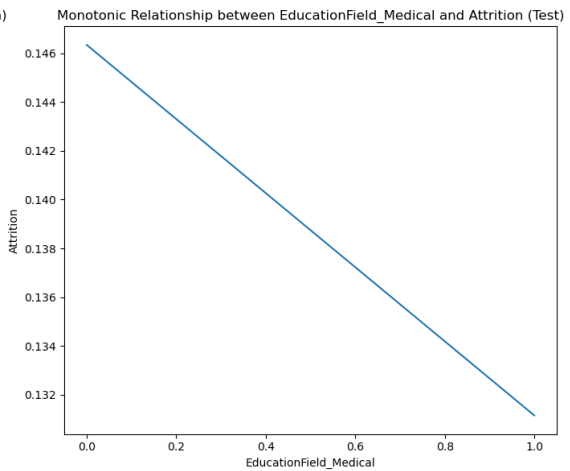
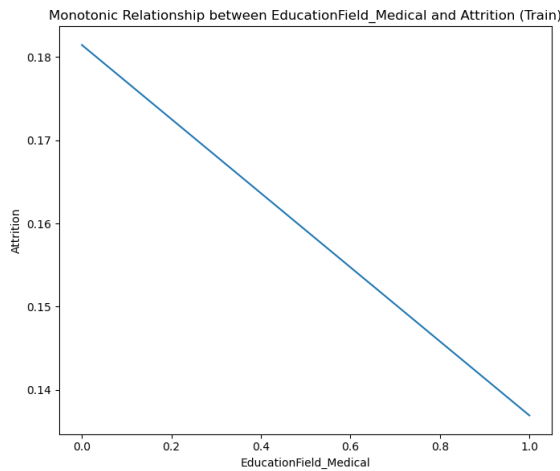
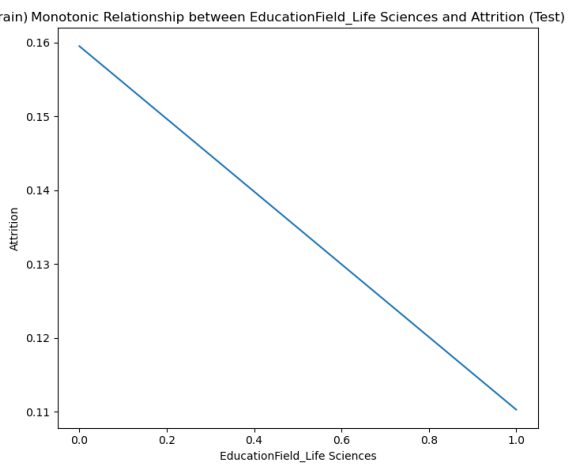
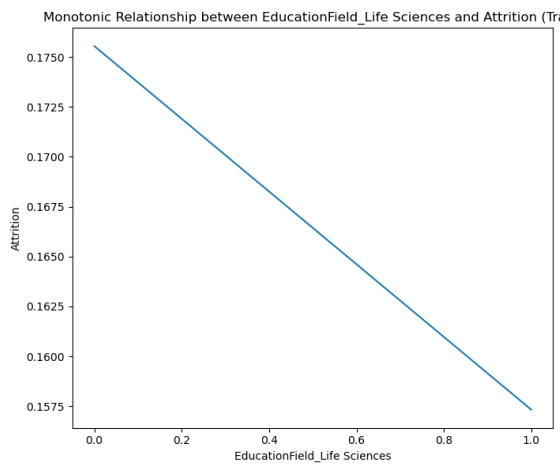
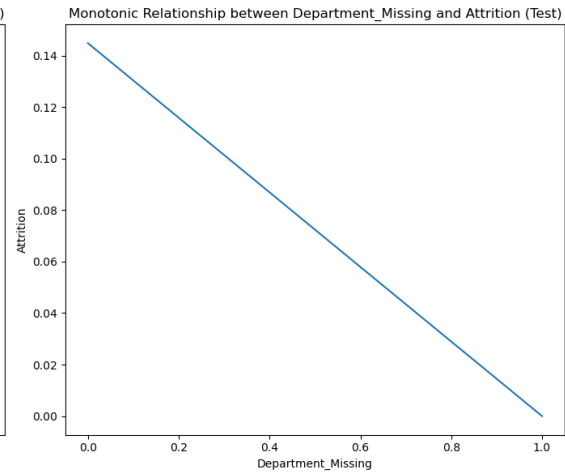
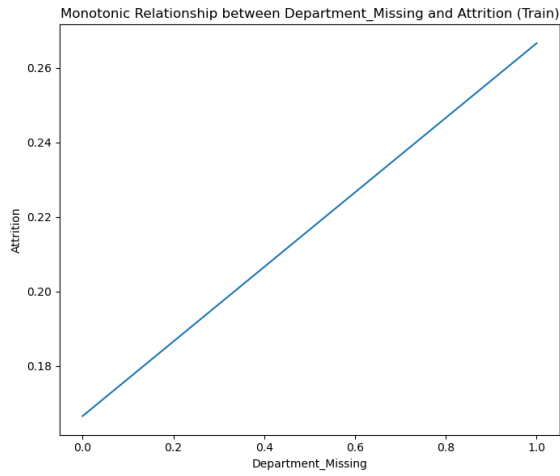


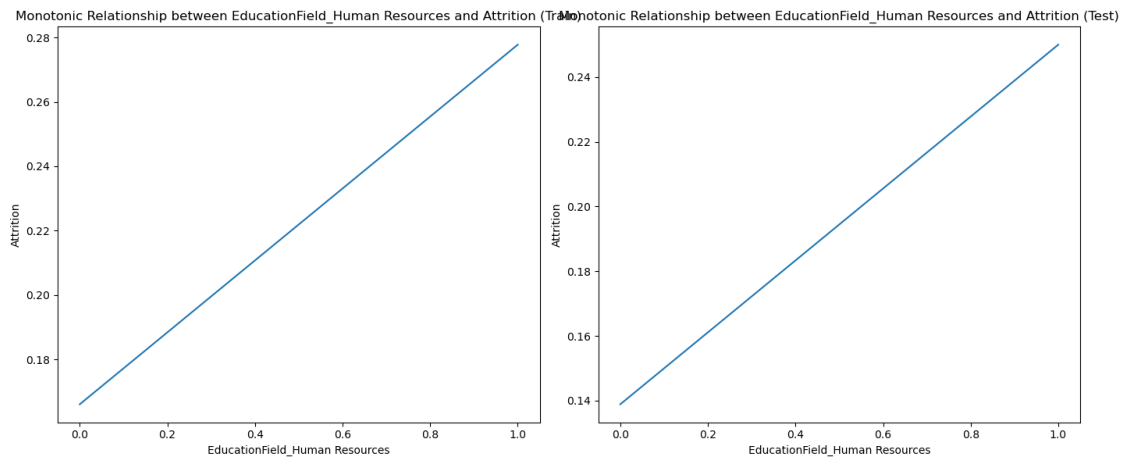
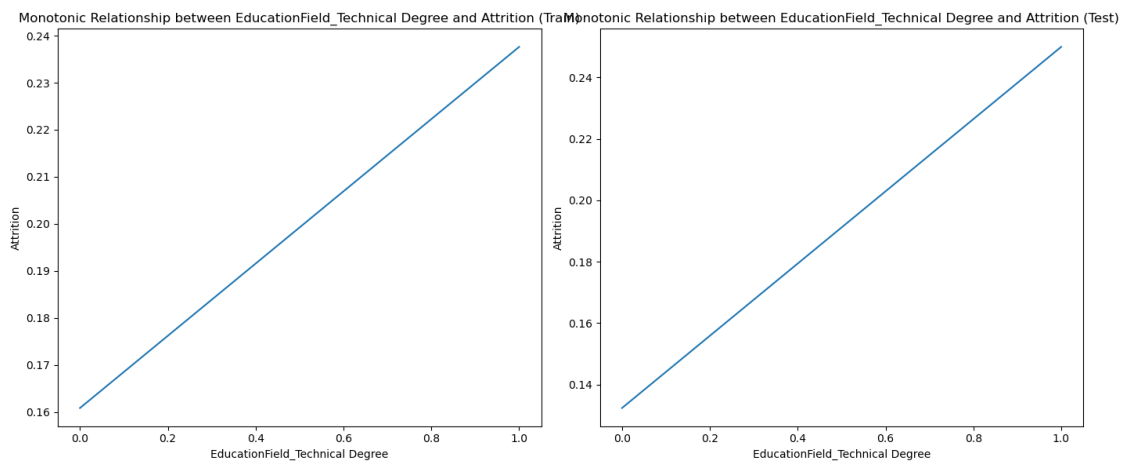
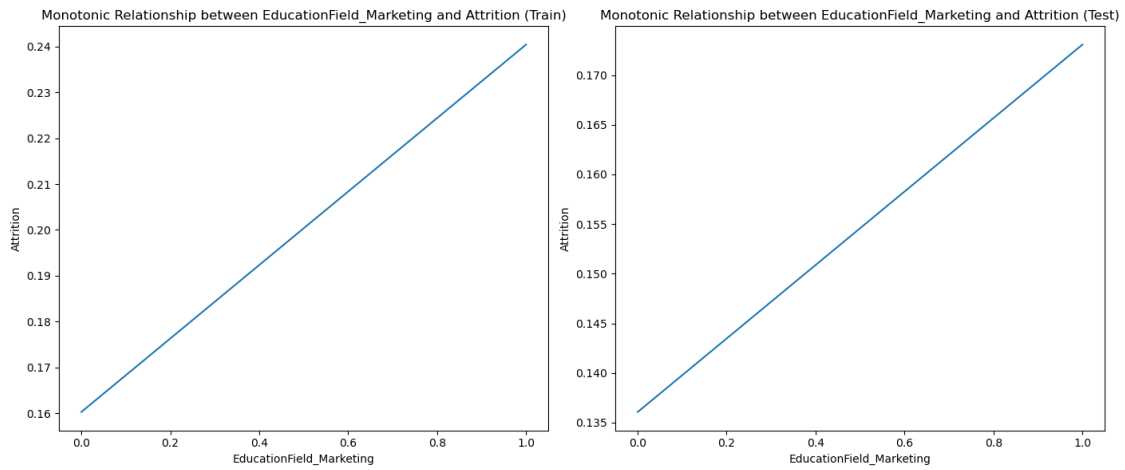
Monotonic Relationship between Department_Sales and Attrition (Train) Monotonic Relationship between Department_Sales and Attrition (Test)

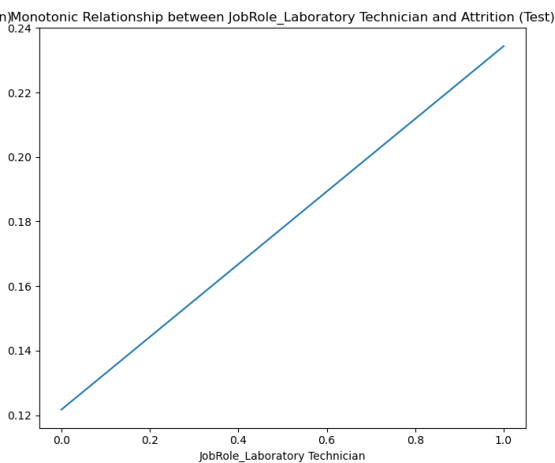
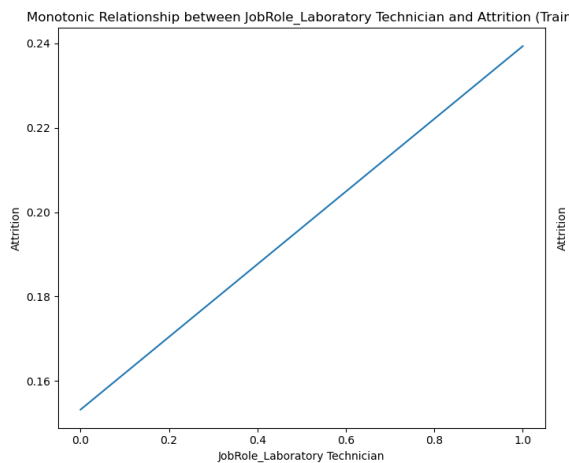
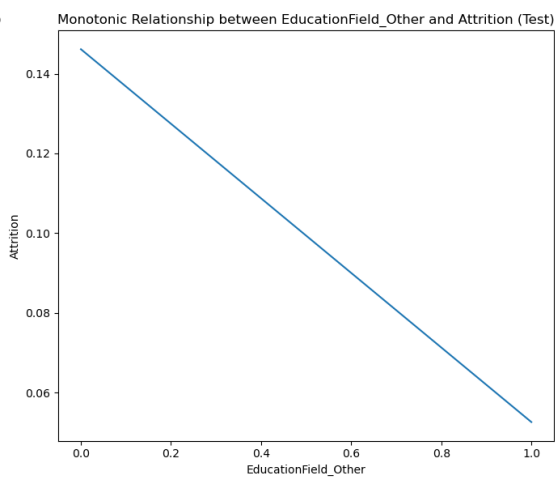
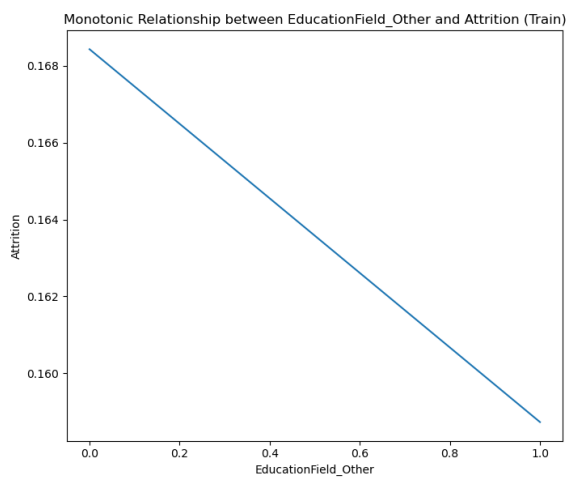
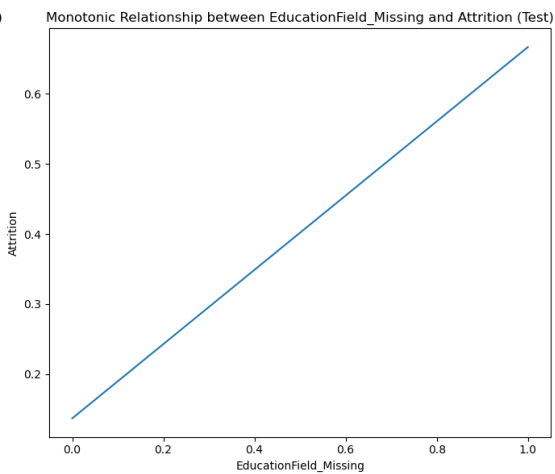
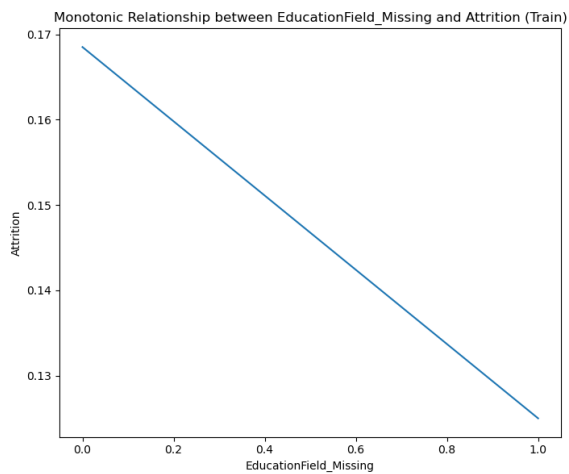


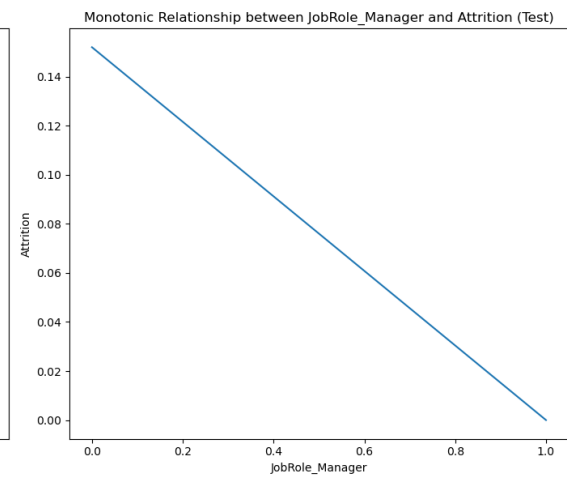
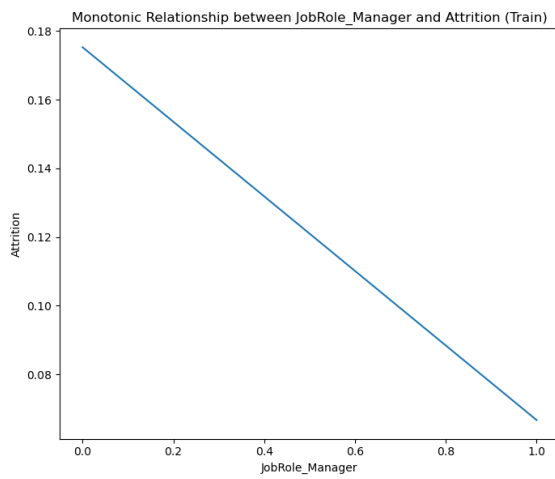
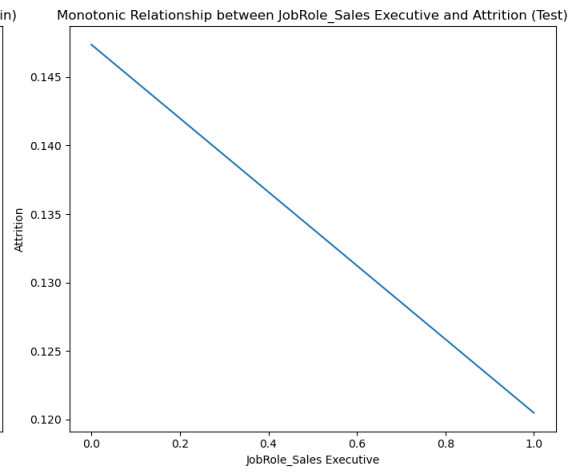
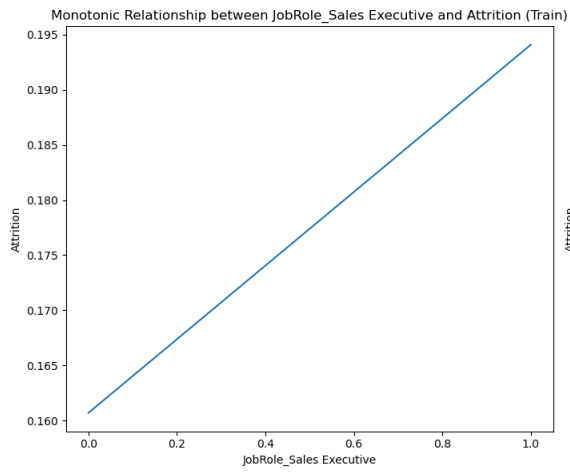
Monotonic Relationship between Department_Human Resources and Attrition (Train) Monotonic Relationship between Department_Human Resources and Attrition (Test)

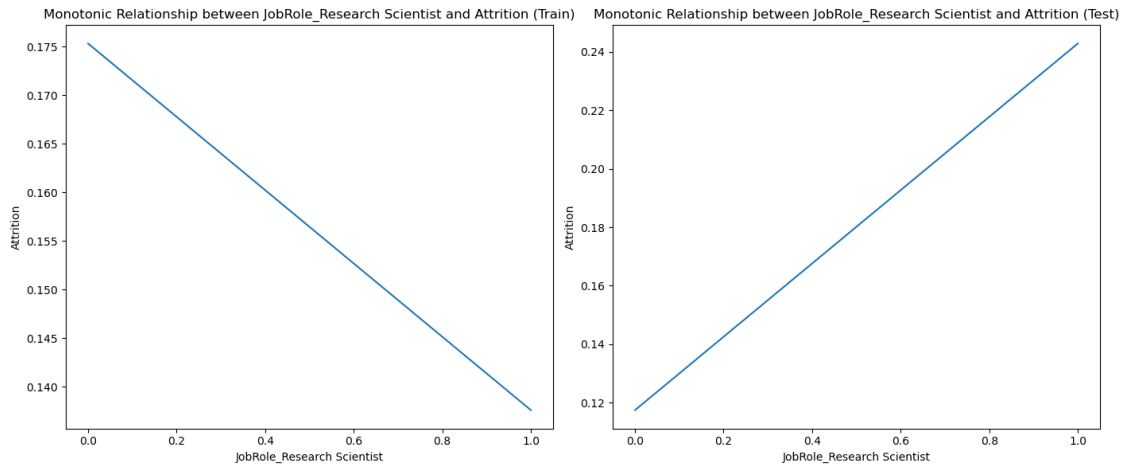
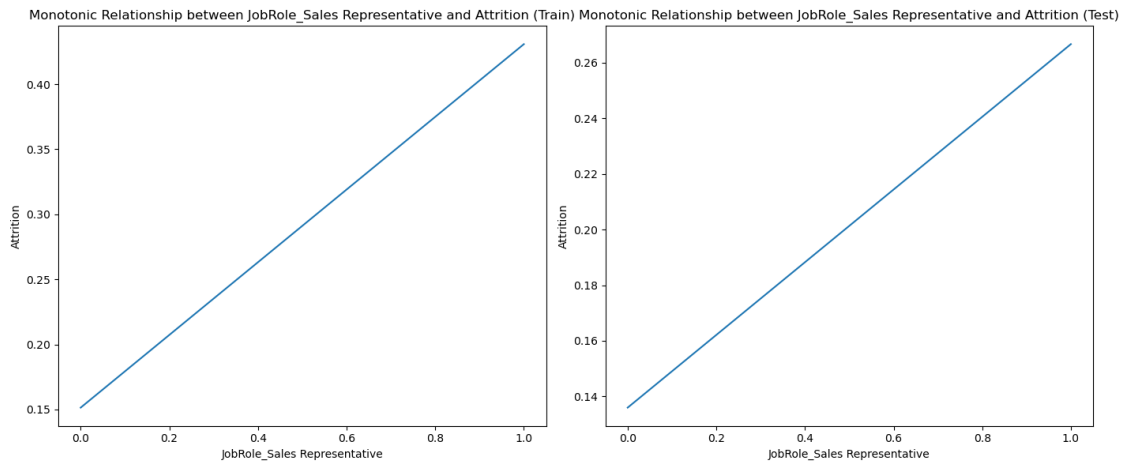
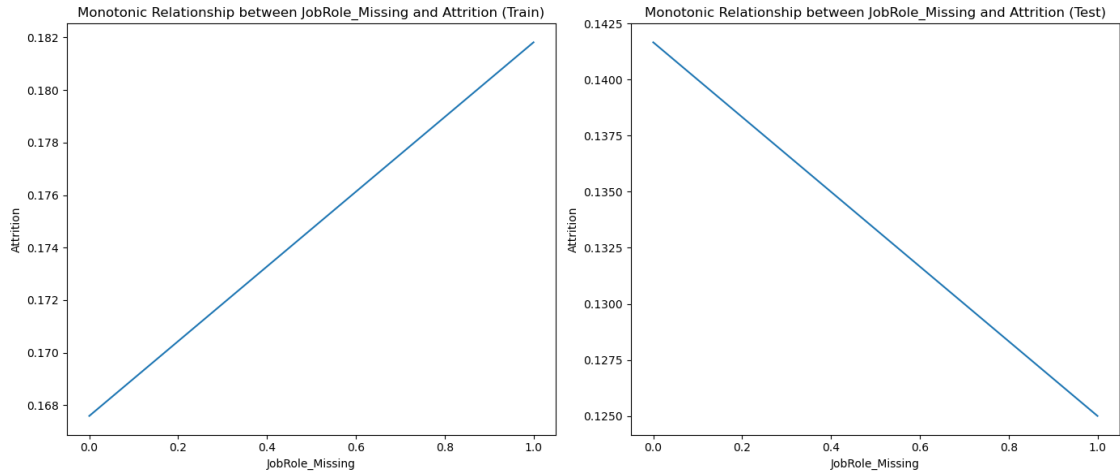




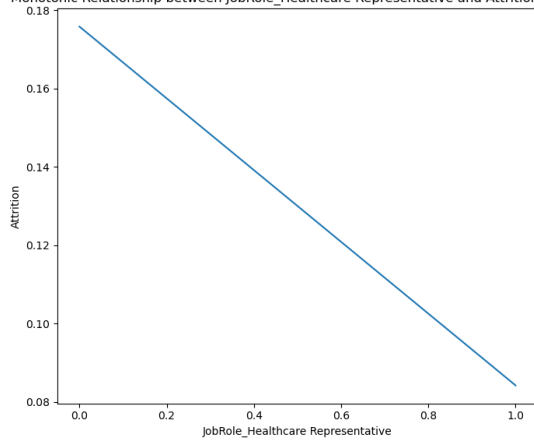




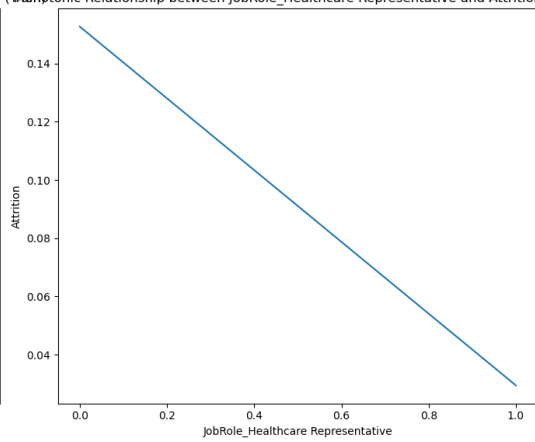




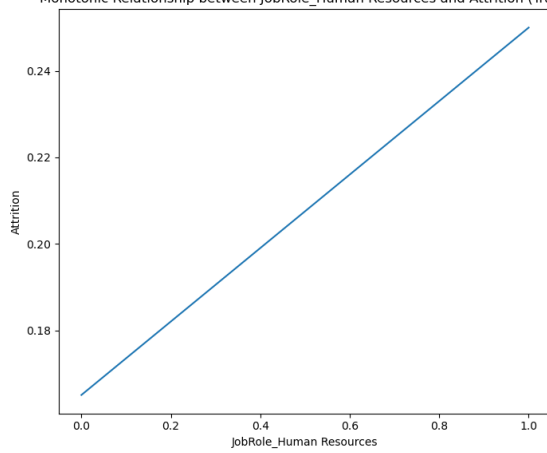
Monotonic Relationship between JobRole_Healthcare Representative and Attrition (Train)



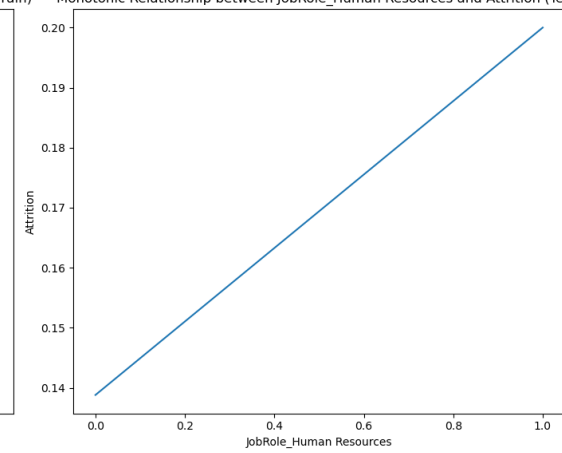
Monotonic Relationship between JobRole_Healthcare Representative and Attrition (Test)



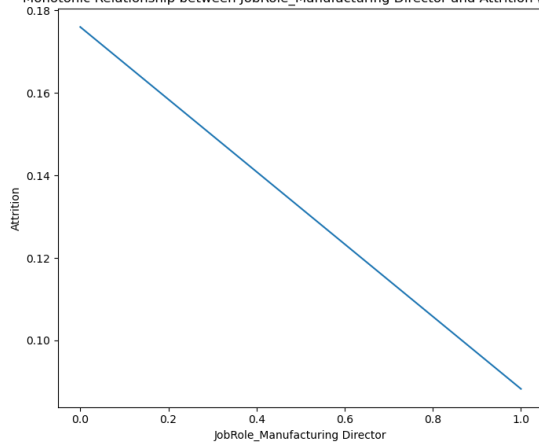
Monotonic Relationship between JobRole_Human Resources and Attrition (Train)



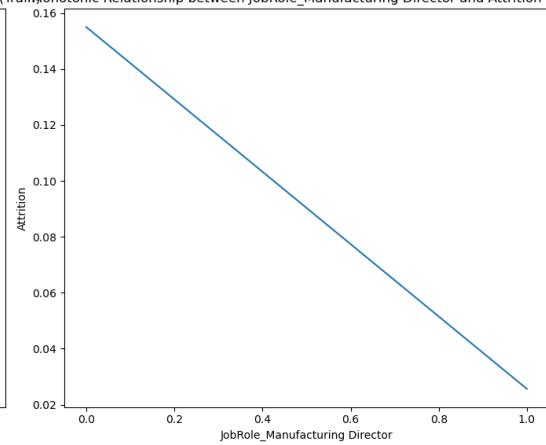
Monotonic Relationship between JobRole_Human Resources and Attrition (Test)

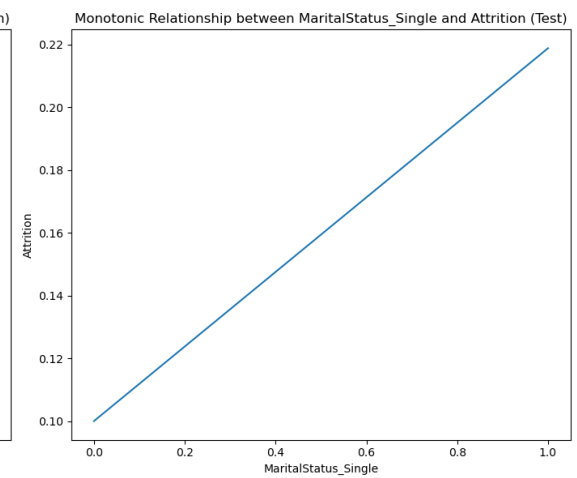
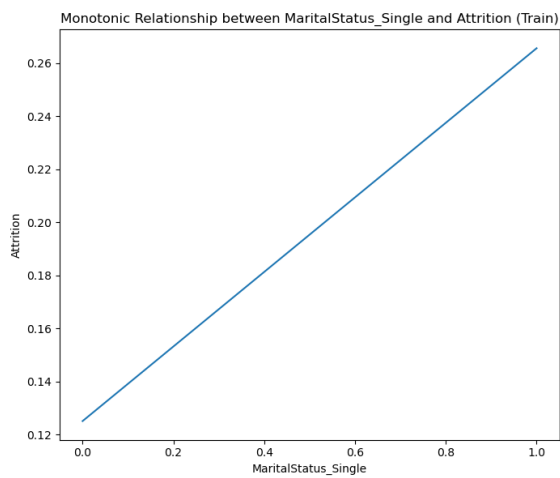
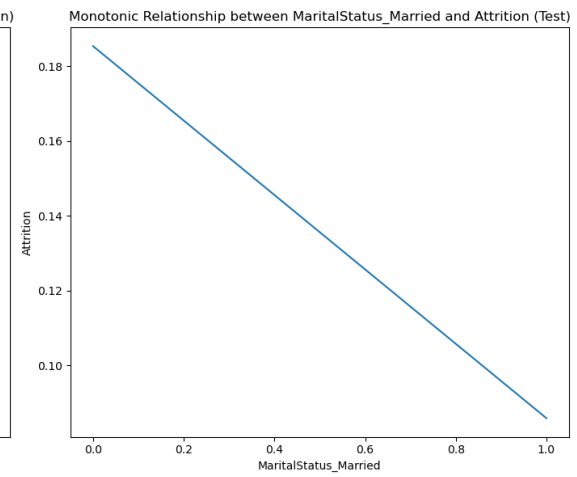
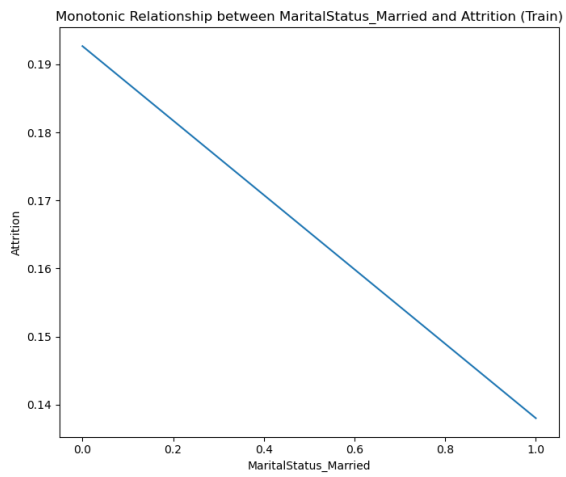
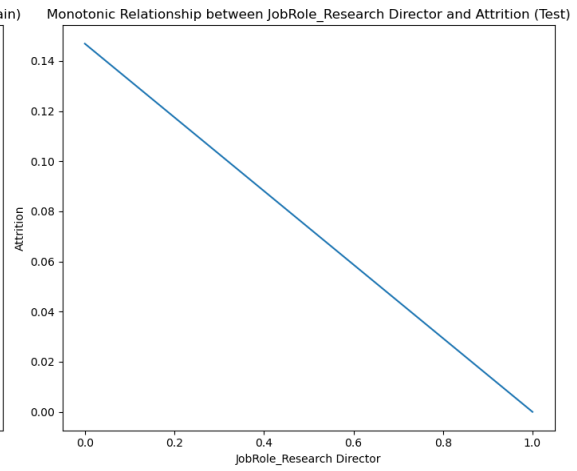
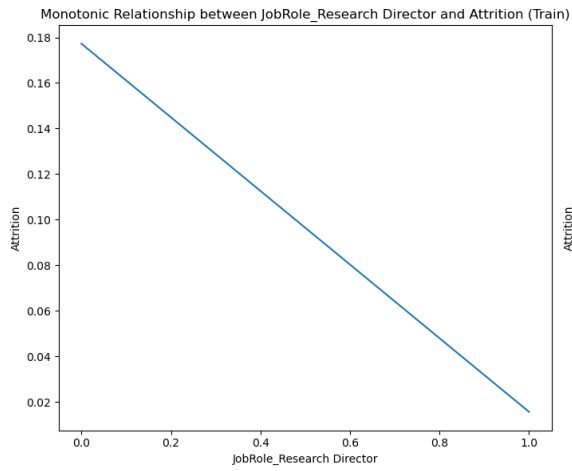


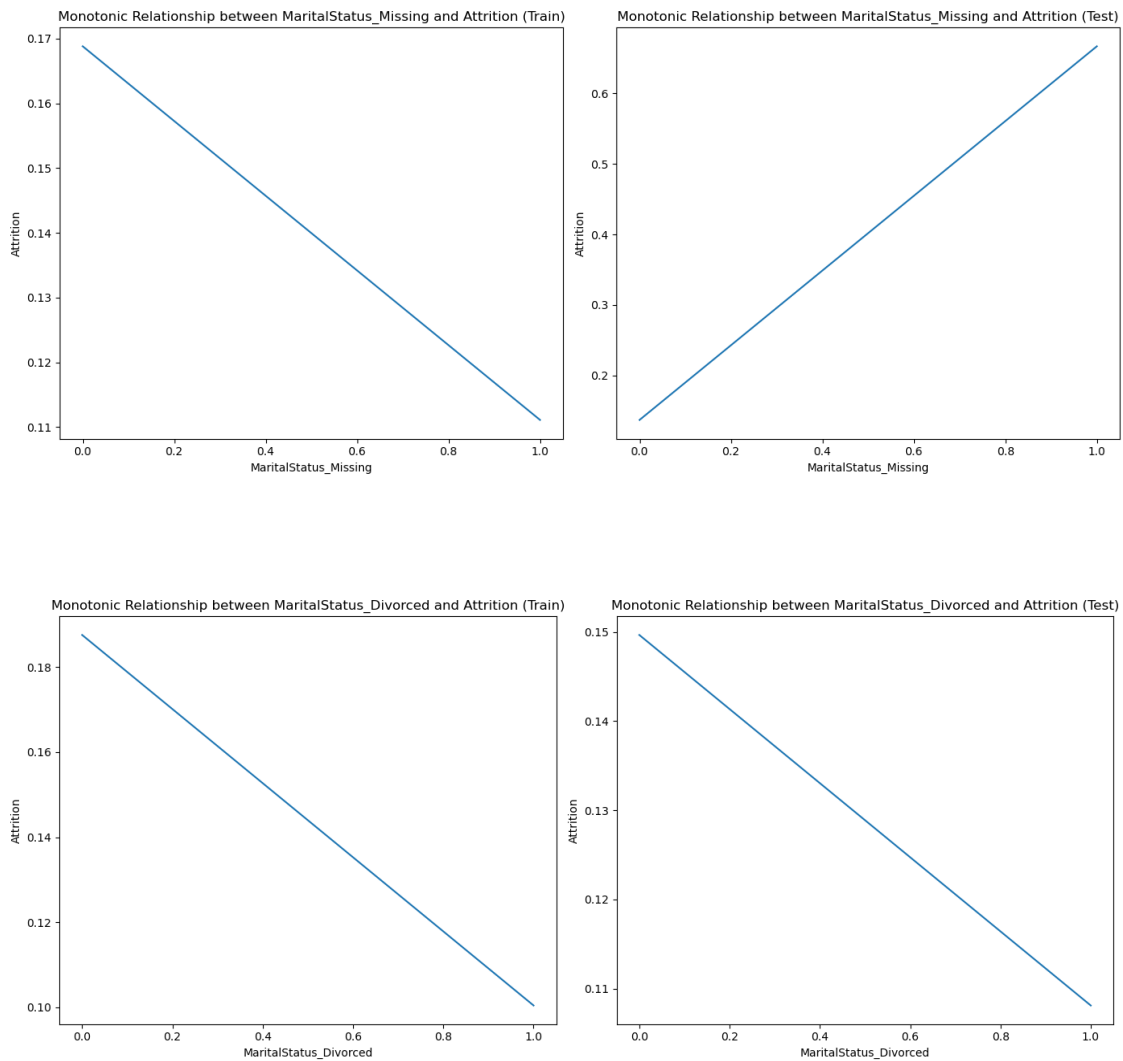
Monotonic Relationship between JobRole_Manufacturing Director and Attrition (Train)



Monotonic Relationship between JobRole_Manufacturing Director and Attrition (Test)







3. Training ML Models with default parameters

```
[137]: classifiers = {
    'KNeighbor': KNeighborsClassifier(),
    'RandomForest': RandomForestClassifier(),
    'SVC': SVC(),
    'Decision Tree': DecisionTreeClassifier(),
    'AdaBoost': AdaBoostClassifier(algorithm = 'SAMME'),
    'BaggingClf': BaggingClassifier(),
    'ExtraTreeClf': ExtraTreesClassifier(),
    'XGBoost': XGBClassifier(),
    'GaussianNB': GaussianNB(),
    'SGDC': SGDClassifier(),
```

```

}

results = {} # Dictionary for save results

for name, clf in classifiers.items():
    clf.fit(tmp_mean_train, y_train)
    score = clf.score(tmp_mean_test, y_test)
    results[name] = score

sorted_results = dict(sorted(results.items(), key=lambda item: item[1],
↪reverse=True))

print("\nResults:")
for name, score in sorted_results.items():
    print(f"{name}: {score:.2f}")

```

[87]: `def evaluate_performance(model, X_train, y_train, X_test, y_test):`

```

    # train model
    model.fit(X_train, y_train)

    # predict label
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    # predict probabilities
    y_pred_proba_train = model.predict_proba(X_train)[: , 1]
    y_pred_proba_test = model.predict_proba(X_test)[: , 1]

    # evaluate metrics performance
    roc_auc_train = roc_auc_score(y_train, y_pred_proba_train)
    roc_auc_test = roc_auc_score(y_test, y_pred_proba_test)

    accuracy_train = accuracy_score(y_train, y_pred_train)
    accuracy_test = accuracy_score(y_test, y_pred_test)

    recall_train = recall_score(y_train, y_pred_train)
    recall_test = recall_score(y_test, y_pred_test)

    f1_train = f1_score(y_train, y_pred_train)
    f1_test = f1_score(y_test, y_pred_test)

    precision_train = precision_score(y_train, y_pred_train)
    precision_test = precision_score(y_test, y_pred_test)

    # Output metrics

```

```

print("ROC AUC Train:", roc_auc_train)
print("ROC AUC Test:", roc_auc_test)
print()
print("Accuracy Train:", accuracy_train)
print("Accuracy Test:", accuracy_test)
print()
print("Recall Train:", recall_train)
print("Recall Test:", recall_test)
print()
print("F1 Score Train:", f1_train)
print("F1 Score Test:", f1_test)
print()
print("Precision Train:", precision_train)
print("Precision Test:", precision_test)
print()

# evaluate mean metrics with cross_val_score
roc_auc_cv_scores = cross_val_score(model, X_train, y_train, cv=10,
↪scoring='roc_auc')
accuracy_cv_scores = cross_val_score(model, X_train, y_train, cv=10,
↪scoring='accuracy')

roc_auc_cv_mean = roc_auc_cv_scores.mean()
accuracy_cv_mean = accuracy_cv_scores.mean()

# evaluate Standard Deviation
roc_auc_cv_std = roc_auc_cv_scores.std()
accuracy_cv_std = accuracy_cv_scores.std()

# Output mean metrics and standard deviation
print("ROC AUC (cross-validation): Mean =", roc_auc_cv_mean)
print("Accuracy (cross-validation): Mean =", accuracy_cv_mean)
print()
print("ROC AUC Std =", roc_auc_cv_std)
print("Accuracy Std =", accuracy_cv_std)
print()

# Confusion Matrix
print("Confusion Matrix for Train:")
print(confusion_matrix(y_train, y_pred_train))
print()
print("Confusion Matrix for Test:")
print(confusion_matrix(y_test, y_pred_test))
print()

```

3.1 Preprocessing Pipeline for ML models

```
[41]: def build_pipeline(classifier):
    pipeline = Pipeline(steps = [
        ('imputer_cat', CategoricalImputer(variables=categorical_feat,
        ↪imputation_method='missing')),
        ('imputer_num', MeanMedianImputer(variables=numerical_feat,
        ↪imputation_method='mean')),
        ('enc', fe_OneHot(variables=categorical_feat)),
        ('outlier', Winsorizer()),
        ('disc', EqualFrequencyDiscretiser(q=7, variables=continuous)),
        ('scaler', MaxAbsScaler()),
        ('clf', classifier)
    ])
    return pipeline

[42]: def tune_model(X_train, y_train, classifier, param_grid, cv = 10):

    pipeline = build_pipeline(classifier)
    grid_search = GridSearchCV(pipeline, param_grid, cv = cv, n_jobs = -1,
    ↪scoring = 'accuracy', error_score = 'raise')
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_
    best_model.fit(X_train, y_train)

    return best_model

[43]: def get_best_model(X_train, y_train, classifier, param_grid, cv=10):
    return tune_model(X_train, y_train, classifier, param_grid, cv)
```

1 AdaBoostClassifier

```
[44]: classifier_ADA = AdaBoostClassifier()

param_ADA = {
    'imputer_cat__imputation_method': ['missing', 'frequent'],
    'imputer_num__imputation_method': ['mean', 'median'],
    'outlier__tail': ['right', 'left'],
    'outlier__fold': [1.5, 2, 3],
    'disc__q': [5, 7, 9],
    'scaler': [MinMaxScaler(), MaxAbsScaler()],
    'clf__n_estimators': [50, 100, 150],
    'clf__learning_rate': [0.2, 0.3],
    'clf__algorithm': ['SAMME']
}
best_model_ADA = get_best_model(X_train, y_train, classifier_ADA, param_ADA)
```

```
[45]: y_pred_Ada_train = best_model_ADA.predict(X_train)
      y_pred_Ada_test = best_model_ADA.predict(X_test)
```

```
[88]: evaluate_performance(best_model_ADA, X_train, y_train, X_test, y_test)
```

2 ExtraTreesClassifier

```
[47]: classifier_Extra = ExtraTreesClassifier()

      param_Extra = {
          'imputer_cat__imputation_method': ['missing', 'frequent'],
          'imputer_num__imputation_method': ['mean', 'median'],
          'outlier__tail': ['right', 'left'],
          'outlier__fold': [1.5, 2, 3],
          'disc__q': [5, 7, 9],
          'scaler': [StandardScaler(), MinMaxScaler()],
          'clf__n_estimators': [100, 150, 200],
          'clf__criterion': ['gini', 'entropy'],
          'clf__max_depth': [3, 5]
      }
      best_model_Extra = get_best_model(X_train, y_train, classifier_Extra,
          ↪param_Extra)
```

```
[48]: y_pred_Extra_train = best_model_Extra.predict(X_train)
      y_pred_Extra_test = best_model_Extra.predict(X_test)
```

```
[89]: evaluate_performance(best_model_Extra, X_train, y_train, X_test, y_test)
```

3 BaggingClassifier

```
[50]: classifier_Bagging = BaggingClassifier()

      param_Bagg = {
          'imputer_cat__imputation_method': ['missing', 'frequent'],
          'imputer_num__imputation_method': ['mean', 'median'],
          'outlier__tail': ['right', 'left'],
          'outlier__fold': [1.5, 2, 3],
          'disc__q': [5, 8],
          'scaler': [MaxAbsScaler(), StandardScaler()],
          'clf__n_estimators': [50, 100, 150],
          'clf__max_samples': [0.8, 1],
          'clf__max_features': [0.8, 1]
      }
      best_model_bagg = get_best_model(X_train, y_train, classifier_Bagging,
          ↪param_Bagg)
```

```
[51]: y_pred_Bagg_train = best_model_bagg.predict(X_train)
      y_pred_Bagg_test = best_model_bagg.predict(X_test)
```

```
[90]: evaluate_performance(best_model_bagg, X_train, y_train, X_test, y_test)
```

4 XGBClassifier

```
[53]: classifier_XGB = XGBClassifier()

param_XGB = {
    'imputer_cat__imputation_method': ['missing', 'frequent'],
    'imputer_num__imputation_method': ['mean', 'median'],
    'outlier__tail': ['right', 'left'],
    'outlier__fold': [1.5, 2, 3],
    'disc__q': [5, 8],
    'scaler': [MinMaxScaler(), StandardScaler()],
    'clf__n_estimators': [100, 200, 300],
    'clf__max_depth': [4, 6],
    'clf__learning_rate': [0.2, 0.3]
}

best_model_XGB = get_best_model(X_train, y_train, classifier_XGB, param_XGB)
```

```
[54]: y_pred_XGB_train = best_model_XGB.predict(X_train)
      y_pred_XGB_test = best_model_XGB.predict(X_test)
```

```
[91]: evaluate_performance(best_model_XGB, X_train, y_train, X_test, y_test)
```

5 RandomForestClassifier

```
[56]: classifier_RF = RandomForestClassifier()

param_RF = {
    'imputer_cat__imputation_method': ['missing', 'frequent'],
    'imputer_num__imputation_method': ['mean', 'median'],
    'outlier__tail': ['right', 'left'],
    'outlier__fold': [1.5, 2, 3],
    'disc__q': [5, 8],
    'scaler': [MaxAbsScaler(), StandardScaler()],
    'clf__n_estimators': [100, 200, 300],
    'clf__criterion': ['gini', 'entropy'],
    'clf__max_depth': [4, 6]
}

best_model_RF = get_best_model(X_train, y_train, classifier_RF, param_RF)
```

```
[57]: y_pred_RF_train = best_model_RF.predict(X_train)
      y_pred_RF_test = best_model_RF.predict(X_test)
```

```
[92]: evaluate_performance(best_model_RF, X_train, y_train, X_test, y_test)
```

```
[93]: y_pred_Ada_test = (y_pred_Ada_test > 0.5)
      y_pred_Ada_train = (y_pred_Ada_train > 0.5)

      y_pred_Bagg_test = (y_pred_Bagg_test > 0.5)
      y_pred_Bagg_train = (y_pred_Bagg_train > 0.5)

      y_pred_Extra_test = (y_pred_Extra_test > 0.5)
      y_pred_Extra_train = (y_pred_Extra_train > 0.5)

      y_pred_RF_test = (y_pred_RF_test > 0.5)
      y_pred_RF_train = (y_pred_RF_train > 0.5)

      y_pred_XGB_test = (y_pred_XGB_test > 0.5)
      y_pred_XGB_train = (y_pred_XGB_train > 0.5)
```

3.2 Separate for training and test split to train ANN

```
[60]: X_train, X_test, y_train, y_test = train_test_split(
      dataset.drop(labels = ['Attrition'], axis = 1),
      dataset['Attrition'],
      test_size = 0.25
      )
      X_train.shape, X_test.shape
```

```
[61]: # Function for demonstrate model performance
      def plot_performance(history):
          plt.figure(figsize=(12, 6))

          # Loss
          plt.subplot(1, 2, 1)
          plt.plot(history.history['loss'], label='Train Loss')
          plt.plot(history.history['val_loss'], label='Validation Loss')
          plt.xlabel('Epoch')
          plt.ylabel('Loss')
          plt.title('Loss')
          plt.legend()

          # Accuracy
          plt.subplot(1, 2, 2)
          plt.plot(history.history['accuracy'], label='Train Accuracy')
          plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
          plt.xlabel('Epoch')
          plt.ylabel('Accuracy')
```

```
plt.title('Accuracy')
plt.legend()

plt.show()
```

3.3 Preprocessing Pipeline for ANN

```
[62]: # pipeline for ANN
pipe_preprocessing = Pipeline(steps = [
    ('imputer_cat', CategoricalImputer(variables = categorical_feat,
    ↪imputation_method = 'missing')),
    ('imputer_num', MeanMedianImputer(variables = numerical_feat,
    ↪imputation_method = 'mean')),
    ('enc', fe_OneHot(variables = categorical_feat)),
    ('outlier', Winsorizer()),
    ('disc', EqualFrequencyDiscretiser(q = 7, variables = continuous)),
    ('scaler', MaxAbsScaler())
])
X_train_preprocessed = pipe_preprocessing.fit_transform(X_train)
X_test_preprocessed = pipe_preprocessing.transform(X_test)
```

3.4 Early stopping

```
[63]: early_stopping = EarlyStopping(
    monitor = 'val_accuracy',
    min_delta = 0.001,
    patience = 50,
    restore_best_weights = True,
    verbose = 1,
    mode = 'auto'
)
callbacks = early_stopping
```

3.5 Build the ANN

```
[64]: input_shape = (X_train_preprocessed.shape[1],)

def create_model(optimizer = 'adam'):
    classifier = Sequential()

    classifier.add(Input(shape = input_shape))

    classifier.add(Dense(units = 300, kernel_initializer = 'he_uniform',
    ↪activation = 'relu', kernel_regularizer = l2(0.001)))
    classifier.add(BatchNormalization())
    classifier.add(Dropout(0.2))
```

```

        classifier.add(Dense(units = 300, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
        classifier.add(BatchNormalization())
        classifier.add(Dropout(0.2))

        classifier.add(Dense(units = 250, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
        classifier.add(BatchNormalization())
        classifier.add(Dropout(0.3))

        classifier.add(Dense(units = 200, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
        classifier.add(BatchNormalization())
        classifier.add(Dropout(0.2))

        classifier.add(Dense(units = 1, activation = 'sigmoid', kernel_initializer
↪= 'glorot_uniform'))
        classifier.compile(optimizer = optimizer, loss = 'binary_crossentropy',
↪metrics = ['accuracy'])
        return classifier

hist_model = create_model()
history1 = hist_model.fit(
    X_train_preprocessed,
    y_train,
    epochs = 300,
    batch_size = 128,
    validation_split = 0.2,
    callbacks = callbacks
)
hist_model.summary()

```

```

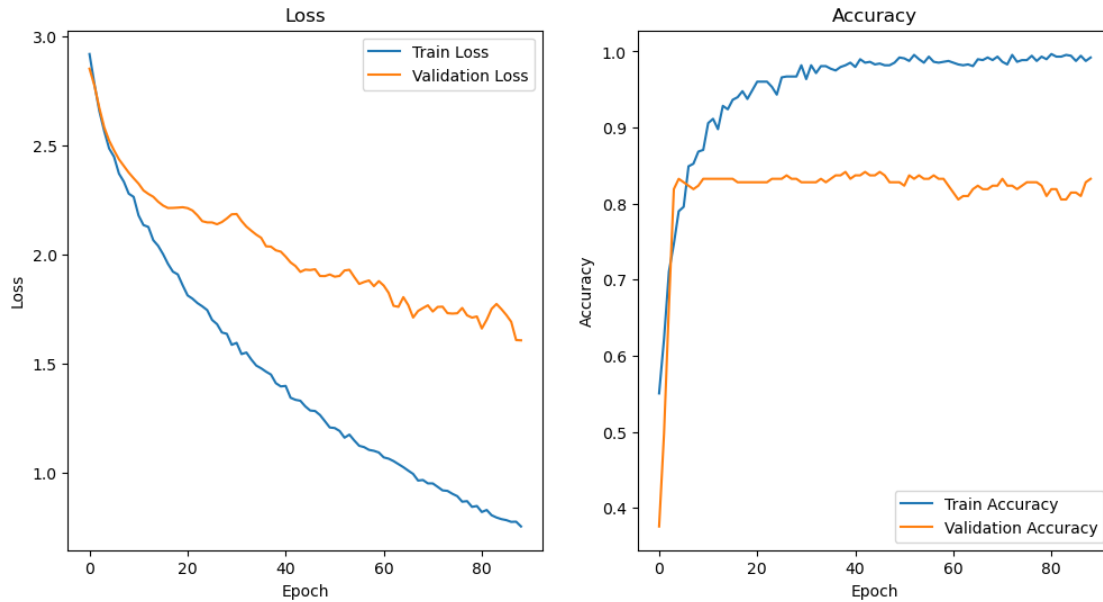
[65]: y_pred_ann_train = hist_model.predict(X_train_preprocessed)
      y_pred_ann_test = hist_model.predict(X_test_preprocessed)

```

```

[66]: plot_performance(history1)

```



3.6 GridSearchCV

```
[73]: pipeline = Pipeline(steps=[
    ('preprocessing', pipe_preprocessing),
    ('model', KerasClassifier(model=create_model,
                              epochs = 100,
                              batch_size = 128,
                              optimizer = 'adam',
                              metrics=['val_accuracy'],
                              validation_split = 0.2,
                              callbacks = callbacks
                              ))
])

# GridSearch parameters
parameters = {
    'model__batch_size': [64, 128, 256],
    'model__epochs': [100, 200, 250],
    'model__optimizer': ['adam', 'SGD']
}

grid = GridSearchCV(
    estimator = pipeline,
    param_grid = parameters,
    n_jobs = -1,
    cv = 3,
    error_score = 'raise'
```

```
)

grid_results = grid.fit(X_train, y_train, model__validation_split = 0.2,
    ↪model__callbacks = callbacks)

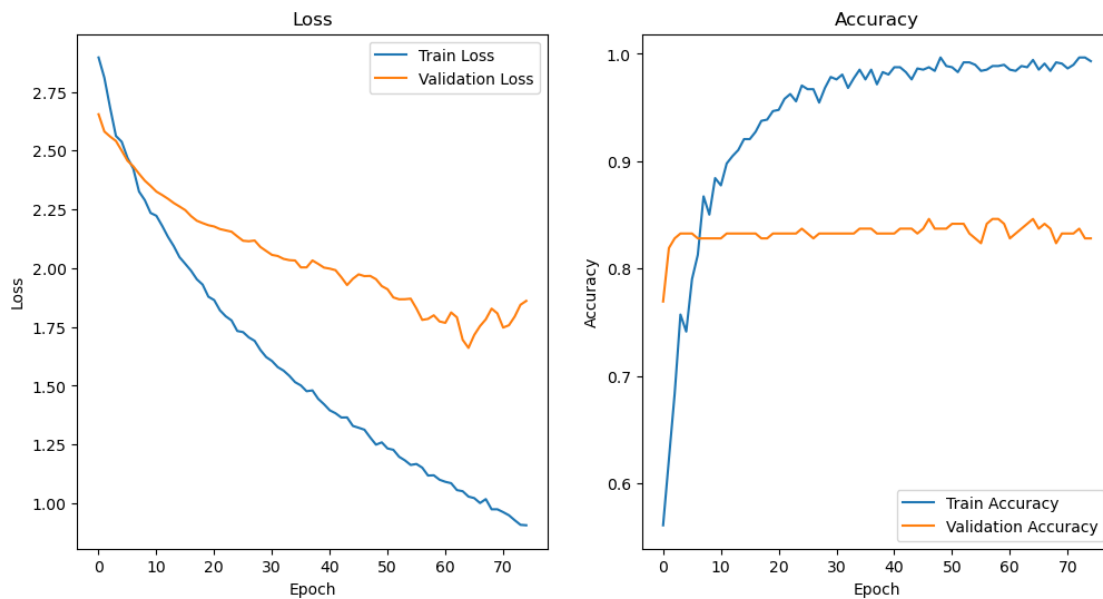
best_model = grid_results.best_estimator_
print("Best Parameters:", grid_results.best_params_)
```

```
[74]: best_params = grid_results.best_params_
print('Best Parameters:', best_params)
```

```
[75]: best_model = create_model(optimizer = best_params['model__optimizer'])

history = best_model.fit(X_train_preprocessed, y_train,
    validation_split = 0.2,
    batch_size = best_params['model__batch_size'],
    epochs = best_params['model__epochs'],
    callbacks = [EarlyStopping(monitor = 'val_loss',
    ↪patience = 10, verbose = 1)])
```

```
[77]: plot_performance(history)
```



```
[78]: y_pred_ann_train = best_model.predict(X_train_preprocessed)
y_pred_ann_test = best_model.predict(X_test_preprocessed)

y_pred_ann_train = (y_pred_ann_train > 0.5)
y_pred_ann_test = (y_pred_ann_test > 0.5)
```

4. Results

```
[108]: models = {
        'Ada Boost': best_model_ADA,
        'BaggingClf': best_model_bagg,
        'ExtraTreeClf': best_model_Extra,
        'Random Forest': best_model_RF,
        'XGBoostClf': best_model_XGB
    }

[132]: # Results all metrics in Data Frame
def calculate_performance_metrics(models, X_train, y_train, X_test, y_test,
    ↪model_names):
    metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'ROC AUC']
    metric_scores = {metric: [] for metric in metrics}

    for model_name, model in models.items(): # iterate through model names and
    ↪corresponding models
        train_predictions = model.predict(X_train)
        test_predictions = model.predict(X_test)

        accuracy = accuracy_score(y_test, test_predictions)
        precision = precision_score(y_test, test_predictions)
        recall = recall_score(y_test, test_predictions)
        f1 = f1_score(y_test, test_predictions)
        roc_auc = roc_auc_score(y_test, test_predictions)

        metric_scores['Accuracy'].append(accuracy)
        metric_scores['Precision'].append(precision)
        metric_scores['Recall'].append(recall)
        metric_scores['F1 Score'].append(f1)
        metric_scores['ROC AUC'].append(roc_auc)

    return pd.DataFrame(metric_scores, index=model_names).
    ↪sort_values(by='Accuracy', ascending=False)

[74]: # Confusion matrix
def plot_cm(y_train, y_test, y_train_pred, y_test_pred, model_name):
    # Creating subgraphs for confusion matrix for train and test set
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Confusion Matrix for train set
    train_cm = confusion_matrix(y_train, y_train_pred)
    sns.heatmap(train_cm, annot=True, cmap='Blues', fmt='d', ax=axes[0])
    axes[0].set_title('Confusion Matrix - Train Set ({}).format(model_name))
    axes[0].set_xlabel('Predicted')
    axes[0].set_ylabel('Actual')
```

```

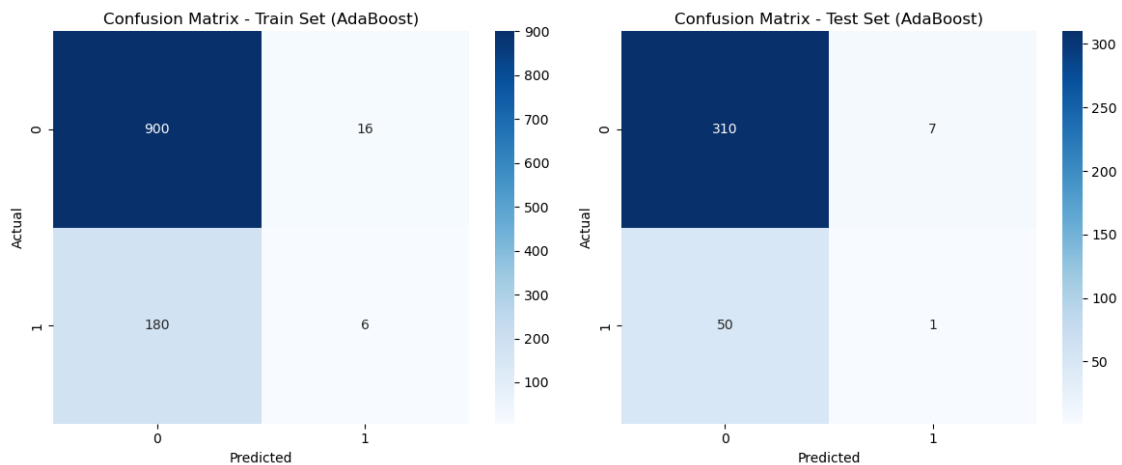
# Confusion Matrix for test set
test_cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(test_cm, annot=True, cmap='Blues', fmt='d', ax=axes[1])
axes[1].set_title('Confusion Matrix - Test Set ({}').format(model_name))
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

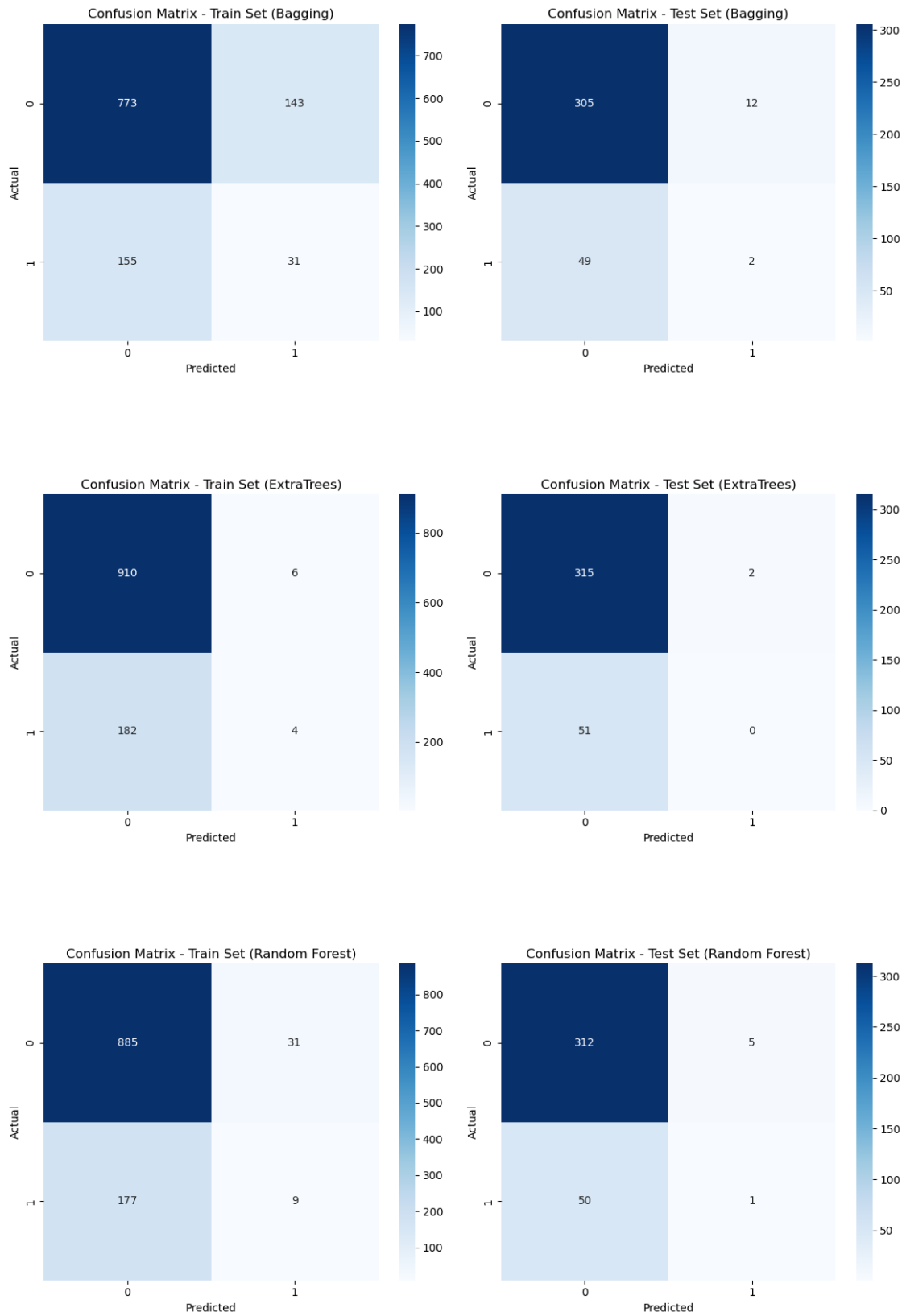
plt.tight_layout()
plt.show()

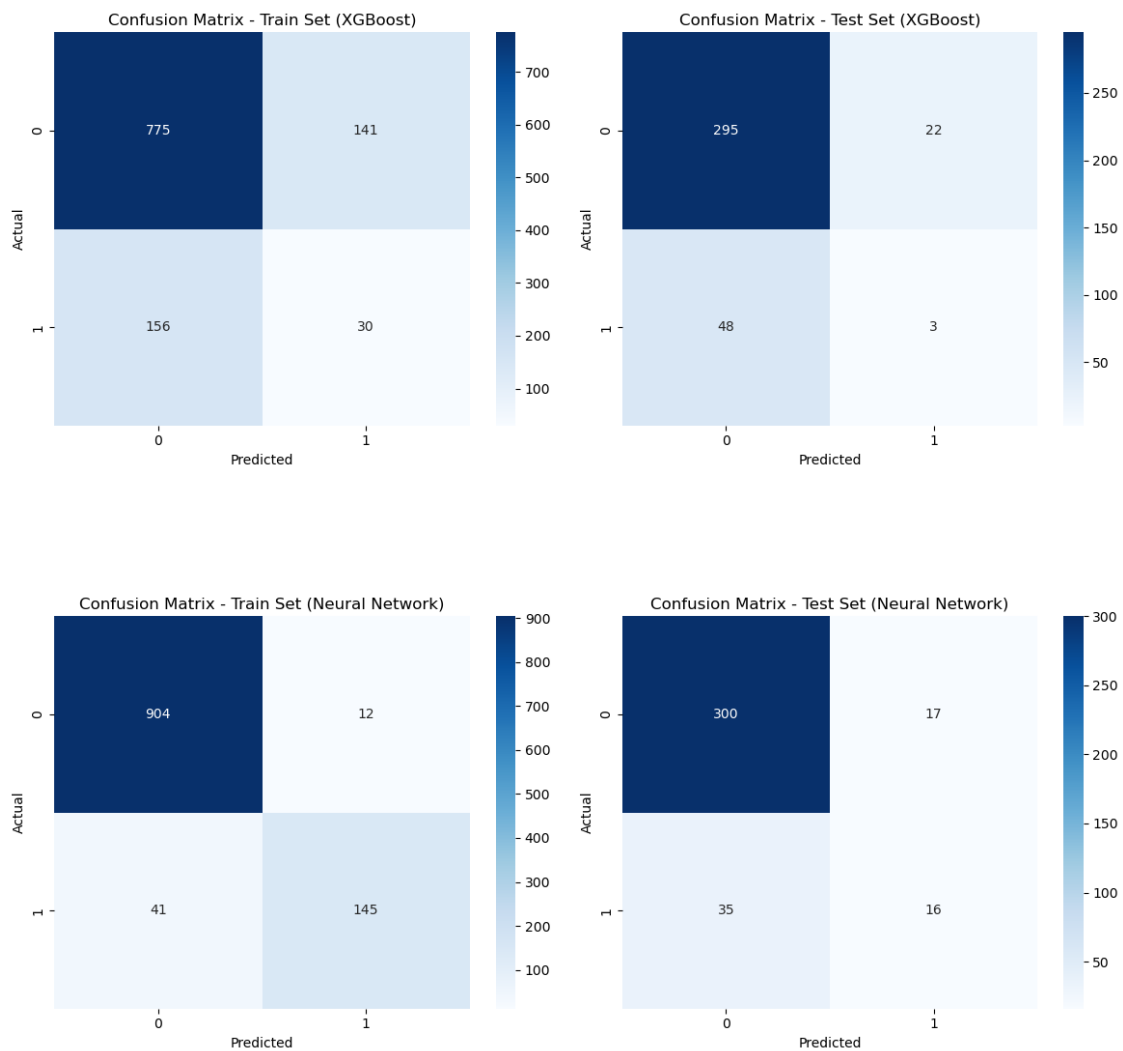
# Define model names and corresponding predictions
model_names = ['AdaBoost', 'Bagging', 'ExtraTrees', 'Random Forest', 'XGBoost',
               ↪ 'Neural Network']
y_pred_train = [y_pred_Ada_train, y_pred_Bagg_train, y_pred_Extra_train,
               ↪ y_pred_RF_train, y_pred_XGB_train, y_pred_ann_train]
y_pred_test = [y_pred_Ada_test, y_pred_Bagg_test, y_pred_Extra_test,
               ↪ y_pred_RF_test, y_pred_XGB_test, y_pred_ann_test]

# Build confusion matrix for each model
for y_train_pred, y_test_pred, model_name in zip(y_pred_train, y_pred_test,
               ↪ model_names):
    plot_cm(y_train, y_test, y_train_pred, y_test_pred, model_name)

```







```
[117]: # Initialize a list to store model results
model_results = []

# Calculate metrics for each model
for model_name, model in models.items():
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    if hasattr(model, 'predict_proba'):
        y_train_proba = model.predict_proba(X_train)[: , 1]
        y_test_proba = model.predict_proba(X_test)[: , 1]
    else:
        y_train_proba = y_train_pred
```

```

y_test_proba = y_test_pred

results = {
    'Model': model_name,
    'Train ROC AUC': roc_auc_score(y_train, y_train_proba),
    'Test ROC AUC': roc_auc_score(y_test, y_test_proba),
    'Train Accuracy': accuracy_score(y_train, y_train_pred),
    'Test Accuracy': accuracy_score(y_test, y_test_pred),
    'Train Precision': precision_score(y_train, y_train_pred),
    'Test Precision': precision_score(y_test, y_test_pred),
    'Train Recall': recall_score(y_train, y_train_pred),
    'Test Recall': recall_score(y_test, y_test_pred),
    'Train F1 Score': f1_score(y_train, y_train_pred),
    'Test F1 Score': f1_score(y_test, y_test_pred)
}

model_results.append(results)

# Convert the results to a DataFrame
model_results_df = pd.DataFrame(model_results)

# Reshape the DataFrame for easier plotting
model_results_melted = model_results_df.melt(id_vars='Model',
                                              value_vars=['Train ROC AUC', 'Test_
↳ROC AUC',
                                                         'Train Accuracy',
↳'Test Accuracy',
                                                         'Train Precision',
↳'Test Precision',
                                                         'Train Recall', 'Test_
↳Recall',
                                                         'Train F1 Score',
↳'Test F1 Score'],
                                              var_name='Metric',
↳value_name='Score')

# Function to plot results for a specific metric
def plot_metric(metric):
    plt.figure(figsize=(12, 6))
    data = model_results_melted[model_results_melted['Metric'].str.
↳contains(metric)]
    data = data.sort_values(by='Score', ascending=False)
    sns.barplot(data=data, x='Score', y='Model', hue='Metric', dodge=True)
    plt.title(f'{metric} Scores', fontsize=14)
    plt.ylabel('Model')
    plt.xlabel(f'{metric} Score')

```

```

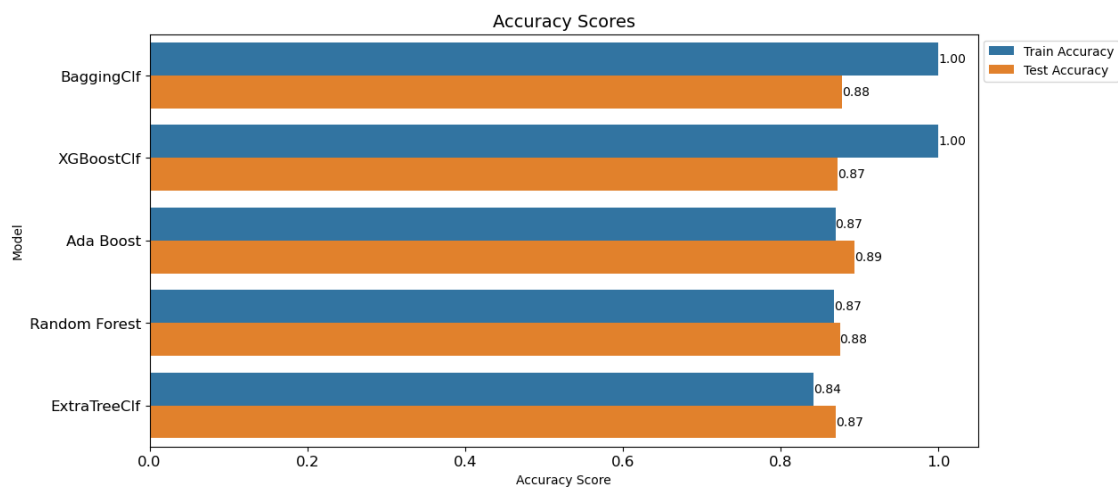
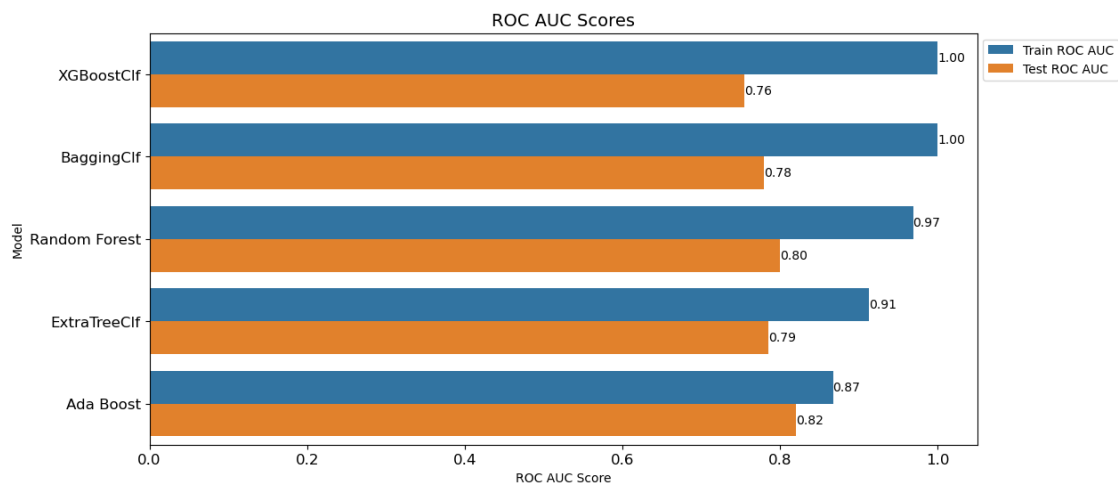
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))

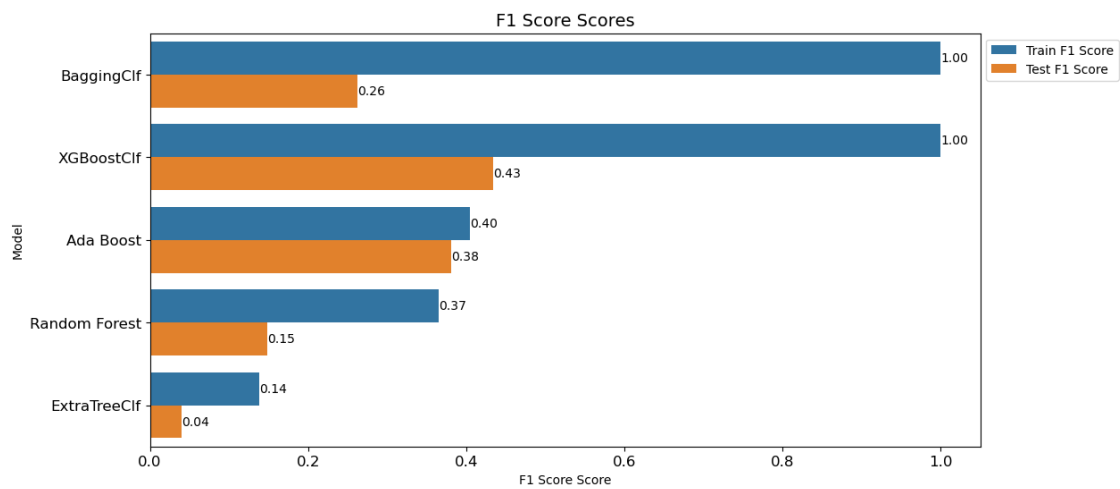
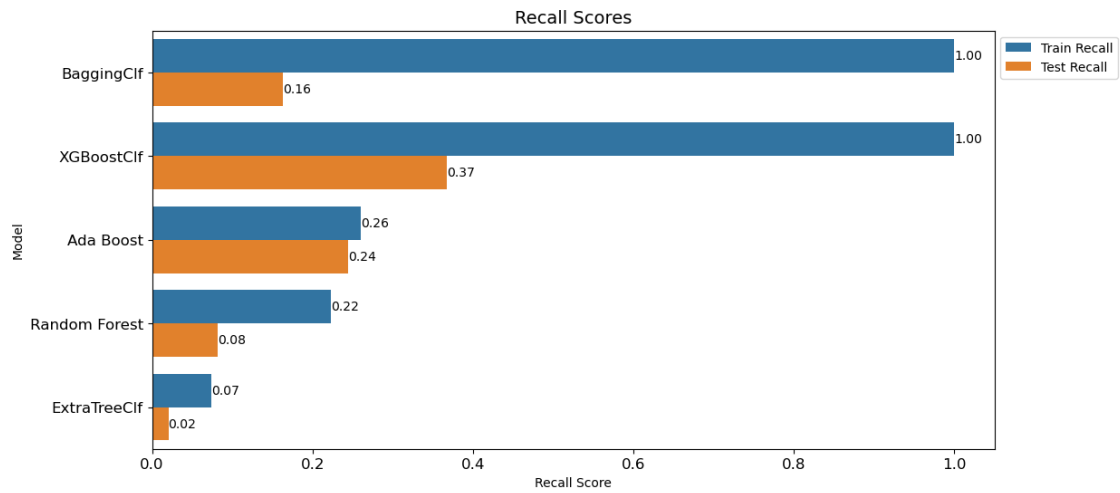
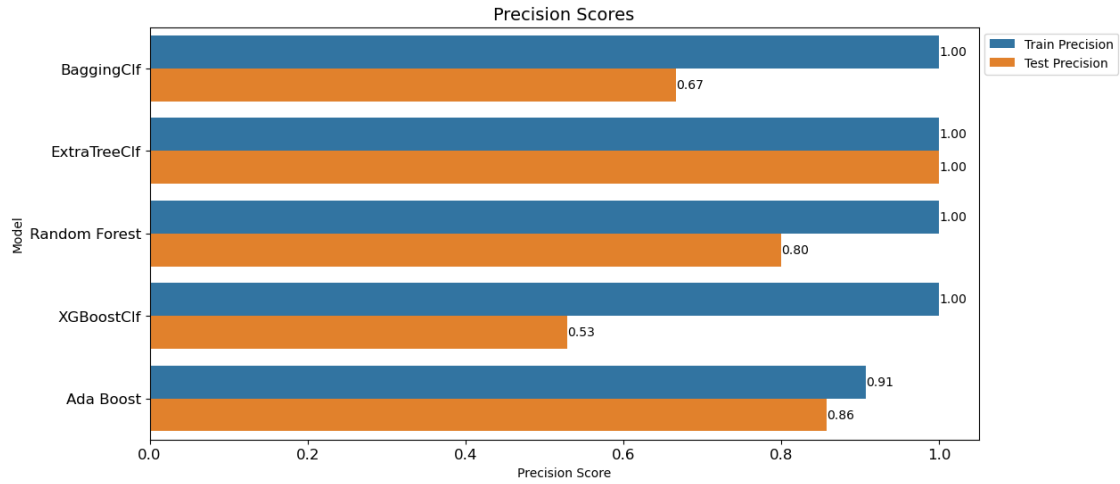
for container in plt.gca().containers:
    plt.gca().bar_label(container, fmt='%.2f', label_type='edge')

plt.show()

# Plot each metric separately
metrics = ['ROC AUC', 'Accuracy', 'Precision', 'Recall', 'F1 Score']
for metric in metrics:
    plot_metric(metric)

```





```
[128]: # Results Neural Network
y_train_pred_proba = best_model.predict(X_train_preprocessed)
y_train_pred = (y_train_pred_proba > 0.5).astype(int)

y_test_pred_proba = best_model.predict(X_test_preprocessed)
y_test_pred = (y_test_pred_proba > 0.5).astype(int)

train_accuracy = accuracy_score(y_train, y_train_pred)
train_roc_auc = roc_auc_score(y_train, y_train_pred_proba)
train_recall = recall_score(y_train, y_train_pred)
train_precision = precision_score(y_train, y_train_pred)
train_f1 = f1_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)
test_roc_auc = roc_auc_score(y_test, y_test_pred_proba)
test_recall = recall_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)

train_metrics = [train_accuracy, train_roc_auc, train_recall, train_precision,
                 ↪train_f1]
test_metrics = [test_accuracy, test_roc_auc, test_recall, test_precision,
                 ↪test_f1]
metric_names = ['Accuracy', 'ROC AUC', 'Recall', 'Precision', 'F1 Score']

x = np.arange(len(metric_names))
width = 0.35 #

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, train_metrics, width, label='Train',
               ↪color='skyblue')
bars2 = ax.bar(x + width/2, test_metrics, width, label='Test',
               ↪color='lightgreen')

for bar in bars1:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.2%}',
            ↪ha='center', va='bottom', color='blue', fontsize=10)
for bar in bars2:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.2%}',
            ↪ha='center', va='bottom', color='green', fontsize=10)

ax.set_ylabel('Score')
```

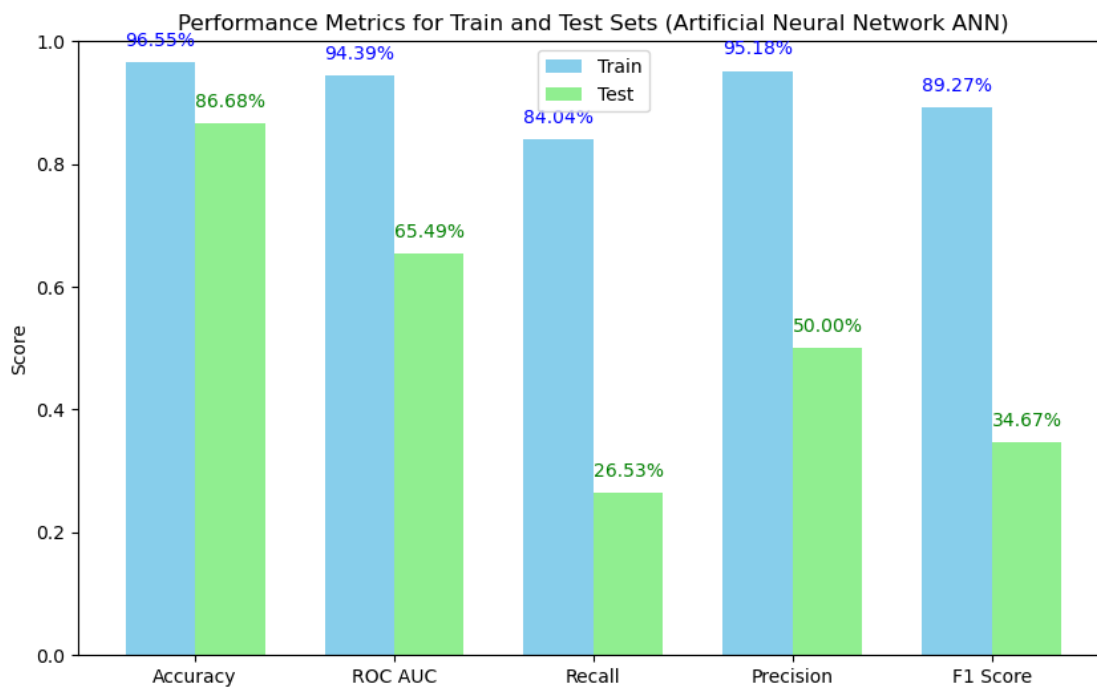
```

ax.set_title('Performance Metrics for Train and Test Sets (Artificial Neural_
↳Network ANN)')
ax.set_xticks(x)
ax.set_xticklabels(metric_names)
ax.set_ylim(0, 1)
ax.legend()

plt.show()

for metric_name, metric_value in zip(metric_names, test_metrics):
    print(f"{metric_name}: {metric_value:.2%}")

```



```

[131]: # Results ML models
calculate_performance_metrics(models, X_train, y_train, X_test, y_test,
↳list(models.keys()))

```

5. Conclusion

From the results, it is evident that the model with the highest scores in terms of accuracy and ROC-AUC is the neural network. However, we suspect overfitting. The other machine learning models showed more stable results in terms of accuracy, ranging from 84% to 89% on average on both training and testing datasets. Overfitting is observed in other metrics such as recall, precision, and F1, particularly in neural networks. Test data values range from 30% to 92% for neural networks and from 10% to 40% for other machine learning models.

accuracy ROC-AUC — accuracy , 84
 89 99% , recall, precision F1, ,
 30 92% 10
 40%

Recommendations: To improve model accuracy, we need a balanced dataset, as our current dataset is unbalanced—the target variable is unevenly distributed. Specifically, 1 represents those who left the company and 0 represents those who stayed. There are 237 observations of 1 and 1233 observations of 0. This is one of the reasons for the low metric scores. Additionally, we can try processing the variables differently and specifying more parameters for the models.

: , 1 , 0 — , 1 — 237
 , 0 — 1233 .